Anatomía de Tor

El protocolo del anonimato, byte a byte

Alessandro Greco

Primera edición — versión 1.0.0

Anatomía de Tor — El protocolo del anonimato, byte a byte
Primera edición, versión 1.0.0. Traducción al español.

Copyright © 2026 Alessandro Greco.

Esta obra es libre: el texto, las figuras, las fuentes LaTeX y el código de los laboratorios pueden redistribuirse y modificarse según los términos de la Licencia Pública General de GNU publicada por la Free Software Foundation, versión 3 o (a su elección) cualquier versión posterior (GPL-3.0-or-later). La obra se distribuye con la esperanza de que sea útil, pero *sin ninguna garantía*, ni siquiera la garantía implícita de comerciabilidad o de idoneidad para un propósito particular; los detalles están en la licencia, `https://www.gnu.org/licenses/gpl-3.0.html`.

La forma fuente modificable (fuentes LaTeX, figuras TikZ, código y red de laboratorio) se publica en `https://github.com/aleff-github/onion-routing-anatomy`.

El libro se distribuye gratuitamente; los derechos de uso y de redistribución son los que establece la licencia indicada.

Esta traducción se generó íntegramente con herramientas de inteligencia artificial a partir de la edición original italiana, que sigue siendo el texto de referencia de la obra.

«Tor» y el logotipo de Tor son marcas de The Tor Project, Inc. Este libro es una obra independiente y no está aprobado ni patrocinado por el Tor Project.

Los nombres de productos y servicios citados pertenecen a sus respectivos titulares y se usan solo con fines descriptivos.

Índice general

Prefacio **xi**

I Historia y fundamentos **1**

1 Anonimato, vigilancia y metadatos **3**
1.1 Contenido y metadatos . . . 3
1.1.1 Qué revelan los metadatos . . . 4
1.2 Análisis de tráfico . . . 4
1.3 Un vocabulario preciso . . . 5
1.4 ¿Quién necesita anonimato? . . . 5
1.4.1 El anonimato necesita compañía . . . 6

2 Los orígenes: redes mix, remailers, Crowds y Freedom **9**
2.1 El mix de Chaum (1981) . . . 9
2.2 Alta y baja latencia . . . 10
2.3 Remailers . . . 10
2.4 Hacia la baja latencia . . . 11

3 Del encaminamiento cebolla del NRL al Tor Project **15**
3.1 En el Naval Research Laboratory . . . 15
3.2 Tres generaciones . . . 15
3.3 La red Tor . . . 17
3.4 Una organización y una comunidad . . . 17

4 El modelo de amenazas de Tor **19**
4.1 Objetivos . . . 19
4.2 No objetivos . . . 20
4.3 El adversario . . . 20
4.3.1 Confirmación y análisis . . . 20
4.4 Qué protege Tor y qué queda fuera de su alcance . . . 21

II El protocolo a bajo nivel **23**

5 Criptografía esencial para Tor **25**
5.1 Cinco funciones . . . 25
5.2 Cifradores . . . 26
5.2.1 AES en modo contador . . . 26
5.2.2 TLS . . . 26
5.3 Funciones hash . . . 26

5.4 MAC y derivación de claves . . . 27
5.4.1 HMAC . . . 27
5.4.2 Funciones de derivación de claves . . . 27
5.5 Intercambio de claves: X25519 . . . 28
5.6 Firmas: Ed25519 y RSA . . . 28
5.7 Criptografía reciente . . . 28

6 Arquitectura de la red **31**
6.1 Funciones de los nodos . . . 31
6.2 Capas del protocolo . . . 32
6.3 Una página web, de principio a fin . . . 33
6.4 Implementaciones y versiones . . . 34
6.4.1 Dos implementaciones . . . 34
6.4.2 Subprotocolos . . . 34

7 Canales y protocolo de enlace **37**
7.1 ¿Qué es un canal? . . . 37
7.2 La identidad de un repetidor . . . 38
7.3 Resumen del intercambio inicial . . . 39
7.4 La capa TLS . . . 39
7.5 CERTS: demostrar la identidad . . . 41
7.5.1 La celda . . . 41
7.5.2 El certificado Ed25519 de Tor . . . 42
7.5.3 Verificación . . . 42
7.5.4 La cadena RSA heredada . . . 43
7.5.5 Qué envía realmente un repetidor . . . 43
7.6 AUTH_CHALLENGE y AUTHENTICATE . . . 44
7.6.1 ¿Quién se autentica? . . . 44
7.6.2 El desafío . . . 44
7.6.3 La respuesta . . . 45
7.7 NETINFO: hora y direcciones . . . 45
7.8 Después del intercambio inicial . . . 46
7.9 Laboratorio: abrir un canal verificado . . . 47

8 Las celdas de Tor **51**
8.1 ¿Por qué utilizar celdas? . . . 51
8.2 Del flujo TLS a las celdas . . . 52
8.3 Formatos de celda . . . 52
8.3.1 Constantes . . . 52
8.3.2 Celdas de longitud fija . . . 53
8.3.3 Celdas de longitud variable . . . 53
8.3.4 Cómo distinguir ambos formatos . . . 53
8.3.5 El caso especial de VERSIONS . . . 54
8.4 CircID . . . 54
8.4.1 ¿Quién elige el CircID? . . . 55
8.5 Comandos de celda . . . 55
8.5.1 DESTROY . . . 56
8.5.2 RELAY_EARLY . . . 57
8.6 Dentro de las celdas RELAY . . . 57
8.6.1 Recognized y Digest: un filtro de dos etapas . . . 58
8.6.2 Relleno de los mensajes relay . . . 58

8.6.3 Comandos relay . . . 59
8.7 Hacia un nuevo formato: propuesta 340 y CGO . . . 59
8.8 Anatomía de una sesión . . . 60
8.9 Laboratorio: codificador de celdas . . . 62

9 Construcción de circuitos: TAP, CREATE_FAST, ntor y ntor-v3 67
9.1 Un salto cada vez . . . 67
9.2 Celdas y mensajes . . . 69
9.2.1 CREATE2 y CREATED2 . . . 69
9.2.2 EXTEND2 y EXTENDED2 . . . 69
9.3 Qué debe garantizar un intercambio . . . 69
9.4 TAP: el primer intercambio . . . 70
9.5 CREATE_FAST: cuando no hace falta una clave pública . . . 70
9.6 ntor . . . 71
9.6.1 Los elementos . . . 71
9.6.2 El protocolo . . . 71
9.7 ntor-v3 . . . 72
9.7.1 Qué cambia . . . 72
9.7.2 El protocolo . . . 72
9.7.3 Extensiones . . . 73
9.7.4 ¿Qué intercambio utiliza tor actualmente? . . . 73
9.8 De los secretos negociados a las claves del circuito . . . 73
9.9 Laboratorio: construir circuitos reales . . . 74

10 La cebolla: cifrado de celdas relay y CGO 79
10.1 El esquema tradicional . . . 79
10.1.1 Sentido de ida: del cliente al circuito . . . 80
10.1.2 Sentido de vuelta: del circuito al cliente . . . 80
10.1.3 El circuito como «tubería con salidas intermedias» . . . 80
10.2 Reconocer una celda . . . 80
10.3 Debilidades de tor1 . . . 81
10.3.1 Maleabilidad . . . 81
10.3.2 El ataque de marcado . . . 81
10.3.3 Otras limitaciones . . . 82
10.4 Counter Galois Onion . . . 82
10.4.1 Propiedades . . . 82
10.4.2 Cómo funciona, a grandes rasgos . . . 82
10.4.3 CGO en la práctica . . . 83
10.5 Laboratorio: un circuito de tres saltos . . . 83

11 Flujos, SENDME y control de congestión 89
11.1 Flujos . . . 89
11.1.1 Abrir un flujo: BEGIN y CONNECTED . . . 89
11.1.2 Transferir datos: DATA . . . 91
11.1.3 Cerrar un flujo: END . . . 91
11.1.4 DNS en la salida . . . 91
11.1.5 Flujos de directorio: BEGIN_DIR . . . 91
11.2 Control de flujo original . . . 91
11.2.1 El problema . . . 91
11.2.2 Ventanas y SENDME . . . 92
11.2.3 Mensajes SENDME autenticados . . . 92

11.3 Control de congestión 92
11.3.1 La señal: tiempo de ida y vuelta 93
11.3.2 Los algoritmos 93
11.3.3 Los parámetros: ¿por qué 31? 93
11.3.4 Adiós a las ventanas de flujo: XON y XOFF 93
11.4 Laboratorio: descargar un archivo mediante Tor 94

12 El sistema de directorios **99**
12.1 El problema 99
12.2 Los participantes 100
12.3 Los documentos 100
12.4 Votación 102
12.4.1 El calendario 102
12.4.2 Combinar los votos 102
12.4.3 Indicadores 103
12.4.4 Ancho de banda 103
12.5 Anatomía de un consenso 103
12.6 Verificar un consenso 105
12.7 Cómo se mantiene actualizado un cliente 105
12.8 Laboratorio: descargar y verificar un consenso 106

13 Selección de rutas, guardias y vanguards **109**
13.1 Orden y restricciones 109
13.2 Ponderación por ancho de banda 110
13.2.1 Por qué importa la ponderación 110
13.2.2 Las posiciones del circuito no son equivalentes 110
13.2.3 Pesos de la red real 111
13.3 Guardias 111
13.3.1 El problema que resuelven 111
13.3.2 Tres, uno, dos 112
13.3.3 El algoritmo 112
13.4 Vanguards 113
13.5 Laboratorio: examinar la red pública 114

14 Servicios onion v3 **117**
14.1 Qué ofrece un servicio onion 117
14.2 La dirección 118
14.3 Claves cegadas 119
14.3.1 El problema 119
14.3.2 La solución 119
14.3.3 Credenciales y claves fuera de línea 119
14.4 Localizar el descriptor 120
14.4.1 Períodos temporales 120
14.4.2 El anillo HSDir 120
14.4.3 Solo solicitudes anónimas 121
14.5 El descriptor 121
14.6 Introducción y encuentro 122
14.7 Descubrimiento restringido 123
14.8 Defensas 124
14.9 Laboratorio: localizar y leer un descriptor 124

15 Defensas contra DoS y gestión de recursos **127**
15.1 Por qué DoS también amenaza el anonimato . . . 127
15.2 Recursos físicos . . . 128
15.2.1 Memoria . . . 128
15.2.2 CPU . . . 128
15.2.3 Ancho de banda . . . 128
15.3 Límites por dirección . . . 128
15.3.1 El código y el consenso . . . 129
15.4 Servicios onion bajo ataque . . . 129
15.4.1 Límites en los puntos de introducción . . . 130
15.4.2 Prueba de trabajo . . . 130
15.4.3 Esfuerzo lineal . . . 131
15.4.4 Ajustar el esfuerzo . . . 131
15.5 Laboratorio: activar las defensas . . . 131

16 Censura, puentes y transportes conectables **135**
16.1 Cómo se bloquea Tor . . . 135
16.2 Puentes . . . 136
16.2.1 El problema de la distribución . . . 136
16.2.2 Lox . . . 137
16.3 Transportes conectables . . . 137
16.3.1 La arquitectura . . . 137
16.3.2 El protocolo entre tor y el PT . . . 137
16.3.3 Lyrebird . . . 138
16.4 Estrategias . . . 138
16.4.1 obfs4: no parecerse a nada reconocible . . . 138
16.4.2 meek: detrás de un gran servicio . . . 139
16.4.3 Snowflake: numerosos proxies efímeros . . . 139
16.4.4 WebTunnel: parecerse a un sitio web . . . 139
16.4.5 Conjure: ocultarse en el espacio de direcciones . . . 139
16.5 Laboratorio: un puente obfs4 visto por un censor . . . 140

III Tor en la práctica **143**

17 Instalación y torrc **145**
17.1 ¿Qué Tor conviene instalar? . . . 145
17.2 El archivo torrc . . . 146
17.2.1 Comprobar antes de utilizar . . . 146
17.3 Una configuración de cliente . . . 146
17.3.1 Aislamiento de flujos . . . 147
17.3.2 Elegir repetidores: precaución . . . 147
17.3.3 Otros puertos . . . 148
17.4 El puerto de control . . . 148
17.5 Registros . . . 148

18 Puerto de control y Stem **151**
18.1 El protocolo de control . . . 151
18.1.1 Los circuitos vistos por tor . . . 152
18.1.2 Comandos principales . . . 153
18.2 Stem . . . 153

18.3 Laboratorio: aislamiento de flujos en directo 154

19 Una red Tor privada: Chutney y Shadow **157**
19.1 ¿Por qué una red privada? 157
19.2 ¿Emulación o simulación? 158
19.3 El laboratorio del libro 159
19.3.1 Requisitos 159
19.3.2 Construir la imagen e iniciar la red 159
19.4 Topología de red 160
19.5 Una red de prueba no es la red real 161
19.6 Dentro del directorio de un nodo 162
19.7 Utilizar la red 163
19.7.1 Navegar a través de la red privada 163
19.7.2 Acceder al servicio onion 163
19.7.3 Observar los circuitos con Stem 164
19.7.4 Trabajar dentro del contenedor 164
19.8 Primer contacto con un repetidor 164
19.9 Shadow: una red dentro de un simulador 166

20 Observar el tráfico: Wireshark y registros **169**
20.1 Qué puede ver un observador 169
20.2 Las herramientas 170
20.2.1 Descifrar tu propia conexión TLS 170
20.3 Laboratorio: celdas dentro de TLS 171
20.4 Los registros de tor 172

21 Publicar un servicio onion **175**
21.1 Tu primer servicio onion 175
21.1.1 Evitar revelar la ubicación del servidor 176
21.2 Onion-Location 177
21.3 Descubrimiento restringido 177
21.4 Otros requisitos 179
21.4.1 Direcciones personalizadas 179
21.4.2 Servicios de un solo salto 179
21.4.3 Onionbalance 179

22 Administrar un repetidor o un puente **181**
22.1 ¿Qué tipo de nodo? 181
22.2 Configuración esencial 182
22.2.1 La vida de un repetidor nuevo 182
22.2.2 Claves 183
22.3 Política de salida 183
22.4 Laboratorio: comprobar una política de salida 184
22.5 Familias de repetidores 185
22.6 Administrar un puente 186

23 Tor Browser, Tails y Whonix **189**
23.1 Por qué no basta con un proxy 189
23.2 Tor Browser 190
23.2.1 Un circuito por sitio 190
23.2.2 Resistencia a las huellas digitales 191

23.2.3 Niveles de seguridad . . . 191
23.3 Más allá del navegador . . . 192
23.3.1 Tails . . . 192
23.3.2 Whonix . . . 193

24 Medir la red: Tor Metrics y OONI 195
24.1 Fuentes de datos . . . 195
24.2 Contar usuarios anónimos . . . 196
24.2.1 Detectar la censura . . . 197
24.3 Rendimiento . . . 197
24.4 Laboratorio: leer un consenso real . . . 197
24.5 OONI . . . 198

25 Aspectos legales y éticos en Italia y la UE 201
25.1 Una herramienta y sus usos . . . 201
25.2 Operadores de repetidores . . . 202
25.2.1 Mera transmisión . . . 202
25.2.2 Responsabilidad civil e investigaciones penales . . . 203
25.2.3 Conservación de datos . . . 203
25.2.4 Los registros son datos personales . . . 203
25.3 Investigadores . . . 204

IV Ataques y defensas 207

26 Correlación de tráfico, identificación de sitios y ataques Sybil 209
26.1 La posición del adversario . . . 209
26.2 Correlación de extremo a extremo . . . 210
26.3 Laboratorio: correlacionar tráfico propio . . . 211
26.4 Identificación de sitios web . . . 212
26.5 Sybil: multiplicar identidades de repetidores . . . 213
26.6 Descubrir el guardia . . . 213

27 Casos reales de ataques contra la red 215
27.1 2014: señales mediante tipos de celda . . . 215
27.2 Laboratorio: el límite de RELAY_EARLY . . . 216
27.3 2020–2021: KAX17 y la salud de la red . . . 217
27.4 2022–2023: denegación de servicio . . . 218
27.5 Lecciones . . . 218

28 Errores de seguridad operativa y desanonimización: lecciones 221
28.1 Cuatro capas . . . 221
28.2 Errores de aplicación . . . 222
28.3 Laboratorio: tor detecta un indicio de filtración DNS . . . 222
28.4 Errores de contenido . . . 223
28.5 Errores de comportamiento: un conjunto de anonimato demasiado pequeño . 224
28.6 Una lista de comprobación . . . 224

V Investigación y futuro **227**

29 Investigación académica sobre Tor **229**
29.1 Dónde se publica investigación sobre Tor . . . 229
29.2 Medir el anonimato . . . 230
29.3 Cuatro métodos . . . 230
29.4 Laboratorio: tiempo hasta la primera exposición . . . 231
29.5 Investigar de forma segura . . . 233

30 Arti: Tor en Rust **235**
30.1 ¿Por qué reescribir tor? . . . 235
30.2 Arquitectura . . . 236
30.2.1 Una comparación: escribir una celda . . . 237
30.3 Funciones admitidas . . . 237
30.4 Laboratorio: Arti en la red privada . . . 238

31 Problemas abiertos y proyectos en curso **243**
31.1 Cómo cambia el protocolo . . . 243
31.2 Ordenadores cuánticos . . . 244
31.3 Mensajes más grandes y más pequeños . . . 245
31.4 Rendimiento: múltiples rutas . . . 245
31.5 Escalabilidad: Walking Onions . . . 245
31.6 Otros frentes abiertos . . . 246
31.7 Cómo contribuir . . . 246

A Glosario de términos de Tor **249**

B Criptografía complementaria **253**
B.1 Una curva, dos formas . . . 253
B.1.1 Ajuste de bits de claves X25519 . . . 253
B.1.2 El punto en el infinito . . . 254
B.2 De Montgomery a Edwards: el certificado cruzado ntor . . . 254
B.3 Maleabilidad del cifrado de flujo . . . 255

C Guía de las especificaciones torspec **257**
C.1 Dónde encontrarlas . . . 257
C.2 Convenciones . . . 257
C.3 Mapa de documentos . . . 258
C.4 Cómo leer una sección . . . 258
C.5 Arti y las especificaciones . . . 259

D Soluciones de los ejercicios **261**

Bibliografía **285**

Índice analítico **289**

Prefacio

Millones de personas utilizan Tor cada día y, sin embargo, casi siempre se habla de él en función de sus usos, reales o supuestos: eludir la censura, proteger las fuentes de información o acceder a la «web oscura». Este libro adopta otro enfoque: explica cómo funciona Tor. Parte de los bytes que intercambian dos repetidores y avanza, capítulo a capítulo, hasta las decisiones de diseño, los ataques, los casos reales y las preguntas que siguen abiertas. Además, no pide al lector que acepte sus afirmaciones sin comprobarlas: cada mecanismo descrito en la parte técnica se construye y se pone a prueba en un laboratorio, dentro de una red Tor privada que puede ejecutarse en un ordenador corriente.

Origen, método e invitación a revisar

Este libro parte de apuntes y documentos de estudio que he elaborado a lo largo de los años, desde mi trabajo de fin de grado. Para convertir esos materiales en una obra técnica coherente, dirigí el uso de herramientas de inteligencia artificial que contribuyeron de forma sustancial a desarrollar las explicaciones, argumentar y redactar el texto, y crear figuras, ejercicios, código y laboratorios. La IA no se utilizó solo para pulir mi redacción ni sustituye una revisión independiente de expertos.

A pesar de las comprobaciones técnicas y de mi propia revisión, puede haber errores, simplificaciones o información desactualizada. Invito a quien lea el libro a señalarlos y proponer correcciones mediante el repositorio del proyecto. He decidido ofrecer gratuitamente la obra por los canales oficiales, junto con sus fuentes bajo la licencia GNU GPL-3.0-or-later, para favorecer la «diffusione dei saperi senza fondare poteri»: difundir conocimientos sin instaurar poderes. La licencia regula también la redistribución comercial.

A quién va dirigido este libro

Este libro está pensado para estudiantes de informática y ciberseguridad, profesionales que encuentran Tor en su trabajo, personas que administran o desean administrar un repetidor e investigadores. Para seguirlo conviene contar con conocimientos básicos de redes (TCP/IP y los fundamentos de TLS), cierta familiaridad con la línea de comandos y, para realizar los laboratorios, Python. La criptografía necesaria se presenta en capítulo 5, con explicaciones adicionales en apéndice B.

Cómo leer este libro

La obra está dividida en cinco partes. Puede leerse de principio a fin o siguiendo uno de estos recorridos:

Recorrido histórico y conceptual Parte I (capítulos 1–4) y parte V (capítulos 29–31): qué significa el anonimato, de dónde procede Tor, frente a qué adversarios ofrece protección y hacia dónde evoluciona.

Recorrido por el protocolo Parte II (capítulos 5–16), que conviene leer en orden y acompañar con los laboratorios: constituye el núcleo de la obra.

Recorrido práctico Parte III (capítulos 17–25): configurar `tor`, observar su tráfico, publicar un servicio onion, administrar un repetidor, utilizar Tor Browser, medir la red y comprender sus implicaciones jurídicas. El capítulo 19 es un requisito previo para todos los laboratorios.

Recorrido de seguridad Parte IV (capítulos 26–28), precedida del capítulo 4: ataques examinados a través de investigaciones, casos reales y errores operativos.

Convenciones

Cada capítulo comienza con sus objetivos de aprendizaje y una indicación de *Última verificación*, que especifica la fecha y las versiones del software y de las especificaciones utilizadas para comprobar el contenido. Tor evoluciona, y el lector debe saber a qué versión se refiere el texto. A lo largo del libro, *Tor* designa la red y el proyecto; *tor*, el programa escrito en C; y *Arti*, la implementación en Rust. Las referencias a las especificaciones se presentan como [document § section]; las propuestas de modificación del protocolo se identifican por su número. Los términos técnicos empleados en esta edición se definen en el apéndice A.

Los recuadros separan el hilo principal de las digresiones: *explicaciones en profundidad*, *notas históricas* y *advertencias*. Los ejercicios se clasifican mediante estrellas: una para comprensión, dos para aplicación y tres para una actividad de laboratorio o un pequeño proyecto de investigación. Sus soluciones aparecen en el apéndice D. Los laboratorios están numerados del 1 al 20. En la parte II se construye progresivamente un pequeño cliente Tor en Python (`torlab`); el código y la red de laboratorio acompañan al libro, y las pruebas automatizadas pueden volver a ejecutarse tras las actualizaciones. Todos estos recursos —el paquete `torlab`, los veinte laboratorios, la red de pruebas con Docker y las herramientas de verificación— se encuentran en el repositorio del libro: `https://github.com/aleff-github/onion-routing-anatomy`.

Nota ética

Todos los experimentos de este libro se realizan en la red Tor privada del laboratorio o utilizan la red pública únicamente como lo haría un cliente ordinario, sin observar ni modificar el tráfico ajeno. Este es el principio recomendado a los investigadores por el Tor Research Safety Board y también tiene implicaciones jurídicas (capítulo 25). Los capítulos sobre ataques adoptan una perspectiva analítica y defensiva: explican qué se ha hecho y cómo protegerse, no cómo atacar a otras personas.

Agradecimientos

Este libro se apoya en el trabajo del Tor Project y de su comunidad. Las especificaciones abiertas, el código fuente de `tor` y Arti, las propuestas de modificación del protocolo y la documentación son la base de las afirmaciones técnicas que siguen. Su disponibilidad pública permite contrastarlas con las fuentes en lugar de exigir al lector un acto de confianza. También merecen reconocimiento quienes desarrollan y mantienen las herramientas libres empleadas

para escribir el libro y comprobar sus laboratorios: LaTeX, Python, Docker, Chutney, Wireshark y los demás proyectos citados a lo largo de la obra.

Parte I

Historia y fundamentos

Capítulo 1

Anonimato, vigilancia y metadatos

Última verificación: 23 de septiembre de 2026 — datos de Tor Metrics disponibles hasta el 20 de septiembre de 2026

Objetivos del capítulo

Al terminar este capítulo, el lector podrá:

- distinguir entre confidencialidad, anonimato, seudonimato, desvinculación y no observabilidad;
- explicar por qué los metadatos de una comunicación pueden revelar tanta información como su contenido, e incluso más;
- describir el análisis de tráfico y explicar por qué el cifrado por sí solo no puede impedirlo;
- identificar quién utiliza Tor y por qué la diversidad de sus usuarios constituye, en sí misma, una forma de protección.

Este libro trata sobre un sistema diseñado para proteger un tipo concreto de información: no *qué* decimos en Internet, sino *con quién* nos comunicamos. Antes de examinar su funcionamiento, debemos comprender el problema que resuelve y por qué es distinto del problema, más conocido, al que responde el cifrado.

1.1 Contenido y metadatos

Cuando dos ordenadores se comunican por Internet, cada paquete transporta dos tipos de información. El **contenido** es lo que intercambian las aplicaciones: el texto de un mensaje o los datos de una página web. Los **metadatos** describen la comunicación *en sí misma*: quién la envía, a quién, cuándo, con qué frecuencia y qué cantidad de datos transporta. Algunos metadatos son indispensables para el funcionamiento de la red, porque los encaminadores necesitan saber a dónde entregar los paquetes. La dirección IP de destino y la dirección IP de origen, a la que deben dirigirse las respuestas, aparecen sin cifrar en cada paquete.

El cifrado, hoy ampliamente extendido gracias a HTTPS, protege el contenido, pero deja visibles los metadatos. Quien observe la red, desde el proveedor de acceso a Internet de una persona hasta el operador de un gran punto de intercambio de tráfico, todavía puede ver que esa persona se conecta a un sitio concreto, cuándo se establece la conexión, cuánto dura y qué volumen de tráfico genera.

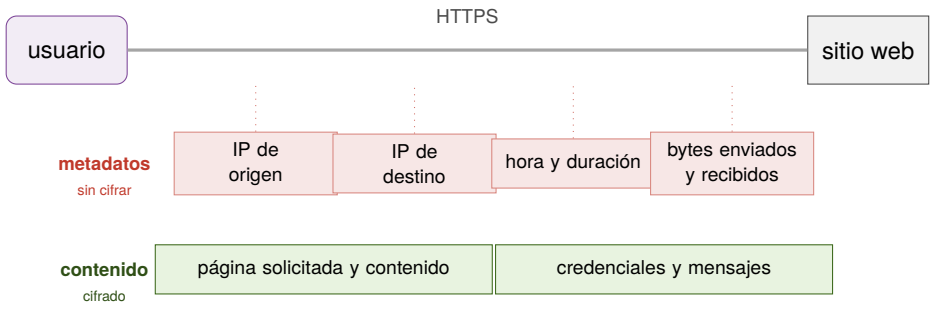


Figura 1.1: Qué puede observarse en una conexión HTTPS. El contenido está cifrado, pero los metadatos siguen siendo visibles; proteger esta información es precisamente lo que Tor intenta conseguir.

1.1.1 Qué revelan los metadatos

Puede resultar tentador considerar los metadatos inofensivos: «Solo saben con quién hablo, no lo que digo». Esa impresión es engañosa. Algunos ejemplos permiten entender por qué:

- una persona visita a las dos de la madrugada el sitio web de un centro de apoyo a víctimas de violencia doméstica y después el de un abogado especializado en divorcios;
- en los días previos a la publicación de una investigación, una periodista intercambia mensajes con un empleado del ministerio al que se refiere su trabajo;
- un trabajador visita regularmente el sitio web de un sindicato o el de una asociación de pacientes dedicada a una determinada enfermedad.

En ninguno de estos casos hace falta leer el contenido para deducir una gran cantidad de información. Para quienes recopilan metadatos, estos presentan además una ventaja: tienen una estructura definida, resultan fáciles de almacenar y analizar a gran escala y pueden cruzarse con otras fuentes. El general Michael Hayden, antiguo director de la NSA y de la CIA, resumió esta idea en una declaración que se hizo célebre durante un debate en la Universidad Johns Hopkins, el 1 de abril de 2014: «Matamos a personas basándonos en metadatos» (traducción propia), antes de matizar de inmediato: «pero eso no es lo que hacemos con estos metadatos» [Jus14]. La matización no cambia la cuestión central: quienes recopilan metadatos consideran que pueden bastar para fundamentar decisiones de semejante gravedad.

1.2 Análisis de tráfico

El **análisis de tráfico** estudia los metadatos de las comunicaciones para inferir información sin necesidad de leer el contenido. Es una práctica tan antigua como las telecomunicaciones. Incluso durante las guerras del siglo XX, observar con qué frecuencia transmitía una estación de radio enemiga, a quién y con qué ritmo podía revelar la estructura de un ejército sin descifrar ninguno de sus mensajes.

En Internet, el análisis de tráfico adopta dos formas principales:

Análisis de direcciones Quien observa un enlace ve las direcciones IP de origen y destino. Es la forma más sencilla y aquella contra la que Tor actúa de manera más directa: ningún punto individual de la red conoce simultáneamente ambos extremos.

Análisis del volumen y del tiempo Incluso sin disponer de toda la información sobre las direcciones, un observador puede comparar flujos según su tamaño y sus tiempos. Si una persona descarga 3 MB a las 10:15:02 y un servidor situado en otro punto de la red envía 3 MB en ese mismo instante, es probable que ambas observaciones correspondan a la misma comunicación. Esta **correlación de tráfico** es la principal limitación de Tor, a la que volveremos en numerosas ocasiones (capítulos 4 y 26).

1.3 Un vocabulario preciso

La palabra «anonimato» se utiliza con distintos significados, y confundirlos ha dado lugar a errores de diseño a lo largo de la historia de estos sistemas. La literatura científica, en particular la terminología propuesta por Pfitzmann y Hansen [PH10], distingue varios conceptos:

Confidencialidad Una parte no autorizada no puede leer el contenido. Es la propiedad que proporciona el cifrado.

Anonimato No es posible identificar a un sujeto dentro de un conjunto de sujetos, denominado **conjunto de anonimato**. El anonimato nunca es absoluto: existe «entre» otras personas. Cuanto mayor y más diverso sea el conjunto, mayor será la protección.

Seudonimato Una persona actúa bajo un identificador distinto de su nombre, que puede ser estable en el tiempo. Una cuenta con un nombre inventado representa un seudónimo, no una forma de anonimato: todas sus acciones pueden vincularse entre sí.

Desvinculación Un observador no puede determinar si dos elementos de interés (dos mensajes, una persona y un sitio web, o dos visitas) están relacionados. Esta es la propiedad que Tor trata de ofrecer entre los usuarios y sus destinos.

No observabilidad Un observador ni siquiera puede determinar *si* se está produciendo una comunicación. Es una propiedad mucho más fuerte, que Tor no ofrece: quien observa a un usuario puede ver que está utilizando Tor, salvo que se emplee un transporte conectable apropiado (capítulo 16).

Nota histórica: Separar la identificación del encaminamiento

Los tres investigadores del Naval Research Laboratory que idearon el encaminamiento cebolla en 1995 —David Goldschlag, Michael Reed y Paul Syverson— preferían no hablar de anonimato. Su objetivo, tal como escribieron en un artículo de 1996 al que Syverson volvería al recordar aquella época, era «separar la identificación del encaminamiento» [Syv11]. Una persona puede querer identificarse ante el sistema con el que se comunica, por ejemplo el ordenador de su oficina, sin que todos los puntos intermedios de la red sepan que alguien ubicado en su lugar de origen se está conectando a esa oficina.

1.4 ¿Quién necesita anonimato?

Según las estimaciones de Tor Metrics, entre 2,8 y 3,5 millones de usuarios se conectaban cada día a la red Tor durante agosto y septiembre de 2026 sin utilizar puentes, con una media aproximada de 3,2 millones. Para Italia, las estimaciones diarias oscilaban entre 44 000 y 83 000, con una media cercana a 60 000. Estos datos presentan una incertidumbre considerable porque la red *estima* el número de usuarios, no los cuenta directamente (capítulo 24). ¿Quiénes son? Pertenecen a grupos muy distintos:

- personas corrientes que no desean ser rastreadas por anunciantes, proveedores de acceso a Internet o administradores de las redes que utilizan;
- periodistas y sus fuentes, activistas y defensores de los derechos humanos, especialmente en países con gobiernos autoritarios;
- personas que viven bajo censura y utilizan Tor para acceder a sitios bloqueados (capítulo 16);
- víctimas de violencia o persecución que necesitan ocultar su ubicación;
- empresas que no quieren revelar a sus competidores sus intereses de investigación;
- agentes de las fuerzas de seguridad y personal militar que necesitan visitar sitios o recopilar información sin revelar su identidad: el encaminamiento cebolla nació precisamente con este propósito en un laboratorio de la Marina de Estados Unidos.

Una pequeña proporción de usuarios emplea Tor para actividades ilegales, como sucede con cualquier herramienta de comunicación. El debate público suele concentrarse en ese grupo. Examinaremos la cuestión con pruebas y casos concretos en la parte IV y en el capítulo 25.

1.4.1 El anonimato necesita compañía

Existe una razón más profunda por la que importa la diversidad de usuarios. Un sistema de anonimato utilizado por un único grupo —por ejemplo, las fuerzas armadas— no protegería a nadie dentro de él: al observar que alguien se conecta a la red, un tercero sabría de inmediato que pertenece a las fuerzas armadas, y el tráfico que saliera de ella se reconocería como tráfico militar. Syverson señala exactamente este problema al hablar de una red que la Marina estadounidense podría haber construido para uso exclusivo: todo lo que saliera de ella sería identificable como tráfico de la Marina [Syv11].

El anonimato de cada persona depende de cuántas otras utilizan el sistema y de lo diversas que sean. La expresión «el anonimato necesita compañía», que también da título a un artículo de Dingledine y Mathewson, resume este principio [DM06]. Por ello Tor está diseñado para estar al alcance de cualquiera, ser gratuito y tener código abierto: cuantos más usuarios lo utilicen, mayor será la protección potencial para todos.

Resumen

- El cifrado protege el contenido; los metadatos (quién, con quién, cuándo y cuánto) siguen siendo visibles y pueden revelar mucho.
- El análisis de tráfico infiere información a partir de las direcciones, el volumen y los tiempos de las comunicaciones.
- El anonimato significa no ser identificable dentro de un conjunto; debe distinguirse del seudonimato, la desvinculación y la no observabilidad.
- Tor sirve a millones de usuarios muy diferentes, y esa diversidad forma parte de su protección.

Ejercicios

Ejercicio 1.1 ⋆ Enumera los metadatos que puede observar un proveedor de acceso a Internet cuando uno de sus clientes lee un artículo en un sitio web de noticias mediante HTTPS. ¿Qué información sigue siendo visible si utiliza una VPN? ¿Y si utiliza Tor?

Ejercicio 1.2 ⋆ Clasifica cada uno de los casos siguientes como anonimato, seudonimato o ninguno de los dos: una cuenta en una red social con un nombre inventado; un mensaje sin firma dejado en un tablón público; una carta firmada con iniciales; una donación en efectivo depositada en una caja de recaudación.

Ejercicio 1.3 ⋆⋆ En el transcurso de una hora, un sitio web recibe visitas de 50 usuarios de Tor diferentes. Uno de ellos publica un comentario con un error ortográfico característico de los hablantes de italiano, a las tres de la madrugada en Italia. ¿Cómo cambia esto su conjunto de anonimato? ¿Qué enseña el ejemplo sobre el papel del comportamiento del usuario?

Ejercicio 1.4 ⋆⋆ Compara una VPN comercial y Tor en cuanto a quién puede relacionar a los usuarios con sus destinos. ¿Dónde se concentra la confianza en cada caso?

Lecturas complementarias

- A. Pfitzmann, M. Hansen, *A terminology for talking about privacy by data minimization* [PH10]: terminología de referencia.
- P. Syverson, *A Peel of Onion* [Syv11]: historia del encaminamiento cebolla narrada por uno de sus creadores.
- Tor Metrics, `https://metrics.torproject.org/`: estimaciones de usuarios por país.

Capítulo 2

Los orígenes: redes mix, remailers, Crowds y Freedom

Última verificación: 23 de septiembre de 2026 — fuentes históricas citadas en este capítulo

Objetivos del capítulo

Al terminar este capítulo, el lector podrá:

- explicar la idea de *mix* propuesta por David Chaum y por qué es fundamental para los sistemas de anonimato;
- distinguir los sistemas de alta latencia destinados al correo electrónico de los sistemas de baja latencia adecuados para navegar por la web;
- seguir la evolución de los remailers desde anon.penet.fi hasta Mixminion;
- situar Crowds, PipeNet y Freedom en la historia que condujo a Tor.

Tor no apareció de la nada. Representa, al menos por ahora, la etapa más reciente de más de cuarenta años de investigación y sistemas puestos en funcionamiento. Algunos tuvieron éxito y muchos fracasaron, pero todos dejaron enseñanzas. Este capítulo recorre los principales avances anteriores al surgimiento del encaminamiento cebolla.

2.1 El mix de Chaum (1981)

En 1981, David Chaum publicó un artículo de cinco páginas que sentaría las bases de todo un campo de investigación: *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81]. La cuestión se refería al correo electrónico: ¿cómo enviar un mensaje sin que un observador pudiera relacionar al remitente con el destinatario, incluso si se vigilara toda la red?

La solución era el **mix**: un servidor que recibe mensajes cifrados con su clave pública, los descifra, los reúne en un grupo (o *lote*) y los reenvía juntos en un orden distinto del de llegada. Tres técnicas impiden relacionar un mensaje entrante con uno saliente:

- el mensaje de salida tiene un aspecto completamente distinto porque el mix ha retirado una capa de cifrado;
- todos los mensajes tienen la misma longitud;
- los mensajes se mezclan y salen juntos, de modo que tampoco puedan vincularse por sus tiempos de salida.

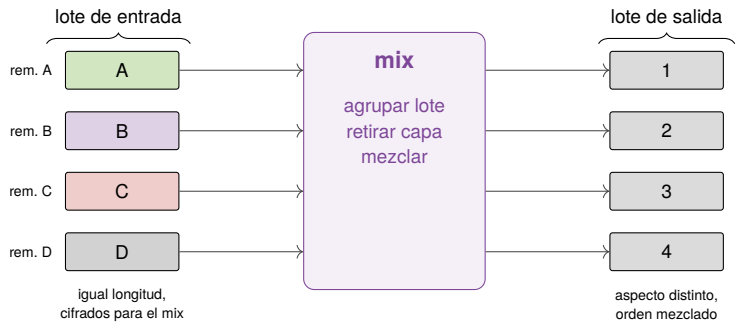


Figura 2.1: Un mix de Chaum. Un observador que vigile tanto la entrada como la salida no puede vincular los mensajes: todos tienen la misma longitud, su aspecto cambia al retirarse una capa de cifrado y salen juntos en un orden diferente.

En lugar de confiar en un único mix, el remitente puede enviar el mensaje a través de una **cadena** de mixes, cifrándolo por capas con la clave de cada uno. Cada mix elimina su propia capa y solo conoce la identidad del siguiente. Si al menos uno de los mixes de la cadena es honesto, el vínculo sigue oculto. Aquí se encuentra el germen del encaminamiento cebolla.

En el mismo artículo, Chaum propuso *direcciones anónimas de respuesta*, que permiten contestar sin conocer al remitente, además de seudónimos digitales. En 1988 añadió un segundo mecanismo, las *DC-nets* [Cha88], que proporcionan anonimato incondicional a un coste muy elevado y cuya influencia ha sido principalmente teórica.

2.2 Alta y baja latencia

El mix de Chaum ofrece una protección sólida, pero es **lento**: un mensaje debe esperar en cada mix hasta que se forme un lote y puede tardar minutos u horas en llegar. Esto resulta aceptable para el correo electrónico, pero no para la navegación web, donde se espera que una página se cargue en segundos.

Esta es la división fundamental del campo:

Sistemas de alta latencia Los mixes y remailers retrasan y reordenan los mensajes. Pueden resistir a un observador que vigile toda la red, pero solo sirven para comunicaciones que admitan demoras.

Sistemas de baja latencia El encaminamiento cebolla y Tor reenvían los datos en cuanto llegan, lo que permite navegar, conversar y mantener conexiones interactivas. Sin embargo, un observador capaz de vigilar ambos extremos puede correlacionar los tiempos y volúmenes de tráfico.

Tor elige deliberadamente el segundo enfoque: un sistema que casi nadie utiliza por ser demasiado lento no protege a nadie (apartado 1.4).

2.3 Remailers

En la década de 1990, las ideas de Chaum se transformaron en servicios reales llamados **remailers**: servidores que reciben un mensaje de correo y lo reenvían después de retirar la información del remitente.

anon.penet.fi (1993–1996). El remailer más conocido fue operado en Finlandia por Johan «Julf» Helsingius entre 1993 y 1996. No era un mix: asignaba un *seudónimo* a cada usuario y conservaba una tabla que relacionaba los seudónimos con las direcciones reales, para poder entregar las respuestas. Llegó a procesar miles de mensajes al día. Aquella misma tabla terminó contribuyendo a su cierre. En 1995 y 1996, la Iglesia de la Cienciología, con la intervención de la policía finlandesa, obtuvo las identidades de algunos usuarios que habían distribuido documentos de esa organización. Helsingius cerró el servicio en 1996, alegando la incertidumbre jurídica que rodeaba a Internet en Finlandia. La lección que retomaron expresamente los diseñadores del encaminamiento cebolla [Syv11] es clara: un sistema que conserva una correspondencia entre usuarios e identidades crea un punto único de fallo, pues puede obligarse a su operador a revelarla.

Cypherpunk, Mixmaster y Mixminion. Mientras tanto, la comunidad *cypherpunk* desarrolló remailers sin tablas de correspondencia. Los remitentes cifraban los mensajes por capas para una cadena de servidores. Las generaciones sucesivas, de Mixmaster a Mixminion [DDM03], incorporaron mensajes de longitud fija, agrupación real por lotes y mecanismos de defensa frente a la repetición de mensajes. Roger Dingledine y Nick Mathewson figuraron entre los autores de Mixminion, publicado en 2003, cuando también trabajaban en el diseño de Tor.

2.4 Hacia la baja latencia

PipeNet (1995). En un mensaje enviado a la lista de correo cypherpunk en febrero de 1995, Wei Dai propuso PipeNet, un sistema de baja latencia en el que todos los participantes enviarían tráfico continuamente a una velocidad constante, de manera que no quedaran variaciones temporales que correlacionar. Resistiría el análisis temporal, pero a un precio inaceptable: si un solo participante dejara de transmitir, toda la red tendría que detenerse [Syv11]. PipeNet nunca llegó a implementarse, aunque sigue siendo un punto de referencia sobre lo que haría falta para contrarrestar la correlación de tráfico.

Encaminamiento cebolla (1996). Ese mismo año comenzaba a tomar forma el encaminamiento cebolla en el Naval Research Laboratory. Contaremos su historia en el capítulo 3.

Crowds (1998). Michael Reiter y Aviel Rubin propusieron Crowds [RR98]. Los usuarios forman una «multitud» y cada solicitud web se reenvía a uno de sus miembros, elegido al azar. Con cierta probabilidad, ese miembro envía la solicitud al sitio web; en caso contrario, la transmite a otro miembro. El sitio ve llegar la petición desde un miembro cualquiera y ningún integrante puede saber con certeza si quien le pasó una solicitud fue también su autor original. Crowds no utiliza cifrado por capas y ofrece protección frente al sitio web y a otros miembros, pero no frente a un observador de la red. Con todo, introdujo una forma rigurosa de expresar el anonimato mediante probabilidades, así como un análisis de los ataques de participantes que actúan en connivencia y que anticipó parte de la investigación relacionada con los guardias.

Freedom (1999–2001). La mayor red de anonimato anterior a Tor fue Freedom, administrada por la empresa canadiense Zero-Knowledge Systems. Anunciada a finales de 1998, estuvo operativa desde finales de 1999 hasta finales de 2001 [Syv11]. Era un servicio comercial: los usuarios pagaban una suscripción y los servidores eran operados por la empresa y sus socios. Cerró porque los ingresos de las suscripciones no bastaban para mantenerlo. Su historia sugiere dos lecciones que Tor ha seguido en la práctica: una red de anonimato es más resistente

cuando la administran numerosos participantes independientes en lugar de una sola empresa, y su modelo de financiación importa tanto como su criptografía (apartado 3.4).

En profundidad: Muchas cebollas

Durante el mismo período se propusieron numerosos sistemas de encaminamiento cebolla, entre ellos Tarzan y MorphMix, completamente descentralizados, y Cebolla, del que Tor tomó la idea de construir circuitos salto a salto mediante un intercambio Diffie–Hellman independiente para cada salto [Syv11]. Otros proyectos surgieron en paralelo a Tor y siguieron caminos distintos: I2P, orientado principalmente a los servicios internos y todavía activo, y AN.ON, nacido en la Universidad de Dresde con el cliente JAP (y ofrecido también como servicio comercial con el nombre de JonDonym), que utiliza cadenas fijas de mixes administrados por organizaciones conocidas. Cada sistema establece un equilibrio diferente entre descentralización, confianza y rendimiento.

Resumen

- El mix de Chaum (1981) oculta la relación entre mensajes entrantes y salientes al modificar su aspecto, igualar sus longitudes y reordenarlos. Una cadena de mixes con cifrado por capas anticipa la idea de la cebolla.
- Los sistemas de alta latencia resisten a observadores poderosos, pero son lentos; los de baja latencia permiten actividades interactivas, aunque son vulnerables a la correlación temporal.
- anon.penet.fi mostró el peligro de almacenar las identidades de los usuarios; Mixmaster y Mixminion desarrollaron la tecnología mix aplicada al correo electrónico.
- PipeNet, Crowds y Freedom exploraron distintas soluciones de baja latencia. Freedom puso de manifiesto la importancia de una red administrada por múltiples actores y de un modelo de financiación sostenible.

Ejercicios

Ejercicio 2.1 $\star$ Un mix reúne lotes de 100 mensajes. Un observador ve entrar los 100 mensajes y salir los 100. Si conoce al remitente de uno de los mensajes de entrada, ¿cuál es la probabilidad de acertar su correspondiente mensaje de salida eligiendo al azar? ¿Qué ocurriría si el observador hubiera enviado 99 de los 100 mensajes?

Ejercicio 2.2 $\star\star$ Diseña a grandes rasgos un remailer que permita enviar respuestas a los usuarios sin conservar una tabla que relacione seudónimos con direcciones. ¿Qué ideas de Chaum necesitarías?

Ejercicio 2.3 $\star\star$ En Crowds, si la probabilidad de reenvío es $p_f = 3/4$, ¿por cuántos miembros pasa en promedio una solicitud antes de llegar al sitio web? ¿Por qué el miembro que recibe una solicitud no puede estar seguro de que quien se la transmitió sea su originador?

Lecturas complementarias

- D. Chaum, *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms* [Cha81].

- M. Reiter, A. Rubin, *Crowds: Anonymity for Web Transactions* [RR98].
- G. Danezis, R. Dingledine, N. Mathewson, *Mixminion: Design of a Type III Anonymous Remailer Protocol* [DDM03].
- La *Anonymity Bibliography* de Free Haven, `https://www.freehaven.net/anonbib/`: colección de referencia de trabajos científicos sobre anonimato.

Capítulo 3

Del encaminamiento cebolla del NRL al Tor Project

Última verificación: 23 de septiembre de 2026 — Syverson, A Peel of Onion *(2011); página histórica del Tor Project*

Objetivos del capítulo

Al terminar este capítulo, el lector podrá:

- describir los orígenes del encaminamiento cebolla en el Naval Research Laboratory y sus tres generaciones;
- explicar qué distingue a Tor de las generaciones anteriores;
- reconstruir las principales etapas de la historia del Tor Project desde 2002 hasta la actualidad;
- explicar de dónde procede el nombre «Tor».

3.1 En el Naval Research Laboratory

En 1995, tres investigadores del Naval Research Laboratory (NRL) de Estados Unidos —David Goldschlag, Michael Reed y Paul Syverson— se preguntaron si era posible establecer conexiones por Internet sin revelar quién se comunicaba con quién [Thea]. Su objetivo no era el anonimato como fin en sí mismo, sino, como hemos visto, «separar la identificación del encaminamiento» (apartado 1.3). Presentaron su primer diseño en el *Information Hiding Workshop* de mayo de 1996 [GRS96]. Antes de terminar ese año desarrollaron un segundo diseño que separaba la aplicación del usuario, encargada de construir los circuitos, de los nodos que transportaban el tráfico [Syv11].

En 1996 el laboratorio ya había obtenido autorización para publicar el código, antes de que se extendiera la expresión *código abierto*. Sus diseñadores entendían que nadie confiaría en una red cuyo código no pudiera examinarse.

El nombre procede de la estructura de los datos: un mensaje envuelto en capas de cifrado que los nodos van retirando una a una, como quien pela una cebolla.

3.2 Tres generaciones

Syverson distingue tres generaciones de los diseños desarrollados en el NRL [Syv11].

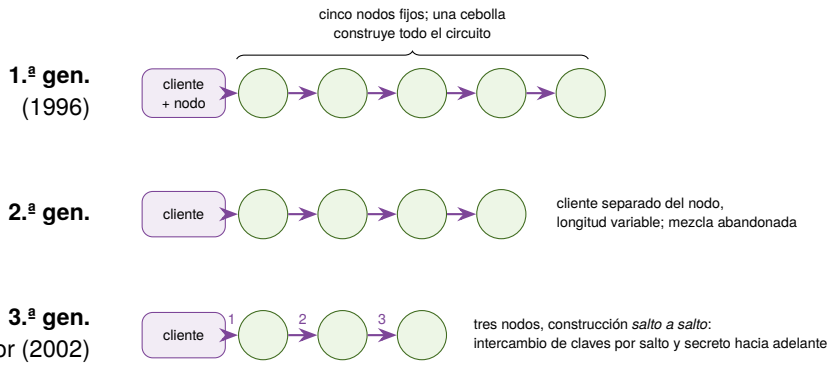


Figura 3.1: Las tres generaciones del encaminamiento cebolla. El cambio decisivo de Tor no fue el número de saltos, sino la manera de construir los circuitos: en lugar de preparar toda la cebolla en el cliente, se extiende el circuito un salto cada vez y se realiza un intercambio de claves independiente con cada nodo.

Primera generación. Utilizaba circuitos de longitud fija de cinco nodos, concebidos en parte para conectar las redes protegidas de dos organizaciones. Con tres saltos, un nodo intermedio hostil podría ver inmediatamente ambas redes. Con cuatro, un atacante que comprometiera el nodo adyacente a uno de los extremos sabría qué otro nodo atacar para reconstruir todo el circuito. Con cinco, ninguno de esos atajos era posible sin comprometer antes otro nodo. El cliente estaba integrado en el propio nodo. La red era estática y contaba con veinte o cien nodos operados por organizaciones estables, con configuraciones y claves distribuidas manualmente. Cada aplicación necesitaba su propio proxy, ya fuera para navegar, iniciar sesiones remotas o enviar correo electrónico, porque SOCKS todavía no estaba extendido. Un circuito se construía como una única cebolla cifrada con las claves públicas de todos los nodos.

Segunda generación. Se separó el cliente del nodo, la longitud del circuito pasó a ser variable y se introdujeron mecanismos de mezcla y de tráfico de relleno entre circuitos, que terminaron abandonándose. También se proyectó distribuir información sobre la red, aunque esa función nunca llegó a implementarse por completo.

Tercera generación: Tor. El cambio más importante fue la construcción de circuitos **salto a salto**, mediante un intercambio Diffie–Hellman con cada nodo (apartado 9.1), una idea tomada del proyecto Cebolla. Así se obtiene secreto hacia adelante y se puede verificar cada salto durante la extensión del circuito. Tor también incorporó servidores de directorio, compatibilidad con SOCKS, servicios ocultos con puntos de encuentro y, sobre todo, un diseño concebido para desplegarse y funcionar gracias a voluntarios de todo el mundo.

Nota histórica: El nombre

El título del artículo de 2004, *Tor: The Second-Generation Onion Router*, dio lugar a dos malentendidos que Syverson reconocería posteriormente [Syv11]. Tor pertenece a la *tercera* generación, no a la segunda, y la letra «r» de Tor no significa *router*. Cuando Roger Dingledine empezó a trabajar en el proyecto del NRL, existían diversos sistemas de encaminamiento cebolla. Si alguien le preguntaba «¿cuál de ellos?», respondía «*the onion routing*», es decir, el sistema original. Rachel Greenstadt observó que sus iniciales formaban un acrónimo adecuado. Dingledine señaló que también permitían una interpretación

recursiva como *Tor's onion routing* y decidió que el nombre debía escribirse «Tor», no «TOR»: una palabra corriente que, además, en alemán significa *puerta*, como la de una ciudad. En el resto del libro utilizamos *Tor* para la red y el proyecto, y *tor* en minúsculas para el programa.

3.3 La red Tor

Roger Dingledine se incorporó al proyecto del NRL a comienzos de la década de 2000 y Nick Mathewson lo hizo poco después. La red Tor comenzó a funcionar en **octubre de 2002**, con el código publicado desde el principio bajo una licencia de software libre. A finales de 2003 tenía aproximadamente una docena de nodos operados por voluntarios, casi todos en Estados Unidos y uno en Alemania [Thea]. En agosto de 2004, Dingledine, Mathewson y Syverson presentaron el proyecto en el USENIX Security Symposium [DMS04].

Los principales hitos posteriores fueron los siguientes [Thea]:

Año	Acontecimiento
2002	Comienza a funcionar la red Tor, con código fuente disponible libremente
2004	La Electronic Frontier Foundation empieza a financiar el trabajo de Dingledine y Mathewson
2006	Se funda The Tor Project, Inc., organización estadounidense sin ánimo de lucro (501(c)(3))
2007	Comienza el desarrollo de puentes resistentes a la censura (capítulo 16)
2008	Comienza el desarrollo de Tor Browser (capítulo 23)
2010–2011	Tor se convierte en una herramienta importante durante las revueltas de la Primavera Árabe
2013	Revelaciones de Edward Snowden sobre la vigilancia masiva, en las que Tor desempeña un papel central
2017	Servicios onion v3 (capítulo 14)
2021	Se deshabilitan los servicios onion v2
2022–2023	Control de congestión (apartado 11.3) y prueba de trabajo para servicios onion (capítulo 15)
2024	WebTunnel; migración de la distribución de puentes a rdsys
2025–2026	CGO en tor (apartado 10.4); Arti como cliente y para servicios onion (capítulo 30)

3.4 Una organización y una comunidad

The Tor Project, Inc. desarrolla el software, mantiene las especificaciones, opera autoridades de directorio junto con operadores independientes, mide la red y presta apoyo a la comunidad. Pero no es propietaria de la red: miles de voluntarios —particulares, universidades y asociaciones— administran repetidores en decenas de países. Esta decisión de diseño se apoya en la experiencia de Freedom (apartado 2.4): ninguna entidad individual debería poder controlar la red ni verse obligada a comprometer a sus usuarios.

La financiación de la organización ha sido un tema recurrente del debate público. A lo largo de los años ha recibido fondos de organismos gubernamentales, especialmente estadounidenses, fundaciones privadas y donantes individuales. La organización publica sus estados financieros anualmente. La pregunta «¿puede confiarse en un programa financiado parcialmente por un gobierno?» es ante todo técnica, no política: el código y las especificaciones son públicos y verificables, las decisiones de diseño se debaten abiertamente en las propuestas (capítulo 31) y la seguridad del sistema no depende de confiar en el Tor Project. Depende, más bien, de distribuir la red entre numerosos operadores independientes (capítulo 4).

En profundidad: Las revelaciones de 2013

Entre los documentos divulgados por Snowden en 2013 figuraban presentaciones internas de la NSA sobre Tor. Reconocían lo difícil que resultaba desanonimizar sistemáticamente a sus usuarios y describían ataques dirigidos contra personas concretas que explotaban vulnerabilidades del navegador, más que defectos de la red. Esto respalda indirectamente un principio al que volveremos con frecuencia: las debilidades más peligrosas suelen encontrarse no en el propio protocolo, sino en los sistemas que lo rodean (capítulo 28).

Resumen

- El encaminamiento cebolla nació en 1995 en el Naval Research Laboratory (Goldschlag, Reed y Syverson) para separar la identificación del encaminamiento.
- La primera generación utilizaba circuitos fijos de cinco nodos y una red estática. Tor, perteneciente a la tercera generación, construye circuitos salto a salto y añade secreto hacia adelante, directorios y puntos de encuentro.
- El nombre «Tor» procede de *the onion routing*; solo la primera letra se escribe en mayúscula.
- La red funciona desde 2002; The Tor Project, Inc. existe desde 2006; los repetidores están operados por voluntarios independientes.

Ejercicios

Ejercicio 3.1 ⋆ ¿Por qué la primera generación utilizaba circuitos de cinco nodos? Reconstruye el argumento sobre los recorridos de tres y cuatro saltos expuesto en este capítulo y explica por qué Tor utiliza solo tres.

Ejercicio 3.2 ⋆⋆ Compara la construcción de un circuito como una única cebolla (primera generación) con la construcción telescópica de Tor, tanto en términos de secreto hacia adelante como de capacidad para identificar de inmediato un nodo que no responde.

Ejercicio 3.3 ⋆⋆ Redacta un argumento breve, de aproximadamente una página, que responda a la pregunta «¿podemos confiar en Tor teniendo en cuenta quién lo ha financiado?». Distingue lo que puede verificarse técnicamente de lo que exige confianza.

Lecturas complementarias

- P. Syverson, *A Peel of Onion* [Syv11].
- D. Goldschlag, M. Reed, P. Syverson, *Hiding Routing Information* [GRS96].
- Página de historia del Tor Project [Thea].

Capítulo 4

El modelo de amenazas de Tor

Última verificación: 23 de septiembre de 2026 — Dingledine, Mathewson y Syverson (2004), sección 3

Objetivos del capítulo

Al terminar este capítulo, el lector podrá:

- explicar qué es un modelo de amenazas y por qué todos los sistemas de seguridad necesitan uno;
- enumerar los objetivos y los «no objetivos» establecidos por los diseñadores de Tor;
- describir al adversario frente al cual Tor pretende ofrecer protección y a aquel frente al que no puede resistir;
- distinguir lo que protege Tor de lo que sigue siendo responsabilidad de los usuarios y las aplicaciones.

Ningún sistema de seguridad protege frente a todo. Un **modelo de amenazas** describe explícitamente a quién o a qué pretende proteger un sistema, frente a qué adversarios y amenazas se ha diseñado y qué supuestos adopta. Constituye el criterio con el que debe evaluarse el sistema: un ataque exitoso dentro del modelo revela un defecto, mientras que un ataque expresamente excluido de él no lo demuestra. Este último caso sigue siendo, sin embargo, una limitación que el usuario debe comprender. El capítulo presenta el modelo de amenazas de Tor tal como lo formularon sus diseñadores en 2004 [DMS04] y explica cómo ha evolucionado desde entonces.

4.1 Objetivos

El objetivo principal consiste en impedir que un adversario pueda **vincular** a las dos partes de una comunicación o relacionar varias comunicaciones realizadas por un mismo usuario. Cuatro criterios recogidos en el artículo de 2004 guiaron el diseño:

Facilidad de despliegue El sistema debe funcionar en el mundo real: no debe resultar excesivamente costoso para los voluntarios, exponer a los operadores a responsabilidades desproporcionadas, exigir cambios en los sistemas operativos ni obligar a los sitios web a instalar componentes adicionales.

Facilidad de uso Un sistema complicado atrae a menos usuarios y, cuantos menos usuarios haya, menor será el anonimato. Como explican sus autores, la facilidad de uso no es solo una comodidad, sino «un requisito de seguridad».

Flexibilidad El protocolo debe estar bien especificado para servir de base a futuras investigaciones.

Simplicidad Solo deben emplearse técnicas conocidas y suficientemente estudiadas. Cada funcionalidad adicional complica el análisis de seguridad.

4.2 No objetivos

Igual de importante es aquello que Tor declara explícitamente que *no* pretende conseguir:

No es una red entre pares Tor no pretende operar con miles de nodos de vida muy corta, muchos de los cuales podrían ser hostiles. Utiliza un conjunto de repetidores enumerados y verificados por las autoridades de directorio (capítulo 12).

No resiste los ataques de extremo a extremo Tor no afirma que pueda derrotar la correlación temporal ni los ataques de intersección realizados por quien observa los dos extremos de una comunicación.

No normaliza los protocolos Tor transporta los datos de las aplicaciones tal como son. Si una aplicación revela la identidad del usuario, Tor no puede impedirlo. En sus comienzos era necesario un proxy de filtrado; hoy esa función la cumple Tor Browser (capítulo 23).

No es esteganográfico Tor por sí solo no esconde que el usuario está conectado a la red. Los transportes conectables, desarrollados posteriormente, atenúan esta limitación (capítulo 16).

4.3 El adversario

Muchos análisis teóricos de los sistemas de anonimato suponen un **adversario pasivo global**: alguien capaz de observar todo el tráfico de Internet. Ningún sistema práctico de baja latencia protege frente a ese adversario, y Tor lo reconoce expresamente. En cambio, su modelo supone un adversario con visibilidad parcial de la red, aunque con capacidades activas relativamente amplias en otros aspectos [DMS04]. Puede:

- observar una *parte* del tráfico de la red;
- generar, modificar, eliminar o retrasar tráfico;
- operar sus propios repetidores;
- comprometer algunos repetidores administrados por terceros.

Con estas capacidades, el adversario puede intentar muchos ataques: correlacionar el tráfico de entrada y salida observando los extremos de la red; comprometer repetidores o claves; repetir mensajes; provocar una denegación de servicio contra repetidores honestos para empujar a los usuarios hacia otros maliciosos (capítulo 15); marcar el tráfico para reconocerlo en otro punto (apartado 10.3); ofrecer a distintos usuarios vistas inconsistentes de la red mediante el sistema de directorios (capítulo 12); o degradar el servicio hasta conseguir que las personas abandonen Tor y utilicen alternativas menos seguras.

4.3.1 Confirmación y análisis

Los diseñadores de Tor distinguen dos grandes clases de ataques basados en el tráfico. En un ataque de **confirmación**, el adversario ya sospecha una relación («Alicia se comunica con Bob») y observa ambos extremos para comprobarla. Como otros sistemas de baja latencia, Tor no puede proporcionar protección completa contra este tipo de ataque. En un ataque de

análisis, el adversario estudia el tráfico para descubrir *dónde* debe mirar, es decir, qué puntos de la red le interesa atacar. Tor está diseñado principalmente para resistir esta segunda clase. Todo el proceso de selección de rutas, incluidos los guardias y los pesos según el ancho de banda (capítulo 13), busca reducir la probabilidad de que un mismo adversario ocupe ambos extremos de un circuito.

Advertencia: El principio que conviene recordar

Tor protege frente a quien observa una *parte* de la red, no frente a quien consigue observar *los dos extremos* de una comunicación. Un adversario que controle u observe tanto la conexión del usuario con el guardia como la conexión de la salida con el sitio web puede correlacionar el tiempo y el volumen del tráfico. Para una organización con acceso a las redes de todo un país, tanto el usuario como el sitio web podrían encontrarse en ese mismo país. Es un escenario práctico, no únicamente teórico.

4.4 Qué protege Tor y qué queda fuera de su alcance

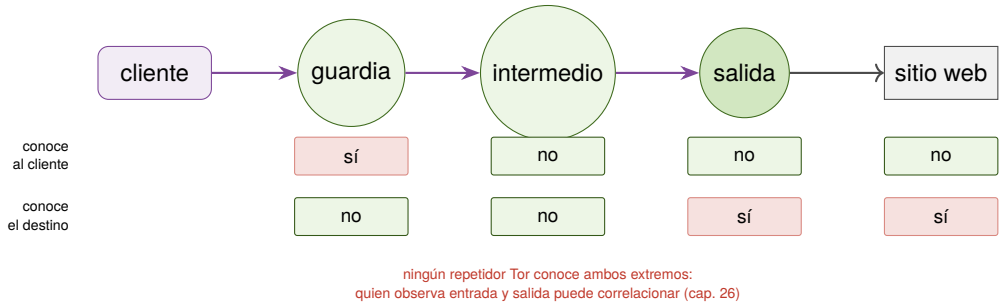


Figura 4.1: Qué sabe cada punto del recorrido. La protección de Tor depende de separar la información: el guardia conoce al cliente, pero no su destino; la salida conoce el destino, pero no al cliente.

Tor protege frente a	Tor no protege frente a
un sitio web que intenta conocer la dirección IP de sus visitantes	los datos que revela el propio usuario, al iniciar sesión o comunicar información personal
un observador de la red local o un proveedor de Internet que solo ve una conexión a Tor	un observador que vigila los dos extremos de la comunicación
un repetidor individual que conoce, como máximo, uno de los extremos	un adversario que controla a la vez el guardia y la salida de un circuito
una salida que intenta identificar quién envió una solicitud	una salida que lee o modifica tráfico sin cifrar, cuando no se utiliza HTTPS
la censura basada en direcciones IP cuando se utilizan puentes	una aplicación o un navegador vulnerable, comprometido o mal configurado

El modelo ha evolucionado para abordar problemas que el artículo de 2004 no podía prever: la censura a escala nacional (capítulo 16), las medidas frente a la *identificación mediante huellas digitales* del navegador (capítulo 23), las defensas contra la denegación de servicio (capítulo 15) y la futura amenaza de los ordenadores cuánticos (capítulo 31). Un principio permanece constante: comprender el modelo de amenazas es el primer paso para utilizar Tor de forma segura.

Resumen

- Un modelo de amenazas identifica los adversarios y ataques frente a los que un sistema está diseñado para proteger.
- Tor intenta evitar que los usuarios y sus destinos puedan vincularse, y prioriza la facilidad de despliegue, la facilidad de uso, la flexibilidad y la simplicidad.
- Tor no es una red entre pares, no resiste los ataques de extremo a extremo, no normaliza los protocolos de aplicación y no oculta su uso sin transportes conectables.
- El adversario puede observar parte de la red, operar y comprometer repetidores y actuar sobre el tráfico. Tor está diseñado sobre todo para resistir ataques de análisis y ofrece menos protección frente a ataques de confirmación.

Ejercicios

Ejercicio 4.1 ⋆ Indica si Tor protege al usuario en cada uno de estos escenarios y explica por qué: (a) el administrador del Wi-Fi de una cafetería quiere saber qué sitios visita un cliente; (b) un sitio web intenta averiguar las direcciones IP de sus visitantes; (c) una persona inicia sesión en su cuenta personal de una red social utilizando Tor; (d) una agencia que controla las redes de un país sospecha que alguien está comunicándose con un periódico ubicado en el mismo país.

Ejercicio 4.2 ⋆⋆ Propón un ejemplo que muestre por qué la facilidad de uso constituye un requisito de seguridad, y no solo una comodidad, en un sistema de anonimato.

Ejercicio 4.3 ⋆⋆ Redacta un modelo de amenazas para una periodista que utiliza Tor Browser para comunicarse con una fuente situada en otro país. Identifica a los posibles adversarios, qué pueden observar y qué aspectos quedan fuera de la protección de Tor.

Lecturas complementarias

- R. Dingledine, N. Mathewson, P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], secciones 3 y 7.
- R. Dingledine, N. Mathewson, *Anonymity Loves Company: Usability and the Network Effect* [DM06].

Parte II

El protocolo a bajo nivel

Capítulo 5

Criptografía esencial para Tor

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; commit de torspec `8e1f245`

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- identificar las primitivas criptográficas que emplea Tor y la función de cada una;
- distinguir entre cifrado, integridad, derivación de claves, intercambio de claves y firmas digitales;
- comprender por qué Tor utiliza distintas familias de funciones hash (SHA-1, SHA-2, SHA-3 y Blake2b) y en qué se diferencian;
- interpretar con seguridad las fórmulas de los capítulos siguientes.

Este capítulo es una caja de herramientas, no un curso de criptografía. Presenta únicamente la teoría indispensable de las técnicas que encontraremos en la parte II, e indica dónde se utiliza cada una. El apéndice B contiene explicaciones más detalladas para quienes deseen profundizar. Todos los ejemplos pueden probarse en Python mediante el módulo estándar `hashlib` y la biblioteca `cryptography`, las mismas herramientas utilizadas en los laboratorios.

5.1 Cinco funciones

En un protocolo como Tor, la criptografía cumple cinco funciones distintas:

Confidencialidad Nadie salvo los destinatarios previstos debe poder leer los datos: *cifradores*.

Integridad Nadie debe poder modificar los datos sin que se detecte: *funciones hash* y *códigos de autenticación de mensajes* (MAC).

Acuerdo de claves Dos partes que no comparten ningún secreto deben establecer uno mientras se comunican por un canal público: intercambio de claves *Diffie–Hellman*.

Autenticación Una parte debe demostrar que posee una identidad determinada: *firmas digitales*.

Derivación Deben obtenerse varias claves independientes a partir de un secreto compartido: *funciones de derivación de claves* (KDF).

Una buena regla de diseño, que Tor aplica con un rigor cada vez mayor, consiste en no reutilizar nunca la misma clave para tareas o contextos diferentes.



Figura 5.1: Las cinco funciones criptográficas en Tor, las primitivas que las realizan y los capítulos en los que veremos su utilización.

5.2 Cifradores

5.2.1 AES en modo contador

Tor cifra casi todo mediante **AES**, el cifrador de bloques estándar, que trabaja con bloques de 16 bytes y claves de 128 o 256 bits. Un cifrador de bloques, por sí solo, cifra un bloque cada vez. Para cifrar flujos de longitud arbitraria, Tor emplea el **modo contador** (CTR): AES cifra un contador que aumenta en una unidad por cada bloque y el flujo de clave resultante (*keystream*) se combina con los datos mediante XOR:

$$C = P \oplus \mathrm{Trunc}_{|P|}\big(\mathrm{AES}_K(IV) \,\|\, \mathrm{AES}_K(IV+1) \,\|\, \cdots\big)$$

El descifrado es la misma operación. Hay dos propiedades del modo contador que nunca deben olvidarse:

- el mismo par clave–contador **no puede reutilizarse jamás**: dos mensajes cifrados con el mismo flujo de clave revelan el XOR de sus textos en claro;
- el modo contador es **maleable**: al modificar un bit del texto cifrado, cambia el bit correspondiente del texto en claro sin necesidad de conocer la clave.

AES-128-CTR cifra las celdas relay en el esquema tradicional (apartado 10.1); su maleabilidad permite los ataques de marcado (*tagging*). AES-256-CTR cifra los mensajes ntor-v3 (apartado 9.7) y los descriptores de servicios onion (apartado 14.5). AES-128 es uno de los componentes básicos de CGO (apartado 10.4).

5.2.2 TLS

El canal entre dos nodos está protegido por TLS (capítulo 7), que negocia de forma independiente algoritmos de cifrado autenticado, como AES-GCM o ChaCha20-Poly1305. No obstante, Tor no confía en TLS para identificar al repetidor: los repetidores demuestran su identidad mediante los certificados propios del protocolo Tor.

5.3 Funciones hash

Una función hash criptográfica transforma una entrada de longitud arbitraria en una huella de longitud fija. Está diseñada para hacer computacionalmente inviable encontrar dos entradas con la misma huella (*resistencia a colisiones*) o una entrada que corresponda a una huella determinada. Tor utiliza cuatro familias que reflejan su evolución histórica:

Función	Salida	Dónde aparece
SHA-1	20 bytes	Huellas RSA de repetidores, resumen acumulativo de celdas relay (tor1), KDF-TOR. Se considera débil frente a colisiones, pero permanece donde esa propiedad no es necesaria
SHA-256	32 bytes	ntor y su KDF, certificados Ed25519 y TLS, firmas del consenso y huellas de microdescriptores
SHA3-256, SHAKE-256	32 bytes / variable	ntor-v3, servicios onion v3 (direcciones, claves cegadas, anillo HSDir, descriptores), resúmenes de circuitos de encuentro
Blake2b	variable	Prueba de trabajo de servicios onion (capítulo 15)

SHA-3 no es una versión mejorada de SHA-2, sino una construcción completamente distinta, denominada *esponja* (*sponge*). SHAKE-256 pertenece a la misma familia y es una función de salida extensible: produce tantos bytes como se necesiten, por lo que resulta conveniente para derivar claves. En los protocolos más recientes de Tor, cada uso de SHA-3 incorpora una etiqueta de dominio como prefijo (por ejemplo, `"ntor3-curve25519-sha3_256-1:kdf_phase1"`). Así se evita que un hash calculado para un propósito coincida con otro calculado para una finalidad distinta.

En profundidad: Resúmenes acumulativos

Muchas bibliotecas permiten añadir datos de forma incremental a una función hash y consultar su resumen en cualquier momento sin volver a comenzar. Tor aprovecha esta posibilidad para calcular el *resumen acumulativo* de las celdas relay (apartado 10.2), que resume las celdas intercambiadas con un salto concreto. En Python, `hashlib.sha1()` permite hacerlo mediante los métodos `update`, `digest` y `copy`.

5.4 MAC y derivación de claves

5.4.1 HMAC

Un MAC es una huella que depende también de una clave: solo quien conoce esa clave puede calcularla o verificarla. **HMAC** construye un MAC a partir de una función hash. Tor utiliza HMAC-SHA256 en el intercambio ntor (apartado 9.6) y en su función de derivación de claves. En los protocolos basados en SHA-3 puede obtenerse un MAC de forma más sencilla calculando el hash de la clave, precedida por su longitud, seguida del mensaje. La longitud permite determinar de forma inequívoca dónde termina la clave y dónde comienza el mensaje [rend-spec § Cryptographic building blocks]. En ntor-v3 también se antepone la longitud de la etiqueta de dominio (apartado 9.7).

Un detalle práctico: el MAC debe compararse siempre en *tiempo constante*, mediante `hmac.compare_digest` en Python. De otro modo, las diferencias de tiempo en la comparación podrían revelar cuántos bytes son correctos.

5.4.2 Funciones de derivación de claves

Un secreto compartido debe producir varias claves: una para el cifrado en sentido de ida, otra para el cifrado de vuelta y dos para la integridad (apartado 9.8). Una KDF expande el secreto hasta obtener un flujo de la longitud necesaria, del que se extraen las claves individuales. Tor utiliza tres:

KDF-TOR $\mathrm{SHA1}(K_0\|0)\,\|\,\mathrm{SHA1}(K_0\|1)\,\|\,\ldots$, construcción histórica que aún utiliza `CREATE_FAST`;

HKDF-SHA256 construcción estándar definida en RFC 5869, utilizada por ntor;

SHAKE-256 función empleada directamente como KDF en ntor-v3 y los servicios onion.

5.5 Intercambio de claves: X25519

El intercambio de claves Diffie–Hellman permite a dos partes establecer un secreto compartido intercambiando únicamente valores públicos. La versión de curva elíptica utilizada por Tor es **X25519**, que realiza el intercambio sobre *Curve25519*. Cada parte posee una clave privada de 32 bytes x y una clave pública $X = x \cdot G$, donde G es un punto fijo de la curva y el producto representa la multiplicación de un punto por un entero. Con las claves x, X e y, Y, ambas partes calculan el mismo valor:

$$x \cdot Y = y \cdot X = xy \cdot G,$$

que un observador que solo conozca X e Y no puede obtener. Las claves tienen 32 bytes y el intercambio es muy rápido. Estas dos ventajas llevaron a Tor a abandonar el Diffie–Hellman modular de 1024 bits utilizado por el intercambio TAP (apartado 9.4).

Una clave empleada para un único intercambio y eliminada después se denomina **efímera**. La combinación de claves efímeras y a largo plazo permite que los intercambios de Tor proporcionen a la vez autenticación y *secreto hacia adelante* (apartado 9.3).

5.6 Firmas: Ed25519 y RSA

Una firma digital se genera con una clave privada y se verifica mediante su clave pública correspondiente. Tor utiliza principalmente **Ed25519**, un esquema de firmas sobre una curva equivalente a Curve25519, con claves públicas de 32 bytes y firmas de 64 bytes: aparece en las identidades de los repetidores, los certificados de Tor (apartado 7.5) y los descriptores de servicios onion. Ed25519 tiene una propiedad que Tor aprovecha de manera particular: multiplicar una clave pública A por un factor h genera otra clave pública válida, $A' = h \cdot A$, cuya clave privada correspondiente puede calcular quien posea la clave privada original. Esta es la base de las claves cegadas de los servicios onion (apartado 14.3).

RSA, con claves de 1024 bits, es una herencia de los primeros años de Tor: las identidades antiguas de los repetidores y las claves con las que las autoridades de directorio firman el consenso. Las claves de las autoridades son más largas, y los dos tipos tienen requisitos diferentes: la especificación exige al menos **3072** bits para la clave de identidad (`KP_auth_id_rsa`) y al menos 2048 para la clave de firma (`KP_auth_sign_rsa`) [dir-spec § Directory authority operation and formats]. Tor está abandonando gradualmente RSA.

5.7 Criptografía reciente

En los desarrollos recientes aparecen otras dos construcciones especializadas:

- **POLYVAL**, una función hash universal que también se utiliza en AES-GCM-SIV, constituye junto con AES una de las bases de CGO, el nuevo esquema de cifrado de celdas relay (apartado 10.4);

- **Elligator2** codifica un punto de Curve25519 de modo que resulte indistinguible de una cadena aleatoria; obfs4 lo emplea para ocultar las claves públicas del intercambio de claves (apartado 16.4).

La criptografía *poscuántica*, que deberá sustituir a X25519 cuando los ordenadores cuánticos se conviertan en una amenaza práctica, constituye un área activa de investigación para Tor y se estudia en el capítulo 31.

Resumen

- AES en modo contador cifra los datos; un flujo de clave no debe reutilizarse nunca y el modo contador es maleable.
- SHA-1, SHA-256, SHA3-256/SHAKE-256 y Blake2b coexisten en Tor: la elección depende de cuándo se diseñó el protocolo y de la finalidad del hash.
- HMAC autentica los mensajes, mientras que las KDF (KDF-TOR, HKDF y SHAKE) derivan claves; los MAC deben compararse en tiempo constante.
- X25519 se utiliza para el intercambio de claves en todos los protocolos modernos; Ed25519 firma identidades, certificados y descriptores; RSA se está retirando.

Ejercicios

Ejercicio 5.1 ⋆ Con AES-CTR, un adversario conoce el texto en claro P_1 de un mensaje e intercepta su texto cifrado C_1 y el texto cifrado C_2 de otro mensaje, que por error se cifró con el mismo flujo de clave. ¿Cómo puede recuperar P_2?

Ejercicio 5.2 ⋆ En Python, calcula `hashlib.shake_256(b"Tor").hexdigest(16)` y `hexdigest(32)`. ¿Qué relación existe entre las dos salidas? ¿Qué implica esto para el uso de SHAKE como KDF?

Ejercicio 5.3 ⋆⋆ Utilizando la biblioteca `cryptography`, genera dos pares de claves X25519 y comprueba que producen secretos compartidos idénticos. ¿Qué sucede si una de las partes utiliza 32 bytes de valor cero como clave pública de la otra? Relaciona el resultado con la comprobación del «punto en el infinito» de ntor.

Ejercicio 5.4 ⋆⋆ ¿Por qué los protocolos de Tor basados en SHA-3 incluyen una etiqueta de dominio al principio de cada entrada? Describe un posible ataque si dos usos diferentes de la misma función no se separaran.

Lecturas complementarias

- J.-P. Aumasson, *Serious Cryptography* (No Starch Press): introducción práctica y moderna a todas las primitivas de este capítulo.
- D. J. Bernstein y otros, trabajos originales sobre Curve25519 y Ed25519 y RFC 7748 (X25519) y 8032 (Ed25519).
- Sección «Preliminaries» de la especificación de Tor [Thee], con convenciones sobre cifradores y funciones hash.

Capítulo 6

Arquitectura de la red

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; consenso público del 23 de septiembre de 2026, 15:00 UTC

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- enumerar las funciones de los nodos de la red Tor y sus responsabilidades;
- describir las capas del protocolo, desde el canal TLS hasta el flujo de una aplicación;
- seguir, a grandes rasgos, lo que ocurre cuando alguien abre una página web mediante Tor;
- comprender cómo conviven diferentes implementaciones y versiones en una misma red.

Antes de examinar los detalles de los capítulos siguientes necesitamos una visión general de la red. Este capítulo es un mapa: volveremos sobre todos sus elementos con mayor profundidad y con referencias a las secciones correspondientes.

6.1 Funciones de los nodos

La red Tor está formada por programas que desempeñan diferentes funciones, a menudo más de una al mismo tiempo. La figura 6.1 ofrece una visión de conjunto.

Cliente Programa del usuario, como el incluido en Tor Browser. Elige rutas, construye circuitos y ofrece a las aplicaciones un puerto SOCKS. No acepta conexiones entrantes desde la red ni se autentica ante ella (capítulo 7).

Repetidor Nodo que reenvía tráfico de terceros. Según los indicadores que le asignen las autoridades de directorio y su configuración, puede desempeñar las funciones siguientes:

- **guardia**: primer salto de los circuitos de un cliente; se selecciona y mantiene durante un período prolongado (apartado 13.3);
- **intermedio**: salto que no conoce ni el origen ni el destino;
- **salida**: último salto, que establece conexiones con Internet según su política de salida (capítulo 11);
- **HSDir**: almacena descriptores de servicios onion (apartado 14.4);
- **caché de directorio**: redistribuye el consenso y los descriptores (capítulo 12).

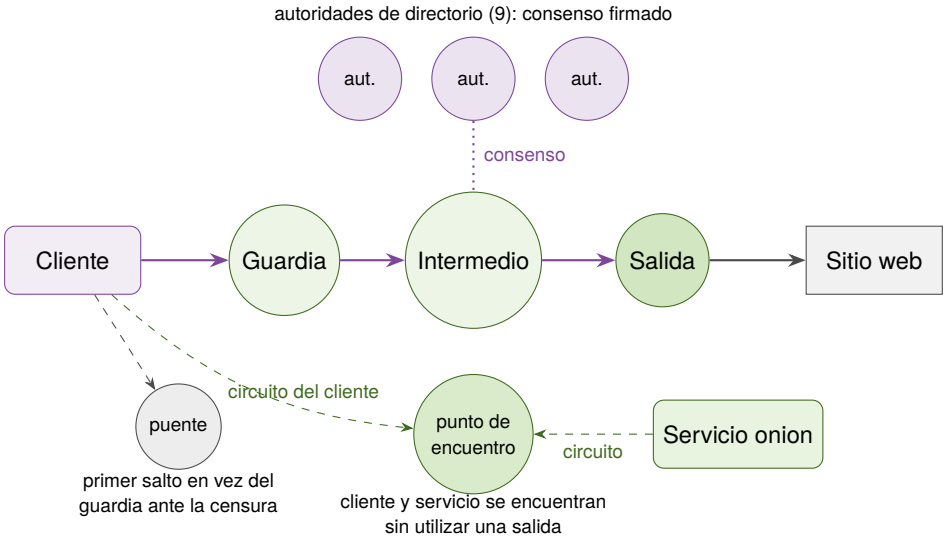


Figura 6.1: Funciones dentro de la red Tor. El cliente construye un circuito a través de un guardia, un repetidor intermedio y una salida. Las autoridades de directorio publican el consenso que enumera los repetidores. Los puentes sustituyen a los guardias para quienes sufren censura. A los servicios onion se accede mediante un punto de encuentro, sin utilizar una salida.

Autoridad de directorio Nueve servidores de confianza que votan cada hora sobre el estado de la red y firman el consenso; algunos también miden el ancho de banda de los repetidores.

Puente Repetidor no publicado, utilizado como primer salto por personas que no pueden acceder a los repetidores públicos, a menudo mediante un transporte conectable (capítulo 16).

Servicio onion Servicio accesible únicamente a través de Tor, sin revelar su ubicación (capítulo 14).

En el consenso público del 23 de septiembre de 2026 aparecen 9358 repetidores: 5306 con el indicador `Guard` y 3177 con el indicador `Exit` (1615 tienen ambos). Los guardias representan el 70 % del ancho de banda total, mientras que las salidas concentran el 33 % (capítulo 13).

6.2 Capas del protocolo

El protocolo Tor puede describirse como una pila de capas, cada una construida sobre la inferior (figura 6.2).

Canal Conexión TLS entre dos nodos adyacentes, mediante la que se verifica la identidad del repetidor (capítulo 7).

Celdas Unidades de transmisión de 514 bytes transportadas por un canal (capítulo 8).

Circuito Ruta de múltiples saltos construida uno a uno mediante intercambios que derivan claves distintas para cada salto (capítulo 9).

Cifrado por capas Mensajes relay envueltos en una capa de cifrado por cada salto (capítulo 10).

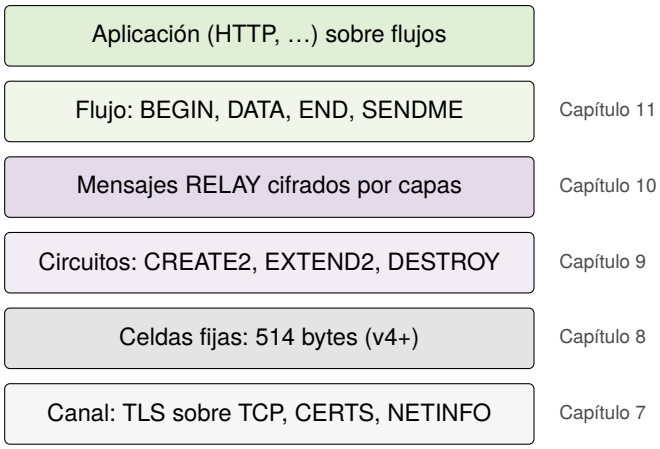


Figura 6.2: Capas del protocolo Tor y capítulos que las explican. Cada capa utiliza la inferior: los flujos viajan como mensajes relay, estos mensajes se transportan dentro de celdas cifradas por capas y las celdas circulan por canales TLS.

Flujo Conexión TCP de aplicación abierta por la salida y gestionada mediante ventanas y control de congestión (capítulo 11).

Junto a esta pila están el sistema de directorios, que proporciona a los clientes la lista de repetidores, y el protocolo de servicios onion, que utiliza circuitos para reunirse en un punto de encuentro.

6.3 Una página web, de principio a fin

Sigamos, en términos generales, lo que sucede cuando alguien inicia Tor Browser por primera vez y abre una página.

1. **Inicio.** `tor` contacta con una autoridad de directorio o con uno de los aproximadamente doscientos *directorios de reserva* (*fallback directories*) incluidos en su código fuente y descarga el consenso, cuyas firmas verifica. A continuación descarga los microdescriptores de los repetidores (capítulo 12).

2. **Selección de guardias.** Selecciona al azar una pequeña muestra de guardias, con probabilidades ponderadas por ancho de banda, y utiliza dos de ellos (apartado 13.3).

3. **Canal.** Abre una conexión TLS con un guardia y verifica su identidad mediante celdas **`CERTS`** (capítulo 7).

4. **Circuito.** Selecciona una salida apropiada para el destino y un repetidor intermedio, y construye un circuito de tres saltos con **`CREATE2`** y `EXTEND2` (capítulos 9 y 13). En la práctica, `tor` construye algunos circuitos por adelantado para tenerlos listos cuando hagan falta.

5. **Flujo.** A través del puerto SOCKS, el navegador solicita una conexión a `www.example.com:443`. `tor` envía un mensaje `BEGIN` a la salida, que resuelve el nombre y abre una conexión TCP (capítulo 11).

6. **Datos.** El navegador se comunica con el sitio web mediante HTTPS a través del flujo. La salida conoce el nombre del sitio y ve el tráfico TLS, pero no quién lo solicitó. El guardia sabe quién hizo la solicitud, pero no qué se pidió. El repetidor intermedio no conoce ninguno de los extremos.

El proceso completo de inicio normalmente tarda solo unos segundos. Muchos de los mecanismos descritos más adelante, entre ellos el relleno, el control de congestión y los Vanguards, funcionan discretamente alrededor de esta secuencia.

6.4 Implementaciones y versiones

6.4.1 Dos implementaciones

La implementación histórica es **tor**, escrita en C, y desempeña todas las funciones: cliente, repetidor, autoridad y servicio onion. Junto a ella, el Tor Project desarrolla **Arti**, una reimplementación en Rust (capítulo 30), que actualmente se utiliza como cliente y para servicios onion. En el consenso público del 23 de septiembre de 2026, ningún repetidor utiliza Arti: 6027 ejecutan tor 0.4.9.12, 2520 ejecutan 0.4.9.11 y la mayoría de las versiones restantes corresponden a versiones anteriores de la serie 0.4.9. Cuarenta y siete repetidores ejecutan la versión de desarrollo 0.5.0.0-alpha-dev.

6.4.2 Subprotocolos

¿Cómo puede una red de miles de nodos, actualizados en momentos diferentes, introducir nuevas funcionalidades? Tor no se basa en los números de versión del programa para resolverlo. En su lugar, utiliza **subprotocolos**: cada funcionalidad pertenece a un subprotocolo con su propio número de versión y cada repetidor declara los números que admite en la línea `pr` del consenso [tor-spec § Subprotocol-based versioning]. Por ejemplo, un repetidor de la red pública declara:

```
pr Conflux=1 Cons=1-2 Desc=1-4 DirCache=2 FlowCtrl=1-2 HSDir=2 HSIntro=4-5
   HSRend=1-2 Link=3-5 LinkAuth=3 Microdesc=1-3 Padding=2 Relay=2-6
```

Un cliente que quiera utilizar ntor-v3 con un repetidor comprueba si anuncia `Relay=4`; CGO exige `Relay=6` y el control de congestión requiere `FlowCtrl=2`. Estos números identifican *capacidades* independientes, no versiones acumulativas: `Relay=6` no implica formalmente `Relay=4` (Propuesta 346, que prefiere por ello la expresión «capacidades de subprotocolo»). En la práctica, `tor` las anuncia como un intervalo continuo porque implementa las funcionalidades en orden. El consenso también enumera los subprotocolos *obligatorios*: si una versión de `tor` no los admite, reconoce que es demasiado antigua y se niega a funcionar, en lugar de poner en peligro a sus usuarios.

Resumen

- La red consta de clientes, repetidores (guardias, intermedios, salidas, HSDir y cachés de directorio), autoridades de directorio, puentes y servicios onion.
- El protocolo se organiza por capas: canales TLS, celdas, circuitos, cifrado por capas y flujos.
- Un guardia conoce al usuario, pero no el destino; una salida conoce el destino, pero no al usuario.
- `tor` (C) y Arti (Rust) conviven. Las funcionalidades se negocian mediante subprotocolos anunciados en la línea `pr` del consenso.

Ejercicios

Ejercicio 6.1 ⋆ Para cada función (guardia, intermedio, salida, HSDir y autoridad de directorio), indica qué sabe sobre un usuario que visita un sitio web mediante Tor.

Ejercicio 6.2 ⋆ Según la línea `pr` mostrada en este capítulo, ¿qué funcionalidades de los capítulos 9–11 admite el repetidor? ¿Podría utilizarse como salida con CGO?

Ejercicio 6.3 ⋆⋆ Se descarga mediante Tor una página web de 100 KB. Estima, capa por capa, cuántos bytes atraviesan el enlace entre el cliente y el guardia, teniendo en cuenta las celdas, los encabezados de los mensajes relay y SENDME.

Lecturas complementarias

- R. Dingledine, N. Mathewson y P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04]: la arquitectura original, que en gran parte sigue siendo válida.
- Tor Metrics, `https://metrics.torproject.org/`: cifras actualizadas de repetidores, usuarios y versiones.

Capítulo 7

Canales y protocolo de enlace

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; Arti 2.6.0; protocolo de enlace 4–5

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar qué es un canal y por qué Tor lo establece sobre TLS;
- describir las claves que constituyen la identidad de un repetidor y cómo se vinculan entre sí;
- seguir el intercambio inicial del canal, celda a celda: VERSIONS, CERTS, AUTH_CHALLENGE, AUTHENTICATE y NETINFO;
- leer y verificar un certificado Ed25519 de Tor;
- comprender por qué un cliente no se autentica y qué implica esto para el anonimato;
- abrir un canal verificado con un repetidor de la red de laboratorio (laboratorio 1).

El circuito, la construcción central de Tor, es una cadena de saltos. Sin embargo, cada salto se apoya en algo más sencillo: una conexión directa y cifrada entre dos nodos adyacentes. La especificación denomina **canal** a esta conexión. En este capítulo veremos cómo se establece y, sobre todo, cómo verifica cada extremo la identidad del interlocutor al que pretendía llegar. El formato de las celdas que se intercambian por el canal se estudia en el capítulo 8, al que nos referiremos con frecuencia.

7.1 ¿Qué es un canal?

Un canal es una conexión directa y cifrada entre dos repetidores o entre un cliente y un repetidor [tor-spec § Channels]. Técnicamente, es una sesión TLS sobre TCP que transporta celdas. Lo caracterizan tres propiedades:

Funciones asimétricas Cualquiera puede *abrir* un canal (el *iniciador* (initiator)), pero solo un repetidor puede *recibirlo* (el *respondedor* (responder)). Un cliente no acepta conexiones: no tiene ningún puerto a la escucha de conexiones procedentes de la red Tor.

Identidad verificada El respondedor siempre demuestra criptográficamente que posee una o varias identidades de repetidor. Por lo general, el iniciador ya sabe qué identidad debe encontrar: la obtiene del consenso, de la lista de autoridades compilada en el código fuente o, cuando abre el canal para extender un circuito, del propio mensaje `EXTEND2`. Si la identidad presentada no coincide con la esperada, debe cerrarse el canal.

Uso compartido Un canal transporta numerosos circuitos, diferenciados por sus CircID (apartado 8.4). Los nodos prefieren reutilizar un canal existente a abrir un segundo canal hacia el mismo repetidor. Cuando un canal no contiene circuitos, se cierra tras un período de inactividad elegido aleatoriamente, cuyo intervalo depende de su uso (`channelpadding_get_channel_idle_timeout` en `src/core/or/channelpadding.c`): de 3 a 4,5 minutos en los canales de clientes y de 45 a 75 minutos entre repetidores, donde el valor central es el parámetro de consenso `nf_conntimeout_relays` (una hora). Mientras un canal está en uso, `KeepalivePeriod` provoca en cambio el envío de una celda de relleno cada cinco minutos para evitar que los cortafuegos lo cierren.

En profundidad: ¿Canal o conexión?

La documentación antigua y muchos artículos científicos denominan *conexiones* a los canales (*OR connections*). La especificación abandonó esta denominación porque podía confundirse con las conexiones TCP que abren los repetidores de salida para los flujos salientes. Ambos términos sobreviven en el código de `tor`: `or_connection_t` es la conexión TLS, mientras que `channel_t` es la abstracción que la utiliza.

7.2 La identidad de un repetidor

Antes de examinar el intercambio inicial debemos saber *qué* se necesita demostrar. Un repetidor se identifica mediante claves de larga duración. La especificación les asigna nombres sistemáticos: el prefijo `KP_` designa una clave pública y `KS_` su clave secreta correspondiente [torspec § Relay keys and identities]. Las claves relevantes para el canal son:

`KP_relayid_ed` Identidad Ed25519, también llamada «clave de identidad maestra». No caduca y tiene una única función: certificar la clave de firma. Puede mantenerse fuera de línea.

`KP_relayid_rsa` Identidad RSA heredada, de 1024 bits. Sigue siendo la base de las huellas de cuarenta dígitos hexadecimales que muestra Tor Metrics, pero la seguridad del protocolo ya no depende de ella.

`KP_relaysign_ed` Clave de firma de duración intermedia, certificada por la identidad. Es la clave que el repetidor utiliza normalmente mientras está conectado.

`KP_link_ed` Clave de corta duración que solo se utiliza cuando un repetidor inicia un canal y necesita autenticarse (apartado 7.6).

La figura 7.1 muestra cómo los certificados vinculan estas claves entre sí. La idea consiste en una *cadena de confianza* que comienza en la identidad y llega hasta el certificado TLS utilizado por la conexión.

En el laboratorio, estas claves son archivos situados en el directorio `keys/` de cada repetidor: `ed25519_master_id_public_key` y la clave secreta de identidad correspondiente, `ed25519_signing_cert` y `ed25519_signing_secret_key` para la clave de firma, y `secret_id_key` para la identidad RSA. El archivo `fingerprint-ed25519`, ubicado en el directorio del nodo, contiene su identidad Ed25519 codificada en base64.

Nota histórica: De RSA a Ed25519

Hasta 2015, la única identidad de un repetidor era su clave RSA de 1024 bits, una longitud que hoy se considera insuficiente. La Propuesta 220 introdujo las identidades Ed25519 y

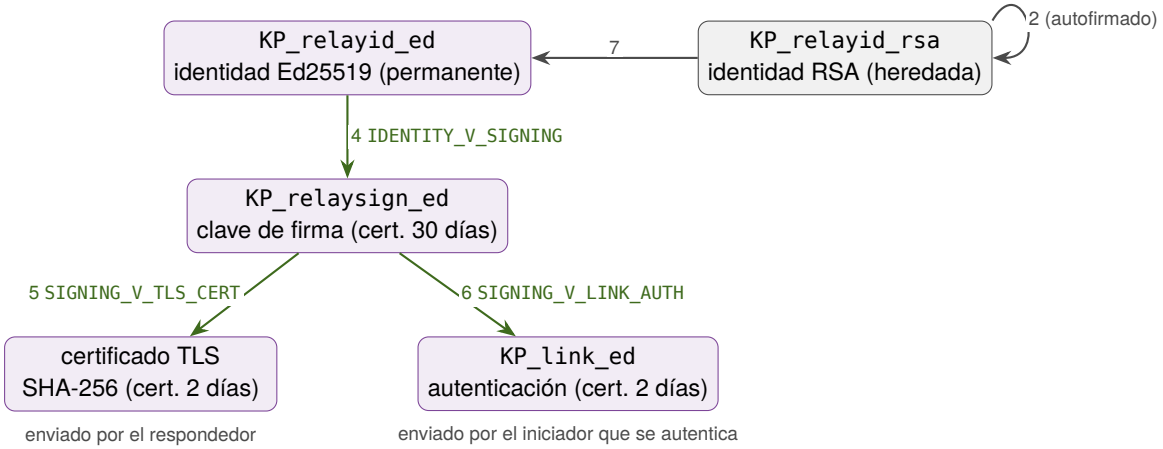


Figura 7.1: Cadena de certificados que vincula la identidad Ed25519 de un repetidor con el certificado TLS de la conexión. Cada flecha representa un certificado: la clave de origen firma la clave (o huella) de destino. El número identifica el CertType utilizado en la celda CERTS. A la derecha aparece la cadena RSA heredada, que vincula la antigua identidad RSA con la identidad Ed25519.

el formato de certificados descrito en este capítulo; la compatibilidad comenzó con tor 0.2.7.2-alpha. La clave RSA se conservó por compatibilidad y se vinculó con la Ed25519 mediante un certificado cruzado.

7.3 Resumen del intercambio inicial

La figura 7.2 resume cómo se establece un canal [tor-spec § Negotiating and initializing channels].

1. El iniciador abre una conexión TLS.
2. Ambas partes envían una celda **`VERSIONS`** y acuerdan una versión del protocolo de enlace (apartado 8.3).
3. El respondedor envía **`CERTS`**, con los certificados que demuestran su identidad; **`AUTH_CHALLENGE`**, un desafío que el iniciador firmará si desea autenticarse; y **`NETINFO`**, con la hora y las direcciones.
4. El iniciador verifica los certificados. Si es un repetidor público, responde con **`CERTS`** y **`AUTHENTICATE`**; en todos los casos termina con **`NETINFO`**.

La celda **`NETINFO`** marca el final del intercambio inicial: una vez enviada, esa parte ya no transmite más celdas del intercambio. Así sabe el respondedor si el iniciador ha decidido no autenticarse. Entre unas celdas y otras pueden aparecer tantas **`VPADDING`** como sea necesario, pero ningún otro tipo de celda.

7.4 La capa TLS

Tor utiliza TLS de una manera poco habitual. En un sitio web, el certificado del servidor está firmado por una autoridad de certificación y contiene un nombre de dominio; el navegador

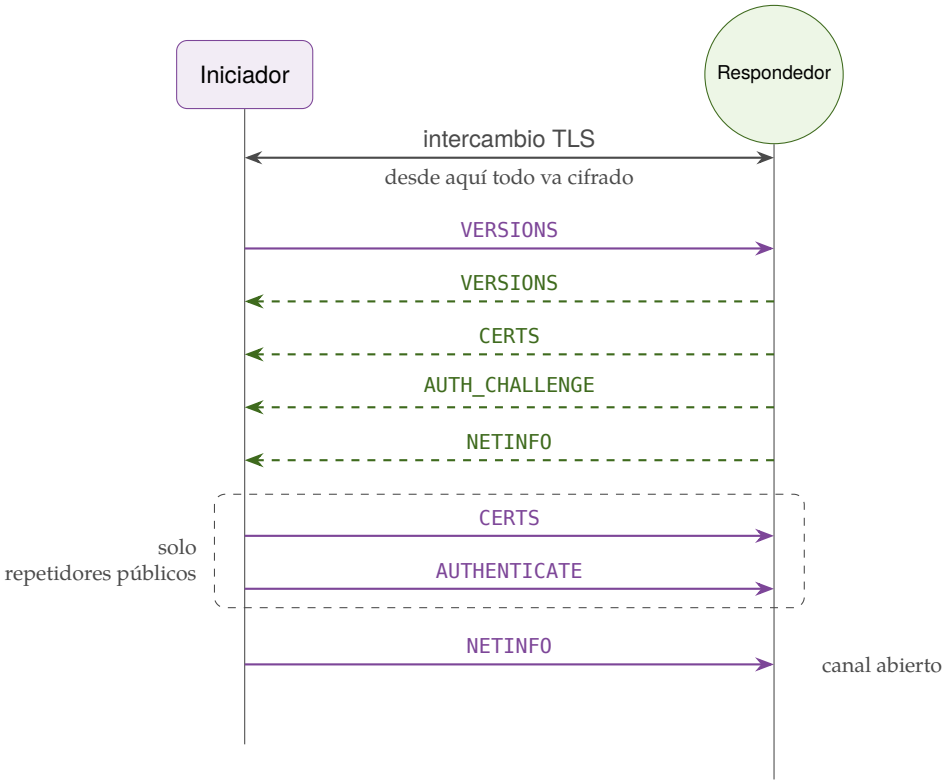


Figura 7.2: Intercambio inicial del canal. Una vez establecido TLS, todos los mensajes viajan cifrados. El respondedor envía siempre CERTS, AUTH_CHALLENGE y NETINFO, en ese orden. El recuadro discontinuo se aplica solo cuando el iniciador es un repetidor público que necesita demostrar su identidad; un cliente o un puente envía únicamente NETINFO.

comprueba ambos elementos. Un repetidor, en cambio, presenta un certificado **autofirmado** sin un nombre significativo, y el cliente no lo valida según las reglas habituales de TLS. En el código del laboratorio 1 esto se traduce en dos líneas que serían un error de seguridad grave en casi cualquier otro contexto:

```
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

Aquí no son un error porque la autenticación se realiza *después*, dentro del canal, mediante la celda **CERTS**. TLS proporciona cifrado e integridad al canal y demuestra la posesión de la clave asociada al certificado TLS; las celdas establecen las demás garantías.

La especificación también establece varias recomendaciones [tor-spec § The TLS handshake]:

- las implementaciones deberían admitir TLS 1.3 y los repetidores también TLS 1.2, con suites modernas (AES-GCM, ChaCha20-Poly1305 e intercambio de claves mediante X25519 o P-256);
- no debería permitirse la *reanudación de sesión* (*session resumption*), porque debilita el secreto hacia adelante y ha facilitado ataques como el *Triple Handshake* (2013);
- no debería utilizarse compresión TLS.

En la red de laboratorio, un cliente Python con OpenSSL 3.5 negocia TLS 1.3 con `tor` y utiliza la suite `TLS_AES_256_GCM_SHA384`.

Nota histórica: ¿Por qué un intercambio tan poco habitual?

Las primeras versiones del protocolo de enlace colocaban los certificados de Tor directamente en el intercambio TLS (versión 1). Después los ocultaban mediante la renegociación TLS (versión 2), indicando la variante seleccionada con la lista de suites de cifrado ofrecidas. La razón era la censura: antes de los transportes conectables (capítulo 16), Tor intentaba parecer una conexión HTTPS corriente entre un navegador y un servidor web. Con TLS 1.2, los certificados viajaban sin cifrar y resultaban fáciles de reconocer. En la versión 3 pasaron al interior del canal cifrado, dentro de celdas **CERTS**. Hoy la resistencia a la censura se confía a los transportes conectables y las variantes antiguas están obsoletas. La especificación señala que en marzo de 2025 `tor` todavía aceptaba algunas, aunque ya estaba prevista su retirada.

7.5 CERTS: demostrar la identidad

7.5.1 La celda

CERTS es una celda de longitud variable (comando 129) cuyo cuerpo contiene una lista de certificados:

Campo	Bytes	Contenido
N	1	número de certificados
se repite N veces:		
CertType	1	tipo de certificado
CertLen	2	longitud del certificado
Certificate	CertLen	el certificado

Una celda **CERTS** no puede contener dos certificados del mismo tipo. Los tipos definidos se enumeran en la cuadro 7.1 [cert-spec § Certificate types (CERT_TYPE field)].

Tipo	Nombre	Formato	Qué certifica
1	`TLS_LINK_X509`	X.509	clave TLS, firmada por la identidad RSA (obsoleto)
2	`RSA_ID_X509`	X.509	identidad RSA autofirmada
3	`LINK_AUTH_X509`	X.509	clave RSA de autenticación (obsoleto)
4	`IDENTITY_V_SIGNING`	Ed	clave de firma, firmada por la identidad Ed25519
5	`SIGNING_V_TLS_CERT`	Ed	huella del certificado TLS, firmada por la clave de firma
6	`SIGNING_V_LINK_AUTH`	Ed	clave `KP_link_ed`, firmada por la clave de firma
7	`RSA_ID_V_IDENTITY`	RSA→Ed	identidad Ed25519, firmada por la identidad RSA
8–12	servicios onion (8, 9, 11), certificado cruzado ntor (10), familias (12)		

Cuadro 7.1: Tipos de certificado. «Ed» indica el formato de certificados Ed25519 de Tor.

7.5.2 El certificado Ed25519 de Tor

Tor no utiliza X.509 para sus certificados Ed25519. Emplea un formato propio mucho más sencillo [cert-spec § Tor Ed25519 Certificates], representado en la figura 7.3.

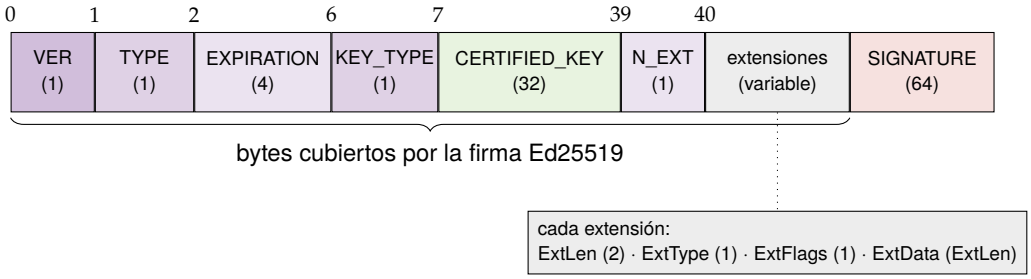


Figura 7.3: Certificado Ed25519 de Tor. La firma cubre todos los campos que la preceden. La caducidad se expresa en *horas* desde la época Unix: un campo sin signo de cuatro bytes no se desbordará hasta dentro de cientos de miles de años.

VERSION Tiene el valor 1.

CERT_TYPE Tiene el mismo valor que CertType en la cuadro 7.1.

EXPIRATION_DATE Expresa la fecha de caducidad en horas desde el 1 de enero de 1970.

CERT_KEY_TYPE Indica qué contiene el campo siguiente: 1 significa una clave Ed25519 y 3 indica la huella SHA-256 de un certificado X.509. Las versiones de `tor` hasta 0.4.5.1-alpha escribían siempre 1, y la especificación exige que las implementaciones lo toleren.

CERTIFIED_KEY Contiene los 32 bytes certificados.

Extensiones Cada una contiene longitud, tipo, indicadores y datos. La única extensión definida actualmente es la de tipo 4, *signed-with-ed25519-key*, que incluye la clave firmante del certificado. El indicador 1 (`AFFECTS_VALIDATION`) determina cómo tratar una extensión desconocida: si está activado, debe rechazarse el certificado; en caso contrario, se ignora la extensión.

SIGNATURE Es la firma Ed25519 que cubre todos los campos anteriores.

7.5.3 Verificación

El respondedor debe enviar exactamente un certificado de tipo 4 y otro de tipo 5. El iniciador los verifica del siguiente modo [tor-spec § Authenticating the responder from its CERTS]:

1. El certificado de tipo 4 contiene una clave Ed25519 en la extensión 4: la identidad declarada `KP_relayid_ed`. Debe estar firmado con esa misma clave (es *autofirmado*); la clave certificada es `KP_relaysign_ed`.
2. El certificado de tipo 5 debe estar firmado con `KP_relaysign_ed`, y su clave certificada debe ser la huella SHA-256 del certificado TLS recibido durante el intercambio TLS.
3. Ninguno de los dos certificados puede haber caducado.
4. Por último, la identidad obtenida debe compararse con la identidad esperada.

¿Por qué demuestra algo esta cadena? El segundo paso vincula la clave de firma con el certificado TLS de *esta* conexión. El intercambio TLS ya ha demostrado que el respondedor posee la clave privada correspondiente al certificado. El primer paso vincula la clave de firma con la identidad. Un impostor podría copiar las celdas **`CERTS`** de un repetidor legítimo, porque no contienen secretos, pero no podría utilizarlas: tendría que presentar el certificado TLS de ese repetidor legítimo en el intercambio TLS y, por tanto, poseer su clave privada.

Advertencia: Comprobar la identidad esperada

Una cadena completamente válida solo demuestra que el respondedor posee *alguna* identidad. Si el iniciador omite el paso 4, cualquiera que disponga de una identidad propia podría hacerse pasar por el repetidor previsto. El laboratorio 1 incluye el parámetro `expected_identity` precisamente por eso, y una de sus pruebas comprueba que se rechaza a un repetidor que presenta una identidad distinta.

7.5.4 La cadena RSA heredada

El iniciador también puede verificar de forma opcional la identidad RSA. El certificado de tipo 2 contiene la clave RSA de 1024 bits (autofirmada, exponente 65537); el certificado de tipo 7, en formato de «certificado cruzado», vincula esa clave con la identidad Ed25519 ya verificada. La firma RSA de dicho certificado se calcula sobre la huella SHA-256 de la cadena `"Tor TLS RSA/Ed25519 cross-certificate"` seguida por los campos del certificado. El prefijo impide reutilizar la misma firma en un contexto distinto.

7.5.5 Qué envía realmente un repetidor

En la red de laboratorio, el repetidor `test003r` envía una celda **`CERTS`** que contiene cinco certificados:

```
type 1  584 bytes
type 2  442 bytes
type 4  140 bytes  expires 2026-10-24 16:00  key_type=1  ext=[4]
type 5  104 bytes  expires 2026-09-29 11:00  key_type=3  ext=[]
type 7  165 bytes
```

Además de la cadena Ed25519 (tipos 4 y 5) y de la RSA (tipos 2 y 7), el repetidor continúa enviando el antiguo certificado de tipo 1 para clientes anteriores. Este certificado no solo tiene la misma longitud que el certificado TLS, sino también su huella. Al comparar sus bytes se comprueba que son *exactamente* el mismo certificado. La longitud no es un dato que merezca memorizarse, y conviene entender por qué. El nombre común del certificado es un dominio ficticio que `tor` construye con `www.`, seguido de entre 4 y 25 caracteres aleatorios y de `.com` (`crypto_random_hostname` en `src/lib/tls/tortls_openssl.c`). Solo eso hace que la longitud varíe unos veinte bytes entre repetidores.

También cambia *con el tiempo* para un mismo repetidor, porque `tor` renueva el contexto TLS y, con él, el certificado, cada dos horas (`MAX_SSL_KEY_LIFETIME_INTERNAL` en `src/core/or/or.h`). Cuando repetimos la medición cuarenta minutos después, el cuerpo de la celda **`CERTS`** tenía 1456 bytes en vez de 1451. Si tus mediciones difieren, es de esperar: lo que debe coincidir es la *estructura*, y la suma de las longitudes de los certificados, más un byte de recuento y tres bytes de cabecera por certificado, debe ser igual a la longitud del cuerpo de la celda.

Las dos fechas de caducidad muestran la distinta duración de las claves: la clave de firma se certifica durante treinta días, mientras que su vinculación con el certificado TLS dura solo dos días. Son los valores predeterminados de `tor` (opciones `SigningKeyLifetime` y `TestingLinkCertLifetime`, esta última configurable únicamente en redes de prueba), por lo que también se aplican en la red pública. La clave de identidad permanece constante, mientras las claves que la representan en línea se renuevan con frecuencia, lo que limita el daño si el repetidor resulta comprometido.

7.6 AUTH_CHALLENGE y AUTHENTICATE

7.6.1 ¿Quién se autentica?

La autenticación del iniciador es opcional. La regla que determina quién debería realizarla es una de las decisiones de diseño más importantes del protocolo [tor-spec § Channels]:

- Un **repetidor público** que abre un canal hacia otro repetidor *debería* autenticarse: así, el respondedor sabe que está hablando con un repetidor de la red y no con un cliente.
- Un **cliente** o un **puente** *no deberían* autenticarse.

La razón de la segunda regla es el anonimato. Un cliente carece de identidad de repetidor, y crear una sería perjudicial: un guardia podría reconocer al mismo cliente cuando volviera a conectarse desde otra dirección. Por su parte, un puente no debe revelar a sus usuarios que es un repetidor de red. Como contrapartida, un repetidor puede inferir que ocupa el primer salto de un circuito si el canal no está autenticado (capítulo 8, apartado 8.6). Esta es una de las razones por las que los repetidores se niegan a actuar como salida de circuitos de un solo salto.

7.6.2 El desafío

El respondedor envía siempre una celda **`AUTH_CHALLENGE`**. Contiene 32 bytes aleatorios y una lista de métodos de autenticación admitidos:

Campo	Bytes	Contenido
Challenge	32	valor aleatorio, distinto para cada canal
N_Methods	2	número de métodos
Methods	2× N_Methods	lista de métodos

Los métodos definidos son 1 (`RSA-SHA256-TLSSecret`, obsoleto), 2 (nunca implementado) y 3 (`Ed25519-SHA256-RFC5705`). La celda recibida de los repetidores de laboratorio tiene 36 bytes: $32 + 2 + 2$, lo que corresponde a un único método, el número 3.

7.6.3 La respuesta

Un repetidor que desea autenticarse responde con su propia celda **`CERTS`** (tipos 4 y 6, en vez de 4 y 5, ya que no necesariamente ha presentado un certificado TLS como iniciador) y luego con **`AUTHENTICATE`**. Con el método 3, el cuerpo de **`AUTHENTICATE`** es un documento firmado de 352 bytes con la clave `KP_link_ed`:

Campo	Bytes	Contenido
TYPE	8	cadena `AUTH0003`
CID, SID	32 + 32	huellas SHA-256 de las identidades RSA del iniciador y del respondedor
CID_ED, SID_ED	32 + 32	identidades Ed25519 del iniciador y del respondedor
SLOG	32	SHA-256 de todos los bytes *recibidos* por el canal hasta AUTH_CHALLENGE, inclusive
CLOG	32	SHA-256 de todos los bytes *enviados* por el canal antes de AUTHENTICATE
SCERT	32	SHA-256 del certificado TLS del respondedor
TLSSECRETS	32	valor derivado de los secretos de la sesión TLS (*keying material exporter*)
RAND	24	bytes aleatorios
SIG	64	firma Ed25519 de todos los campos anteriores

Cada campo vincula la firma con algún aspecto de *esta* conexión: las identidades implicadas (CID, SID y otros), la conversación mantenida hasta ese momento (SLOG y CLOG, que incluyen el desafío aleatorio), el certificado TLS (SCERT) y la propia sesión TLS (TLSSECRETS). Una firma de este tipo no puede reutilizarse en otra conexión, ni tampoco por un intermediario que retransmita mensajes entre dos sesiones TLS distintas (ataque de *intermediario*, o *man-in-the-middle*), porque TLSSECRETS y SCERT serían diferentes a ambos lados.

En profundidad: Un error conservado en la especificación

TLSSECRETS se calcula mediante el *exportador* TLS (RFC 5705 y, para TLS 1.3, RFC 8446, §7.5), utilizando la etiqueta `"EXPORTER FOR TOR TLS CLIENT BINDING AUTH0003"` y un valor de contexto. La intención era utilizar como contexto la identidad Ed25519 del iniciador, pero la implementación usó por equivocación CID, la huella de identidad RSA. Dado que esto no debilita la seguridad, se decidió documentar el comportamiento real en vez de modificarlo. Es un ejemplo útil de cómo, en un protocolo con miles de implementaciones ya desplegadas, el comportamiento efectivo del código puede convertirse en la norma.

7.7 NETINFO: hora y direcciones

La última celda del intercambio inicial, enviada por ambas partes, es **`NETINFO`**. Es una celda de longitud fija (comando 8), con el siguiente cuerpo:

Campo	Bytes	Contenido
TIME	4	segundos desde la época Unix, en orden de bytes de mayor a menor peso
OTHERADDR	2 + ALEN	dirección de la otra parte tal como la ve el remitente
NMYADDR	1	número de direcciones propias del remitente
MYADDR	NMYADDR × (2 + ALEN)	direcciones propias del remitente

Cada dirección se codifica con un tipo (un byte: `0x04` para IPv4 o `0x06` para IPv6), una longitud (4 o 16) y un valor. Estos campos cumplen dos funciones:

- **La hora** permite detectar una configuración incorrecta del reloj: `tor` registra una advertencia si la hora recibida de un repetidor difiere de la suya en más de una hora (constante `NETINFO_NOTICE_SKEW` en `src/core/or/channeltls.c`).
- **Las direcciones** permiten a un repetidor descubrir desde qué dirección pública resulta visible y al iniciador comprobar si ha alcanzado al repetidor a través de una de sus direcciones *canónicas*, es decir, publicadas en su descriptor.

Advertencia: Un cliente no revela su reloj

Los repetidores envían sus propias horas y direcciones. Un cliente, sin embargo, debería enviar `00 00 00 00` como hora y no transmitir ninguna dirección propia. El motivo es la *identificación por huella*: el reloj de un ordenador suele tener una pequeña desviación característica. Si un cliente revelara su hora a cada guardia, esa desviación podría contribuir a reconocerlo a lo largo del tiempo. Este ejemplo ilustra una regla general que volverá a aparecer: un cliente Tor no debe divulgar información que no sea estrictamente necesaria.

El campo TIME de cuatro bytes sin signo se desbordará en 2106; la Propuesta 338 estudia ampliarlo a ocho bytes. El destinatario de una celda **`NETINFO`** no debería confiar ciegamente en sus valores: la otra parte puede mentir sobre sus direcciones y su hora.

Esta es la celda **`NETINFO`** recibida del repetidor `test003r`, descodificada en el laboratorio 1:

```
Netinfo(timestamp=1790511791, peer_address='127.0.0.1',
        my_addresses=['127.0.0.1'])
```

La hora corresponde al 27 de septiembre de 2026, a las 12:23:11 UTC. Ambas direcciones son de bucle local porque toda la red funciona en el mismo ordenador. Nuestra respuesta, construida por `client_netinfo_cell`, tiene como cuerpo `00000000 04 04 7f000001 00`: hora cero, dirección del repetidor y ninguna dirección propia.

7.8 Después del intercambio inicial

Llegados a este punto, el canal está *abierto* y puede transportar circuitos. Un experimento sencillo permite observar un detalle interesante. Si escuchamos después de abrir el canal sin crear ningún circuito, el repetidor no envía *nada*: en el laboratorio no llega ni una sola celda durante quince segundos.

Podríamos esperar celdas **`PADDING`**: desde el protocolo de enlace 5, los clientes y los guardias intercambian celdas de relleno en canales inactivos a intervalos aleatorios de entre 1,5 y 9,5 segundos, de modo que un observador de la red (por ejemplo, un encaminador que registre estadísticas NetFlow) no pueda distinguir los períodos activos de los silenciosos [padding-spec § Connection-level padding]. El código fuente de `tor` explica por qué aquí no ocurre. La función que decide si añadir relleno (`channelpadding_decide_to_pad_channel`, en `src/core/or/channelpadding.c`) lo activa únicamente en canales *ya utilizados* para circuitos completos o tráfico de usuario.

Desde el punto de vista del repetidor, un canal solo se considera utilizado de esta forma cuando lo atraviesa un circuito de varios saltos: la primera celda **`RELAY_EARLY`** (extensión hacia el segundo salto) o una celda **`RELAY`** en un circuito que continúa más allá del repetidor (`src/core/or/relay.c`). Un canal nuevo permanece silencioso, igual que uno que transporta

únicamente circuitos de un salto, como descubriremos en el capítulo 9. El relleno aparecerá en el capítulo 10, cuando nuestro cliente extienda un circuito.

Este pequeño episodio deja una enseñanza metodológica: *la ausencia de una observación no constituye una prueba*. Antes de concluir que el relleno no funciona, debemos verificar que realmente se cumplen las condiciones en las que debería aparecer.

7.9 Laboratorio: abrir un canal verificado

Laboratorio 1: Abrir un canal hacia un repetidor y verificar su identidad

Objetivo. Escribir `labs/es/codigo/torlab/channel.py`, que abre un canal como un cliente Tor: TLS, **`VERSIONS`**, verificación de **`CERTS`**, lectura de **`AUTH_CHALLENGE`** y **`NETINFO`**, y envío de su propia celda **`NETINFO`**.
Requisitos previos. Laboratorio 2 (`cells.py`, capítulo 8): la manipulación de celdas es la base del canal, por lo que tiene sentido implementarla primero, aunque se presente en el capítulo siguiente. La explicación de arriba abajo (primero el canal, después las celdas) y la implementación de abajo arriba (primero las celdas, después el canal) siguen aquí recorridos opuestos, algo completamente normal. También se necesita una red de laboratorio en funcionamiento (capítulo 19) y la biblioteca `cryptography` para Ed25519.
Tiempo estimado. 2–3 horas.

Paso 1: descodificar un certificado Ed25519. Siguiendo la figura 7.3, leemos la cabecera de longitud fija y después las extensiones, y conservamos los bytes que cubre la firma:

```
def decode_ed_certificate(data):
    if len(data) < 40 + 64 or data[0] != 1:
        raise ChannelError("certificado Ed25519 malformado")
    _, kind, expiration, key_type = struct.unpack(">BBIB", data[:7])
    key = data[7:39]
    extension_count, pos, extensions = data[39], 40, {}
    for _ in range(extension_count):
        length, extension_type, flag = struct.unpack(">HBB", data[pos:pos+4])
        body = data[pos+4:pos+4+length]
        if extension_type != 4 and flag & 1:     # AFFECTS_VALIDATION
            raise ChannelError("extensión desconocida que afecta a la validación")
        extensions[extension_type] = (flag, body)
        pos += 4 + length
    return EdCertificate(kind, expiration, key_type, key,
                         extensions, data[:pos], data[pos:])
```

Paso 2: verificar la cadena. Esta función aplica las reglas de la apartado 7.5.3 y devuelve la identidad cuya posesión se ha demostrado:

```
def verify_responder_identity(certificates, cert_tls_der):
    id_v_sign = decode_ed_certificate(certificates[4])
    sign_v_tls = decode_ed_certificate(certificates[5])
    identity = id_v_sign.signing_key               # extensión 4
    id_v_sign.verify_signature(identity)             # autofirmado
    sign_v_tls.verify_signature(id_v_sign.certified_key)
    if sign_v_tls.certified_key != sha256(cert_tls_der).digest():
        raise ChannelError("el certificado TLS no coincide")
    for c in (id_v_sign, sign_v_tls):
        if c.expired():
```

```
            raise ChannelError("certificado caducado")
    return identity
```

Obtenemos el certificado TLS del socket en formato DER mediante `sock.getpeercert(binary_form=True)`. Python lo devuelve incluso aunque, en este caso, no lo hayamos validado.

Paso 3: intercambio inicial. La función `open_channel` envía **`VERSIONS`**, ofreciendo las versiones 4 y 5 (igual que Arti, omite la 3). Después lee celdas hasta que recibe **`NETINFO`**: ignora **`VPADDING`** y las **`VERSIONS`** posteriores a la primera, y rechaza cualquier otra celda inesperada. Finalmente verifica los certificados, compara la identidad con la esperada y envía su propia **`NETINFO`**. El código completo está en el repositorio; su bucle central es:

```
while netinfo is None:
    cell, buffer = _read(sock, buffer, version)
    if cell.command == CellCommand.VERSIONS:
        if version:
            continue
        version = negotiate_version(SUPPORTED_VERSIONS, cell.body)
    elif cell.command == CellCommand.CERTS:
        certificates = decode_certs(cell.body)
    elif cell.command == CellCommand.AUTH_CHALLENGE:
        ...                                     # métodos aceptados
    elif cell.command == CellCommand.NETINFO:
        netinfo = decode_netinfo(cell.body)
    elif cell.command != CellCommand.VPADDING:
        raise ChannelError("celda inesperada durante el intercambio inicial")
```

Paso 4: pruebas sin red. En `labs/es/codigo/torlab/test/test_channel.py` generaremos dos claves Ed25519, construiremos una cadena de certificados válida y comprobaremos que el código la acepta. Después la alteraremos de todas las maneras descritas por la especificación (firma modificada, clave firmante no certificada, certificado TLS diferente, certificado caducado y tipo duplicado) y comprobaremos que se detecta cada defecto. Una prueba que comprueba únicamente el caso válido no demuestra nada: un verificador que acepte todos los casos también la superaría.

Paso 5: ejecutar en la red de laboratorio. Con la red en marcha:

```
from torlab.channel import open_channel, identity_from_file

expected = identity_from_file("labs/es/laboratorio/net/nodes/003r/fingerprint-ed25519")
c = open_channel("127.0.0.1", 5103, expected_identity=expected)
print(c.version, c.ed_identity.hex(), c.netinfo, c.auth_methods)
print(c.registry)
```

En una de nuestras ejecuciones:

```
5 785d3199f21d20ad...  Netinfo(time=1790511793, ...)  [3]
['TLS TLSv1.3 TLS_AES_256_GCM_SHA384', '<- VERSIONS (6 byte)',
 '<- CERTS (1451 byte)', '<- AUTH_CHALLENGE (36 byte)',
 '<- NETINFO (509 byte)', '-> NETINFO: channel open']
```

En cambio, si se conecta al puerto 5104 (repetidor `test004r`) esperando la identidad de otro repetidor, el código rechaza la conexión, como debe ser, con el diagnóstico «la identidad del repetidor es distinta de la esperada».

Resumen

- Un canal es una sesión TLS entre dos nodos adyacentes. Solo los repetidores reciben canales y un canal transporta muchos circuitos.
- La identidad de un repetidor es su clave Ed25519 `KP_relayid_ed`, que certifica la clave de firma `KP_relaysign_ed`; la identidad RSA permanece por compatibilidad.
- El certificado TLS de un repetidor está autofirmado. La autenticación se realiza dentro del canal, mediante **`CERTS`**, que vincula la identidad, la clave firmante y la huella del certificado TLS.
- El iniciador debe verificar las firmas, las fechas de caducidad, la vinculación con TLS y la **identidad esperada**.
- Los repetidores que inician canales se autentican mediante **`AUTHENTICATE`**, una firma vinculada a la sesión TLS y a la conversación. Los clientes y puentes no se autentican para evitar crear identidades reconocibles.
- **`NETINFO`** termina el intercambio inicial. Un cliente envía una marca temporal cero y ninguna dirección propia.
- El relleno del enlace necesita que el canal ya haya transportado un circuito *completo* o tráfico de usuario. Un canal nuevo, o uno usado solo por circuitos de un salto, permanece silencioso. Superado ese umbral, el relleno también puede comenzar antes del primer tráfico de aplicación (`nf_pad_before_usage`, cuyo valor predeterminado es 1).

Ejercicios

Ejercicio 7.1 ⋆ ¿Por qué un cliente Tor no necesita un puerto de escucha accesible desde la red? ¿En qué circunstancias inicia canales un repetidor?

Ejercicio 7.2 ⋆ Un certificado Ed25519 tiene un campo EXPIRATION_DATE igual a `0x000772F7`. ¿Cuándo caduca? ¿Por qué la especificación eligió horas en vez de segundos?

Ejercicio 7.3 ⋆ Una celda **`AUTH_CHALLENGE`** tiene un cuerpo de 38 bytes. ¿Cuántos métodos ofrece? ¿Es válida una celda de 36 bytes con N_Methods igual a 2?

Ejercicio 7.4 ⋆⋆ Un adversario graba la celda **`CERTS`** de un repetidor legítimo y la reproduce sin cambios en su propio servidor, pero presenta su propio certificado TLS. ¿Qué paso de verificación falla? ¿Qué ocurriría si también presentara el certificado TLS del repetidor legítimo?

Ejercicio 7.5 ⋆⋆ Escribe en hexadecimal el cuerpo de la celda **`NETINFO`** enviada por un repetidor situado en `203.0.113.5`, a las 12:00:00 UTC del 1 de enero de 2026, a un cliente que se ha conectado desde `198.51.100.7`.

Ejercicio 7.6 ⋆⋆ Supongamos que los clientes Tor se autenticaran ante los guardias mediante una clave generada una sola vez y guardada en disco. Describe dos maneras concretas en que esto perjudicaría el anonimato de los usuarios.

Ejercicio 7.7 ⋆⋆ Amplía `labs/es/codigo/torlab/channel.py` para verificar opcionalmente la identidad RSA (tipos 2 y 7), siguiendo la apartado 7.5. Compara la huella SHA-1 de la clave RSA obtenida con el archivo `fingerprint` del repetidor. *Pista:* la huella es el SHA-1 de la codificación DER de la clave pública.

Ejercicio 7.8 ⋆⋆⋆ Con la red de laboratorio en funcionamiento, abre un canal mediante `labs/es/codigo/torlab/channel.py` y mantenlo abierto mientras el cliente `test008c` descarga una página a través de la red. ¿Recibirás celdas **`PADDING`** por *tu* canal? Formula una hipótesis, ponla a prueba y explica el resultado a la luz de `channelpadding_decide_to_pad_channel`.

Ejercicio 7.9 ⋆⋆⋆ Explica por qué el campo TLSSECRETS de **`AUTHENTICATE`** impide que un intermediario retransmita la autenticación del repetidor A al repetidor B mediante otra sesión TLS, aunque reenvíe fielmente todas las celdas. ¿Qué ocurriría si no existiera TLSSECRETS?

Lecturas complementarias

- *Tor Protocol Specification*, capítulos «Channels», «Negotiating and initializing channels» y «Relay keys», y *Tor certificate formats* (cert-spec) [Thee].
- Propuesta 220, *Migrate server identity keys to Ed25519*: razones para sustituir RSA.
- *Tor channel padding*, de la especificación de relleno: modelo de adversario basado en NetFlow y selección de intervalos.
- RFC 5705 y RFC 8446 (§7.5): funciones *keying material exporter* de TLS.

Capítulo 8

Las celdas de Tor

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; Arti 2.6.0; protocolo de enlace 3–5

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir byte a byte los formatos de celdas de longitud fija y variable;
- explicar por qué Tor utiliza unidades de tamaño fijo, qué protege esta elección y qué no protege;
- interpretar los CircID y las reglas para elegirlos en cada canal;
- identificar todos los comandos de celda y los principales comandos relay;
- leer la cabecera de un mensaje relay y explicar las funciones de `Recognized` y `Digest`;
- implementar en Python un codificador y descodificador de celdas (laboratorio 2).

En el capítulo anterior vimos cómo dos nodos Tor abren un *canal*: una conexión TLS en la que acuerdan una versión del protocolo y se autentican. Ahora estudiaremos *qué circula* por ese canal. La respuesta es sencilla: **celdas**. Todo lo que transporta Tor, desde la construcción de un circuito hasta la descarga de páginas web, se divide en celdas. Comprenderlas equivale a aprender el alfabeto con el que está escrito todo el protocolo.

8.1 ¿Por qué utilizar celdas?

Un sistema de anonimato de baja latencia no puede ocultar *que* dos nodos se comunican, pero sí puede intentar ocultar *qué* intercambian y *cuánta información* pueden conocer los nodos intermedios. Si cada nodo reenviara los datos exactamente como llegaran, en paquetes de tamaños diferentes, un observador podría seguir un mensaje de 1337 bytes por la red simplemente buscando un mensaje de esa longitud en cada enlace.

El diseño original de encaminamiento cebolla, seguido por Tor [DMS04], adoptó por ello una unidad de transmisión de **tamaño fijo**: la celda. Cuando todas las celdas tienen el mismo tamaño:

- la longitud de un mensaje individual no revela su contenido ni la identidad del flujo;
- los repetidores pueden administrar las colas y los búferes de forma sencilla y eficiente;
- una cabecera mínima permite reenviar una celda después de leer muy pocos bytes.

Advertencia: Lo que las celdas *no* ocultan

Las celdas normalizan el *tamaño* de cada unidad, no la *cantidad* de unidades ni sus *tiempos*. Un adversario que observe los dos extremos de un circuito aún puede correlacionar los volúmenes de tráfico y los tiempos. Se trata de una limitación fundamental de Tor, que analizaremos en el capítulo 26. Algunas defensas parciales, como el relleno a nivel de canal y de circuito, se explican en capítulos 7 y 26 y en la parte V.

Nota histórica: 512 bytes

El artículo que presentó Tor en 2004 describe celdas de 512 bytes: un identificador de circuito de 2 bytes, un comando de 1 byte y 509 bytes de datos [DMS04]. El cuerpo de 509 bytes se ha conservado hasta nuestros días. Sin embargo, el identificador de circuito creció hasta 4 bytes con el protocolo de enlace 4, aumentando el tamaño total a 514 bytes.

8.2 Del flujo TLS a las celdas

Una vez completado el intercambio TLS, las dos partes intercambian celdas *secuencialmente*, una tras otra, como un flujo continuo de bytes [tor-spec § Cells (messages on channels)]. Los límites de los registros TLS son independientes de los límites de las celdas: una celda puede dividirse entre varios registros y un registro puede contener varias celdas o fragmentos de ellas. No obstante, la especificación exige que la segmentación de los registros TLS no revele información sobre el tipo ni el contenido de las celdas.

Esto tiene una consecuencia práctica para quienes implementan el protocolo: el receptor no puede esperar leer «una celda por llamada» al socket. Debe acumular bytes en un búfer y extraer una celda únicamente cuando esté completa, conservando los bytes sobrantes. Nuestro descodificador del laboratorio 2 funcionará exactamente así.

8.3 Formatos de celda

Existen dos formatos: **longitud fija** y **longitud variable**. Ambos empiezan por los mismos campos, `CircID` y `Command`.

8.3.1 Constantes

La especificación define las siguientes longitudes [tor-spec § Message lengths], donde v es la versión del protocolo de enlace negociada en el canal:

Nombre	Bytes	Significado
`CELL_BODY_LEN`	509	cuerpo de celda de longitud fija
`CIRCID_LEN`(v), $v < 4$	2	longitud de CircID
`CIRCID_LEN`(v), $v \geq 4$	4	
`CELL_LEN`(v), $v < 4$	512	longitud total de celda fija
`CELL_LEN`(v), $v \geq 4$	514	

Para cualquier v, `CELL_LEN`$(v) = 1 +$ `CIRCID_LEN`$(v) +$ `CELL_BODY_LEN`.

En profundidad: Una denominación antigua

El código y la documentación anteriores llaman `PAYLOAD_LEN` a `CELL_BODY_LEN`. Ambos nombres representan el mismo valor: 509.

8.3.2 Celdas de longitud fija

La figura 8.1 muestra una celda de longitud fija con el protocolo de enlace actual ($v \geq 4$).

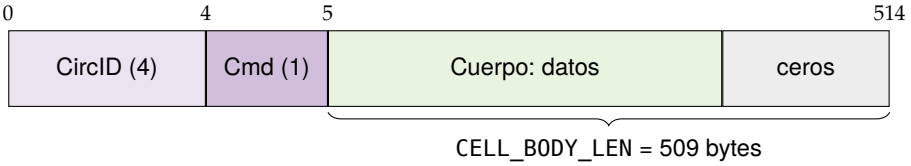


Figura 8.1: Celda de longitud fija (protocolo de enlace ≥ 4): CircID de 4 bytes, comando de 1 byte y cuerpo de 509 bytes, para un total de 514 bytes. Si el contenido útil es menor que 509 bytes, el resto del cuerpo se completa con ceros. Los números indican desplazamientos en bytes desde el inicio de la celda.

Cuando la información ocupa menos de 509 bytes, el remitente **debe** completar con ceros los bytes restantes y el destinatario **debe** ignorarlos [tor-spec § Padding fixed-length cell bodies].

8.3.3 Celdas de longitud variable

Algunas celdas, en particular las utilizadas durante el intercambio inicial del canal, necesitan más de 509 bytes (por ejemplo, para transportar un certificado) o muchos menos. El formato de longitud variable resuelve estos casos (figura 8.2) añadiendo un campo entero `Length` de 2 bytes en orden *big-endian*, sin añadir relleno.

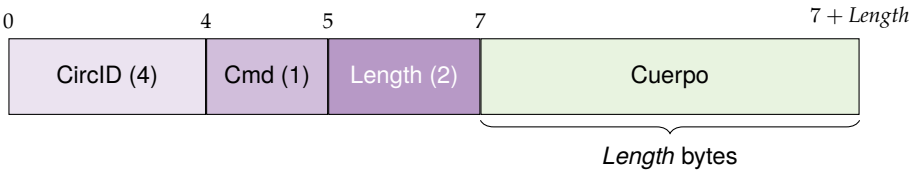


Figura 8.2: Celda de longitud variable (protocolo de enlace ≥ 4): después de CircID y del comando aparece un campo Length de 2 bytes en orden big-endian, que indica el número de bytes del cuerpo. Este puede contener entre 0 y 65 535 bytes.

8.3.4 Cómo distinguir ambos formatos

El receptor necesita saber cuántos bytes debe leer *antes* de haber leído la celda completa. La regla depende exclusivamente del comando [tor-spec § Cells (messages on channels)]:

- el comando 7 (**`VERSIONS`**) identifica una celda de longitud variable;
- todos los demás comandos menores que 128 identifican celdas de longitud fija;
- todos los comandos mayores o iguales a 128 identifican celdas de longitud variable.

En otras palabras, el bit más significativo del byte del comando funciona como indicador de formato, con una única excepción histórica: **`VERSIONS`**.

Nota histórica: La excepción VERSIONS

En el protocolo de enlace 1 todas las celdas tenían longitud fija. La versión 2 introdujo la negociación de versiones, y **`VERSIONS`**, que recibió el valor 7 después de los comandos ya existentes (0–6), se convirtió en la única celda de longitud variable. La versión 3 reservó el intervalo ≥ 128 para comandos variables. Las versiones 1 y 2 están obsoletas y las implementaciones no deberían admitirlas.

8.3.5 El caso especial de VERSIONS

Existe un problema circular: la longitud de CircID depende de la versión negociada, pero esa versión se negocia mediante la propia celda **`VERSIONS`**. La solución es enviar **`VERSIONS`** **siempre** como si $v = 0$, con un CircID de 2 bytes. Su cuerpo consiste en una secuencia de enteros de 2 bytes en orden big-endian, uno por cada versión admitida. Un cliente compatible con las versiones 3, 4 y 5 envía:

Bytes	Campo	Significado
`00 00`	CircID	0, CircID de 2 bytes
`07`	Command	**`VERSIONS`**
`00 06`	Length	cuerpo de 6 bytes
`00 03 00 04 00 05`	Body	versiones 3, 4 y 5

Ambas partes eligen la **versión más alta que aparezca en las dos** celdas **`VERSIONS`**. Si no hay ninguna versión común o el cuerpo contiene una cantidad impar de bytes, deben cerrar la conexión [tor-spec § Negotiating versions with VERSIONS cells].

Versión	Descripción
1	(obsoleta) intercambio inicial basado en certificados
2	(obsoleta) intercambio con renegociación TLS; introduce celdas variables
3	(obsoleta) intercambio «dentro del protocolo», al estilo actual (capítulo 7)
4	CircID de 4 bytes
5	compatibilidad con relleno del enlace y su negociación

En las implementaciones actuales, `tor` ofrece las versiones 3, 4 y 5 (arreglo `or_protocol_versions` en `src/core/or/connection_or.c`), mientras Arti ofrece solo 4 y 5. Dos instancias actualizadas negociarán por tanto la versión 5.

8.4 CircID

El campo `CircID` identifica el circuito al que pertenece una celda. Su valor es 0 para las celdas que no pertenecen a ningún circuito, como **`VERSIONS`**, **`NETINFO`** y **`PADDING`**.

Lo esencial es que los **CircID son locales a cada canal**. Un circuito de tres saltos atraviesa tres canales (cliente–guardia, guardia–intermedio e intermedio–salida) y tiene un CircID *distinto* en cada uno. Cada repetidor mantiene una tabla de reenvío que asocia el par entrante (canal, CircID) con el par saliente (canal, CircID), como muestra la figura 8.3.

Esta elección presenta dos ventajas: los nodos no necesitan coordinar la selección de identificadores, y CircID no se convierte en una etiqueta global con la que relacionar distintos segmentos del mismo circuito.

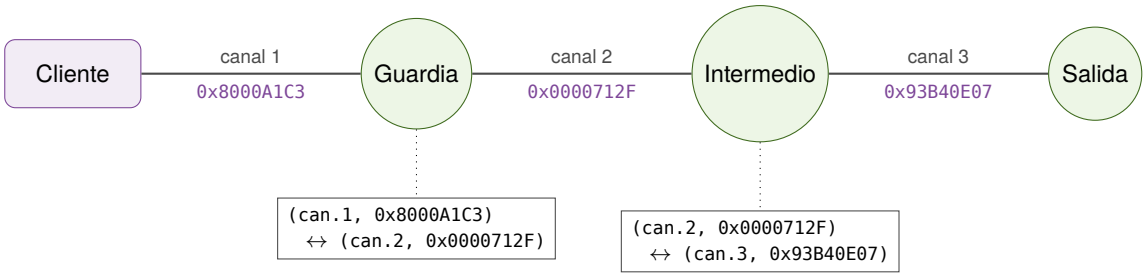


Figura 8.3: El mismo circuito utiliza distintos CircID en cada canal. El guardia y el repetidor intermedio traducen el identificador mediante una tabla de reenvío. Por ello, un observador de un enlace no puede identificar el mismo circuito en otro enlace comparando los CircID. Los valores son ilustrativos, pero cumplen la regla del bit más significativo descrita a continuación: en el canal 2 el bit vale cero, pues el repetidor intermedio abrió la conexión y el guardia la está reutilizando.

8.4.1 ¿Quién elige el CircID?

El CircID lo elige el nodo que envía la celda de creación de circuito (**`CREATE2`**, capítulo 9). Como ambos extremos de un canal pueden crear circuitos, deben evitar elegir el mismo valor a la vez. La regla depende de la versión [tor-spec § Choosing circuit IDs in create cells]:

- **protocolo de enlace** ≥ 4: el nodo que *abrió* la conexión utiliza CircID cuyo bit más significativo (MSB) es 1; el otro extremo utiliza CircID cuyo MSB es 0;
- **protocolo de enlace** ≤ 3: las dos mitades del espacio se asignaban comparando las claves públicas de identidad de los repetidores (el repetidor con la clave menor utilizaba MSB igual a 0).

El valor 0 está reservado; ningún otro valor lo está. Las implementaciones existentes eligen CircID **aleatoriamente** entre los valores disponibles, y la especificación recomienda que las nuevas implementaciones hagan lo mismo para evitar ser distinguibles. `tor` deja de intentar crear un circuito nuevo en un canal tras 64 intentos consecutivos de encontrar un identificador libre.

En profundidad: ¿Por qué 4 bytes?

Con 2 bytes y solo la mitad del espacio disponible, un canal puede transportar poco más de 32 000 circuitos por extremo, y la selección aleatoria provoca colisiones mucho antes de agotar el espacio. La Propuesta 214 (2012) expone el problema de forma directa: «los repetidores se están quedando sin identificadores de circuito» (traducción propia). La propuesta introdujo el protocolo de enlace 4 con CircID de 4 bytes, implementado en tor 0.2.4.11-alpha.

8.5 Comandos de celda

El byte `Command` determina cómo interpretar el cuerpo. Las tablas siguientes enumeran todos los valores definidos [tor-spec § Interpreting the fields: Command]. En la columna C, «sí» significa que el comando requiere un CircID distinto de cero y «no» significa que necesita un CircID igual a cero. La columna P indica la primera versión del protocolo de enlace compatible con el comando (en blanco significa como mínimo versión 3).

Valor	Comando	C	P	Finalidad
0	**PADDING**	no		relleno del enlace
1	**CREATE**	sí		crear circuito (TAP, obsoleto)
2	**CREATED**	sí		respuesta a CREATE (obsoleto)
3	**RELAY**	sí		datos de extremo a extremo
4	**DESTROY**	sí		destruir un circuito
5	**CREATE_FAST**	sí		crear circuito sin criptografía de clave pública
6	**CREATED_FAST**	sí		respuesta a CREATE_FAST
8	**NETINFO**	no		hora y direcciones
9	**RELAY_EARLY**	sí		como RELAY, con cantidad limitada
10	**CREATE2**	sí		crear circuito (intercambio extensible)
11	**CREATED2**	sí		respuesta a CREATE2
12	**PADDING_NEGOTIATE**	sí	5	negociación del relleno

Cuadro 8.1: Comandos de celda de longitud fija.

Valor	Comando	C	Finalidad
7	**VERSIONS**	no	negociación del protocolo de enlace
128	**VPADDING**	no	relleno de longitud variable
129	**CERTS**	no	certificados
130	**AUTH_CHALLENGE**	no	desafío de autenticación
131	**AUTHENTICATE**	no	autenticación del iniciador
132	**AUTHORIZE**	no	reservado

Cuadro 8.2: Comandos de celda de longitud variable.

Comandos desconocidos. No se admiten otros valores. El destinatario de un comando desconocido *puede* descartar silenciosamente la celda y *puede* cerrar el canal. Esta severidad obedece a que cada comando nuevo se introduce con una versión nueva del protocolo de enlace: una implementación correcta jamás enviará un comando que el otro extremo no haya declarado compatible. En cambio, antes de que el protocolo de enlace tuviera versiones, los comandos desconocidos se ignoraban suponiendo que procedían de una versión más reciente de Tor.

Los comandos pueden agruparse según su función:

Administración del canal **VERSIONS**, **CERTS**, **AUTH_CHALLENGE**, **AUTHENTICATE**, **NETINFO**: el intercambio inicial del capítulo 7.

Relleno **PADDING**, **VPADDING**, **PADDING_NEGOTIATE**: tráfico ficticio destinado a dificultar el análisis de tráfico de un enlace.

Ciclo de vida del circuito **CREATE2**/**CREATED2** (y sus variantes históricas), **DESTROY**: creación y destrucción de circuitos (capítulo 9).

Transporte **RELAY** y **RELAY_EARLY**: todo lo que circula *dentro* de un circuito.

8.5.1 DESTROY

El cuerpo de una celda **DESTROY** contiene un único byte que expresa la *razón* para cerrar el circuito [tor-spec § Tearing down circuits]. Cuando un nodo recibe **DESTROY**, libera los recursos del circuito y, si no es uno de los extremos, propaga otra celda **DESTROY** al siguiente nodo.

0	NONE	7	OR_IDENTITY
1	PROTOCOL	8	CHANNEL_CLOSED
2	INTERNAL	9	FINISHED
3	REQUESTED	10	TIMEOUT
4	HIBERNATING	11	DESTROYED
5	RESOURCELIMIT	12	NOSUCHSERVICE
6	CONNECTFAILED		

Advertencia: El código de razón como canal lateral

La especificación recomienda enviar **siempre** la razón NONE y no propagar el código de razón recibido. Si un repetidor reenviara la razón original en ambos sentidos, dos repetidores que colaboraran maliciosamente en un mismo circuito podrían utilizarla como señal para detectar que están en él: un pequeño ejemplo de cómo incluso un byte puede convertirse en un *canal lateral*.

8.5.2 RELAY_EARLY

`RELAY_EARLY` tiene el mismo formato que **`RELAY`**, pero su propósito es **limitar la longitud del circuito** [tor-spec § Handling RELAY_EARLY cells]. Las reglas son:

- un cliente debe enviar los mensajes de extensión de circuito (`EXTEND2`) únicamente dentro de celdas **`RELAY_EARLY`**; desde la versión 0.2.3.11-alpha, los repetidores rechazan extensiones recibidas de otro modo;
- un repetidor que reciba más de 8 celdas **`RELAY_EARLY`** hacia delante en un circuito debería cerrarlo; si recibe incluso una sola celda *hacia atrás*, en dirección al cliente, debe cerrar el circuito inmediatamente;
- para evitar revelar la longitud del circuito, los clientes también deberían enviar como **`RELAY_EARLY`** aproximadamente las primeras 8 celdas que no estén destinadas al primer salto.

Sin este límite, un adversario podría construir circuitos extremadamente largos que atravesaran repetidamente los mismos repetidores, amplificando de esta forma su propio tráfico. Es el ataque de congestión mediante rutas largas descrito por Evans, Dingledine y Grothoff [EDG09].

Nota histórica: Una señal inesperada

La regla que prohíbe las celdas **`RELAY_EARLY`** hacia atrás surgió de un incidente de 2014: algunos repetidores enviaron estas celdas hacia los clientes como canal encubierto para marcar los circuitos de usuarios de servicios onion. Examinaremos el incidente en detalle en el capítulo 27.

8.6 Dentro de las celdas RELAY

Las celdas **`RELAY`** y **`RELAY_EARLY`** transportan **mensajes relay**: los comandos intercambiados entre el cliente y los nodos del circuito, desde la extensión hasta los datos de los flujos TCP. Sus cuerpos de 509 bytes están cifrados en capas, una por salto. Es la «cebolla» del capítulo 10. Desde la perspectiva del canal, el cuerpo de una celda RELAY aparece por tanto completamente ocupado: no hay relleno de ceros visible.

Cuando se han retirado todas las capas de cifrado, el cuerpo en claro tiene el formato tradicional representado en la figura 8.4 [tor-spec § Relay cells].

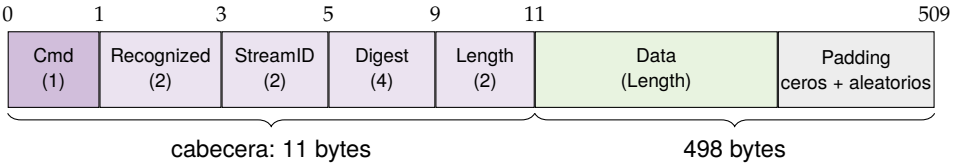


Figura 8.4: Cuerpo *descifrado* de una celda RELAY tradicional: cabecera de 11 bytes, hasta 498 bytes de datos y relleno opcional hasta 509 bytes. La especificación recomienda hasta cuatro bytes iniciales a cero, seguidos de bytes aleatorios; si quedan menos de cuatro bytes, todos son cero, y con 498 bytes de datos no hay relleno.

Comando relay (1 byte) Tipo de mensaje (cuadro 8.3).

Recognized (2 bytes) Vale cero en un mensaje en claro. Permite que un repetidor determine rápidamente si la celda está dirigida a él: si, después de quitar su capa de cifrado, el campo no vale cero, la celda sigue cifrada y debe reenviarse.

StreamID (2 bytes) Identificador del flujo al que se refiere el mensaje, elegido por el cliente; cero en mensajes que afectan al circuito completo.

Digest (4 bytes) Los primeros 4 bytes de un resumen *acumulativo* de todos los mensajes intercambiados con ese salto (SHA-1 para circuitos ordinarios y SHA3-256 para circuitos de servicios onion). Confirma que la celda está destinada a ese repetidor y que no se ha modificado.

Length (2 bytes) Cantidad de bytes significativos del campo Data.

Data Contenido del mensaje.

Padding Espacio restante hasta completar 509 bytes.

Una cabecera de 11 bytes deja un máximo de $509 - 11 = 498$ bytes de datos por celda.

8.6.1 Recognized y Digest: un filtro de dos etapas

¿Por qué hacen falta dos campos? Calcular el resumen de cada celda en tránsito resultaría costoso. Recognized es un filtro barato: después de descifrar una celda que todavía contiene capas de cifrado, sus 2 bytes son seudoaleatorios y solo valen cero con probabilidad 2^{-16}. El repetidor calcula el resumen únicamente cuando Recognized vale cero. Si el resumen coincide, la celda se «reconoce» y procesa; de lo contrario, se reenvía. Analizaremos el cálculo y sus debilidades en el capítulo 10.

8.6.2 Relleno de los mensajes relay

El relleno de los mensajes relay es diferente del relleno de ceros de las celdas de longitud fija. La especificación recomienda 4 bytes cero seguidos de **bytes aleatorios** [tor-spec § Relay cells]. Los bytes aleatorios hacen impredecible el contenido de las celdas. Esto resulta necesario para los mensajes SENDME autenticados de la Propuesta 289, que prueban que los datos se recibieron realmente incluyendo el resumen de las celdas recibidas (capítulo 11). Los destinatarios no deben depender del contenido del relleno.

8.6.3 Comandos relay

La cuadro 8.3 enumera los comandos relay definidos. La columna Dir. indica la dirección: F (*forward*, únicamente desde el cliente hacia el circuito), B (*backward*, únicamente hacia el cliente), o F/B (ambas direcciones). C indica un mensaje de *control*, que debe llevar un StreamID igual a cero.

Valor	Comando	Dir.	Finalidad
Protocolo básico			
1	BEGIN	F	abrir un flujo
2	DATA	F/B	datos del flujo
3	END	F/B	cerrar un flujo
4	CONNECTED	B	flujo abierto correctamente
5	SENDME	F/B, C?	confirmación de recepción (control de flujo)
6–7	EXTEND/EXTENDED	F/B, C	extensión mediante TAP (obsoleta)
8–9	TRUNCATE/TRUNCATED	F/B, C	truncamiento de circuito (sin uso)
10	DROP	F/B, C	relleno de extremo a extremo
11–12	RESOLVE/RESOLVED	F/B	resolución DNS remota
13	BEGIN_DIR	F	flujo hacia una caché de directorio
14–15	EXTEND2/EXTENDED2	F/B, C	extensión de circuito
16–18	*reservados*		UDP sobre Tor (Propuesta 339)
Conflux			
19–22	CONFLUX_...	F/B, C	LINK, LINKED, LINKED_ACK, SWITCH: varios circuitos (Propuesta 329)
Servicios onion			
32	ESTABLISH_INTRO	F, C	establecer un punto de introducción
33	ESTABLISH_RENDEZVOUS	F, C	establecer un punto de encuentro
34–35	INTRODUCE1/INTRODUCE2	F/B, C	solicitud de introducción
36–37	RENDEZVOUS1/RENDEZVOUS2	F/B, C	unir los extremos en el punto de encuentro
38–40	..._ESTABLISHED, INTRODUCE_ACK	B, C	confirmaciones
Relleno de circuito y control de flujo			
41–42	PADDING_NEGOTIATE/...D	F/B, C	relleno de circuito
43–44	XOFF/XON	F/B	control del flujo (Propuesta 324)

Cuadro 8.3: Comandos relay. SENDME es un mensaje de control solamente cuando afecta al circuito y no a un flujo individual (capítulo 11).

Si una celda es reconocida, pero su comando relay es desconocido, debe descartarse; su contenido, sin embargo, cuenta para el resumen y las ventanas de control de flujo.

8.7 Hacia un nuevo formato: propuesta 340 y CGO

El formato tradicional presenta dos limitaciones conocidas. La primera se refiere a la eficiencia: un mensaje grande ocupa al menos una celda completa, mientras que varios mensajes pequeños no pueden compartir una misma celda. La segunda afecta a la seguridad: el resumen de 4 bytes y el cifrado AES-CTR sin autenticación permiten ataques de *marcado* (*tagging*), que veremos en el capítulo 10.

La Propuesta 340 (*Packed and fragmented relay messages*) separa los conceptos de *celda* y *mensaje*: un mensaje puede fragmentarse entre varias celdas y varios mensajes pueden empaquetarse en una sola. La Propuesta 359 (*Counter Galois Onion, Updated*) introduce un

nuevo esquema de cifrado relay, CGO, y adopta el formato de celda de la propuesta 340. Una celda descifrada mediante CGO tiene esta estructura:

Campo	Bytes
marca de autenticación	16
cuerpo	$509 - 16 = 493$
dentro del cuerpo: comando	1
longitud	2
StreamID (solo para ciertos comandos)	2
datos	*longitud*
relleno (4 ceros y después bytes aleatorios)	resto

StreamID solo aparece en comandos que lo necesitan (BEGIN, DATA, END, CONNECTED, RESOLVE, RESOLVED, BEGIN_DIR, XON y XOFF). Desaparecen Recognized y Digest, sustituidos por la marca de autenticación. Un cliente utiliza CGO con un repetidor únicamente si ambos lo admiten (subprotocolo `Relay=6`) y lo negocian durante la creación del circuito.

En profundidad: ¿Cuántos datos caben en una celda CGO?

El formato muestra que un mensaje DATA dispone de $493 - 1 - 2 - 2 = 488$ bytes para datos, diez menos que en el formato tradicional: es el coste de la marca de autenticación de 16 bytes.

CGO ya no es solo una propuesta: `tor` lo admite desde la serie 0.4.9 (introducido en 0.4.9.3-alpha, septiembre de 2025). En Arti, la compatibilidad está siempre habilitada; la opción de compilación `counter-galois-onion` se conserva solo por compatibilidad. El repositorio de especificaciones sigue clasificando ambas propuestas como «Open»: el estado de una propuesta refleja el proceso de estandarización y no siempre coincide con el estado del código.

8.8 Anatomía de una sesión

Ahora podemos reunir las piezas y seguir, celda a celda, qué envía un cliente a su guardia para construir un circuito de tres saltos y descargar una página web. La figura 8.5 resume la secuencia; los capítulos siguientes examinan cada paso con detalle.

1. **Intercambio inicial del canal**: **`VERSIONS`**, seguida de **`CERTS`**, **`AUTH_CHALLENGE`** y **`NETINFO`** por parte del guardia; el cliente responde con **`NETINFO`**. CircID siempre es 0.

2. **Primer salto**: **`CREATE2`** con un nuevo CircID (MSB 1 porque el cliente abrió la conexión); el guardia responde con **`CREATED2`**.

3. **Segundo y tercer salto**: dos celdas **`RELAY_EARLY`** que contienen `EXTEND2`; el guardia y después el intermedio las convierten en celdas **`CREATE2`** dirigidas al siguiente nodo. Las respuestas regresan como celdas **`RELAY`** que contienen `EXTENDED2`.

4. **Flujo**: una celda con `BEGIN` destinada a la salida, que responde con `CONNECTED`; después `DATA` en ambos sentidos, intercalado con `SENDME`.

5. **Cierre**: `END` para el flujo y, al finalizar la vida del circuito, **`DESTROY`**.

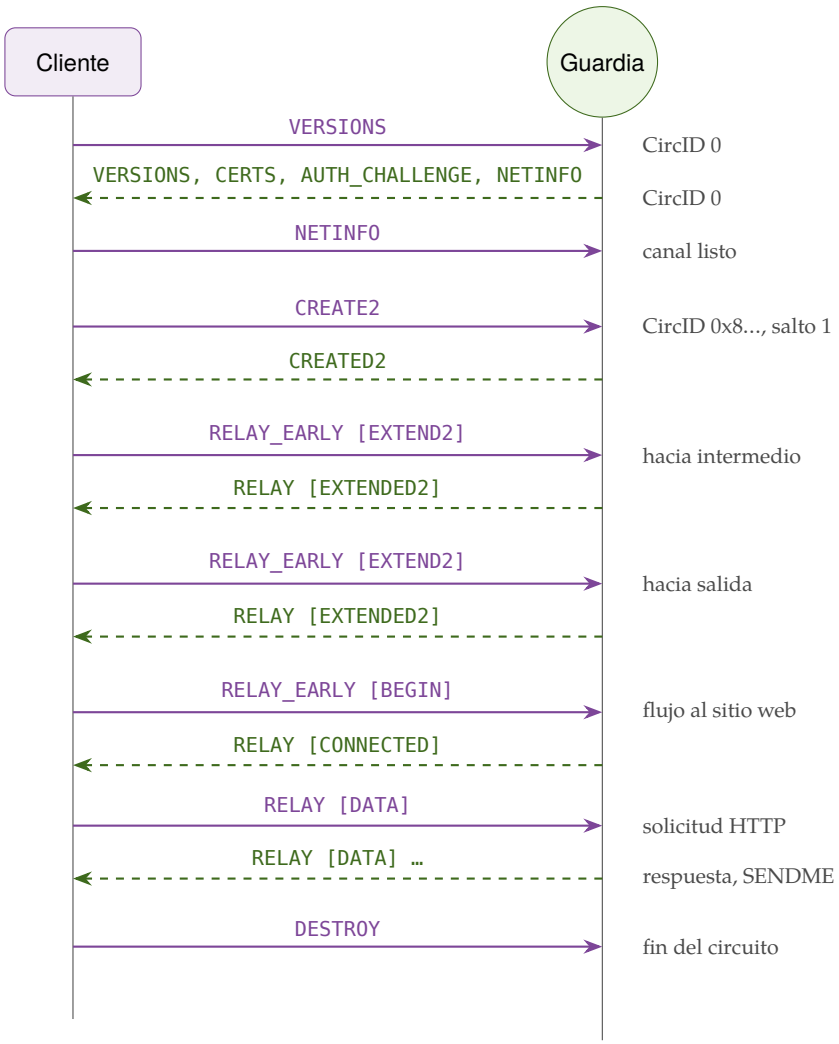


Figura 8.5: Celdas intercambiadas entre un cliente y su guardia por el canal TLS para crear un circuito de tres saltos y abrir un flujo. El guardia solo ve las celdas de su propio canal: no puede leer el contenido de las celdas RELAY destinadas al repetidor intermedio y a la salida, porque están cifradas con claves que no posee.

8.9 Laboratorio: codificador de celdas

Laboratorio 2: Codificar y descodificar celdas

Objetivo. Implementar el módulo `labs/es/codigo/torlab/cells.py`, que se utilizará en los laboratorios posteriores.
Requisitos previos. Python 3.10 o posterior y `pytest`. No se necesita red: este laboratorio opera sobre bytes construidos manualmente.
Tiempo estimado. 1–2 horas.

Paso 1: constantes y comandos. Convertimos las tablas de la especificación en enumeraciones de Python, de modo que cada valor numérico aparezca en un único lugar:

```
CELL_BODY_LEN = 509
RELAY_HEADER_LEN = 11
RELAY_DATA_MAX = CELL_BODY_LEN - RELAY_HEADER_LEN   # 498

class CellCommand(IntEnum):
    PADDING = 0
    CREATE = 1
    # ... todos los valores de las tablas 8.1 y 8.2
    AUTHORIZE = 132

def circid_len(version):
    return 2 if version < 4 else 4

def is_variable_length(command):
    return command == CellCommand.VERSIONS or command >= 128
```

Paso 2: codificación. Esta función recibe CircID, comando, cuerpo y versión del protocolo de enlace y genera los bytes que deben enviarse. Obsérvese el tratamiento especial de **`VERSIONS`** y del relleno de ceros:

```
def encode(cell, version):
    if cell.command == CellCommand.VERSIONS:
        version = 0                 # CircID siempre tiene 2 bytes
    n = circid_len(version)
    header = cell.circ_id.to_bytes(n, "big") + bytes([cell.command])
    if is_variable_length(cell.command):
        return (header + struct.pack(">H", len(cell.body))
                + cell.body)
    return header + cell.body.ljust(CELL_BODY_LEN, b"\x00")
```

Paso 3: descodificación de un flujo. Dado que los bytes llegan fragmentados desde el socket TLS, el descodificador devuelve `None` hasta disponer de una celda completa. Cuando la tiene, devuelve la celda y los bytes no utilizados:

```
def decode(buffer, version):
    n = circid_len(version)
    if len(buffer) < n + 1:
        return None
    circ_id = int.from_bytes(buffer[:n], "big")
    command = buffer[n]
    if is_variable_length(command):
```

```
        if len(buffer) < n + 3:
            return None
        (length,) = struct.unpack(">H", buffer[n+1:n+3])
        start, end = n + 3, n + 3 + length
    else:
        start, end = n + 1, n + 1 + CELL_BODY_LEN
    if len(buffer) < end:
        return None
    return Cell(circ_id, command, buffer[start:end]), buffer[end:]
```

Paso 4: VERSIONS y negociación de versiones.

```
def negotiate_version(ours, received_body):
    if len(received_body) % 2:
        raise CellError("VERSIONS contiene un número impar de bytes")
    theirs = {v for (v,) in struct.iter_unpack(">H", received_body)}
    common = set(ours) & theirs
    if not common:
        raise CellError("no existe una versión de enlace compartida")
    return max(common)
```

Los dos diagnósticos son los de la implementación en `labs/es/codigo/torlab/cells.py`: la longitud del cuerpo **`VERSIONS`** debe ser par y las versiones anunciadas deben tener una intersección no vacía.

Paso 5: mensaje relay. Por último, construimos la cabecera de 11 bytes. La cadena de formato `">BHH4sH"` de `struct` corresponde exactamente a los campos de la figura 8.4:

```
def encode_relay(msg, random_bytes=os.urandom):
    header = struct.pack(">BHH4sH", msg.command, msg.recognized,
                                msg.stream_id, msg.digest, len(msg.data))
    space = CELL_BODY_LEN - RELAY_HEADER_LEN - len(msg.data)
    padding = b"\x00" * min(4, space) + random_bytes(max(0, space - 4))
    return header + msg.data + padding
```

El parámetro `random_bytes` permite que las pruebas reemplacen los bytes aleatorios por valores previsibles.

Verificación. El repositorio contiene las pruebas en `labs/es/codigo/torlab/test/test_cells.py`. Ejecútalas desde la raíz del repositorio con `labs/es/codigo` en `PYTHONPATH`:

```
$ PYTHONPATH=labs/es/codigo python3 -m pytest -q labs/es/codigo/torlab/test/test_cells
    .py
........                                                              [100%]
8 passed
```

(Si se omite el nombre de archivo, `pytest` ejecuta las pruebas de todos los laboratorios que contiene el repositorio.) Por ejemplo, una prueba comprueba que la celda **`VERSIONS`** con versiones 3, 4 y 5 coincide byte a byte con el ejemplo de apartado 8.3: `0000 07 0006 0003 0004 0005`.

Ampliaciones. En el laboratorio 1 (capítulo 7), este módulo se utilizará para comunicarse con un repetidor real de la red privada. En el laboratorio 4 añadiremos el cálculo de resúmenes y el cifrado por capas.

Resumen

- Las celdas son las unidades de transmisión de Tor dentro de un canal. Desde el protocolo de enlace 4, las de longitud fija contienen 514 bytes (CircID 4 + comando 1 + cuerpo 509); las versiones anteriores utilizaban 512.
- Las celdas de longitud variable añaden un campo Length de 2 bytes. **`VERSIONS`** (7) y todos los comandos ≥ 128 tienen longitud variable.
- **`VERSIONS`** siempre usa un CircID de 2 bytes; ambas partes eligen la versión común más alta.
- Los CircID son locales a cada canal. El extremo que abrió la conexión elige valores aleatorios con MSB igual a 1.
- **`DESTROY`** debería transportar siempre la razón NONE para evitar canales laterales; **`RELAY_EARLY`** limita la longitud del circuito.
- El cuerpo descifrado de una celda RELAY incluye una cabecera de 11 bytes (comando, Recognized, StreamID, Digest y Length) y como máximo 498 bytes de datos. Su relleno consta de 4 bytes cero y después bytes aleatorios.
- Las propuestas 340 y 359 (CGO) introducen un nuevo formato con una marca de autenticación de 16 bytes y StreamID opcional.

Ejercicios

Ejercicio 8.1 ⋆ ¿Cuántos bytes ocupa una celda **`CREATE2`** en un canal que ha negociado el protocolo de enlace 3? ¿Y con la versión 5? ¿Cuántos bytes ocupa una celda **`VERSIONS`** cuyo cuerpo sea `00 04 00 05`?

Ejercicio 8.2 ⋆ Indica si una celda es de longitud fija o variable para cada uno de los comandos con valores 0, 7, 9, 12, 129 y 200. ¿Cuál de esos valores no está permitido?

Ejercicio 8.3 ⋆ Un cliente abre un canal con un guardia mediante el protocolo de enlace 5. ¿Cuáles de los siguientes CircID puede utilizar para crear un circuito nuevo: `0x00000000`, `0x12345678` o `0x92345678`? ¿Por qué?

Ejercicio 8.4 ⋆⋆ Descodifica manualmente la siguiente secuencia de bytes recibida en un canal con protocolo de enlace 4. Indica CircID, comando y longitud del cuerpo de cada celda:
`00 00 00 00 81 00 03 aa bb cc 80 00 00 07 04 00 …`
(La segunda celda continúa con el resto del cuerpo, que no se muestra).

Ejercicio 8.5 ⋆⋆ Escribe en hexadecimal los primeros 11 bytes del cuerpo en claro de un mensaje relay DATA del flujo 5, que contiene 300 bytes de datos, antes de calcular el resumen. ¿Cuántos bytes de relleno siguen y cuántos de ellos serán necesariamente cero?

Ejercicio 8.6 ⋆⋆ Un repetidor intermedio procesa 1 000 000 celdas que no están destinadas a él. Por término medio, ¿para cuántas deberá calcular el resumen? ¿Por qué el campo Recognized no basta por sí solo para determinar si una celda está dirigida a ese repetidor?

Ejercicio 8.7 ⋆⋆ Describe un caso concreto en el que dos repetidores que colaboran maliciosamente podrían aprovechar el código de razón de una celda **`DESTROY`** para averiguar que pertenecen al mismo circuito. ¿Por qué enviar siempre NONE evita este caso?

Ejercicio 8.8 ⋆⋆ Un cliente descarga un archivo de 1 MiB (1 048 576 bytes) a través de un circuito. ¿Cuántas celdas DATA hacen falta con el formato tradicional? ¿Cuántas con CGO? En cada caso, ¿qué porcentaje de los datos útiles se añade como sobrecarga debida a las cabeceras de las celdas (protocolo de enlace 4, sin contar TLS)?

Ejercicio 8.9 ⋆⋆⋆ Amplía el laboratorio 2 con una prueba *basada en propiedades* (*property-based testing*), por ejemplo con la biblioteca Hypothesis, que genere celdas aleatorias de longitud fija y variable, las codifique y concatene y divida el resultado en fragmentos de longitudes aleatorias. Comprueba que el descodificador devuelve exactamente las celdas originales.

Ejercicio 8.10 ⋆⋆⋆ Busca las funciones que codifican y descodifican celdas de canal en el código fuente de Arti (crate `tor-cell`). Compara su tratamiento de una «celda incompleta» con la función `decode` del laboratorio 2.

Lecturas complementarias

- *Tor Protocol Specification*, secciones «Cells», «Relay cells» y «Negotiating channels» [Thee]: fuente principal de este capítulo.
- R. Dingledine, N. Mathewson y P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], sección 4: el formato original de las celdas y su justificación.
- N. Evans, R. Dingledine y C. Grothoff, *A Practical Congestion Attack on Tor Using Long Paths* [EDG09]: el ataque que motivó **`RELAY_EARLY`**.
- Propuestas 340 y 359 de torspec: el nuevo formato de celdas.

Capítulo 9

Construcción de circuitos: TAP, CREATE_FAST, ntor y ntor-v3

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; propuestas 216, 332 y 350

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir cómo el cliente construye un circuito salto a salto mediante CREATE2 y EXTEND2;
- explicar las garantías que debe proporcionar un intercambio de creación de circuitos: autenticación del repetidor, confidencialidad y secreto hacia adelante;
- seguir paso a paso los intercambios CREATE_FAST, ntor y ntor-v3, y entender por qué se abandonó TAP;
- derivar las cuatro claves de un salto (Df, Db, Kf y Kb) a partir del material negociado;
- verificar una implementación mediante los vectores de prueba oficiales;
- construir circuitos reales hacia un repetidor de la red de laboratorio (laboratorio 3).

Con un canal abierto (capítulo 7) y el alfabeto de celdas a nuestra disposición (capítulo 8), tenemos todo lo necesario para construir la estructura central de Tor: el **circuito**. Veremos cómo el cliente establece un secreto compartido con cada repetidor de la ruta, desconocido para cualquier otra parte, y cómo convierte ese secreto en claves para cifrar el tráfico. Su utilización en el cifrado por capas será el tema del capítulo 10.

9.1 Un salto cada vez

El cliente construye el circuito *incrementalmente*, o mediante «construcción telescópica» [DMS04]:

1. Elige una ruta, por ejemplo un guardia R_1, un intermedio R_2 y una salida R_3 (capítulo 13).

2. En su canal hacia R_1, envía una celda **`CREATE2`** que contiene la primera mitad de un intercambio criptográfico. R_1 responde con **`CREATED2`**; a partir de ese momento, el cliente y R_1 comparten material de claves.

3. Envía un mensaje `EXTEND2`, cifrado para R_1, con el que le pide extender el circuito hasta R_2. El mensaje contiene la primera mitad de un nuevo intercambio, esta vez destinado a R_2.

4. R_1 abre un canal hacia R_2, o reutiliza uno existente, y le envía una celda **`CREATE2`** con esos datos. La respuesta **`CREATED2`** vuelve a R_1, que la reenvía al cliente dentro de un mensaje `EXTENDED2`.

5. El cliente completa el intercambio con R_2 y repite el proceso para R_3, ahora a través de R_1 y R_2.

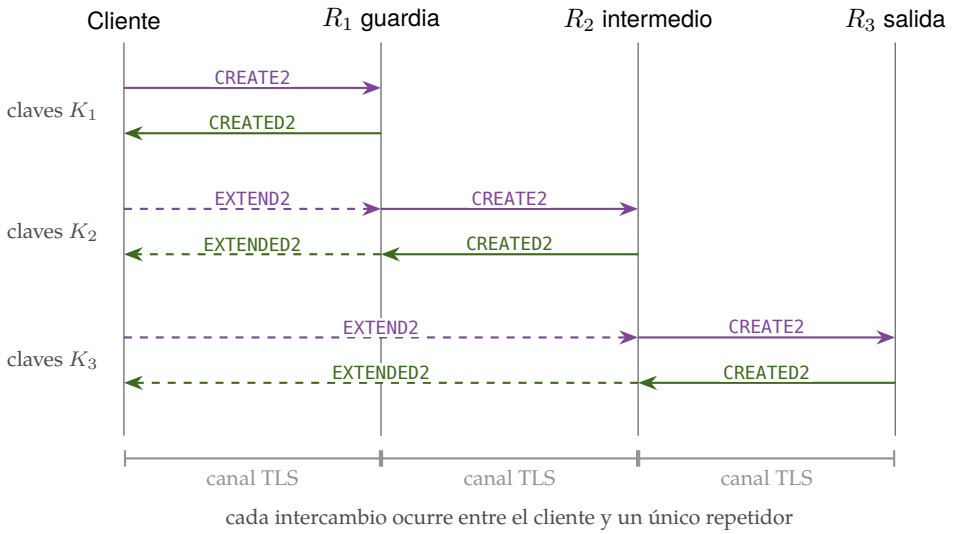


Figura 9.1: Construcción telescópica de un circuito de tres saltos. Cada intercambio se produce entre el cliente y un solo repetidor; los intermedios se limitan a transportar los mensajes. Las flechas discontinuas representan mensajes que circulan dentro del circuito, cifrados con las claves de los saltos ya establecidos.

Esta organización tiene una consecuencia fundamental. El intercambio con R_3 pasa por R_1 y R_2, pero está destinado a la clave pública de R_3: ni R_1 ni R_2 pueden derivar su secreto compartido. Y, como cada salto utiliza un secreto distinto, ningún repetidor puede leer lo que el cliente intercambia con los demás. El guardia R_1 conoce al cliente, pero no el destino del tráfico; la salida R_3 conoce el destino, pero no quién lo envió.

Nota histórica: Encaminamiento cebolla de primera generación

En el sistema original del Naval Research Laboratory, el cliente preparaba una sola «cebolla»: un mensaje cifrado en capas mediante las claves públicas de todos los repetidores. Cada repetidor retiraba una capa para descubrir el siguiente salto y su propia clave de sesión (capítulo 3). Un repetidor comprometido en el futuro podría haber descifrado cebollas previamente registradas. La construcción telescópica de Tor, con un intercambio Diffie–Hellman efímero por salto, introdujo el *secreto hacia adelante*: una vez eliminadas las claves de sesión, ni siquiera la posesión posterior de las claves a largo plazo permite descifrar el tráfico pasado [DMS04].

9.2 Celdas y mensajes

9.2.1 CREATE2 y CREATED2

El formato de **CREATE2** es deliberadamente genérico [tor-spec § Create and Created cells]: un tipo de intercambio (HTYPE, 2 bytes), una longitud (HLEN, 2 bytes) y los datos (HDATA). **CREATED2** solo contiene HLEN y HDATA, porque el tipo ya se conoce. Todo debe caber en una celda de longitud fija, por lo que los datos del intercambio no pueden superar $509 - 4 = 505$ bytes en **CREATE2**.

HTYPE	Intercambio	Estado
`0x0000`	TAP	obsoleto; eliminado en tor 0.4.9.1-alpha
`0x0001`	—	reservado
`0x0002`	ntor	en uso
`0x0003`	ntor-v3	en uso; preferido

Si el intercambio tiene éxito, el repetidor responde con **CREATED2**; en caso de fallo, responde con **DESTROY**. El CircID de la celda nueva obedece las reglas de apartado 8.4: desde el protocolo de enlace 4, quien abrió el canal utiliza valores cuyo bit más significativo es 1.

9.2.2 EXTEND2 y EXTENDED2

El mensaje relay `EXTEND2` (comando relay 14) contiene, además de los campos de **CREATE2**, una lista de *especificadores de enlace* que indican al repetidor cómo llegar al siguiente salto e identificarlo [tor-spec § Extend and Extended messages]:

Tipo	Contenido	Longitud
`00`	dirección IPv4 y ORPort	6
`01`	dirección IPv6 y ORPort	18
`02`	identidad RSA heredada (SHA-1, 20 bytes)	20
`03`	identidad Ed25519	32

El repetidor que extiende el circuito debe verificar *todas* las identidades indicadas y no debe extenderlo si el repetidor alcanzado no demuestra poseerlas durante el intercambio inicial del canal descrito en capítulo 7. Así puede asegurarse el cliente de que su intercambio llega al repetidor previsto, aunque nunca se comunique directamente con R_2. El repetidor también rechaza las solicitudes de extensión hacia sí mismo o hacia el repetidor del que procede la solicitud.

Dos detalles muestran el cuidado que dedica el protocolo a la *indistinguibilidad*. Primero, la especificación recomienda enviar siempre los especificadores en el mismo orden (`00`, `02`, `03`, `01`), para que este no revele la implementación utilizada. Segundo, `EXTEND2` siempre viaja dentro de una celda **RELAY_EARLY** (apartado 8.5); el número limitado de estas celdas impide construir circuitos de longitud arbitraria.

`EXTENDED2` (comando relay 15) tiene exactamente el mismo cuerpo que **CREATED2**.

9.3 Qué debe garantizar un intercambio

Antes de examinar los intercambios uno por uno, fijemos qué esperamos de ellos. Al terminar, el cliente y el repetidor deben compartir un secreto con las siguientes propiedades:

Autenticación del repetidor El cliente debe saber que comparte el secreto con *ese repetidor concreto* y con ningún otro. En cambio, el repetidor no debe conocer información sobre la identidad del cliente: el intercambio se autentica *en una sola dirección*; de lo contrario, cada repetidor conocería la identidad del cliente.

Confidencialidad Un observador de la red, o un repetidor anterior del circuito que reenvía los mensajes, no debe poder derivar el secreto.

Secreto hacia adelante Incluso quien consiga en el futuro las claves de larga duración del repetidor no debe poder reconstruir los secretos de sesiones anteriores.

Eficiencia Un repetidor con mucho tráfico realiza miles de intercambios por segundo, por lo que importa el coste de cálculo.

Cada intercambio que examinaremos establece compromisos distintos entre estos requisitos.

9.4 TAP: el primer intercambio

El intercambio original de Tor, TAP (*Tor Authentication Protocol*), combinaba Diffie–Hellman convencional sobre un grupo modular de 1024 bits con cifrado de clave pública RSA de 1024 bits. El cliente cifraba su mitad de Diffie–Hellman con la clave RSA «onion» del repetidor; solo el repetidor previsto podía descifrarla y responder. A continuación se derivaban las claves mediante KDF-TOR, una función basada en SHA-1 (apartado 9.8).

TAP cumplía los requisitos, pero sus márgenes de seguridad se reducían: 1024 bits resultan insuficientes tanto para RSA como para Diffie–Hellman modular, la construcción de cifrado híbrido se consideraba frágil y los intercambios eran costosos. Desde 2013, TAP convivió con ntor, y con tor 0.4.9.1-alpha los repetidores dejaron de admitirlo por completo (Propuesta 350). Queda un vestigio: por compatibilidad, todavía publican la antigua clave RSA «onion» en sus descriptores. En la red de laboratorio puede observarse en los microdescriptores bajo `onion-key`, junto a la clave `ntor-onion-key` que sí se utiliza.

9.5 CREATE_FAST: cuando no hace falta una clave pública

Existe un caso en el que no es necesario un intercambio de clave pública: el *primer* salto. El cliente acaba de abrir un canal TLS con ese repetidor y de verificar su identidad; por tanto, TLS ya proporciona un canal cifrado y autenticado. **`CREATE_FAST`** aprovecha esta circunstancia [tor-spec § CREATE_FAST/CREATED_FAST cells]:

1. El cliente envía 20 bytes aleatorios X.
2. El repetidor responde con 20 bytes aleatorios Y y 20 bytes KH.
3. Ambos derivan material de claves con KDF-TOR a partir de $K_0 = X \| Y$. Los primeros 20 bytes deben coincidir con KH, lo que demuestra que el repetidor recibió X.

La seguridad de **`CREATE_FAST`** depende por completo del canal TLS: quien pueda leer su tráfico también podrá leer X e Y. Por ello solo debería emplearse en circuitos de un salto. Un caso habitual es el inicio de un cliente que necesita descargar información de directorio desde un repetidor cuya clave onion todavía no conoce. El código de `tor` lo expresa de forma explícita: `should_use_create_fast_for_circuit` (en `src/core/or/circuitbuild.c`) selecciona **`CREATE_FAST`** únicamente cuando el cliente carece de una clave onion utilizable para el primer salto.

Advertencia: CREATE_FAST revela el primer salto

Un repetidor que recibe **`CREATE_FAST`** sabe con certeza que ocupa el primer salto del circuito y, por tanto, que la parte del otro extremo del canal es el cliente. La especificación sugiere que los repetidores aprovechen precisamente esta señal para rechazar circuitos de un único salto que intenten salir hacia Internet.

9.6 ntor

El intercambio ntor, propuesto por Goldberg, Stebila y Ustaoglu [GSU13] e introducido en Tor mediante la Propuesta 216 (tor 0.2.4.8-alpha, 2013), sustituye RSA y Diffie–Hellman modular por **Curve25519**, un intercambio de claves rápido sobre curvas elípticas con claves de solo 32 bytes (capítulo 5).

9.6.1 Los elementos

El repetidor dispone de una clave onion Curve25519 de duración intermedia: la secreta b y la pública $B = b\,G$, publicada en su descriptor como `ntor-onion-key`. El cliente conoce B y la identidad del repetidor (aquí, su huella RSA de 20 bytes, `ID`). Utilizaremos $\mathrm{EXP}(P, k)$ para representar un intercambio Diffie–Hellman Curve25519 entre el punto P y el escalar k, y $H(x, t)$ para el HMAC-SHA256 del mensaje x con la clave t. Las claves t son cadenas constantes derivadas del identificador `"ntor-curve25519-sha256-1"`.

9.6.2 El protocolo

1. El cliente genera un par efímero (x, X) y envía en **`CREATE2`** los elementos `ID` (20 bytes), B (32 bytes, para identificar la clave onion elegida) y X (32 bytes). El total es de 84 bytes.

2. El repetidor comprueba `ID` y B, genera un par efímero (y, Y) y calcula

$$\mathit{secret_input} = \mathrm{EXP}(X, y) \,\|\, \mathrm{EXP}(X, b) \,\|\, \texttt{ID} \,\|\, B \,\|\, X \,\|\, Y \,\|\, \texttt{PROTOID}.$$

   A partir de este valor deriva $\mathit{KEY_SEED} = H(\mathit{secret_input}, t_{key})$ y un valor de confirmación AUTH. Responde con Y (32 bytes) y AUTH (32 bytes).

3. El cliente calcula el mismo *secret_input* utilizando sus secretos, $\mathrm{EXP}(Y, x) \,\|\, \mathrm{EXP}(B, x) \,\|\, \ldots$, y verifica que AUTH coincide.

El núcleo del intercambio son dos operaciones Diffie–Hellman:

- $\mathrm{EXP}(X, y) = \mathrm{EXP}(Y, x)$ combina dos claves **efímeras**, lo que proporciona secreto hacia adelante porque x e y se eliminan inmediatamente después.
- $\mathrm{EXP}(X, b) = \mathrm{EXP}(B, x)$ utiliza la clave onion del repetidor y proporciona **autenticación**: solo quien conozca b puede calcular ese valor y generar el AUTH correcto.

Ninguna de las operaciones basta por sí sola. Sin la primera, quien robara b en el futuro podría reconstruirlo todo; sin la segunda, un intermediario podría hacerse pasar por el repetidor. Mientras tanto, el cliente no utiliza claves a largo plazo: el repetidor solo sabe que comparte el secreto con «quien haya enviado X».

En profundidad: Comprobar el punto en el infinito

La especificación exige que ambas partes verifiquen que ninguna operación Diffie–Hellman produce el «punto en el infinito», representado aquí por un resultado compuesto únicamente por ceros. Un adversario podría enviar como Y un punto de orden pequeño, forzando $\mathrm{EXP}(Y, x)$ a un valor predecible. En Curve25519, esta comprobación sencilla es una alternativa suficiente a verificar exhaustivamente que Y pertenece al grupo correcto. En el laboratorio 3 corresponde a tres líneas de la función `x25519_dh`.

9.7 ntor-v3

ntor presenta una limitación: no transporta nada más que las claves. Con el tiempo surgió la necesidad de *negociar* opciones al establecer un salto, como el control de congestión (capítulo 11) o CGO (capítulo 10). ntor-v3 (Propuesta 332) amplía ntor con **mensajes cifrados** en ambas direcciones e introduce además varias modificaciones criptográficas.

9.7.1 Qué cambia

- La identidad del repetidor pasa a ser su identidad **Ed25519** (32 bytes), en lugar de su huella RSA.
- Las funciones hash pertenecen ahora a la familia SHA-3: SHA3-256 para resúmenes y MAC, y SHAKE-256 para derivación de claves de longitud variable. Cada entrada lleva una etiqueta de dominio codificada junto con su longitud (función ENCAP), evitando confundir un valor calculado para un propósito con otro destinado a un uso diferente.
- Una cadena de verificación `VER`, conocida por las dos partes, interviene en todos los cálculos: un intercambio realizado en un contexto no puede reutilizarse en otro. Para circuitos ordinarios, `tor` utiliza `"circuit extend"` (constante `NTOR3_CIRC_VERIFICATION` en `src/core/crypto/onion_crypto.c`).
- El cliente puede enviar un mensaje CM y el repetidor otro SM, ambos cifrados.

9.7.2 El protocolo

1. El cliente genera (x, X) y calcula inmediatamente $B_x = \mathrm{EXP}(B, x)$. A partir de B_x y los parámetros públicos deriva dos claves de «fase 1»: una para cifrar CM mediante AES-256-CTR y otra para autenticarlo con un MAC. Envía `ID` (32), B (32), X (32), CM cifrado y un MAC (32).
2. El repetidor calcula $\mathrm{EXP}(X, b) = B_x$, deriva las mismas claves de fase 1, verifica el MAC y descifra CM. Después genera (y, Y) y combina las operaciones $\mathrm{EXP}(X, y)$ y $\mathrm{EXP}(X, b)$ en un *key_seed*. Mediante SHAKE-256 deriva una clave para cifrar su respuesta SM y el flujo de claves del circuito. Responde con Y (32), $AUTH$ (32) y SM cifrado.
3. El cliente verifica $AUTH$, descifra SM y deriva el mismo flujo de claves.

Advertencia: El mensaje del cliente no tiene secreto hacia adelante

CM se cifra con claves que dependen únicamente de B_x, derivado mediante la clave onion del repetidor. Alguien que registre el intercambio y obtenga b posteriormente podría descifrar CM. La especificación lo declara explícitamente: solo los datos del repetidor

se benefician del secreto hacia adelante. Por ello, CM debe contener exclusivamente parámetros de negociación, nunca datos del usuario.

9.7.3 Extensiones

Los mensajes CM y SM comparten un formato común [tor-spec § Sending extensions in the circuit extension handshake]: un byte con el número de extensiones y, para cada una, un tipo (1 byte), una longitud (1 byte) y los datos. Las extensiones definidas para crear circuitos son:

Tipo	Dirección	Significado
1	cliente → repetidor	`CC_FIELD_REQUEST`: el cliente solicita control de congestión de la Propuesta 324; cuerpo vacío
2	repetidor → cliente	`CC_FIELD_RESPONSE`: el repetidor acepta; 1 byte, `sendme_inc`
3	cliente → repetidor	solicitud de subprotocolo, por ejemplo `[02 06]` para CGO (`Relay=6`)

9.7.4 ¿Qué intercambio utiliza tor actualmente?

En tor 0.4.9.13, la elección se realiza por separado para cada salto (`circuit_pick_create_handshake`). Se utiliza ntor-v3 cuando el repetidor admite control de congestión y esa función está activada, como suele ocurrir en la red actual, o cuando se desea negociar CGO. De lo contrario, `tor` vuelve a ntor. En la práctica, como observa la especificación, los clientes modernos casi siempre utilizan ntor-v3, y ntor permanece como alternativa de compatibilidad.

9.8 De los secretos negociados a las claves del circuito

Cada intercambio produce un flujo de bytes seudoaleatorios del que se extraen las claves utilizadas para cifrar celdas relay bajo el esquema tradicional (denominado «tor1») [tor-spec § Setting circuit keys]:

Clave	Bytes	Finalidad	Dirección
Df	20	semilla del resumen acumulativo (SHA-1)	cliente → repetidor
Db	20	semilla del resumen acumulativo (SHA-1)	repetidor → cliente
Kf	16	clave AES-128-CTR	cliente → repetidor
Kb	16	clave AES-128-CTR	repetidor → cliente

El total es de 72 bytes (constante `CPATH_KEY_MATERIAL_LEN` del código de `tor`). La forma de producir el flujo depende del intercambio:

KDF-TOR (**`CREATE_FAST`** y anteriormente TAP) $K = \mathrm{SHA1}(K_0 \| \texttt{00}) \,\|\, \mathrm{SHA1}(K_0 \| \texttt{01}) \,\|\, \ldots$. Los primeros 20 bytes forman KH, seguidos por Df, Db, Kf y Kb. El contador de un solo byte limita la salida a 5120 bytes, mucho más de lo necesario.

KDF-RFC5869 (ntor) utiliza HKDF con SHA-256 y `KEY_SEED` como clave; Df, Db, Kf y Kb ocupan los primeros 72 bytes.

SHAKE-256 (ntor-v3) utiliza los primeros 32 bytes para cifrar SM; después aparecen Df, Db, Kf y Kb.

No es un lujo utilizar claves distintas para las dos direcciones. Si cliente y repetidor emplearan la misma clave AES-CTR para ida y vuelta, los flujos cifrados reutilizarían el mismo flujo de clave y el XOR de dos celdas revelaría el XOR de ambos textos en claro.

CGO modifica la distribución de claves; lo examinaremos en el capítulo 10.

9.9 Laboratorio: construir circuitos reales

Laboratorio 3: Intercambios para crear circuitos

Objetivo. Implementar `labs/es/codigo/torlab/ntor.py` con CREATE_FAST, ntor y ntor-v3; verificarlo mediante los vectores de prueba oficiales; construir circuitos de un salto hacia un repetidor de la red de laboratorio.
Requisitos previos. Laboratorios 1 y 2, red de laboratorio activa, biblioteca `cryptography` (X25519, AES) y módulo estándar `hashlib` (SHA-3, SHAKE).
Tiempo estimado. 3–4 horas.

Paso 1: ntor-v3 en pocas líneas. La implementación sigue casi literalmente la especificación. Primero, las funciones auxiliares:

```
NTOR3_PROTOID = b"ntor3-curve25519-sha3_256-1"
def _t(name):  return NTOR3_PROTOID + b":" + name.encode()
def _encap(s): return struct.pack(">Q", len(s)) + s
def _h3(s, t): return hashlib.sha3_256(_encap(t) + s).digest()
def _mac3(k, msg, t):
    return hashlib.sha3_256(_encap(t) + _encap(k) + msg).digest()
def _kdf3(s, t, n): return hashlib.shake_256(_encap(t) + s).digest(n)
```

Después, el lado del cliente:

```
def ntor3_client_start(ID, B, message, ver=b"circuit extend", x=None):
    sk, X = x25519_generate(x)
    Bx = x25519_dh(sk, B)
    phase1 = _kdf3(Bx + ID + X + B + NTOR3_PROTOID + _encap(ver),
                   _t("kdf_phase1"), 64)
    enc_k1, mac_k1 = phase1[:32], phase1[32:]
    encrypted = aes_ctr(enc_k1, message)
    mac = _mac3(mac_k1, ID + B + X + encrypted, _t("msg_mac"))
    state = Ntor3State(sk, X, B, ID, Bx, mac, ver)
    return state, ID + B + X + encrypted + mac

def ntor3_client_finish(st, response, length=72):
    Y, auth, msg = response[:32], response[32:64], response[64:]
    secret_input = (x25519_dh(st.x, Y) + st.Bx + st.ID + st.B + st.X + Y
                    + NTOR3_PROTOID + _encap(st.ver))
    key_seed = _h3(secret_input, _t("key_seed"))
    verify = _h3(secret_input, _t("verify"))
    auth_input = (verify + st.ID + st.B + Y + st.X + st.mac
                  + _encap(msg) + NTOR3_PROTOID + b"Server")
    if not hmac.compare_digest(_h3(auth_input, _t("auth_final")), auth):
        raise HandshakeError("AUTH de ntor-v3 no válido")
    stream = _kdf3(key_seed, _t("kdf_final"), 32 + length)
    return aes_ctr(stream[:32], msg), stream[32:]
```

La comparación de *AUTH* utiliza `hmac.compare_digest`, que tarda lo mismo independientemente de dónde difieran los valores. Una comparación ordinaria podría revelar, a través del tiempo de ejecución, cuántos bytes iniciales son correctos.

Paso 2: vectores de prueba. No basta con verificar un intercambio criptográfico haciéndolo «funcionar»: dos implementaciones que compartan el mismo error podrían funcionar perfectamente entre sí. Utilizaremos en cambio los **vectores de prueba** de la especificación: entradas

fijas, incluidas las claves supuestamente «aleatorias», y salidas esperadas, byte a byte. La Propuesta 332 proporciona un vector completo para ntor-v3, y la Propuesta 216 ofrece varios para la función HKDF de ntor. El archivo `labs/es/codigo/torlab/test/test_ntor.py` los reproduce; por ejemplo:

```
def test_ntor3_vectors_proposal_332():
    st, hdata = ntor3_client_start(h(V["ID"]), h(V["B"]), h(V["CM"]),
                                   ver=h(V["VER"]), x=h(V["x"]))
    assert hdata.hex() == V["client"]
    sm, stream = ntor3_client_finish(st, h(V["server"]), length=256)
    assert sm == h(V["SM"])
    assert stream[:32].hex() == V["keystream_start"]
```

Otras pruebas verifican que alterar *AUTH* o cambiar la cadena `VER` provoque el fallo del intercambio.

Paso 3: claves del repetidor. Para realizar el intercambio necesitamos la identidad Ed25519 del repetidor y su `ntor-onion-key`. Un cliente real las obtiene del consenso y de los microdescriptores (capítulo 12); aquí las tomaremos de esas mismas fuentes, mediante el archivo `cached-microdescs.new` del cliente de laboratorio, en el que cada repetidor tiene una entrada como esta:

```
onion-key
-----BEGIN RSA PUBLIC KEY-----
MIGJAoGBAJet1Oxp+cvg/vgryEmJuVv2s++WxQt5Id1BQ9UNwCOVFcYvEN9KVklM
...
-----END RSA PUBLIC KEY-----
ntor-onion-key gsi2OisnQE96UkjOVvBKtWDLfwRdYG9d4DScg2wz4Qs
id ed25519 Kz1dj/u6/g7WvhHrt9Ds0jVZjKC+KHo26hWViXAJ1Pg
```

Obsérvese la antigua clave TAP, todavía publicada pero sin uso, y las dos líneas base64 que realmente necesitamos.

Paso 4: contra un repetidor real. Abrimos un canal hacia `test003r` mediante el laboratorio 1 y enviamos una celda **`CREATE2`** para cada tipo de intercambio:

```
c = open_channel("127.0.0.1", 5103, expected_identity=ed)
st, hd = ntor3_client_start(ed, B, encode_extensions([]))
c.send(Cell(0x80000001, CellCommand.CREATE2,
              create2_body(HTYPE_NTOR_V3, hd)))
r = c.receive()
sm, stream = ntor3_client_finish(st, hdata_created2(r.body))
keys = CircuitKeys.from_stream(stream)
```

Al repetir la prueba con las distintas variantes, se obtiene:

```
<- CircID=0x80000001 CREATED2 cuerpo=509 bytes
   ntor-v3 correcto, mensaje del repetidor: []
<- CircID=0x80000002 CREATED2 cuerpo=509 bytes
   ntor-v3 con CC_FIELD_REQUEST correcto, respuesta: [(2, b'\x1f')]
<- CircID=0x80000003 CREATED2 cuerpo=509 bytes
   ntor correcto
<- CircID=0x80000004 CREATED_FAST cuerpo=509 bytes
   CREATE_FAST correcto
<- CircID=0x80000005 DESTROY cuerpo=509 bytes     (MAC modificado)
   motivo: 1
```

Todos los intercambios funcionan si no se alteran. En el segundo caso, el repetidor acepta el control de congestión y responde con la extensión 2, con `sendme_inc` igual a 31 (capítulo 11). En el último caso modificamos deliberadamente un bit del MAC: el repetidor rechaza el intercambio con una celda **`DESTROY`** cuyo motivo es 1, `PROTOCOL`. Aquí puede revelarse el motivo real porque el repetidor genera el error; no está reenviando el **`DESTROY`** de otro nodo (apartado 8.5).

Una observación inesperada. En el capítulo 7 vimos que un canal nuevo no recibe celdas **`PADDING`**. Ahora este canal tiene cuatro circuitos abiertos y, sin embargo, durante veinte segundos de espera tampoco llega ninguna celda. La explicación está en el código de `tor` (`src/core/or/relay.c`): un repetidor considera que un canal está «en uso», y por tanto admite relleno, únicamente si transporta un circuito que continúa más allá de ese repetidor. Los circuitos de un solo salto no cuentan. En el capítulo 10, cuando extendamos un circuito, aparecerá el relleno.

Resumen

- Un circuito se construye salto a salto: **`CREATE2`**/**`CREATED2`** con el primer repetidor y `EXTEND2`/`EXTENDED2` a través del circuito para los siguientes. Cada intercambio involucra al cliente y a un único repetidor.
- `EXTEND2` contiene especificadores de enlace (dirección e identidades RSA y Ed25519); el repetidor que extiende debe verificar esas identidades.
- El intercambio de un circuito autentica al repetidor, pero no al cliente, y debe proporcionar confidencialidad y secreto hacia adelante.
- TAP (RSA y DH de 1024 bits) se eliminó en tor 0.4.9.1-alpha. **`CREATE_FAST`** depende de TLS y solo debería emplearse en circuitos de un salto.
- ntor combina dos intercambios Curve25519: efímero–efímero para el secreto hacia adelante y efímero–clave onion para la autenticación.
- ntor-v3 utiliza identidad Ed25519 y la familia SHA-3; añade mensajes cifrados para negociar opciones como el control de congestión y CGO. El mensaje del cliente carece de secreto hacia adelante.
- El flujo de claves aporta Df y Db (20 bytes cada una) y Kf y Kb (16 bytes cada una): una clave y un resumen para cada dirección.

Ejercicios

Ejercicio 9.1 ⋆ Calcula la longitud de HDATA en **`CREATE2`** para ntor y para ntor-v3 con un mensaje de tres bytes. ¿Cuál es el tamaño máximo del mensaje CM de ntor-v3 dentro de una celda **`CREATE2`**?

Ejercicio 9.2 ⋆ ¿Por qué no debe utilizarse **`CREATE_FAST`** para el segundo salto de un circuito? ¿Qué podría hacer el primer repetidor?

Ejercicio 9.3 ⋆ ¿Por qué emplean cliente y repetidor distintas claves AES en ambas direcciones (Kf y Kb)? Explica qué sucedería si utilizaran una sola clave.

Ejercicio 9.4 ⋆⋆ Escribe en hexadecimal la cabecera del cuerpo de un mensaje `EXTEND2` (número de especificadores y especificador IPv4) para extender un circuito hasta el repetidor de laboratorio `test004r`. ¿Cuántos bytes ocuparía el mensaje completo con ntor-v3 y un CM de un byte?

Ejercicio 9.5 ⋆⋆ En ntor, supongamos que el cliente utiliza solo EXP(Y, x) y omite EXP(B, x). Describe un ataque concreto que podría realizar un repetidor intermedio al reenviar el intercambio al siguiente salto. Supón ahora que omite EXP(Y, x): ¿qué propiedad se pierde?

Ejercicio 9.6 ⋆⋆ ¿Qué función cumple la cadena `VER` de ntor-v3? Imagina que otro protocolo distinto de Tor utilizara el mismo intercambio con las mismas claves: ¿qué podría ocurrir sin `VER`?

Ejercicio 9.7 ⋆⋆ Repite el paso 4 del laboratorio con la solicitud de subprotocolo `[02 06]` (CGO), además de `CC_FIELD_REQUEST`. ¿La acepta el repetidor? Según la Propuesta 359, explica por qué CGO requiere también control de congestión.

Ejercicio 9.8 ⋆⋆⋆ El repositorio de `tor` contiene una implementación de referencia de ntor en Python (`src/test/ntor_ref.py`). Utilízala para generar vectores completos del intercambio ntor (cliente y repetidor) e incorpóralos a `test_ntor.py`.

Ejercicio 9.9 ⋆⋆⋆ Implementa el *lado del repetidor* de ntor-v3 (funciones `server_part1` y `server_part2`) y verifícalo mediante los vectores de la Propuesta 332. Después ejecuta un intercambio completo entre el cliente y el repetidor, en memoria.

Lecturas complementarias

- *Tor Protocol Specification*, capítulos «Creating and extending circuits» y «Setting circuit keys» [Thee].
- I. Goldberg, D. Stebila y B. Ustaoglu, *Anonymity and one-way authentication in key exchange protocols* [GSU13]: trabajo que define ntor y establece su seguridad.
- Propuestas 216 (ntor), 332 (ntor-v3) y 350 (retirada de TAP).

Capítulo 10

La cebolla: cifrado de celdas relay y CGO

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; propuesta 359

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir cómo el cliente envuelve un mensaje en una capa de cifrado por cada salto y cómo los repetidores retiran esas capas;
- explicar las funciones de Recognized y del resumen acumulativo, y por qué se necesitan ambos;
- identificar las debilidades del esquema tradicional: maleabilidad y ataques de *marcado* (*tagging*);
- describir el funcionamiento básico de Counter Galois Onion (CGO), el esquema que lo sustituye;
- extender un circuito real a tres saltos con el código propio (laboratorio 4).

En el capítulo 9, el cliente acordó cuatro claves con cada repetidor del circuito: Kf y Kb para cifrar, y Df y Db para verificar la integridad. Ahora vamos a utilizarlas. Este mecanismo da nombre al encaminamiento cebolla: un mensaje se envuelve en tantas capas como repetidores haya en la ruta, y cada repetidor retira una, como al pelar una cebolla.

10.1 El esquema tradicional

El esquema utilizado desde los orígenes de Tor, denominado *tor1* en el código más reciente, combina dos herramientas por salto y dirección:

- un cifrador de flujo, **AES-128 en modo contador** (CTR), con la clave Kf o Kb;
- un **resumen SHA-1 acumulativo**, inicializado con Df o Db.

Ambos mantienen un *estado* durante toda la vida del circuito. El contador de AES-CTR no vuelve a cero por cada celda: la primera utiliza los primeros 509 bytes del flujo de clave, la segunda los 509 siguientes, y así sucesivamente. De igual modo, el resumen no se calcula sobre una sola celda, sino sobre *todas* las intercambiadas hasta ese momento con el mismo salto. Por tanto, el cliente y el repetidor deben procesarlas exactamente en el mismo orden: una celda perdida o duplicada desincronizaría ambos estados. TLS, que garantiza el orden y la integridad del canal, hace que esto sea posible.

10.1.1 Sentido de ida: del cliente al circuito

Para enviar un mensaje al salto d de un circuito, el cliente [tor-spec § Routing relay cells]:

1. prepara el cuerpo en claro del mensaje relay (apartado 8.6), con Recognized y Digest establecidos en cero;
2. actualiza el resumen de ida del salto d con ese cuerpo y escribe sus primeros cuatro bytes en el campo Digest;
3. cifra el cuerpo con Kf del salto d, después con Kf de $d-1$ y así sucesivamente hasta el salto 1: la capa exterior pertenece al primer repetidor.

Cada repetidor que recibe la celda la descifra con su propio Kf, retirando una capa, y comprueba si está destinada a él. Si no lo está, la reenvía al siguiente salto, que retira otra capa. La figura 10.1 muestra el proceso.

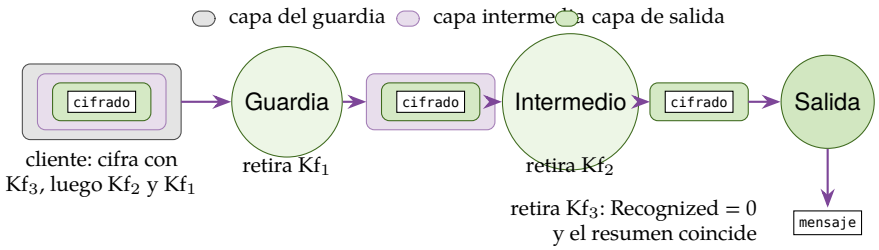


Figura 10.1: Cifrado RELAY tradicional tor1: el cliente aplica Kf_3, Kf_2, Kf_1 y cada repetidor retira su capa. La salida reconoce el mensaje con Recognized $= 0$ y un resumen acumulativo correcto; la aplicación puede seguir cifrada mediante HTTPS. Las cajas internas representan datos todavía cifrados.

10.1.2 Sentido de vuelta: del circuito al cliente

En sentido contrario se invierten las funciones. Un repetidor que origina un mensaje lo firma con su resumen de vuelta y lo cifra con Kb. Cada repetidor posterior en dirección al cliente *añade* otra capa utilizando su propia Kb. El cliente, que conoce todas las claves, retira las capas una a una desde el primer salto y comprueba después de cada capa si ese salto originó la celda.

10.1.3 El circuito como «tubería con salidas intermedias»

El cliente puede enviar un mensaje a *cualquier* salto del circuito, no solo al último, simplemente interrumpiendo el cifrado en ese salto. Así se entrega al segundo repetidor un mensaje `EXTEND2` para extender el circuito hasta el tercero. La especificación denomina esta estructura *leaky pipe topology*: el circuito se parece a una tubería por la que los mensajes pueden salir antes de llegar al extremo. El artículo original sobre Tor [DMS04] también propone aprovechar defensivamente esta propiedad: hacer salir algunos flujos por un repetidor intermedio dificultaría al observador de una salida determinar a qué circuito pertenecen. En la práctica, sin embargo, `tor` dirige los flujos únicamente por el último salto.

10.2 Reconocer una celda

¿Cómo sabe un repetidor, después de retirar su capa, si la celda está destinada a él? No basta con «comprobar que el contenido tiene sentido»: para un programa, tanto una celda

todavía cifrada como una celda en claro son secuencias de bytes. El esquema tradicional realiza dos comprobaciones consecutivas [tor-spec § Relay cells]:

1. **Recognized**: estos dos bytes deben ser cero. Si la celda sigue cifrada, son seudoaleatorios y solo valen cero una vez cada $65\,536$ casos. La comprobación resulta prácticamente gratuita y descarta inmediatamente la inmensa mayoría de celdas en tránsito.

2. **Digest**: cuando Recognized es cero, el repetidor calcula el resumen acumulativo *como si* la celda fuera para él, poniendo el campo Digest a cero, y compara los primeros cuatro bytes con el contenido del campo. Si coinciden, la celda se considera «reconocida»: el repetidor actualiza el estado del resumen y procesa la celda.

Un detalle fácil de pasar por alto: si Recognized vale cero, pero el resumen no coincide, se trata de un falso positivo y **no debe** actualizarse el estado del resumen. En el laboratorio 4 lo conseguimos trabajando con una copia del estado (`self.digest.copy()`) y conservándola solo cuando la comprobación tiene éxito.

El resumen cubre todo el cuerpo, incluidos los bytes de relleno, que por ello son aleatorios (apartado 8.6). En los circuitos de servicios onion, donde el cliente y el servicio comparten las claves de un «salto virtual» (capítulo 14), se utiliza SHA3-256 en lugar de SHA-1.

¿Qué ocurre cuando ningún salto reconoce la celda? Si el último repetidor recibe una celda de ida que no reconoce, se ha producido un error y destruye el circuito con **`DESTROY`**. Si ninguno de los saltos reconoce una celda entrante, el cliente hace lo mismo.

10.3 Debilidades de tor1

10.3.1 Maleabilidad

El modo contador tiene una propiedad problemática: el texto cifrado es simplemente $C = P \oplus S$, donde S es el flujo de clave. Modificar un bit de C modifica ese mismo bit de P sin necesidad de conocer la clave. Si un adversario conoce o puede adivinar parte del texto en claro, puede reemplazarla por un contenido elegido: basta aplicar XOR al texto cifrado con la diferencia entre ambos textos. El laboratorio 4 contiene una prueba que hace exactamente esto, cambiando «paga 10 euros» por «paga 99 euros».

En tor1, la integridad se comprueba solo *en el destino*: el repetidor previsto verifica el resumen, pero los intermedios no. Por ello, una celda alterada puede atravesarlos sin incidentes hasta llegar al destino, donde por fin falla la verificación.

10.3.2 El ataque de marcado

Estas propiedades permiten un ataque concreto denominado *tagging*, o *marcado*. Supongamos que un adversario controla tanto el guardia como la salida de un circuito:

1. El guardia modifica algunos bits de una celda en tránsito según un patrón reconocible (la «marca», o *tag*).

2. La celda atraviesa el intermedio, que no puede detectar la modificación.

3. Antes de comprobar el resumen, la salida busca la marca en la celda. Si la encuentra, sabe que el circuito pasa por su guardia cómplice y puede asociar el destino del tráfico con la dirección del cliente, conocida por el guardia.

Si la salida no coopera, el resumen falla, se destruye el circuito y el cliente construye otro. El adversario puede repetir el intento, y cada fallo aporta información. La marca también puede aplicarse en sentido inverso, desde la salida hacia el cliente.

En profundidad: ¿Hasta qué punto importa el marcado?

Un adversario que controle el guardia y la salida ya puede correlacionar tiempos y volúmenes de tráfico sin ninguna marca (capítulo 26). Por eso los diseñadores de Tor consideraron durante mucho tiempo que el marcado era un problema real, pero de prioridad relativamente menor [DMS04]. Sin embargo, es más fiable que la correlación estadística y funciona incluso con muy poco tráfico. Durante más de una década se propusieron medidas para impedirlo: propuestas 202, 261 (basada en el cifrador AEZ), 295 y 308, seguidas por la 359, que introdujo CGO.

10.3.3 Otras limitaciones

El resumen solo tiene cuatro bytes y su algoritmo subyacente, SHA-1, ya no se considera seguro para aplicaciones nuevas. Además, el esquema carece de secreto hacia adelante *dentro* del circuito: alguien que obtenga las claves de un salto mientras el circuito está activo también puede descifrar todas las celdas anteriores que haya registrado, porque el flujo de clave depende por completo de la clave inicial.

10.4 Counter Galois Onion

CGO (*Counter Galois Onion*) es un esquema de cifrado de celdas relay diseñado por Degabriele, Melloni, Münch y Stam, y adoptado en la Propuesta 359. Aquí presentamos su funcionamiento general; la construcción criptográfica completa se describe en la propuesta y en los artículos de sus autores.

10.4.1 Propiedades

Según la propuesta, CGO introduce tres mejoras:

- Es **no maleable**: cualquier modificación de una celda hace que esta y todas las celdas posteriores del circuito resulten indescifrables. Una marca no puede leerse ni eliminarse: únicamente puede destruir el circuito.
- Proporciona **secreto hacia adelante a nivel de celda**: después de procesar una celda, una parte ya no conserva el material necesario para volver a descifrarla.
- La autenticación ofrece una seguridad de 128 bits, en lugar de depender de la comprobación mucho más corta del esquema tradicional.

Como CGO ya no utiliza SHA-1, también debería ser más rápido en conjunto.

10.4.2 Cómo funciona, a grandes rasgos

Los elementos básicos son AES-128 y POLYVAL, una función hash universal de la misma familia utilizada en el modo AES-GCM-SIV. Con ellos, la propuesta construye un cifrador *ajustable de bloque ancho* (*wide-block tweakable cipher*): opera sobre los 509 bytes completos de una celda como si fueran un solo bloque, de manera que modificar un único bit en cualquier posición altera de forma impredecible todo el resultado. La celda descifrada consta de una *marca de autenticación* (*tag*) de 16 bytes y un cuerpo de 493 bytes, como vimos en apartado 8.7.

Para cada salto y dirección, el cliente y el repetidor comparten:

- una clave K de 64 bytes y un valor N de 16 bytes, derivados del intercambio de creación del circuito;

- la *marca* T' de la última celda procesada, inicialmente cero.

Tres ideas hacen que el esquema resista el marcado:

Reconocimiento mediante marca Cuando el repetidor retira su capa, la celda le pertenece si la marca resultante coincide con N. Se trata de una comprobación de 16 bytes, frente a los dos bytes Recognized y los cuatro bytes Digest.

Encadenamiento La marca de la celda anterior interviene en el procesamiento de la siguiente como parámetro del cifrador. Una celda alterada produce una marca incorrecta que impide descifrar todas las posteriores: el circuito deja de funcionar.

Actualización de claves Cada vez que un salto reconoce u origina una celda, reemplaza K y N por nuevos valores derivados de los anteriores. Los valores antiguos se eliminan, proporcionando secreto hacia adelante a nivel de celda.

El byte de comando de la celda (**`RELAY`** o **`RELAY_EARLY`**) participa en los cálculos como *datos asociados*: un repetidor no puede convertir un comando en el otro sin impedir que la celda se descifre.

Advertencia: Las direcciones no son intercambiables

La construcción subyacente de CGO es no maleable en una dirección y maleable en la otra. La propuesta escoge cuidadosamente qué dirección utilizar para el cliente y cuál para los repetidores, y advierte explícitamente a quienes implementan el protocolo que no deben invertirlas. Esto demuestra cómo una elección que parece un mero detalle de implementación puede comprometer la seguridad de todo un esquema criptográfico.

10.4.3 CGO en la práctica

- **Claves**: del flujo producido por el intercambio, cada dirección recibe K de 64 bytes y N de 16 bytes, en lugar de Df, Db, Kf y Kb.
- **SENDME**: el control de flujo autenticado de la Propuesta 289 utilizaba el resumen acumulativo para demostrar la recepción de los datos. CGO ya no tiene ese resumen, así que SENDME transporta la marca de la última celda confirmada (capítulo 11).
- **Negociación**: el cliente solo utiliza CGO con un repetidor si este anuncia el subprotocolo `Relay=6` y el cliente lo solicita durante el intercambio ntor-v3 (apartado 9.7). En capítulo 9 vimos que el repetidor rechaza la solicitud cuando no va acompañada por otra de control de congestión.
- **Despliegue**: `tor` admite CGO desde la serie 0.4.9 y Arti lo ha tenido siempre activado. Un mismo circuito puede combinar saltos CGO con otros que todavía utilicen tor1: la elección es independiente para cada salto.

10.5 Laboratorio: un circuito de tres saltos

Laboratorio 4: Cifrado por capas y extensión del circuito

Objetivo. Implementar `labs/es/codigo/torlab/encryption.py`, que desarrolla el lado del cliente de tor1, y utilizarlo para extender un circuito real hasta una salida.
Requisitos previos. Laboratorios 1–3; red de laboratorio en funcionamiento.
Tiempo estimado. 3 horas.

Paso 1: una dirección de un salto. En el núcleo del laboratorio hay una pequeña clase que combina un cifrador AES-CTR y el resumen SHA-1 de una dirección:

```
class Direction:
    @classmethod
    def new(cls, key, digest_seed):
        d = hashlib.sha1(); d.update(digest_seed)
        return cls(_aes_ctr(key), d)

    def apply(self, data):             # cifra o descifra: CTR usa la misma operación
        return self.cipher.update(data)

    def sign(self, body):                 # escribe el campo Digest
        self.digest.update(_without_digest(body))
        return body[:5] + self.digest.digest()[:4] + body[9:]

    def recognizes(self, body):
        if body[1:3] != b"\x00\x00":
            return False
        trial = self.digest.copy()
        trial.update(_without_digest(body))
        if trial.digest()[:4] != body[5:9]:
            return False                # falso positivo: no cambia el estado
        self.digest = trial
        return True
```

El objeto de cifrado de la biblioteca `cryptography` conserva el contador entre llamadas; es precisamente el comportamiento que necesitamos.

Paso 2: el cliente. Con un objeto `Direction` por dirección y por salto, cifrar y descifrar se reducen a un bucle:

```
def encrypt(self, msg, destination):
    body = self.hops[destination].forward.sign(encode_relay(msg))
    for i in range(destination, -1, -1):
        body = self.hops[i].forward.apply(body)
    return body

def decrypt(self, body):
    for i, hop in enumerate(self.hops):
        body = hop.backward.apply(body)
        if hop.backward.recognizes(body):
            return i, decode_relay(body)
    raise CellError("celda no reconocida")
```

Paso 3: pruebas en memoria y sus limitaciones. El archivo `labs/es/codigo/torlab/test/test_encryption.py` construye una simulación de circuito con tres repetidores. Comprueba que cada salto reconoce únicamente sus propias celdas, que los repetidores intermedios solo ven bytes seudoaleatorios y que las modificaciones se detectan en el destino; también demuestra la maleabilidad de AES-CTR. Sin embargo, estas pruebas tienen una limitación importante: cliente y repetidores simulados utilizan *el mismo código*. Si, por ejemplo, actualizáramos el resumen en un orden incorrecto, ambas partes cometerían el mismo error y aun así superarían las pruebas. La comprobación definitiva consiste en comunicarse con una implementación independiente.

Paso 4: extender un circuito real. Establecemos el primer salto mediante ntor-v3 hacia `test003r`, como en el laboratorio 3, y lo extendemos hacia `test004r` y la salida `test006e`. El módulo `labs/es/codigo/torlab/laboratory.py` lee por nosotros las identidades y las claves ntor de los repetidores a partir de los archivos de la red de laboratorio:

```
for r, est in ((middle, []), (exit_, [(1, b"")])):
    st, hd = ntor3_client_start(r.id_ed, r.ntor, encode_extensions(est))
    body = extend2_body("127.0.0.1", r.port, r.id_rsa, r.id_ed,
                        HTYPE_NTOR_V3, hd)
    msg = RelayMessage(RelayCommand.EXTEND2, 0, body)
    c.send(Cell(CIRC, CellCommand.RELAY_EARLY, circ.crypto.encrypt(msg)))
    i, response = circ.crypto.decrypt(c.receive().body)
    sm, k = ntor3_client_finish(st, hdata_created2(response.data))
    circ.hops.append(Hop.from_keys(CircuitKeys.from_stream(k)))
```

El mensaje `EXTEND2` tiene StreamID igual a cero (es un mensaje de control) y viaja dentro de una celda **`RELAY_EARLY`**. También solicitamos control de congestión para la salida, igual que `tor`. La ejecución produce:

```
salto 1: 003r
  <- CircID=0x80000100 RELAY cuerpo=509 bytes
  del salto 1 EXTENDED2 67 bytes
salto 2: 004r  extensiones del repetidor: []
  <- CircID=0x80000100 RELAY cuerpo=509 bytes
  del salto 2 EXTENDED2 70 bytes
salto 3: 006e  extensiones del repetidor: [(2, b'\x1f')]
```

El circuito tiene ahora tres saltos. Cada línea «del salto» demuestra que nuestro código cifró correctamente para el repetidor previsto y que este cifró correctamente para nosotros. Una sola clave incorrecta, un byte del resumen diferente o un orden de capas equivocado habría provocado la destrucción del circuito. Las longitudes de `EXTENDED2` concuerdan con apartado 9.7: dos bytes para HLEN, 32 para Y, 32 para $AUTH$ y el mensaje SM, de un byte (sin extensiones) en el intermedio y cuatro bytes (una extensión de un byte) en la salida.

Paso 5: por fin aparece el relleno. Seguimos escuchando el canal después de extender el circuito:

```
  4.1 s  <- CircID=0x00000000 PADDING cuerpo=509 bytes
  7.1 s  <- CircID=0x00000000 PADDING cuerpo=509 bytes
 12.8 s  <- CircID=0x00000000 PADDING cuerpo=509 bytes
 21.5 s  <- CircID=0x00000000 PADDING cuerpo=509 bytes
 28.3 s  <- CircID=0x00000000 PADDING cuerpo=509 bytes
```

Es lo que predijimos en los capítulos 7 y 9: ahora que un circuito de varios saltos atraviesa el canal, el guardia lo considera «en uso» y envía celdas **`PADDING`** a intervalos aleatorios mientras permanece inactivo. Los intervalos observados, de entre tres y 8,7 segundos, entran en la ventana de relleno de 1,5–9,5 segundos de la especificación.

Resumen

- En tor1, cada salto tiene un flujo AES-128-CTR y un resumen SHA-1 acumulativo por dirección, ambos con estado durante la vida del circuito.
- En sentido de ida, el cliente aplica las capas desde el último salto al primero y cada repetidor retira una. En sentido de vuelta, cada repetidor añade una capa y el cliente las retira todas.

- El cliente puede dirigirse a cualquier salto (*leaky pipe*).
- Una celda se reconoce cuando Recognized vale cero y coincide el resumen acumulativo; un falso positivo de Recognized no debe modificar el estado del resumen.
- AES-CTR es maleable y la integridad solo se verifica en el destino: esto permite el marcado.
- CGO impide la maleabilidad, encadena las marcas, actualiza las claves tras cada mensaje y proporciona autenticación de 128 bits; se negocia por separado con cada salto mediante ntor-v3.

Ejercicios

Ejercicio 10.1 ⋆ Un cliente con un circuito de tres saltos quiere enviar un mensaje al segundo. ¿Qué claves utiliza para cifrarlo y en qué orden? ¿Qué ve el tercer salto?

Ejercicio 10.2 ⋆ Después de que el cliente haya enviado 1000 celdas por el circuito, ¿en qué posición del flujo de clave AES-CTR está el primer salto? ¿Por qué no debe reutilizarse la misma clave en un segundo circuito?

Ejercicio 10.3 ⋆⋆ Explica qué ocurriría si un repetidor actualizara su resumen acumulativo incluso cuando Recognized fuera cero pero el resumen no coincidiera. Por término medio, ¿después de cuántas celdas aparecería el problema?

Ejercicio 10.4 ⋆⋆ Un guardia malicioso que coopera con una salida quiere marcar una celda de ida para que esta la reconozca. Dada la estructura de un mensaje `DATA` en claro, ¿qué bytes de la celda sería conveniente modificar? ¿Qué observaría una salida honesta?

Ejercicio 10.5 ⋆⋆ Repite el escenario del ejercicio anterior utilizando CGO. ¿Qué ve la salida cómplice? ¿Qué sucede con las celdas posteriores del circuito?

Ejercicio 10.6 ⋆⋆ En el paso 5 del laboratorio, calcula los intervalos entre las celdas **`PADDING`** observadas. Según la especificación, el retardo se extrae de la distribución del máximo de dos variables aleatorias uniformes. ¿Cuál es el retardo esperado en el intervalo de 1,5 a 9,5 segundos? ¿Concuerda con las observaciones?

Ejercicio 10.7 ⋆ ⋆ ⋆ Amplía el laboratorio enviando un mensaje `DROP` (comando relay 10, relleno de largo alcance) al segundo salto. ¿Responde el repetidor? ¿Cómo puedes verificar que reconoció el mensaje y que no destruyó el circuito?

Ejercicio 10.8 ⋆ ⋆ ⋆ En el laboratorio, invierte deliberadamente un bit de una celda **`RELAY_EARLY`** cifrada que contiene `EXTEND2`. ¿Qué repetidor lo detecta y cómo reacciona? Predice el resultado y luego compruébalo.

Lecturas complementarias

- *Tor Protocol Specification*, capítulos «Relay cells» y «Routing relay cells» [Thee].
- Propuesta 359, *Counter Galois Onion, Updated*, con referencias a los trabajos de Degabriele, Melloni, Münch y Stam.
- Propuestas 202, 261, 295 y 308: larga historia de los intentos de impedir el marcado.

- R. Dingledine, N. Mathewson y P. Syverson, *Tor: The Second-Generation Onion Router* [DMS04], sección 4 (circuitos, tuberías con salidas intermedias y verificación de integridad).

Capítulo 11

Flujos, SENDME y control de congestión

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; propuestas 289 y 324

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- abrir, utilizar y cerrar un flujo TCP a través de un circuito mediante BEGIN, CONNECTED, DATA y END;
- explicar por qué la resolución de nombres se realiza en la salida y cómo funciona RESOLVE;
- describir el control de flujo original de Tor, incluidas las ventanas de circuito y flujo y los mensajes SENDME;
- comprender por qué se autentican los mensajes SENDME y qué demuestran;
- describir el control de congestión de la propuesta 324 y las funciones de XON y XOFF;
- descargar un archivo a través de un circuito propio de tres saltos y medir las ventanas de una salida real (laboratorio 5).

El circuito del capítulo 10 es un conducto cifrado hasta la salida, pero todavía no transporta nada útil. En este capítulo empezaremos a utilizarlo para llevar conexiones TCP hasta Internet, denominadas **flujos** (*streams*). Después examinaremos un problema que todas las redes deben resolver y que adquiere una forma particular en Tor: impedir que un remitente que transmite más deprisa de lo que la red o el destinatario pueden asimilar llene las colas de los repetidores.

11.1 Flujos

Un flujo es una conexión TCP del cliente, por ejemplo hacia un servidor web, transportada por un circuito. Un circuito puede transportar numerosos flujos simultáneamente. Se distinguen mediante el **StreamID** de los mensajes relay (apartado 8.6), elegido por el cliente y necesariamente distinto de cero. La figura 11.1 muestra su ciclo de vida.

11.1.1 Abrir un flujo: BEGIN y CONNECTED

Para abrir un flujo, el cliente envía a la salida un mensaje `BEGIN` (comando relay 1) utilizando un StreamID nuevo. El cuerpo del mensaje indica el destino [tor-spec § Opening streams and transmitting data]:

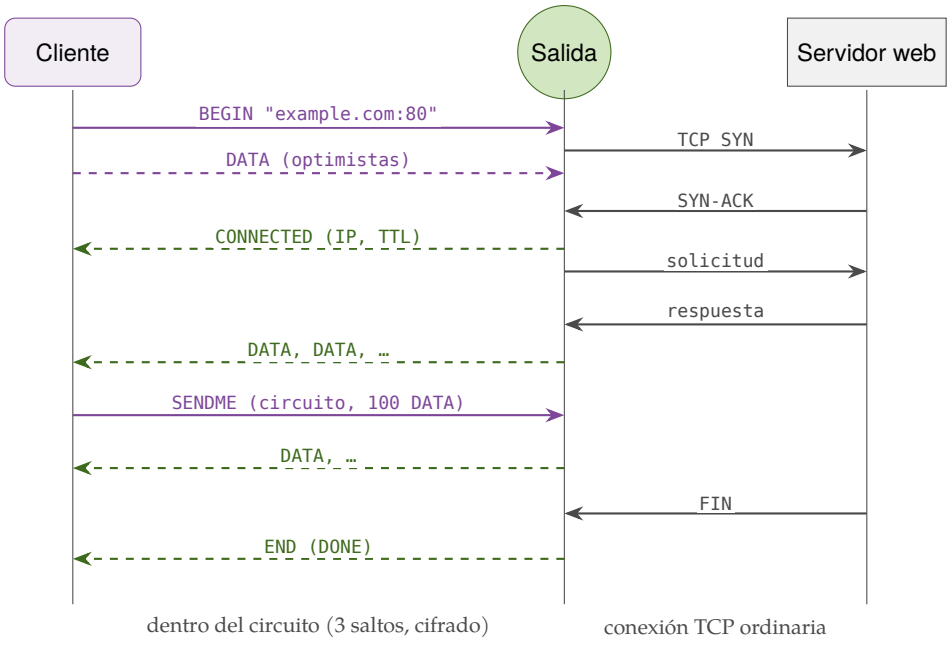


Figura 11.1: Ciclo de vida de un flujo hacia un servidor web. Los mensajes entre cliente y salida viajan dentro del circuito protegidos por cifrado por capas; entre la salida y el servidor existe una conexión TCP ordinaria, sin cifrado si la aplicación no utiliza TLS. El cliente puede enviar datos antes de recibir CONNECTED (*datos optimistas*).

Campo	Longitud	Contenido
ADDRPORT	variable	`dirección:puerto`, terminado con un byte cero
FLAGS	4 (opcional)	preferencias IPv4 e IPv6

La dirección puede ser un nombre DNS, una dirección IPv4 o una IPv6 entre corchetes. Dos reglas pequeñas muestran las prioridades del protocolo: la cadena debe enviarse **en minúsculas**, para que las diferencias de capitalización no permitan distinguir clientes, y FLAGS debe **omitirse** cuando vale cero. Los bits de FLAGS indican si el cliente acepta IPv6 (bit 1), rechaza IPv4 (bit 2) o prefiere IPv6 (bit 3).

Después de recibir `BEGIN`, la salida:

1. comprueba que no es el primer salto del circuito: el canal por el que llegó debe haberlo abierto un repetidor autenticado incluido en el consenso. Tor no debe funcionar como proxy de un solo salto, lo que convertiría las salidas en objetivos todavía más atractivos; se rechaza ese intento con **`DESTROY`**;

2. aplica su *política de salida*, la lista de direcciones y puertos a los que acepta conexiones (capítulo 22);

3. resuelve el nombre si es necesario y abre la conexión TCP.

Si todo va bien, responde con `CONNECTED` (comando relay 4). El cuerpo normalmente contiene la dirección IPv4 alcanzada (4 bytes) y el número de segundos durante los que el cliente puede almacenarla en caché (TTL, 4 bytes). El formato IPv6 es algo diferente y también se permite un cuerpo vacío. El cliente *puede* ignorar la dirección; si la almacena, no debe reutilizarla en otros circuitos. Una salida deshonesta podría mentir o devolver una dirección única para reconocer al cliente posteriormente.

11.1.2 Transferir datos: DATA

Los datos de aplicación viajan en mensajes `DATA` (comando relay 2), cada uno con hasta 498 bytes en el formato tradicional. La salida copia su contenido a la conexión TCP y viceversa. El cliente puede enviar `DATA` inmediatamente después de `BEGIN`, sin esperar `CONNECTED`: la salida pone estos mensajes en cola y los reenvía tan pronto como se abre la conexión. Los *datos optimistas* evitan una ida y vuelta completa por el circuito, que en Tor puede costar fácilmente varios cientos de milisegundos.

11.1.3 Cerrar un flujo: END

La parte que cierra la conexión o encuentra un error envía `END` (comando relay 3) con un byte que representa la razón [tor-spec § Closing streams]. Las más habituales son `DONE` (6, cierre normal), `CONNECTREFUSED` (3), `RESOLVEFAILED` (2), `TIMEOUT` (7) y `EXITPOLICY` (4); esta última va seguida por la dirección rechazada y un TTL. Para los flujos que ha originado, el cliente debería enviar únicamente `MISC` (1): igual que con **`DESTROY`**, no hay motivo para proporcionar al otro extremo información adicional innecesaria.

11.1.4 DNS en la salida

En `BEGIN`, el cliente puede especificar un *nombre* en vez de una dirección: la resolución se realiza en la salida. No es solo una comodidad, sino una decisión de seguridad. Si el cliente resolviera los nombres por sí mismo, consultaría a su propio servidor DNS, fuera de Tor, para conocer la dirección de cada sitio que pretende visitar. Cualquiera que observe esas consultas, empezando por su proveedor de conectividad, sabría hacia dónde se dirige. Es la conocida «filtración DNS» (*DNS leak*), ilustrada en el ejercicio 19.3 y observada desde `tor` en el laboratorio 18 (apartado 28.3).

Cuando el cliente solo necesita una dirección sin abrir una conexión, utiliza `RESOLVE` (comando relay 11), con un nombre terminado en cero. La salida responde mediante `RESOLVED` (12), que contiene una o varias respuestas con tipo, longitud, valor y TTL: tipo `0x04` para IPv4, `0x06` para IPv6, `0x00` para un nombre (en consultas inversas) y `0xF0` y `0xF1` para errores temporales y permanentes [tor-spec § Remote hostname lookup]. Este mecanismo sustenta el puerto DNS que un cliente `tor` puede ofrecer a las aplicaciones locales (opción `DNSPort`).

11.1.5 Flujos de directorio: BEGIN_DIR

`BEGIN_DIR` (comando relay 13) abre un flujo no hacia Internet, sino hacia el servicio de directorio del propio repetidor. Así descargan los clientes el consenso y los descriptores (capítulo 12) sin abrir conexiones sin cifrar. Su cuerpo está vacío y no se aplica la política de salida, porque el flujo nunca abandona el proceso `tor`.

11.2 Control de flujo original

11.2.1 El problema

Imaginemos una salida con una conexión rápida a Internet y un cliente lento. Si la salida leyera el servidor y enviara todo lo recibido al circuito, los datos se acumularían en las colas de los repetidores intermedios, consumirían memoria y ralentizarían otros circuitos. TCP resuelve el problema en cada enlace individual, pero un circuito es una cadena de conexiones TCP diferentes: la contrapresión TCP entre el intermedio y el cliente no llega hasta la salida. Hace falta un mecanismo *de extremo a extremo*.

11.2.2 Ventanas y SENDME

El mecanismo original de Tor utiliza **ventanas**, medidas en celdas `DATA` [tor-spec § Flow control]:

- En el **circuito**, un emisor tiene una *ventana de envío* (*package window*) que comienza en 1000 celdas (o en el parámetro de consenso `circwindow`). Cada celda `DATA` enviada la reduce en una unidad; cuando llega a cero, el envío se detiene. El receptor mantiene una *ventana de entrega* simétrica y, por cada 100 celdas recibidas, envía un mensaje `SENDME` (comando relay 5) con StreamID cero, que devuelve 100 celdas a la ventana del emisor.
- En el **flujo**, las ventanas comienzan en 500 celdas y aumentan en 50 con cada `SENDME` que contiene el StreamID correspondiente.

Como el cliente puede intercambiar datos con cualquier salto (apartado 10.1), mantiene una pareja de ventanas por salto. Las ventanas cuentan solo celdas originadas y consumidas, no las que se limitan a reenviarse.

En consecuencia, en un instante determinado puede haber como máximo 1000 celdas en tránsito por circuito y 500 por flujo: aproximadamente 500 KB y 250 KB. Esto limita el uso de memoria de los repetidores, pero también la velocidad. Un circuito con medio segundo de ida y vuelta no puede superar 1000 celdas cada medio segundo, alrededor de 1 MB/s, independientemente del ancho de banda disponible. Si la red está congestionada, incluso 1000 celdas en tránsito son demasiadas: crecen las colas y la latencia.

11.2.3 Mensajes SENDME autenticados

Un cliente malicioso podría enviar mensajes `SENDME` sin haber recibido nada. La salida continuaría enviando datos que se acumularían en la red, permitiendo al cliente agotar la memoria de los repetidores o amplificar tráfico con muy poco esfuerzo. Desde la versión 0.4.1.1-alpha, los mensajes `SENDME` de circuito se **autentican** (Propuesta 289). Sus cuerpos transportan una versión (1), una longitud (20) y el **resumen acumulativo** de la celda `DATA` que desencadenó el `SENDME`. El destinatario conoce el resumen porque lo calculó al enviar la celda y puede compararlos. Si no coinciden, cierra el circuito.

El resumen depende del contenido de todas las celdas recibidas, pero un cliente podría intentar predecirlo si conoce ese contenido de antemano, por ejemplo al descargar un archivo conocido. Por eso al menos una celda de cada grupo de 100 debe incluir bytes aleatorios: es una de las razones del relleno aleatorio de los mensajes relay (apartado 8.6).

En profundidad: ¿Qué resumen exactamente?

La especificación habla del «resumen de la celda anterior al `SENDME`». En el código de `tor`, se trata del resumen SHA-1 completo, de 20 bytes, del flujo de vuelta después de la última celda reconocida, guardado en el campo `sendme_digest` por `tor1_relay_digest_matches_v0` (`src/core/crypto/relay_crypto_tor1.c`). Con CGO lo sustituye la marca de la última celda (apartado 10.4).

11.3 Control de congestión

Las ventanas fijas presentan dos defectos opuestos: resultan demasiado pequeñas para circuitos rápidos y demasiado grandes para circuitos congestionados. La Propuesta 324, de Mike Perry, introdujo en Tor un verdadero **control de congestión**, análogo al de TCP, adoptado a partir de tor 0.4.7.7 (abril de 2022).

11.3.1 La señal: tiempo de ida y vuelta

¿Cómo reconoce la salida la congestión en un circuito? Utilizando los mensajes `SENDME` como reloj. Registra cuándo envía la celda `DATA` que desencadenará un `SENDME`; cuando llega ese mensaje, calcula el **tiempo de ida y vuelta** (RTT) del circuito. Si el RTT aumenta con respecto al mínimo observado, significa que se están llenando las colas del circuito y el emisor debería reducir su velocidad. El emisor ajusta una *ventana de congestión* (*cwnd*) e intenta mantenerla cerca del producto ancho de banda–retardo del circuito: la cantidad de datos que este puede transportar sin crear colas.

11.3.2 Los algoritmos

La propuesta describe tres algoritmos inspirados en TCP: Tor Westwood, Tor Vegas y Tor NOLA. Tras evaluarlos mediante simulaciones de Shadow (capítulo 19) y en la red real, el algoritmo predeterminado de `tor` es **Vegas** (constante `CC_ALG_DFLT` de `src/core/or/congestion_control_common.c`). Como TCP Vegas, estima cuántas celdas están «en cola» comparando la ventana actual con la necesaria al RTT mínimo observado y luego aumenta o reduce la ventana para mantener la estimación dentro de un umbral.

11.3.3 Los parámetros: ¿por qué 31?

El control de congestión se negocia salto a salto mediante ntor-v3 (apartado 9.7). El cliente lo solicita con la extensión `CC_FIELD_REQUEST`, y el repetidor responde con `CC_FIELD_RESPONSE`, que contiene `sendme_inc`: el número de celdas `DATA` tras el cual solicita un `SENDME`. En capítulo 9, la salida de laboratorio devolvió 31. No es un valor arbitrario: el código de `tor` lo define como el número de celdas que caben en un registro TLS de tamaño máximo:

$$\texttt{TLS_RECORD_MAX_CELLS} = \left\lfloor \frac{16384}{514} \right\rfloor = 31,$$

y la ventana inicial es $4 \times 31 = 124$ celdas. El envío más frecuente de `SENDME` —uno cada 31 celdas en lugar de cada 100— permite medir el RTT con mayor frecuencia y reaccionar más rápidamente.

11.3.4 Adiós a las ventanas de flujo: XON y XOFF

Con el control de congestión desaparecen los mensajes `SENDME` a nivel de flujo. Las ventanas de flujo planteaban un dilema: unas pequeñas limitaban la velocidad de cada descarga; unas grandes permitían que un cliente malicioso obligara a la salida a almacenar una ventana entera de datos por cada uno de sus numerosos flujos. La propuesta las sustituye por dos mensajes:

XOFF (comando relay 43): «deja de enviar en este flujo»; se envía cuando la cola hacia la aplicación supera cierto umbral.

XON (comando relay 44): «reanuda el envío»; contiene un campo de 4 bytes que indica a qué velocidad se vacía la cola, para que el otro extremo se adapte. El campo se llama `kbps_ewma` y la Propuesta 324 no especifica su unidad. El código de `tor` lo calcula e interpreta en unidades de **1000 bytes por segundo**, no de 1000 bits (`src/core/or/congestion_control_flow.c`): el nombre induce a error y leerlo literalmente produce una diferencia de un factor de ocho.

En cambio, los mensajes `SENDME` de circuito se envían tan pronto como se descifran y reconocen las celdas, antes de entregarlas a la aplicación. Miden el comportamiento de la red, no el de la aplicación.

Advertencia: Control de congestión y anonimato

Cualquier mecanismo que responda al tráfico puede convertirse en un canal lateral. Un adversario que observe el comportamiento de la ventana podría inferir información sobre el circuito. La Propuesta 324 dedica una sección a estos riesgos y a sus medidas de mitigación, como limitar la frecuencia de XON y XOFF. Es la misma clase de precaución que, en el capítulo 8, lleva a enviar siempre la razón NONE en una celda **`DESTROY`**: cualquier variación observable que dependa de un estado interno filtra información. Los ataques que aprovechan señales derivadas del tráfico, y que esta precaución pretende no facilitar, se estudian en el capítulo 26.

11.4 Laboratorio: descargar un archivo mediante Tor

Laboratorio 5: Flujos y control de flujo

Objetivo. Implementar `labs/es/codigo/torlab/circuit.py`, que reúna los laboratorios anteriores en una clase `Circuit`, y `labs/es/codigo/torlab/stream.py`, que abra flujos y transfiera datos. Descargar un archivo de un servidor web local a través de un circuito de tres saltos y medir las ventanas de la salida.
Requisitos previos. Laboratorios 1–4, red de laboratorio en ejecución y servidor web local (`python3 -m http.server 8080`).
Tiempo estimado. 3 horas.

Paso 1: un circuito en una línea. La función `build` abre el canal con el guardia, crea el primer salto y extiende el circuito, solicitando control de congestión únicamente en el último salto, como hace `tor`:

```
circ = build([relay("003r"), relay("004r"), relay("006e")])
print(circ.exit_extensions)            # [(2, b'\x1f')]: sendme_inc = 31
```

Paso 2: abrir el flujo.

```
def begin_body(address, port, flag=0):
    body = f"{address.lower()}:{port}".encode() + b"\x00"
    return body + (struct.pack(">I", flag) if flag else b"")

s = Stream(circ, 1)
s.open("127.0.0.1", 8080)       # BEGIN, esperar CONNECTED o END
```

La salida responde con `CONNECTED` y un cuerpo de 8 bytes, `7f000001 00000708`: dirección 127.0.0.1 y TTL `0x708` = 1800 segundos.

Paso 3: mensajes SENDME. El circuito cuenta cada celda `DATA` recibida de la salida y envía un `SENDME` autenticado cada `sendme_inc` celdas:

```
def count_received_data(self):
    est = dict(self.exit_extensions)
    sendme_inc = est[2][0] if 2 in est else 100
```

```
    self.data_to_ack += 1
    if self.data_to_ack == sendme_inc:
        self.data_to_ack = 0
        digest = self.crypto.hops[-1].backward.digest.digest()
        body = struct.pack(">BH", 1, 20) + digest
        self.send(RelayMessage(RelayCommand.SENDME, 0, body))
```

Advertencia: Un error instructivo

En la primera versión de este laboratorio, el contador era un atributo del *flujo*. El primer flujo, una página de dos celdas, funcionó; el segundo, un archivo de 1 MB, fue interrumpido por la salida mediante **DESTROY**. La razón: el SENDME de circuito cuenta las celdas DATA de *todos* los flujos del circuito. La salida ya había contado dos celdas del primer flujo y esperaba SENDME después de la celda 29 del segundo. Nosotros lo enviamos después de la celda 31, con un resumen distinto del esperado, y la salida hizo exactamente lo que exige la Propuesta 289: cerró el circuito. Trasladar el contador a la clase `Circuit` resolvió el problema.

Paso 4: la descarga. Enviamos una solicitud HTTP y leemos hasta END:

```
s.send(b"GET /grande.bin HTTP/1.0\r\nHost: 127.0.0.1\r\n\r\n")
response = s.receive_all()
```

Con un archivo aleatorio de $1\,048\,576$ bytes, estos son los datos históricos del laboratorio de referencia, con las etiquetas de diagnóstico **traducidas editorialmente** (no representan una ejecución de esta edición):

```
flujo 1: 209 bytes en 0.05 s, 2 celdas DATA, 0 SENDME de circuito,
         otros ['END DONE']
flujo 2: 1048781 bytes en 0.18 s, 2120 celdas DATA, 68 SENDME de circuito, otros ['END
     DONE']
```

La huella SHA-256 del archivo recibido coincide con la del original. Los 68 mensajes SENDME corresponden exactamente a $\lfloor (2 + 2120)/31 \rfloor$: un contador por circuito, como aprendimos. Se reciben 2120 celdas en lugar del mínimo $\lceil 1048781/498 \rceil = 2106$: la salida no espera necesariamente a acumular 498 bytes antes de enviar cada celda, sino que agrupa los datos tal como los lee del servidor.

Paso 5: observar las ventanas. Realizamos ahora un experimento que un cliente corriente no puede ejecutar: descargar el mismo archivo *sin enviar nunca* SENDME y contar cuántas celdas transmite la salida antes de detenerse. Repetimos con y sin control de congestión. En una de nuestras ejecuciones:

```
circ = build(path, cc=cc)
circ.count_received_data = lambda: None      # no enviar SENDME
```

```
control de congestión=True:  la salida se detiene tras 124 celdas DATA
control de congestión=False: la salida se detiene tras 500 celdas DATA
```

Con control de congestión, la salida se detiene al llegar a 124 celdas: la ventana de congestión inicial, 4×31. Sin él, se detiene después de 500 celdas y no después de 1000: se agota primero la ventana del *flujo* (500), antes que la del circuito (1000), porque el control de flujo original requiere además mensajes SENDME del flujo. En ambos casos, la salida simplemente espera: los datos permanecen en su propio búfer, no en la red.

Resumen

- Un flujo se abre con `BEGIN` (`dirección:puerto`, en minúsculas), se confirma con `CONNECTED`, transporta `DATA` y se cierra con `END`. El cliente puede enviar datos antes de `CONNECTED`.
- Los nombres se resuelven en la salida para evitar revelar destinos al servidor DNS local; `RESOLVE` y `RESOLVED` permiten resolver nombres sin abrir conexiones.
- El control de flujo original emplea ventanas de 1000 celdas por circuito y 500 por flujo, con `SENDME` cada 100 y 50 celdas, respectivamente.
- Los `SENDME` de circuito se autentican mediante el resumen acumulativo de la celda que provoca la confirmación (Propuesta 289).
- Con control de congestión (Propuesta 324, algoritmo Vegas predeterminado), la ventana se adapta al RTT medido a través de `SENDME`; `sendme_inc` vale 31, el número de celdas que caben en un registro TLS, y la ventana inicial es 124.
- XON y XOFF sustituyen las ventanas por flujo.

Ejercicios

Ejercicio 11.1 ⋆ Escribe en hexadecimal el cuerpo de un mensaje `BEGIN` dirigido a `Example.COM`, puerto 443, para un cliente que también acepta IPv6.

Ejercicio 11.2 ⋆ Interpreta el cuerpo `c0000201 0000012c` de un mensaje `CONNECTED`. ¿Qué debería hacer el cliente con esa información?

Ejercicio 11.3 ⋆ Estima el tiempo que ahorran los *datos optimistas* en un circuito con un RTT de 600 ms y una página que necesita una sola solicitud HTTP. ¿Por qué debe la salida descartar los datos en cola si falla la conexión?

Ejercicio 11.4 ⋆⋆ Con el control de flujo original, ¿cuál es la velocidad máxima de un solo flujo en un circuito con RTT de 400 ms? ¿Y en un circuito con varios flujos? Compara las cifras con la ventana inicial del control de congestión.

Ejercicio 11.5 ⋆⋆ Describe un ataque de agotamiento de memoria contra una salida basado en mensajes `SENDME` sin autenticar. ¿Por qué es necesario el relleno aleatorio de las celdas `DATA` para que funcione la autenticación?

Ejercicio 11.6 ⋆⋆ Amplía `stream.py` con una función `resolve(name)` que utilice `RESOLVE` e interprete `RESOLVED`. Pruébala en la red de laboratorio con `localhost`. ¿Cómo responde la salida a un nombre inexistente?

Ejercicio 11.7 ⋆⋆⋆ Intenta abrir un flujo hacia un puerto que no acepte la salida de laboratorio, después de modificar su política de salida (`torrc` de `test006e` y luego `chutney hup`). ¿Qué razón indica `END`? Interpreta los campos adicionales.

Ejercicio 11.8 ⋆⋆⋆ Modifica el laboratorio para medir, desde el cliente, el tiempo entre el envío de cada `SENDME` y la recepción de la siguiente celda `DATA`. ¿Qué estás midiendo? Compara los valores con y sin una segunda descarga ejecutada simultáneamente en el mismo circuito.

Lecturas complementarias

- *Tor Protocol Specification*, secciones «Opening streams and transmitting data», «Closing streams», «Remote hostname lookup» y «Flow control» [Thee].
- Propuesta 289, *Authenticating sendme cells to mitigate bandwidth attacks*.
- Propuesta 324, *RTT-based Congestion Control for Tor*: motivación, algoritmos, parámetros y análisis de seguridad.

Capítulo 12

El sistema de directorios

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`*; tor 0.4.9.13; método de consenso 36*

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar por qué Tor necesita un sistema de directorios y qué riesgos debe evitar;
- distinguir sus documentos: descriptores, microdescriptores, votos, consensos y certificados de autoridades;
- describir cómo votan las autoridades de directorio cada hora y combinan sus votos;
- entender los criterios empleados para asignar indicadores;
- leer un consenso línea por línea y verificar sus firmas;
- descargar y verificar el consenso de la red de laboratorio utilizando código propio (laboratorio 6).

En los laboratorios anteriores siempre sabíamos de antemano a qué repetidores conectarnos, qué claves utilizaban y en qué puertos escuchaban: obteníamos esa información de los archivos de la red privada. Un cliente real no goza de esa ventaja. Debe descubrir qué repetidores existen, dónde se encuentran, qué claves utilizan y qué confianza merecen. Esta es la función del **sistema de directorios**, objeto del presente capítulo.

12.1 El problema

Un sistema de directorios para Tor debe satisfacer varios requisitos que compiten parcialmente entre sí:

Corrección La información sobre los repetidores (direcciones y claves) debe ser auténtica. Un adversario que convenciera a un cliente de que un repetidor concreto tiene otra clave podría interceptar su tráfico.

Visión común Todos los clientes deben ver *la misma* lista de repetidores. Si un adversario pudiera proporcionar una lista distinta a cada cliente, quizás con solo unos pocos repetidores bajo su control, podría reconocer a los clientes por los repetidores que utilizan. Esto es un ataque de *partición* o *epistémico*.

Resistencia a abusos Cualquiera puede añadir repetidores a la red. El sistema debe limitar la influencia de alguien que añada numerosos nodos (ataque *Sybil*) y poder excluir a los que se comporten indebidamente.

Escalabilidad Millones de clientes descargan información de directorio varias veces al día. La distribución debe consumir pocos recursos.

La solución de Tor, desarrollada a lo largo de tres versiones del protocolo de directorios, es un conjunto reducido de servidores de confianza: las **autoridades de directorio**. Cada hora, votan y firman conjuntamente un único documento: el **consenso** [dir-spec § Outline].

Nota histórica: Tres generaciones

En la primera versión del protocolo, cada autoridad publicaba su propia lista de repetidores, por lo que una autoridad deshonesta podía engañar a los clientes. En la segunda, los clientes descargaban las listas de todas las autoridades y combinaban ellos mismos las distintas perspectivas, multiplicando el consumo de ancho de banda. La tercera versión, utilizada desde hace más de quince años, delega esa combinación en las autoridades: producen un documento único, firmado por todas, que los clientes solo necesitan verificar. A partir de ese momento, las autoridades también dejaron de firmar con la misma clave que utilizaban como repetidores y que debían mantener en texto claro en la memoria.

12.2 Los participantes

Autoridad de directorio Servidor administrado por una persona de confianza de la comunidad Tor. La red pública cuenta con nueve, más una autoridad independiente para puentes (capítulo 16). Su lista está incorporada a `tor` (`src/app/config/auth_dirs.inc`), de manera que todos los clientes parten del mismo conjunto. Cada autoridad posee una clave de *identidad* de larga duración, mantenida fuera de línea, que certifica una clave de *firma* válida durante 3 a 12 meses. Esta última firma los votos y los consensos.

Caché de directorio Repetidor que descarga consensos y descriptores desde las autoridades y los redistribuye a los clientes, reduciendo la carga de las primeras. Posee el indicador `V2Dir` y sirve flujos `BEGIN_DIR` (capítulo 11).

Directorio de reserva Lista de aproximadamente doscientos repetidores estables, también incorporada a `tor` (`fallback_dirs.inc`, 200 entradas en tor 0.4.9.13). Un cliente recién instalado solicita a uno de ellos su primer consenso, en lugar de sobrecargar las autoridades.

Repetidor Publica periódicamente sus propios descriptores ante las autoridades.

Cliente Descarga el consenso y los microdescriptores, los verifica y los utiliza para seleccionar rutas (capítulo 13).

La figura 12.1 muestra cómo circulan estos documentos.

12.3 Los documentos

Todos los documentos de directorio son texto en el formato que la especificación denomina *netdoc*: una secuencia de líneas (*elementos*), cada una con una palabra clave y argumentos, a veces seguida de un objeto codificado en base64 entre líneas `-----BEGIN ...-----` y `-----END ...-----`. Es un formato sencillo de leer por personas y de analizar por programas, como veremos en el laboratorio.

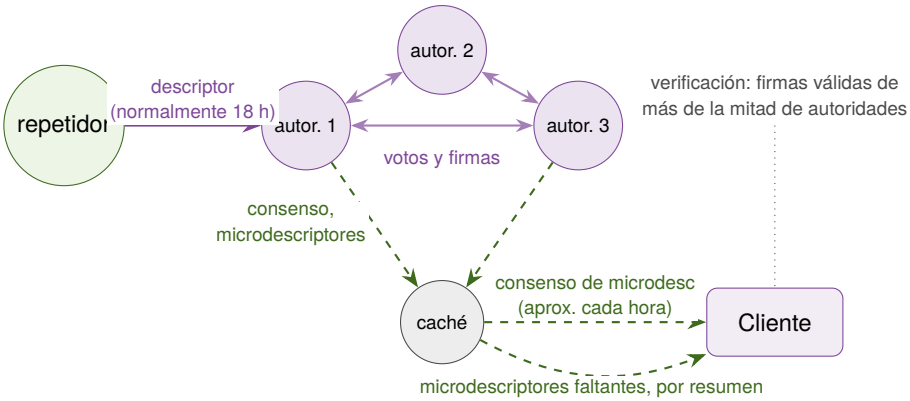


Figura 12.1: Flujo de documentos de directorio. Los repetidores publican descriptores ante las autoridades; estas intercambian votos, calculan el consenso a partir de los votos disponibles e intercambian firmas; las cachés descargan consensos y microdescriptores y los sirven a los clientes.

Descriptor de repetidor (*server descriptor*) Documento creado y firmado por el repetidor, con direcciones y puertos, claves (identidad, clave onion ntor, certificados Ed25519), ancho de banda anunciado, política de salida completa, familia, plataforma y versión. Se vuelve a publicar al menos cada 18 horas (`FORCE_REGENERATE_DESCRIPTOR_INTERVAL` en el código de `tor`), o antes si cambia algún elemento importante.

Documento extra-info Estadísticas que no son necesarias para el funcionamiento del protocolo, como el tráfico de puentes desglosado por país, publicadas por separado para evitar que crezcan demasiado los descriptores. Alimentan Tor Metrics (capítulo 24).

Microdescriptor Subconjunto de un descriptor que el cliente realmente necesita y que cambia poco: clave onion ntor, familia, resumen de política de salida e identidad Ed25519. Lo calculan las autoridades; cada microdescriptor se identifica por el resumen SHA-256 de su texto.

Voto Evaluación de una autoridad sobre el estado de la red: indicadores, ancho de banda y versión de cada repetidor.

Consenso Resultado de combinar los votos, firmado por las autoridades. Tiene dos *variantes*: `ns`, con el resumen del descriptor completo para cada repetidor, y `microdesc`, con el resumen del microdescriptor. Los clientes modernos solo utilizan la segunda, más compacta.

Certificado de clave de autoridad Vincula la clave de firma de una autoridad con su identidad.

En profundidad: ¿Por qué microdescriptores?

Un descriptor completo ocupa varios kilobytes y cambia con frecuencia, por ejemplo cuando varía el ancho de banda anunciado. Descargar todos los descriptores de miles de repetidores varias veces al día consumiría mucho ancho de banda de los clientes, especialmente con conexiones móviles. Un microdescriptor contiene únicamente información que cambia poco. La información que varía a menudo, como el ancho de banda y los indicadores, se incorpora al consenso que el cliente descargará de todas formas. Como el cliente solicita microdescriptores por su resumen y no por la identidad del repetidor, una caché de directorio no puede entregar a un cliente un microdescriptor distinto del que

reciben todos los demás.

12.4 Votación

12.4.1 El calendario

Cada consenso contiene tres marcas temporales [dir-spec § Voting timeline]:

`valid-after` instante a partir del cual se considera válido;

`fresh-until` instante en que deja de ser el consenso más reciente; normalmente coincide con `valid-after` del siguiente;

`valid-until` instante a partir del cual el cliente ya no puede utilizarlo.

En la red pública, el intervalo de votación es de una hora y cada consenso permanece válido durante tres intervalos (opciones `V3AuthVotingInterval` y `V3AuthNIntervalsValid`). Así, en un instante dado pueden coexistir varios consensos válidos y un retraso de las autoridades no tiene por qué bloquear a los clientes. Antes de cada intervalo, las autoridades disponen de cinco minutos para intercambiar votos y otros cinco para intercambiar firmas (`V3AuthVoteDelay` y `V3AuthDistDelay`). En la red de laboratorio todo se acelera: un voto cada 20 segundos y 4 segundos para cada fase (apartado 19.5).

12.4.2 Combinar los votos

Cada autoridad recoge los votos de las demás y calcula el consenso mediante un algoritmo determinista, para que todas obtengan el mismo documento y puedan firmarlo [dir-spec § Computing a consensus from a set of votes]. Las reglas principales son:

- un repetidor aparece en el consenso solo si lo conocen suficientes autoridades y, según los métodos actuales (24 y posteriores; el laboratorio usa 36), solo si posee los indicadores `Running` y `Valid`;
- un repetidor recibe un indicador cuando más de la mitad de las autoridades que votan sobre ese indicador se lo asignan;
- el ancho de banda (`w`) es la mediana inferior de los valores votados, salvo que al menos tres autoridades hayan *medido* su ancho de banda, caso en que se utiliza la mediana de esas mediciones;
- los parámetros de red (`params`) se incluyen si vota por ellos una mayoría del total de autoridades o al menos tres (`i > total_authorities/2 || i >= 3` en el código); el valor elegido es la mediana inferior;
- las marcas temporales son las medianas de los tiempos propuestos.

Como las autoridades pueden ejecutar versiones distintas de `tor`, el documento declara el *método de consenso* empleado: un número que identifica el conjunto exacto de reglas, elegido como el método más alto admitido por suficientes autoridades. El consenso de laboratorio utiliza el método 36.

12.4.3 Indicadores

Los indicadores resumen la valoración de cada autoridad sobre un repetidor [dir-spec § Assigning flags in a vote]. La especificación advierte que los criterios exactos pueden cambiar y los clientes no deben depender de ellos. La cuadro 12.1 resume los criterios actuales.

Indicador	Criterio (resumen)
`Running`	la autoridad alcanzó al repetidor en todos sus ORPorts durante los últimos 45 minutos
`Valid`	versión de tor sin defectos conocidos; repetidor no excluido por las autoridades
`Fast`	ancho de banda entre los siete octavos más rápidos, o al menos 100 KB/s
`Stable`	tiempo medio entre fallos ponderado por duración igual o superior a la mediana, o al menos 7 días
`Guard`	`Fast`, `Stable`, *conocido* (varias semanas en la red), buena disponibilidad, ancho de banda suficiente y `V2Dir`
`Exit`	permite tráfico de salida a al menos un bloque /8 en los puertos 80 y 443
`HSDir`	`Stable` y `Fast`, en funcionamiento durante al menos 96 horas: puede almacenar descriptores onion
`V2Dir`	ofrece información de directorio (caché de directorio)
`Authority`	es una autoridad de directorio
`BadExit`	salida que no debe utilizarse como tal (por ejemplo, porque modifica tráfico)
`MiddleOnly`	debe usarse solo como repetidor intermedio (introducido en 0.4.7.2-alpha)
`StaleDesc`	descriptor publicado hace más de 18 horas
`Sybil`	supera el máximo de repetidores por dirección IP (normalmente 2)

Cuadro 12.1: Indicadores del consenso y resumen de los criterios con los que las autoridades los asignan.

`BadExit` y `MiddleOnly` permiten a las autoridades reaccionar frente a repetidores sospechosos sin eliminarlos por completo. El indicador `Sybil` limita el efecto de que alguien ejecute muchos repetidores desde una misma dirección. En la red de laboratorio, donde todos funcionan en 127.0.0.1, se desactiva el límite mediante `AuthDirMaxServersPerAddr=0`; lo veremos en la línea `params`.

12.4.4 Ancho de banda

¿Cómo se mide la velocidad de un repetidor? El valor que anuncia en su descriptor no resulta fiable: un repetidor malicioso podría declarar un ancho de banda enorme para aumentar su probabilidad de selección. Por ello algunas autoridades, denominadas *autoridades de ancho de banda*, miden los repetidores enviando tráfico a través de ellos con una herramienta específica (actualmente *sbws*, capítulo 24) e incluyen el resultado en sus votos como `Measured`. Si hay al menos tres mediciones, el consenso utiliza su mediana. En caso contrario, la línea `w` incorpora `Unmeasured=1` y un valor limitado. El ancho de banda de consenso no es estrictamente una velocidad, sino un *peso*: determina la probabilidad de seleccionar un repetidor (capítulo 13).

12.5 Anatomía de un consenso

Este es un consenso `microdesc` abreviado de la red de laboratorio, descargado en el laboratorio 6. Los fragmentos de esta sección proceden de una copia conservada en el repositorio (`labs/es/codigo/torlab/test/data/consensus-microdesc.txt`), utilizada por las pruebas. Se pueden examinar línea por línea sin iniciar la red. La copia se obtuvo el 23 de septiembre

de 2026, cuando los repetidores de laboratorio todavía ejecutaban tor 0.4.9.12. Esa es la versión de la línea `v` de la entrada, no la versión de referencia del libro, que utiliza actualmente la red.

Cabecera.

```
network-status-version 3 microdesc
vote-status consensus
consensus-method 36
valid-after 2026-09-23 15:37:20
fresh-until 2026-09-23 15:37:40
valid-until 2026-09-23 15:38:00
voting-delay 4 4
known-flags Authority Exit Fast Guard HSDir ... Running Stable ... Valid
params AuthDirMaxServersPerAddr=0 cc_alg=2
```

La línea `params` contiene los *parámetros de consenso*, utilizados por las autoridades para ajustar el comportamiento de toda la red sin exigir actualizaciones del programa. Además del límite por dirección, aquí indica el algoritmo de control de congestión (`cc_alg=2`, Vegas, apartado 11.3). La red pública tiene decenas de parámetros.

Entrada de un repetidor.

```
r test006e ml1aPg9UFy4VCx85YcpsRs/9shc 2038-01-01 00:00:00 127.0.0.1 5106 0
m FfrgO+s4VYS6WKBq3pjfMDs+uZ2uVsvgG88JilbhIOo
s Exit Fast Guard HSDir Running Stable V2Dir Valid
v Tor 0.4.9.12
pr Conflux=1 Cons=1-2 Desc=1-4 ... Link=3-5 ... Relay=2-6
w Bandwidth=4096
```

`r` sobrenombre, resumen de identidad RSA (base64), campo de fecha que ya no tiene significado en el consenso, dirección IPv4, ORPort y DirPort;

`m` resumen del microdescriptor que debe descargarse;

`s` indicadores;

`v`, `pr` versión de `tor` y *subprotocolos* admitidos: aquí descubre el cliente, por ejemplo, que el repetidor admite CGO (`Relay=6`) o los protocolos de enlace 3–5;

`w` peso de ancho de banda en kilobytes por segundo.

En la variante `ns`, cada entrada incluye también una línea `p` con un resumen de la política de salida por puerto. En `microdesc`, ese resumen se incluye en el microdescriptor y no se repite en el consenso.

Final y firmas.

```
directory-footer
bandwidth-weights Wbd=821 Wbe=0 Wbg=5000 Wbm=10000 Wdb=10000 ...
directory-signature sha256 3776EF73A9214587AEFB3D732FA4A443CFF491BA
                           79E102D226590BC5D1D1EBBAD8A16122EC0F9801
-----BEGIN SIGNATURE-----
...
-----END SIGNATURE-----
```

Los valores de `bandwidth-weights` son coeficientes que el cliente utiliza para equilibrar el uso de repetidores entre las distintas posiciones del circuito (capítulo 13). Después aparecen las firmas, una por autoridad: algoritmo hash, resumen de identidad de la autoridad, resumen de la clave de firma y firma.

12.6 Verificar un consenso

Un cliente acepta un consenso solo si contiene firmas válidas de **más de la mitad** de las autoridades que conoce. En el código de `tor`, el umbral es `n_v3_authorities/2 + 1` (`src/feature/nodelist/networkstatus.c`): 5 firmas de 9 en la red pública y 2 de 3 en el laboratorio. Para verificar una firma, el cliente necesita:

1. el **certificado de la autoridad**, que descarga de `/tor/keys/all` y verifica frente a la identidad de autoridad conocida de antemano;
2. el **texto firmado**: todo el documento desde su principio hasta el espacio inmediatamente posterior al primer término `directory-signature`, inclusive;
3. el esquema de firma: RSA con relleno PKCS#1 v1.5 aplicado directamente al resumen (SHA-1 o SHA-256), *sin* el identificador del algoritmo que normalmente requiere PKCS#1 [dir-spec § Signing documents].

En profundidad: ¿Hasta el espacio o hasta el salto de línea?

La sección general de la especificación sobre documentos firmados indica que la firma cubre el texto hasta el *salto de línea* posterior a la línea de firma. Sin embargo, la sección específica del consenso establece el *espacio* posterior a `directory-signature`, garantizando que todas las autoridades firmen exactamente los mismos bytes, aunque sus líneas de firma sean distintas. En el laboratorio 6 probamos ambas interpretaciones con un consenso real de la red privada. Las tres firmas fueron válidas siguiendo la regla del espacio y ninguna lo fue aplicando la regla del salto de línea. Ante la duda, una comprobación experimental contra una implementación independiente sigue siendo el criterio decisivo.

Advertencia: La lista de autoridades es un ancla de confianza

Todo el sistema descansa sobre la lista de autoridades incluida en el programa. Un usuario puede modificarla, pero la especificación desaconseja hacerlo: un cliente que emplee autoridades diferentes verá un consenso distinto del de los demás y resultará reconocible. Las propias autoridades son objetivos. Si una mayoría quedara comprometida, podría firmar un consenso con repetidores maliciosos. Por eso las administran personas distintas en diferentes países y mantienen las claves de identidad fuera de línea.

12.7 Cómo se mantiene actualizado un cliente

Un cliente no descarga un consenso nuevo exactamente al comienzo de cada hora: si todos lo hicieran a la vez, las cachés de directorio quedarían saturadas. En cambio, elige un instante aleatorio entre tres cuartas partes del primer intervalo desde que el consenso deja de ser reciente y siete octavos del tiempo restante hasta su caducidad [dir-spec § Client operation]. La especificación pone un ejemplo: si un consenso es válido desde las 1:00, reciente hasta las 2:00 y válido hasta las 4:00, el cliente buscará otro en un instante aleatorio entre las 2:45 y las 3:50.

Para ahorrar ancho de banda, puede solicitar únicamente las *diferencias* con respecto al consenso que ya posee (*consensus diff*), identificado por su resumen SHA3-256. Después descarga solamente los microdescriptores que falten, solicitándolos por resumen (`/tor/micro/d/`, seguido por los resúmenes separados con `-`).

12.8 Laboratorio: descargar y verificar un consenso

Laboratorio 6: El consenso de la red de laboratorio

Objetivo. Implementar `labs/es/codigo/torlab/directory.py` para descargar un consenso, certificados y microdescriptores desde una autoridad a través de un circuito, analizarlos y verificar sus firmas.
Requisitos previos. Laboratorios 1–5 y red de laboratorio en funcionamiento.
Tiempo estimado. 3 horas.

Paso 1: descargar mediante BEGIN_DIR. Igual que `tor`, utilizamos un circuito de un salto hacia un directorio, sin control de congestión, y abrimos flujos `BEGIN_DIR`. Nos comunicamos por HTTP/1.0:

```
def download(circ, path, stream_id):
    s = Stream(circ, stream_id)
    circ.send(RelayMessage(RelayCommand.BEGIN_DIR, stream_id, b""))
    _, msg = s._next()                      # CONNECTED, cuerpo vacío
    s.send(f"GET {path} HTTP/1.0\r\n\r\n".encode())
    header, _, body = s.receive_all().partition(b"\r\n\r\n")
    return body

circ = build([relay("000a")], cc=False)
text = download(circ, "/tor/status-vote/current/consensus-microdesc", 1)
```

Paso 2: formato netdoc. Una función de unas veinte líneas, `items`, divide el documento en elementos y recopila los objetos `-----BEGIN/-----END`. A partir de ella se construyen dos analizadores: `parse_consensus`, que produce una entrada por repetidor con sus indicadores, ancho de banda, política y resumen de microdescriptor, y `parse_certificates`.

Paso 3: verificar las firmas.

```
def signed_text(text):
    start = text.index("network-status-version ")
    pos = text.index("\ndirectory-signature ") + len("\ndirectory-signature ")
    return text[start:pos].encode()

def verify_signature(consensus, signature, cert):
    h = hashlib.sha256 if signature.algorithm == "sha256" else hashlib.sha1
    expected = h(signed_text(consensus.text)).digest()
    recovered = cert.signing_key.recover_data_from_signature(
        signature.signature, padding.PKCS1v15(), None)
    return recovered == expected
```

El método `recover_data_from_signature`, con algoritmo `None`, devuelve el resumen incluido en la firma sin exigir identificador de algoritmo: exactamente el esquema de Tor.

Paso 4: los resultados. En una de nuestras ejecuciones en la red de laboratorio (las fechas, los nombres y los recuentos dependen de la instancia de la red):

```
consensus: 6035 bytes; 2026-09-23 15:37:20 -> 2026-09-23 15:38:00
method 36; signatures 3; certificates 3
params: {'AuthDirMaxServersPerAddr': 0, 'cc_alg': 2}
  test006e 127.0.0.1:5106 w=4096 flag=['Exit', 'Fast', 'Guard', ...]
  test000a 127.0.0.1:5100 w=20   flag=['Authority', 'Fast', 'Guard', ...]
  ...
valid signatures: 3 (required: 2)
microdescriptors: 8; match consensus m lines: True
```

El consenso cubre 40 segundos (dos intervalos de 20), está firmado por las tres autoridades y el resumen SHA-256 de cada microdescriptor descargado coincide con su línea `m` del consenso. La cadena de confianza se extiende desde las claves de las autoridades hasta la clave ntor de cada repetidor.

Paso 5: controles negativos. Un verificador que nunca falla no verifica nada. Si cambiamos una cifra del consenso (`Bandwidth=4096` por `4097`), el número de firmas válidas baja a cero; con una sola autoridad conocida, se reduce a una. Los archivos `test/test_directory.py` repiten estas comprobaciones con una copia guardada del consenso, de modo que también pueden ejecutarse sin red.

Paso 6: el consenso real. El mismo código funciona con los documentos de la red pública. Descargamos un consenso y los certificados de autoridad del mismo período desde CollecTor y los verificamos:

```
c = parse_consensus(open("consensus-microdesc-2026-09-23-15").read())
cert = parse_certificates(open("certs").read())
valid, required, _ = verify_consensus(c, cert, 9)
print(valid, required)          # 9 5
```

Las nueve firmas son válidas, por encima de las cinco exigidas. Un detalle suele causar problemas al primer intento: los archivos de CollecTor empiezan con una línea de anotación, `@type network-status-microdesc-consensus-3 1.0`, que no pertenece al documento firmado. Incluirla modifica el resumen y hace que *todas* las firmas fallen. Puede parecer un fallo criptográfico cuando en realidad es un error de formato. Por ello, la función `signed_text` debe comenzar en `network-status-version`, y una prueba debe verificar expresamente ese comportamiento.

Resumen

- El sistema de directorios debe ofrecer a todos los clientes una visión auténtica y compartida de la red, resistir ataques Sybil y escalar a millones de usuarios.
- Nueve autoridades de directorio (más una para puentes), enumeradas en el código, votan cada hora y firman un único consenso, válido durante tres horas.
- Los repetidores publican descriptores; los clientes utilizan el consenso `microdesc` y microdescriptores descargados por resumen.
- Un indicador se asigna si lo vota más de la mitad de las autoridades responsables; el ancho de banda medido por al menos tres autoridades prevalece sobre el anunciado.

- Un cliente acepta un consenso con firmas válidas de más de la mitad de las autoridades; la firma cubre el texto hasta el espacio posterior a `directory-signature`.
- Los parámetros de consenso modifican el comportamiento de la red sin necesidad de actualizar el programa.

Ejercicios

Ejercicio 12.1 ⋆ Un consenso tiene `valid-after` a las 10:00, `fresh-until` a las 11:00 y `valid-until` a las 13:00. ¿En qué intervalo buscará el cliente un sustituto? ¿Qué ocurre si no encuentra un nuevo consenso antes de las 13:00?

Ejercicio 12.2 ⋆ Un repetidor dispone de 500 KB/s de ancho de banda, lleva dos días en la red y tiene la política de salida `accept *:443, reject *:*`. ¿Qué indicadores puede recibir? ¿Por qué no `Exit`?

Ejercicio 12.3 ⋆⋆ Describe un ataque de partición en el que una caché de directorio maliciosa entrega a un cliente concreto un consenso antiguo pero todavía válido. ¿Qué limita el ataque? ¿Por qué ayuda solicitar microdescriptores por su resumen?

Ejercicio 12.4 ⋆⋆ Tres autoridades miden el ancho de banda de un repetidor: 5000, 5200 y 90000. ¿Qué valor aparece en el consenso? ¿Y si la tercera autoridad, que colabora con el repetidor, fuera la única que lo midiera?

Ejercicio 12.5 ⋆⋆ Busca el archivo `v3-status-votes` en el directorio de una de las autoridades de la red de laboratorio. Compara su voto con el consenso: ¿qué líneas son diferentes y por qué?

Ejercicio 12.6 ⋆⋆ Elige un repetidor del consenso descargado en el laboratorio. Calcula manualmente el resumen de su microdescriptor mediante `sha256sum` y compáralo con la línea `m`. ¿En qué línea comienza el texto sobre el que se calcula el hash y dónde termina?

Ejercicio 12.7 ⋆⋆⋆ Detén una de las tres autoridades de la red de laboratorio y descarga de nuevo el consenso transcurrido un minuto. ¿Cuántas firmas contiene? ¿Lo acepta tu verificador? Repite la prueba con dos autoridades detenidas y explica lo que observas cuando el consenso caduca.

Ejercicio 12.8 ⋆ ⋆ ⋆ Descarga también el consenso `ns` (`/tor/status-vote/current/consensus`) y verifica sus firmas. ¿Qué algoritmos hash utiliza? ¿Qué información adicional contiene la línea `r` en comparación con la variante microdesc?

Lecturas complementarias

- *Tor directory protocol, version 3* (dir-spec) [Thee]: formatos, votación, asignación de indicadores y funcionamiento de clientes y cachés.
- Documento del protocolo de aleatoriedad compartida (srv-spec), utilizado por las autoridades para generar un valor aleatorio común cada día; lo necesitaremos para los servicios onion (capítulo 14).
- Tor Metrics y CollecTor, `https://metrics.torproject.org/`: archivos históricos de consensos, votos y descriptores.

Capítulo 13

Selección de rutas, guardias y vanguards

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; consenso público del 23/09/2026 a las 15:00 UTC

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir el orden y las restricciones con que tor selecciona repetidores para un circuito;
- explicar cómo el ancho de banda del consenso y los *bandwidth-weights* distribuyen la carga entre posiciones;
- explicar por qué se emplean guardias y reconstruir su evolución desde tres hasta uno y luego dos;
- describir el algoritmo de guardias: conjuntos muestreados, filtrados, confirmados y primarios;
- explicar qué son los *vanguards* y por qué los necesitan los servicios onion;
- simular la selección de rutas a partir de un consenso real de la red pública (laboratorio 7).

El sistema de directorios (capítulo 12) proporciona al cliente una lista de repetidores. Queda la pregunta que más determina la protección del usuario: *¿cuáles* debe utilizar? Si un adversario controla el primer y el último repetidor del circuito, puede correlacionar el tráfico entrante y saliente y descubrir quién se comunica con quién (capítulo 26). Seleccionar la ruta consiste, en buena medida, en hacer que esto ocurra lo menos posible sin sacrificar el rendimiento.

13.1 Orden y restricciones

En un circuito de tres saltos, tor selecciona **primero la salida**, después el guardia y por último el intermedio [path-spec § Path selection and constraints]. La salida se elige primero porque es el repetidor sometido a más restricciones: su política debe permitir el destino solicitado. Si el cliente solo conoce el nombre del destino, pero no su dirección, selecciona una salida que *podría* aceptarlo, es decir, cuya política no rechaza el puerto solicitado antes de permitirlo.

Salvo algunas excepciones, las rutas cumplen estas restricciones:

- no se utilizan repetidores sin el indicador `Fast`;

- un repetidor no puede aparecer dos veces;
- como máximo un repetidor por red IPv4 /16 (o IPv6 /32): dos repetidores del mismo centro de datos podrían pertenecer al mismo operador;
- como máximo un repetidor por **familia**: los operadores de varios repetidores deberían declararlos como familia, y tor evita combinarlos en el mismo circuito;
- el primer repetidor es un guardia (apartado 13.3);
- no se emplean salidas marcadas como `BadExit`.

Las conexiones que se prevé mantener abiertas mucho tiempo (tor clasifica ciertos puertos así, entre ellos el 22 de SSH y el 6667 de IRC) también requieren repetidores `Stable`. Los servicios onion añaden una restricción independiente del puerto: los circuitos de introducción y de encuentro del lado del servicio se construyen como circuitos `Stable` y sus extremos no pueden ser `MiddleOnly` [path-spec § Path selection and constraints]. Las excepciones mencionadas se refieren principalmente a circuitos onion creados con vanguards (apartado 13.4), donde deliberadamente se relajan ciertas reglas, por ejemplo permitir un guardia en una posición intermedia. Prohibir esas combinaciones revelaría al adversario qué repetidores *no* se seleccionaron y, por tanto, información sobre la selección del guardia.

En profundidad: Dos formas de declarar una familia

Tradicionalmente, cada operador enumera los otros repetidores de su familia en el descriptor de cada uno. Dos repetidores solo pertenecen a la misma familia si se mencionan mutuamente. Con muchos repetidores, las listas crecen demasiado y un pequeño error de configuración puede romper la asociación. Un mecanismo más reciente, *happy families*, asigna una clave a la familia. Cada repetidor demuestra su pertenencia mediante un certificado firmado por esa clave (`family-cert` en el descriptor y `family-ids` en el microdescriptor). Los parámetros de consenso `use-family-lists` y `use-family-ids` determinan qué mecanismo aplican los clientes.

13.2 Ponderación por ancho de banda

13.2.1 Por qué importa la ponderación

Si los clientes escogieran los repetidores con probabilidad uniforme, uno de 1 MB/s recibiría tanto tráfico como uno de 1 GB/s. El primero se saturaría y el segundo quedaría desaprovechado. En cambio, Tor selecciona cada repetidor con probabilidad **proporcional a su ancho de banda** registrado en el consenso (línea `w`, apartado 12.5), distribuyendo la carga según su capacidad.

13.2.2 Las posiciones del circuito no son equivalentes

Existe una complicación: los repetidores con indicador `Exit` son relativamente escasos y su capacidad de salida es el recurso más limitado de la red. Utilizar una salida como guardia o intermedio consumiría capacidad necesaria para acceder a Internet. Lo mismo sucede, aunque en menor medida, con los guardias. Por ello las autoridades calculan los **bandwidth-weights** de cada consenso: coeficientes que multiplican el ancho de banda según el *tipo* de repetidor (solo guardia, solo salida, ambos o ninguno) y la *posición* para la que se selecciona [path-spec § Weighting node selection].

Cada peso se llama `Wpt`: *p* indica la posición (`g` guardia, `m` intermedio, `e` salida y `b` solicitudes de directorio) y *t* el tipo (`g`, `e`, `d` si es guardia y salida, `m` si no es ninguno). Los valores se expresan en diezmilésimas: `Wmg=3805` significa que al seleccionar un guardia como intermedio solo cuenta el 38,05 % de su ancho de banda.

13.2.3 Pesos de la red real

Los siguientes pesos proceden del consenso público del 23 de septiembre de 2026 a las 15:00 UTC, descargado desde CollecTor en el laboratorio 7:

Posición \ tipo	guardia (`g`)	intermedio (`m`)	salida (`e`)	guardia+salida (`d`)
guardia (`Wg`)	6195	6195	—	0
intermedio (`Wm`)	3805	10000	0	0
salida (`We`)	10000	10000	10000	10000

Las consecuencias son inmediatas. Las salidas y los repetidores que son a la vez guardias y salidas tienen peso **cero** en posiciones de guardia o intermedio. En esta red, el ancho de banda de salida (33 % del total) es tan valioso que se reserva para esa posición. Los guardias, que representan el 70 % del ancho de banda, son abundantes: se ponderan al 62 % como guardias y al 38 % como intermedios. Como mostrará el laboratorio, en el 60 % de los circuitos el intermedio también tiene el indicador `Guard`.

Las fórmulas utilizadas por las autoridades para calcular estos pesos figuran en la especificación de directorios. Distinguen distintos escenarios según el tipo de ancho de banda escaso e intentan equilibrar, cuando sea posible, la capacidad disponible entre las tres posiciones.

13.3 Guardias

13.3.1 El problema que resuelven

Supongamos que un adversario controla una fracción c del ancho de banda de posibles primeros repetidores y salidas. Si el cliente eligiera un guardia y una salida nuevos al azar para cada circuito, la probabilidad de que ambos pertenecieran al adversario sería aproximadamente c^2 por circuito. Después de C circuitos, la probabilidad de que al menos uno hubiera quedado comprometido sería $1 - (1 - c^2)^C$, que se aproxima a 1 [guard-spec § Introduction and motivation]. Quien utiliza Tor todos los días crea miles de circuitos; tarde o temprano, casi con certeza, el adversario observaría parte de su tráfico.

La idea de los **guardias**, llamados *helper nodes* en la literatura, consiste en seleccionar el primer repetidor *una vez* y seguir utilizándolo durante mucho tiempo. Si ese guardia es honesto, ningún circuito queda comprometido por este mecanismo; si es del adversario, quedan expuestos muchos. La probabilidad de verse comprometido *al menos una vez* cae desde casi 1 hasta aproximadamente c.

Circuitos	Sin guardias	Con guardia fijo
1	0.0025	0.0025
100	0.221	0.050
1000	0.918	0.050
10 000	1.000	0.050

Calculada en el laboratorio 7 para $c = 5\,\%$, la tabla ilustra el compromiso. Los guardias no hacen imposible el compromiso: transforman un evento que con el tiempo sería casi inevitable

en otro infrecuente que afecta a una pequeña fracción de usuarios, en lugar de terminar afectando a casi todos. Este modelo es deliberadamente simplificado; por ejemplo, ignora que el guardia puede cambiar con el tiempo.

13.3.2 Tres, uno, dos

Los guardias se introdujeron en 2006, después de que Øverlier y Syverson demostraran cómo localizar un servicio onion provocando repetidamente que se conectara a repetidores controlados por el adversario [ØS06]. Durante años los clientes utilizaron tres guardias, que sustituían tras dos o tres meses. Investigaciones posteriores mostraron que tres guardias y una rotación frecuente aumentaban la exposición. La Propuesta 236 (2014) propuso un único guardia, conservado nueve o diez meses. Sin embargo, uno solo hace más frágil la conectividad y facilita que un adversario fuerce su sustitución. La Propuesta 291 (2018) pasó a **dos** guardias. El valor predeterminado incorporado en `tor` sigue siendo uno (`DFLT_N_PRIMARY_GUARDS_TO_USE` en `src/feature/client/entrynodes.h`), pero el consenso público contiene `guard-n-primary-guards-to-use=2`, que lo sustituye para todos los clientes. Es un ejemplo de cómo las autoridades regulan la red mediante el consenso.

13.3.3 El algoritmo

El algoritmo actual, definido en la Propuesta 271 y la especificación de guardias, administra diversos conjuntos [guard-spec § The algorithm]:

Guardias muestreados Conjunto de guardias elegidos aleatoriamente con ponderación por ancho de banda y conservados en disco. Crece solo cuando es necesario, hasta 60 repetidores o el 20 % de los guardias del consenso. El límite impide que un adversario obligue al cliente a probar uno tras otro todos los guardias de la red hasta alcanzar uno malicioso.

Guardias filtrados Guardias de la muestra que siguen apareciendo en el consenso y pueden utilizarse con la configuración actual, por ejemplo detrás de un cortafuegos que solo permite ciertos puertos.

Guardias confirmados Guardias realmente utilizados para tráfico del usuario, ordenados por preferencia.

Guardias primarios Normalmente tres guardias tomados de los filtrados: primero los confirmados que también están filtrados, según prioridad; si no bastan, se añaden otros filtrados aún no confirmados. El cliente usa el primer primario accesible (o los dos primeros con el parámetro de consenso indicado). Si ninguno responde, prueba otros, pero vuelve a los primarios cuanto antes.

La permanencia de un guardia en la muestra depende de su estado. Un guardia **nunca confirmado** (sin `CONFIRMED_ON_DATE`) se elimina después de `GUARD_LIFETIME`, es decir, 120 días desde la selección. Un guardia **confirmado**, utilizado por el cliente, se elimina a los `GUARD_CONFIRMED_MIN_LIFETIME` = 60 días desde su confirmación. También se elimina si permanece ausente del consenso durante 20 días. Las marcas temporales guardadas (`ADDED_ON_DATE` y `CONFIRMED_ON_DATE`) se retrotraen aleatoriamente hasta `GUARD_LIFETIME`/10 = 12 días. Así, el archivo de estado no revela exactamente cuándo empezó el cliente a utilizar Tor, y la vida efectiva de un guardia confirmado es de 48–60 días. Por tanto, el guardia *confirmado* rota aproximadamente cada dos meses, no cada cuatro. Este detalle importa para la simulación del capítulo 29.

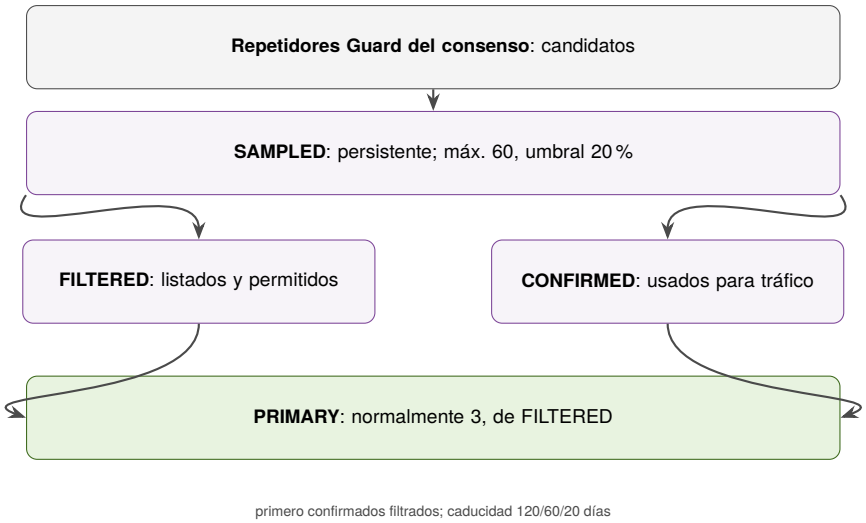


Figura 13.1: Relaciones entre las listas de guardias Tor: la muestra persistente procede de guardias del consenso, mientras que FILTERED y CONFIRMED son subconjuntos distintos que pueden solaparse. Los primarios solo se eligen de FILTERED, priorizando los confirmados y completando con otros filtrados cuando sea necesario. Limitar la muestra reduce la exposición a guardias nuevos.

Advertencia: Un guardia nuevo constituye una pista

Tor distingue cuidadosamente entre un guardia *inaccesible* y un circuito que *ha fallado*. Si el cliente cambiara de guardia cada vez que fallara un circuito, un adversario podría provocar deliberadamente fallos a través de guardias honestos hasta conseguir que seleccionara uno malicioso. Esto es precisamente lo que corrigió la versión de seguridad tor 0.4.9.13, publicada el 23 de septiembre de 2026: ahora el cliente atribuye el fallo al guardia solo cuando no puede establecer la conexión, y no cuando fallan circuitos, flujos o solicitudes de directorio.

13.4 Vanguards

Los guardias protegen el *primer* salto. Los servicios onion (capítulo 14) afrontan otro problema: un cliente malicioso puede inducir al servicio a construir tantos circuitos como desee hasta puntos de encuentro elegidos por el atacante. Si el intermedio de cada circuito se eligiera al azar de nuevo, tarde o temprano pertenecería al adversario, que observaría entonces el guardia del servicio. El siguiente paso podría consistir en atacar ese guardia u obtener sus datos. Los **vanguards** extienden la idea de los guardias a saltos posteriores, manteniendo fijos también los repetidores de la segunda capa y, con ciertas configuraciones, de la tercera.

Vanguards-lite (Propuesta 333, tor 0.4.7.1-alpha) Activados por defecto en clientes y servicios onion. Utilizan cuatro repetidores de segunda capa, no guardados en disco, cada uno con una vida aleatoria. Aquí el código y la especificación difieren. tor 0.4.9.13 utiliza un intervalo de 1 a 12 días (aproximadamente una semana de media), definido por `MIN_SECOND_GUARD_LIFETIME` y `MAX_SECOND_GUARD_LIFETIME` en `src/feature/client/entrynodes.c`. Los parámetros de consenso `guard-hs-l2-lifetime-min` y `-max` pueden modificarlo, pero ninguno aparecía en el consenso del 23 de septiembre de 2026. La especificación de vanguards, en cambio, todavía indica 1–22 días (dos semanas de

media). Este modo está pensado para servicios y usuarios que permanecen activos durante un máximo de unas pocas semanas.

Vanguards completos Repetidores fijos en dos capas (segundo y tercer salto), con mayor duración, para servicios onion de larga vida y alto riesgo. En `tor` se implementan mediante el complemento *vanguards*, que establece `HSLayer2Nodes` y `HSLayer3Nodes`. Arti los implementa directamente con un modo `Full` junto al `Lite`.

En la red de laboratorio (apartado 19.7.3) observamos circuitos con propósito `HS_VANGUARDS`: circuitos preparados con vanguards y listos para extenderse hasta un punto de introducción o encuentro.

13.5 Laboratorio: examinar la red pública

Laboratorio 7: Selección de rutas ponderada por ancho de banda

Objetivo. Implementar `labs/es/codigo/torlab/path_selection.py` y utilizarlo para analizar un consenso real de la red pública.
Requisitos previos. Laboratorio 6 (`directory.py`). Consenso público descargado de CollecTor, archivo público del Tor Project (`https://collector.torproject.org/`). Estos datos se publican específicamente para investigación y descargarlos no implica interactuar con usuarios de la red.
Tiempo estimado. 2 horas.

Paso 1: ponderar un repetidor.

```
def _relay_type(r):  # g = solo guardia, e = solo salida, d = ambos, m = ninguno
    g, e = "Guard" in r.flag, "Exit" in r.flag and "BadExit" not in r.flag
    return {(1, 1): "d", (1, 0): "g", (0, 1): "e", (0, 0): "m"}[(g, e)]

def weight(r, position, weights):
    key = f"W{position[0]}{_relay_type(r)}"          # por ejemplo "Wmg"
    return (r.bandwidth or 0) * weights.get(key, 10000) / 10000
```

Paso 2: seleccionar una ruta. La clase `Selector` calcula los pesos una vez por posición. Aplica las restricciones *mediante rechazo*: selecciona un repetidor y, si incumple una restricción (el mismo repetidor o una misma red /16), selecciona otro.

```
def path(self, rng, guard=None):
    e = self.choose("exit", [], rng)
    g = guard or self.choose("guard", [e], rng)
    m = self.choose("middle", [e, g], rng)
    return g, m, e
```

Una primera versión que filtraba más de nueve mil repetidores y recalculaba pesos y redes en cada selección tardaba 310 segundos en construir 2000 rutas. La versión optimizada construye 20 000 en 7 segundos. En una simulación estadística, el rendimiento no es un detalle menor.

Paso 3: la red pública en cifras. Para el consenso del 23 de septiembre de 2026 a las 15:00 UTC:

```
repetidores: 9358
ancho de banda con Guard: 70.1%    con Exit: 33.1%
guard : aptos 5306; 10 primeros = 1.7%; 100 primeros = 12.6%
middle: aptos 8961; 10 primeros = 2.8%; 100 primeros = 13.7%
exit  : aptos 3126; 10 primeros = 2.3%; 100 primeros = 13.7%
20000 rutas: gge 30.9%  ggd 29.4%  gme 20.4%  gmd 19.3%
```

El número de repetidores *elegibles* es menor que el de aquellos que poseen el indicador correspondiente (capítulo 6 indica 3177 repetidores `Exit`, frente a 3126 aquí). La función `eligible` también exige `Running`, `Valid` y `Fast`, y excluye `BadExit` y `MiddleOnly`. La diferencia comprende 48 salidas sin `Fast` y 3 marcadas como `BadExit`.

La última línea representa una muestra obtenida con semilla aleatoria 7. Sin una semilla fija, los valores varían unas décimas de punto porcentual en cada ejecución y no pueden compararse directamente dos experimentos. Las letras identifican los tipos de guardia, intermedio y salida de cada ruta: `gme`, por ejemplo, es guardia, intermedio sin ninguno de los dos indicadores y salida pura. Tal como predicen los pesos, ninguna ruta utiliza una salida en primera o segunda posición; en el 60 % de los casos, el intermedio tiene indicador `Guard`. La concentración es baja: las diez salidas seleccionadas con mayor frecuencia concentran conjuntamente el 2,3 % de las selecciones y las cien primeras el 13,7 %.

Paso 4: guardias y adversarios. La clase `Adversary` calcula la tabla de apartado 13.3. Prueba a modificar la cuota del adversario: ¿el efecto protector de los guardias sigue siendo evidente para $c = 1\,\%$ o $c = 20\,\%$?

Resumen

- tor selecciona primero la salida, después el guardia y el intermedio, y evita repetidores duplicados o pertenecientes a la misma red /16 o familia.
- La probabilidad de selección de un repetidor es proporcional al ancho de banda multiplicado por un peso que depende de su tipo y de la posición del circuito; en la red estudiada, las salidas solo se emplean como salidas.
- Los guardias convierten el compromiso, casi inevitable a largo plazo con primeros saltos elegidos al azar en cada circuito, en un evento infrecuente; los clientes utilizan actualmente dos.
- El algoritmo de guardias conserva una muestra limitada, guardias confirmados y primarios, y atribuye cuidadosamente los fallos.
- Los vanguards también mantienen fija la segunda capa de los servicios onion; vanguards-lite viene activado por defecto.
- Los parámetros de consenso pueden sustituir los valores predeterminados del programa.

Ejercicios

Ejercicio 13.1 ⋆ ¿Por qué selecciona tor primero la salida? ¿Qué problemas surgirían si eligiera el guardia antes?

Ejercicio 13.2 ⋆ Un repetidor tiene indicadores `Guard` y `Exit` y ancho de banda de consenso 20000. Según los pesos de la red real descritos en este capítulo, ¿qué peso tiene en cada una de las tres posiciones?

Ejercicio 13.3 ⋆⋆ Repite el cálculo de la tabla de guardias para $c = 1\,\%$ y $c = 20\,\%$. ¿Tras cuántos circuitos supera el 50 % la probabilidad de compromiso sin guardias en cada caso?

Ejercicio 13.4 ⋆⋆ Un adversario quiere aumentar la probabilidad de que los clientes seleccionen sus repetidores como salidas. ¿En qué recursos debe invertir y qué mecanismos (mediciones de ancho de banda, familias, límites por dirección y pesos) se lo dificultan?

Ejercicio 13.5 ⋆⋆ ¿Por qué se limita la muestra de guardias a 60 repetidores o al 20 % de todos los guardias? Describe el ataque que evita este límite.

Ejercicio 13.6 ⋆⋆⋆ Amplía `Selector` para aplicar la restricción de familia utilizando las líneas `family` y `family-ids` de los microdescriptores. Descarga desde CollecTor los microdescriptores correspondientes al consenso y mide cuántas selecciones se rechazan por pertenecer a la misma familia.

Ejercicio 13.7 ⋆⋆⋆ Agrupa los repetidores del consenso público por familia declarada y calcula la cuota de probabilidad de selección de salida correspondiente a las diez familias mayores. Compárala con la cuota de las diez salidas individuales más seleccionadas. ¿Qué concluyes sobre el riesgo de los operadores que administran muchos repetidores?

Lecturas complementarias

- *Tor Path Specification* y *Tor Guard Specification* [Thee].
- L. Øverlier y P. Syverson, *Locating Hidden Servers* [ØS06]: el ataque que motivó los guardias.
- Propuestas 236, 271, 291 y 333: del guardia único a dos, el algoritmo actual y vanguards-lite.
- La especificación vanguards-spec, con tablas de plazos para los ataques Sybil.

Capítulo 14

Servicios onion v3

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`*; tor 0.4.9.13; rend-spec v3*

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar qué ofrece un servicio onion y en qué se diferencia de un sitio al que se accede mediante una salida;
- derivar la clave pública de un servicio a partir de su dirección `.onion` y viceversa;
- describir las claves cegadas, los períodos temporales y cómo localiza un cliente los HSDir responsables;
- leer la estructura por capas de un descriptor;
- seguir paso a paso el protocolo de introducción y encuentro;
- descargar, verificar y descifrar el descriptor de un servicio onion de la red de laboratorio (laboratorio 8).

Hasta ahora hemos utilizado Tor para ocultar *quién* se conecta a un servidor. Sin embargo, el propio servidor era público: se conocía su dirección IP y la salida se conectaba directamente con él a través de la red pública, descubriendo el destino. El contenido solo era texto en claro si la aplicación no empleaba cifrado de extremo a extremo, como HTTPS. Los **servicios onion** invierten esta situación. Permiten ofrecer un servicio, por ejemplo un sitio web, sin revelar su ubicación de red. Cliente y servicio se encuentran a mitad de camino, protegidos cada uno por su propio circuito, y ninguno conoce la dirección del otro.

14.1 Qué ofrece un servicio onion

Anonimato del servicio La dirección IP del servidor nunca se expone: el servicio solo se comunica con la red mediante circuitos Tor iniciados por él mismo.

Direcciones autoautenticables La dirección `.onion` *codifica* la clave pública del servicio junto con una suma de comprobación y un byte de versión. Quien se conecta a ella sabe criptográficamente que se comunica con el titular de la clave privada correspondiente, sin necesitar autoridades de certificación.

Cifrado de extremo a extremo El tráfico entre el cliente y el servicio se cifra con claves conocidas únicamente por las dos partes; no existe una salida capaz de observar su contenido en claro.

Accesibilidad El servicio no necesita aceptar conexiones entrantes. Funciona detrás de NAT o de un cortafuegos que permita únicamente conexiones salientes.

El coste es la longitud del recorrido. En el modelo básico, el cliente alcanza un punto de encuentro mediante un circuito de tres saltos y el servicio llega al mismo punto por otro circuito de tres saltos: seis repetidores en total, frente a los tres de una conexión a un sitio convencional. Con vanguards-lite activado por defecto (apartado 13.4), el recorrido deja de ser simétrico. El servicio añade un repetidor fijo de segunda capa y construye un circuito de encuentro de cuatro saltos ($\texttt{S} \to \texttt{G} \to \texttt{L2} \to \texttt{M} \to \texttt{Rend}$), frente a los tres del cliente.

Nota histórica: De v2 a v3

Los *servicios ocultos* existen desde 2004. La primera versión ampliamente utilizada, v2, empleaba direcciones de 16 caracteres derivadas de una clave RSA de 1024 bits, truncando un resumen SHA-1 a 80 bits. También permitía que los HSDir recopilaran las direcciones de los servicios cuyos descriptores alojaban. La versión 3 (Propuesta 224), disponible desde tor 0.3.2.1-alpha (2017), introdujo claves Ed25519, direcciones de 56 caracteres que contienen la clave pública completa, descriptores cifrados y claves cegadas. Los servicios v2 quedaron completamente deshabilitados en octubre de 2021 con tor 0.4.5.11 y 0.4.6.8. También cambió la denominación: hoy se habla de *servicios onion*.

14.2 La dirección

Una dirección v3 codifica en base32 la clave pública de identidad del servicio, una suma de comprobación breve y un número de versión [rend-spec § Encoding onion addresses]:

$$\texttt{address} = \text{base32}(\mathit{PUBKEY} \,\|\, \mathit{CHECKSUM} \,\|\, \mathit{VERSION}) \,\|\, \texttt{".onion"}$$

Aquí *PUBKEY* es la clave Ed25519 de 32 bytes, *VERSION* es el byte `0x03` y *CHECKSUM* son los primeros dos bytes de SHA3-256(`".onion checksum"` ‖ *PUBKEY* ‖ *VERSION*). Los 35 bytes resultantes producen exactamente 56 caracteres base32. La suma de comprobación no protege frente a adversarios, que pueden calcularla como cualquiera, pero permite detectar errores de escritura.

En una ejecución de nuestro laboratorio, el servicio onion tenía la siguiente dirección. Corresponde al ejemplo conservado en `labs/es/codigo/torlab/test/data/`, utilizado por las pruebas de referencia. Por tanto, todos los resultados de esta sección se refieren a ese ejemplo y pueden reproducirse sin iniciar la red. Cuando el lector cree su propia red, se generarán claves nuevas y, por tanto, otra dirección (apartado 19.6).

```
wdjtp7qktdjkkgjjbbx2axvg2gri5p4tuhlc4bq23n7o3uwu4rveppqd.onion
```

La función `key_from_address` del laboratorio 8 extrae la clave pública y verifica la suma de comprobación.

Advertencia: No es posible reconocer una dirección de memoria

Una dirección aleatoria de 56 caracteres no se recuerda fácilmente ni se reconoce de un vistazo. Los usuarios la copian de una fuente que consideran fiable y pueden no detectar una diferencia de unos pocos caracteres. Es el principal vector de *phishing* contra servicios onion. Las *direcciones personalizadas* (*vanity addresses*), generadas probando millones de claves hasta conseguir que sus primeros caracteres formen una palabra, solo ayudan parcialmente: un adversario también puede generar direcciones con el mismo prefijo. Retomaremos este problema en el capítulo 21.

14.3 Claves cegadas

14.3.1 El problema

El servicio necesita publicar en algún lugar información que permita alcanzarlo: el *descriptor*. Si lo publicara directamente bajo su clave de identidad, cualquiera que administrara un HSDir vería las direcciones de todos los servicios que aloja. Podría enumerarlos o relacionar los descriptores de un mismo servicio a lo largo del tiempo. Eso era precisamente lo que sucedía con v2.

14.3.2 La solución

En v3, el servicio firma los descriptores utilizando una **clave cegada**, distinta para cada *período temporal* [rend-spec § Appendix A: Signature scheme with key blinding]. A partir de la clave de identidad $A = aB$, donde B es el punto base Ed25519, calcula el factor

$$h = \text{SHA3-256}(\texttt{"Derive temporary signing key\0"} \parallel A \parallel B \parallel N),$$
$$N = \texttt{"key-blind"} \parallel \textit{period} \parallel \textit{period_length}$$

(con determinados bits forzados, como en las claves Ed25519). La clave cegada es

$$A' = h \cdot A.$$

Esta construcción tiene propiedades importantes:

- cualquiera que conozca la dirección, y por tanto A, puede calcular A' para cada período y sabe dónde buscar el descriptor;
- quien solo conoce A', como un HSDir, no puede recuperar A ni determinar si dos claves cegadas de períodos diferentes pertenecen al mismo servicio;
- el servicio puede firmar bajo A' utilizando la clave privada $a' = h \cdot a$, y la firma se verifica bajo A' como una firma Ed25519 ordinaria.

Un detalle inusual ilustra lo fiel que es la especificación al código: la cadena constante incluye un byte cero final porque `tor` calcula su longitud mediante `sizeof` y no mediante `strlen`. Un comentario del código (`src/feature/hs/hs_common.c`) reconoce que era preferible documentar esa «sorpresa» en la especificación antes que cambiar el código.

14.3.3 Credenciales y claves fuera de línea

La clave de identidad produce una *credencial*, SHA3-256(`"credential"` $\parallel A$). Esta genera para cada período una *subcredencial*, SHA3-256(`"subcredential"` $\parallel$ *credential* $\parallel A'$). La subcredencial se emplea para cifrar el descriptor y en el intercambio de introducción: demuestra conocer la dirección, que un HSDir no posee.

La jerarquía permite conservar la clave de identidad *fuera de línea*. La clave cegada de cada período certifica una clave de firma de descriptores de corta duración (certificado Ed25519 de tipo 8, cuadro 7.1). En principio, el servidor puede recibir anticipadamente claves de firma y certificados de períodos futuros sin poseer nunca la clave privada asociada a su dirección. No todas las implementaciones admiten este modo.

14.4 Localizar el descriptor

14.4.1 Períodos temporales

El tiempo se divide en períodos de un día en la red pública (parámetro de consenso `hsdir-interval`) [rend-spec § Dividing time into periods]. No comienzan a medianoche sino al mediodía UTC. Están desplazados doce horas respecto al protocolo de aleatoriedad compartida de las autoridades, que publica cada día a medianoche un nuevo **valor aleatorio compartido** (*shared random value*, SRV). El número de período se obtiene de los minutos transcurridos desde el 1 de enero de 1970, restando el desplazamiento y dividiendo por la duración. La especificación ofrece un ejemplo: a las 11:15:01 UTC del 13 de abril de 2016, el período es 16903 y comenzó al mediodía del día anterior. El laboratorio 8 comprueba exactamente ese caso.

En redes de prueba, con votaciones cada veinte segundos, esperar un día sería poco práctico. Por eso `tor` reduce el período a una ronda completa del protocolo SRV, equivalente a 24 votaciones (función `get_time_period_length`). En el laboratorio, un período dura ocho minutos y está desplazado cuatro minutos.

14.4.2 El anillo HSDir

¿Qué HSDir almacena el descriptor? Cada repetidor con indicador `HSDir` ocupa una posición en un *anillo* de 256 bits [rend-spec § Where to publish a hidden service descriptor]:

$$\textit{relay_index} = \text{SHA3-256}(\texttt{"node-idx"} \,\|\, \textit{Ed25519 identity} \,\|\, \textit{SRV} \,\|\, \textit{period} \,\|\, \textit{period_length})$$

Para cada una de las dos *réplicas*, el servicio calcula:

$$\textit{service_index}(r) = \text{SHA3-256}(\texttt{"store-at-idx"} \,\|\, A' \,\|\, r \,\|\, \textit{period_length} \,\|\, \textit{period}).$$

El servicio carga su descriptor en los primeros cuatro HSDir situados después de cada índice del anillo (parámetro `hsdir_spread_store`); el cliente elige al azar uno de los tres primeros (`hsdir_spread_fetch`). Incluir el SRV impide que un adversario prediga con demasiada antelación qué repetidores serán responsables del servicio y coloque repetidores maliciosos en las posiciones adecuadas para censurarlo u observar solicitudes de descriptores. El servicio publica simultáneamente dos descriptores para períodos consecutivos, de modo que siga siendo accesible a clientes cuyos consensos estén ligeramente adelantados o atrasados. Los dos períodos utilizados dependen de la fase del ciclo: antes o después de la rotación del SRV, pueden ser el anterior y el actual o el actual y el siguiente [rend-spec § Generating and publishing hidden service descriptors].

En profundidad: ¿Qué SRV debe utilizarse?

Como los períodos y el SRV están desplazados medio día, el cliente debe elegir cuidadosamente su valor. En la media jornada entre el comienzo del período y el siguiente SRV utiliza el actual; en la otra mitad utiliza el anterior. Si el consenso carece del SRV necesario, por ejemplo justo después de iniciar una red, `tor` calcula una alternativa denominada *disaster SRV*: SHA3-256(`"shared-random-disaster"` ‖ *period_length* ‖ *period*). La elección se implementa en `hs_get_current_srv` y `hs_get_previous_srv` (`src/feature/hs/hs_common.c`). Si falta el valor, `tor` utiliza discretamente la alternativa. Sucede en una red de laboratorio recién creada y dura poco: en el registro de su autoridad, el primer SRV aparece 115 segundos después de iniciarse; hasta entonces, el consenso no contiene `shared-rand-current-value`. Para observar el comportamiento alternativo, examina los primeros minutos tras crear la red. Una vez que lleva tiempo activa, como en apartado 14.9, se utilizan los SRV reales.

14.4.3 Solo solicitudes anónimas

El cliente descarga un descriptor mediante una solicitud HTTP a `/tor/hs/3/`, seguida de la clave cegada codificada en base64 sin los caracteres = finales, a través de un flujo `BEGIN_DIR` hacia el HSDir. Existe una condición: la solicitud debe llegar por un circuito **anónimo**. El HSDir rechaza peticiones procedentes de un circuito abierto directamente por el cliente, es decir, de un solo salto. Lo descubrimos en el laboratorio:

```
HTTP/1.0 503 Rejecting single hop HS v3 descriptor request
```

La comprobación está en `connection_dir_is_anonymous` (`src/feature/dircommon/directory.c`). Impide que un HSDir relacione una solicitud de descriptor con la dirección IP del cliente que la hizo.

14.5 El descriptor

El descriptor tiene tres capas [rend-spec § Generating and publishing hidden service descriptors], representadas en figura 14.1.

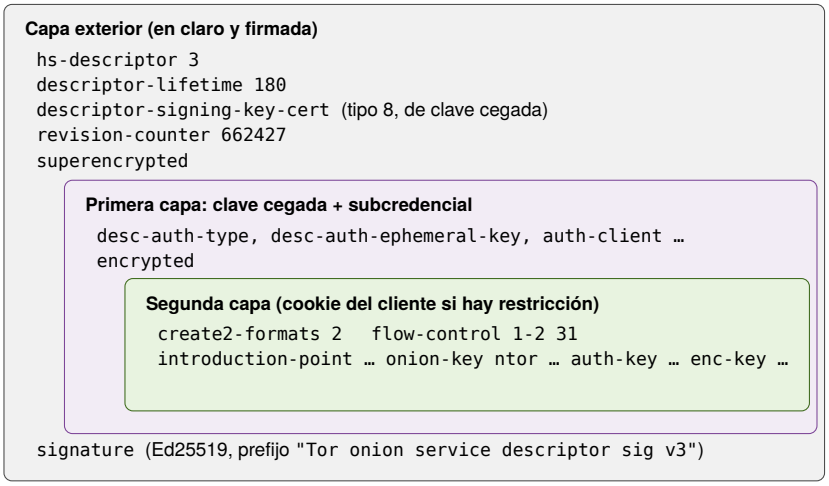


Figura 14.1: Estructura del descriptor onion v3. La envoltura exterior firmada es visible para HSDir; quien conoce la identidad onion puede descifrar la primera capa. Con descubrimiento restringido, el cliente autorizado utiliza su clave X25519 para recuperar el descriptor cookie que permite descifrar la segunda.

La capa exterior está en claro y firmada. Contiene la versión, el período de validez (`descriptor-lifetime`, 180 minutos en el laboratorio), el certificado de la clave firmante emitido por la clave cegada, un contador de revisión, el bloque cifrado `superencrypted` y la firma. Se utiliza Ed25519 con la clave de firma del descriptor sobre texto precedido por `"Tor onion service descriptor sig v3"`. El prefijo impide reutilizar esa firma para otros fines. El contador de revisión debe aumentar con cada versión nueva, para que un HSDir no acepte descriptores anteriores a los que ya posee.

La primera capa cifrada utiliza una clave derivada de la clave cegada y la subcredencial. Un HSDir que desconozca la dirección no puede leerla. Contiene los campos de *descubrimiento restringido* (apartado 14.7) y la segunda capa. Para no revelar si el descubrimiento restringido está activado, los campos están siempre presentes y se rellenan con datos aleatorios cuando no se necesitan.

La segunda capa contiene lo necesario para conectarse: formatos de intercambio admitidos, parámetros de control de flujo, eventuales parámetros anti-DoS y, para cada **punto de introducción**, los especificadores de enlace para alcanzarlo (en base64), su clave ntor, una clave de autenticación certificada y una clave de cifrado. Por defecto, el servicio utiliza tres puntos de introducción.

Ambas capas utilizan el mismo esquema. Una sal aleatoria de 16 bytes, datos secretos y una constante propia de la capa alimentan SHAKE-256 para derivar una clave AES-256, un vector de inicialización y una clave de autenticación. El texto se cifra en modo contador y se autentica mediante SHA3-256. El texto en claro de la primera capa se rellena hasta un múltiplo de 10 000 bytes y las líneas `auth-client` hasta un múltiplo de 16, para revelar lo menos posible sobre el número de puntos de introducción y clientes autorizados.

14.6 Introducción y encuentro

Con el descriptor, el cliente ya puede conectarse. El protocolo, ilustrado en figura 14.2, implica tres repetidores «especiales»: un punto de introducción elegido por el servicio, un punto de encuentro elegido por el cliente y los HSDir.

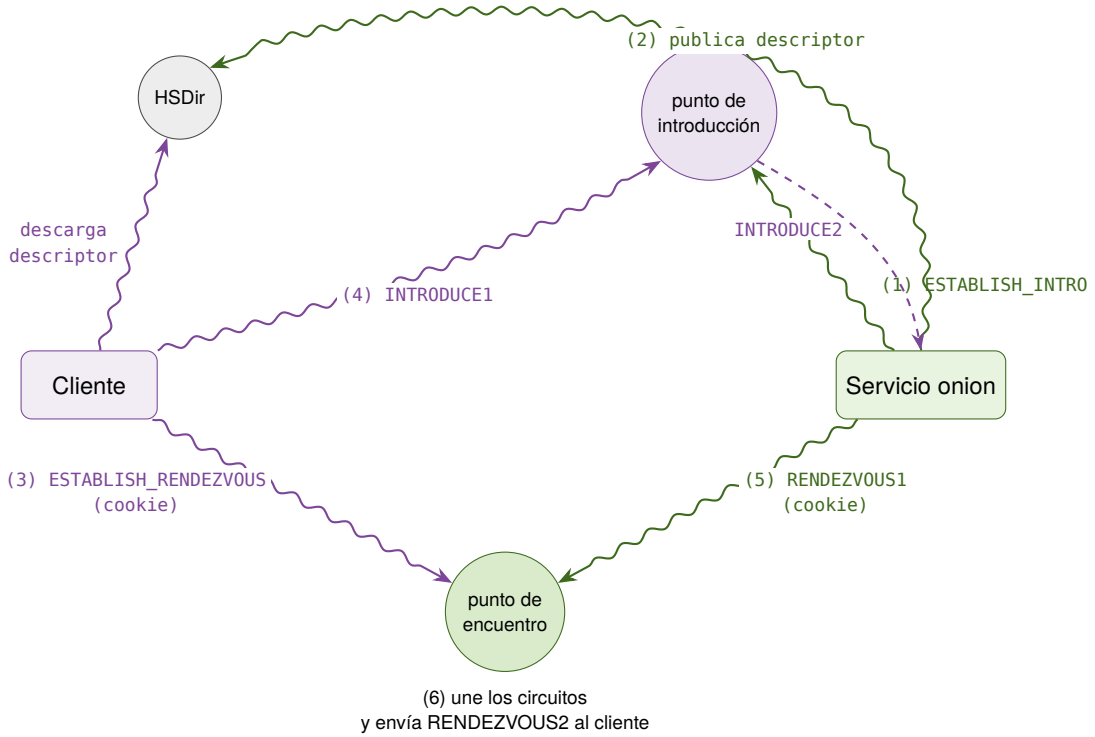


Figura 14.2: Protocolo de encuentro. Cada flecha representa un circuito de varios saltos; no hay conexión directa entre cliente, servicio y puntos intermedios. Los números indican el orden de los pasos explicados en el texto.

1. **El servicio prepara puntos de introducción.** Selecciona varios repetidores, construye un circuito hacia cada uno y envía `ESTABLISH_INTRO` (comando relay 32) con una clave de autenticación distinta y una firma para cada punto. El repetidor responde con `INTRO_ESTABLISHED` (38); a partir de ese momento reenvía al servicio las solicitudes que presentan esa clave.

2. **El servicio publica su descriptor** en los HSDir responsables, como se ha explicado.

3. **El cliente prepara el encuentro.** Selecciona al azar un repetidor como punto de encuentro, crea un circuito hasta él y envía `ESTABLISH_RENDEZVOUS` (33) con una *cookie* aleatoria de 20 bytes. El repetidor responde con `RENDEZVOUS_ESTABLISHED` (39).

4. **El cliente se presenta.** Construye un circuito hacia un punto de introducción y envía `INTRODUCE1` (34). La parte en claro especifica la clave de autenticación del punto. La parte cifrada, legible únicamente por el servicio, contiene la cookie, los especificadores de enlace y la clave ntor del punto de encuentro, y la primera mitad de un intercambio. El punto de introducción envía `INTRODUCE_ACK` (40) al cliente y reenvía el mensaje al servicio como `INTRODUCE2` (35).

5. **El servicio alcanza el punto de encuentro.** Descifra `INTRODUCE2`, comprueba que no duplica una solicitud anterior (*caché antirrepetición*), crea un circuito hasta el punto indicado y envía `RENDEZVOUS1` (36) con la cookie y la segunda mitad del intercambio.

6. **El punto de encuentro une los circuitos.** Localiza el circuito del cliente mediante la misma cookie, reenvía la segunda mitad del intercambio en `RENDEZVOUS2` (37) y, desde ese momento, copia celdas entre ambos circuitos.

El intercambio cliente–servicio, *hs-ntor*, es una variante de ntor (apartado 9.6) que incluye la subcredencial y la clave de autenticación del punto de introducción. Solo quien conoce la dirección puede generar un `INTRODUCE1` aceptable para el servicio. Las claves derivadas añaden otra capa de cifrado, como un *salto virtual* incorporado al circuito del cliente, y emplean SHA3-256 para el resumen acumulativo en lugar de SHA-1 (apartado 10.2). Ni el punto de introducción ni el de encuentro pueden leer el tráfico.

A través del circuito unido, el cliente abre flujos mediante `BEGIN`, igual que haría con una salida (capítulo 11). El servicio reenvía esos flujos a los puertos locales configurados con `HiddenServicePort`.

En profundidad: ¿Por qué tantos repetidores?

El punto de introducción conoce el servicio solo como «la clave de autenticación de un circuito»: no sabe a qué dirección `.onion` corresponde. El punto de encuentro ve una cookie aleatoria y dos circuitos; no conoce qué servicio está conectando con qué cliente. Cada función ve únicamente un fragmento y ninguna conoce las direcciones IP del cliente o del servicio, que permanecen detrás de sus respectivos guardias.

14.7 Descubrimiento restringido

Un servicio puede querer limitar su acceso a determinados clientes, por ejemplo si funciona como panel de administración. En el **descubrimiento restringido** (anteriormente *autorización de clientes*), cada cliente autorizado tiene un par de claves X25519 y el servicio conoce su componente pública. El servicio cifra por separado la clave de la segunda capa del descriptor para cada cliente, mediante las líneas `auth-client` de la primera capa. Alguien que conozca la dirección, pero no posea una clave autorizada, no puede leer la lista de puntos de introducción ni siquiera intentar conectarse. En `tor` se configura añadiendo las claves públicas de los clientes al directorio `authorized_clients` del servicio.

14.8 Defensas

Los servicios onion son objetivos naturales de ataques: no pueden bloquear sencillamente las direcciones IP de clientes hostiles, porque no las conocen. Sus principales defensas, tratadas más ampliamente en otros capítulos, son:

- **vanguards**, que protegen el guardia del servicio frente a ataques que inducen la creación de circuitos a demanda (apartado 13.4);
- la **caché antirrepetición**, que evita hacer repetir el mismo trabajo al servicio;
- parámetros anti-DoS que el servicio puede establecer en los puntos de introducción mediante una extensión de `ESTABLISH_INTRO`, limitando el número de mensajes `INTRODUCE2` por segundo;
- la **prueba de trabajo**, mediante la que un servicio atacado pide a los clientes resolver un pequeño problema computacional antes de atenderlos (capítulo 15).

14.9 Laboratorio: localizar y leer un descriptor

Laboratorio 8: El descriptor del servicio onion de laboratorio

Objetivo. Implementar `labs/es/codigo/torlab/onion.py`: tratamiento de direcciones, clave cegada, anillo HSDir, descarga de descriptores, verificación y descifrado.
Requisitos previos. Laboratorios 1–6; red de laboratorio en funcionamiento.
Tiempo estimado. 4 horas.

Paso 1: la clave cegada. Las bibliotecas criptográficas habituales permiten crear firmas Ed25519, pero no multiplicar el punto por el escalar para obtener $A' = h \cdot A$. Por ello el módulo incorpora unas treinta líneas de aritmética Ed25519 derivadas de RFC 8032, suficientes para esta finalidad, pero inadecuadas para uso general: son lentas y no tienen protección frente a ataques de tiempo.

```
def blinding_factor(A, period, duration_minutes):
    n = b"key-blind" + struct.pack(">QQ", period, duration_minutes)
    h = bytearray(hashlib.sha3_256(
        b"Derive temporary signing key\x00" + A + BASE_STR + n).digest())
    h[0] &= 248; h[31] &= 63; h[31] |= 64
    return int.from_bytes(h, "little")

def blinded_key(A, period, duration_minutes):
    return _encode_point(_multiply(blinding_factor(A, period, duration_minutes),
                                   _decode_point(A)))
```

Paso 2: período y SRV. La función `time_period_info` reproduce el cálculo de `tor` a partir de `valid-after` y del intervalo de votación del consenso, determinando si se necesita el SRV actual o el anterior. Si falta el necesario, utiliza el *disaster SRV*.

Paso 3: investigar un error. En el primer intento, no encontramos el descriptor en ninguno de los ocho HSDir de la red. Como habíamos probado todos, la selección de HSDir no podía explicar el problema. Quedaban la clave cegada y la solicitud. Para distinguirlas, ordenamos al cliente real de la red que descargara el descriptor por el puerto de control (`HSFETCH` y

luego `GETINFO hs/client/desc/id/...`). Leímos la clave cegada realmente utilizada por el servicio en el certificado obtenido: coincidía con la nuestra. El problema estaba, por tanto, en la solicitud, y el código de `tor` explicó por qué: los HSDir rechazan solicitudes de un salto. Con un circuito de tres saltos conseguimos el descriptor. Es un método general útil: cuando un experimento falla, busca un punto intermedio donde comparar tus resultados con una implementación de referencia.

Paso 4: verificar y descifrar. En el ejemplo, `path` representa una lista de tres objetos `RelayInfo` obtenidos de la red de laboratorio, con el HSDir como último repetidor; `A` es la clave de identidad y `Ap` su clave cegada. No se muestra cómo construir esos objetos.

```
d = download_descriptor(Ap, path)
desc = parse_descriptor(d, A, Ap)
```

`parse_descriptor` verifica que el certificado de firma proceda de la clave cegada esperada, comprueba la firma del descriptor y descifra ambas capas. Resultado abreviado de una sesión de laboratorio anterior:

```
duración 180 revisión 662427 puntos de introducción 3
create2-formats 2
flow-control 1-2 31
introduction-point AwAGfwAAARPyAhQXE5GoH+wp3MoxW9M1QV6XSxb0fgM...
onion-key ntor yRURxDb2eeEmF/p5M/bmHA+giwVyxjApKYyNY1KkCDo=
auth-key
-----BEGIN ED25519 CERT-----
...
```

La línea `flow-control 1-2 31` comunica que el servicio admite control de congestión con el mismo `sendme_inc` de 31 observado en capítulo 11.

Paso 5: comprobar la coherencia. Hay ocho HSDir en el laboratorio y el servicio carga el descriptor en $2 \times 4 = 8$: todos ellos. Por tanto, encontrar el descriptor no demuestra que nuestro cálculo del anillo sea correcto. Sin embargo, el registro `info.log` del cliente indica qué HSDir contactó: `test002a`, a las 16:07. Es uno de los tres que nuestro código obtuvo para la réplica correspondiente en ese momento. Una observación no constituye una demostración, pero es coherente. El ejercicio 14.6 propone una prueba más exigente.

Resumen

- Un servicio onion ofrece un servicio sin revelar su dirección IP; su dirección `.onion` contiene y autentica la clave pública del servicio.
- Los descriptores se firman con claves cegadas específicas de cada período, que solo quien conoce la dirección puede relacionar con el servicio.
- Las posiciones de descriptores en el anillo HSDir dependen de la clave cegada, el período y el valor aleatorio compartido; los clientes los descargan mediante solicitudes anónimas.
- El descriptor tiene una capa exterior firmada y dos cifradas; la segunda enumera puntos de introducción.
- Cliente y servicio se encuentran en un punto elegido por el cliente, después de enviar una solicitud a un punto de introducción. El intercambio hs-ntor añade una capa de cifrado extremo a extremo.

- El descubrimiento restringido, los vanguards, las cachés antirrepetición, los límites de frecuencia y la prueba de trabajo protegen los servicios.

Ejercicios

Ejercicio 14.1 ⋆ ¿Cuántos bytes codifica una dirección v3 y por qué produce exactamente 56 caracteres base32? ¿Qué probabilidad tiene un error aleatorio de escritura de superar la comprobación de la suma de comprobación?

Ejercicio 14.2 ⋆ Calcula el número de período de la red pública a las 09:00 UTC del 1 de enero de 2026. ¿Cuándo comenzó ese período?

Ejercicio 14.3 ⋆⋆ Un adversario quiere bloquear un servicio onion asegurándose el control de sus HSDir responsables. ¿Qué debe predecir y con cuánta antelación puede hacerlo? ¿Qué papel cumple el SRV?

Ejercicio 14.4 ⋆⋆ Enumera qué saben el punto de introducción, el punto de encuentro y el HSDir sobre cliente y servicio. ¿Qué función sería más interesante para un adversario que quisiera estimar la popularidad de un servicio?

Ejercicio 14.5 ⋆⋆ ¿Por qué rechaza el HSDir las solicitudes de descriptor que llegan por circuitos de un salto? ¿Qué podría conocer en caso contrario?

Ejercicio 14.6 ⋆⋆⋆ Comprueba el cálculo del anillo con mayor rigor. Añade repetidores a la red de laboratorio (ejercicio 19.7) hasta tener más de ocho HSDir y verifica que el descriptor esté disponible *únicamente* en aquellos elegidos por el código.

Ejercicio 14.7 ⋆⋆⋆ Descodifica los especificadores de enlace de un punto de introducción a partir del descriptor de laboratorio (valor base64 de una línea `introduction-point`). ¿De qué repetidor de laboratorio se trata? ¿Qué contiene el certificado de la línea `auth-key`?

Lecturas complementarias

- *Tor Rendezvous Specification - Version 3* (rend-spec) [Thee]: direcciones, claves cegadas, descriptores, introducción y encuentro, incluidos vectores de prueba de hs-ntor.
- Propuesta 224: diseño de los servicios v3.
- Especificación de aleatoriedad compartida (srv-spec): cómo producen las autoridades el SRV.

Capítulo 15

Defensas contra DoS y gestión de recursos

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`; tor 0.4.9.13; dos-spec, hspow-spec

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar por qué un ataque de denegación de servicio contra Tor también amenaza el anonimato;
- describir cómo protege tor la memoria, la capacidad de CPU y el ancho de banda de los repetidores;
- comprender los límites por dirección aplicados a conexiones y circuitos, y la función de los parámetros de consenso;
- explicar cómo se defienden los servicios onion frente a inundaciones de solicitudes de introducción;
- describir el mecanismo de prueba de trabajo v1 (Equi-X y Blake2b) y cómo se ajusta el esfuerzo;
- activar y observar las defensas de un repetidor y un servicio onion en la red de laboratorio (laboratorio 9).

Una red abierta, anónima y administrada por voluntarios es un objetivo natural para quien quiera inutilizarla. En este capítulo examinamos las defensas de Tor frente a ataques de *denegación de servicio* (DoS): intentos de agotar los recursos de un repetidor o servicio onion para impedir su uso por otras personas. Los incidentes concretos de los últimos años, incluida la oleada de ataques contra servicios onion de 2022–2023, se estudian en capítulo 27. Aquí nos centraremos en los mecanismos.

15.1 Por qué DoS también amenaza el anonimato

En una red ordinaria, DoS sería «solo» un problema de disponibilidad. En Tor puede convertirse también en un arma contra el anonimato [dos-spec § Overview]:

- Un adversario capaz de deshabilitar un repetidor cuando quiera puede obligar a los clientes que lo utilizan como guardia a seleccionar otro, con la esperanza de que elijan uno malicioso. Jansen, Tschorsch, Johnson y Scheuermann demostraron cómo un adversario con pocos recursos podía agotar la memoria de un repetidor y aprovecharlo para desanonimizar servicios onion: el *Sniper Attack* [Jan+14].

- Volver lentos o inaccesibles los repetidores honestos empuja el tráfico hacia los del adversario, aumentando la fracción que puede observar.
- Un ataque que provoca fallos de circuito selectivos puede servir como señal, por ejemplo para determinar qué circuitos atraviesan cierto repetidor.

Por ello, las defensas persiguen algo más que «mantener los repetidores encendidos»: deben evitar comportamientos observables que el adversario pueda explotar.

15.2 Recursos físicos

15.2.1 Memoria

La memoria es el recurso más delicado y cuenta con los mecanismos de protección más maduros [dos-spec § Memory exhaustion]. En Linux, cuando un proceso solicita demasiada memoria no necesariamente recibe un error de inmediato. El sistema promete asignársela y puede terminar el proceso más tarde, cuando intenta utilizarla. Por tanto, esperar a que falle una asignación es ineficaz. tor impone su propio límite, `MaxMemInQueues`, configurado por el operador o calculado como proporción de la memoria del sistema. Contabiliza todo lo que consume memoria: celdas en cola de circuitos, búferes de conexiones, estado de compresión, descriptores onion en caché y consultas DNS almacenadas.

Cuando el uso supera el límite, tor libera memoria en dos fases. Primero expulsa las entradas más antiguas de las cachés que consumen más del 20 % del total, hasta situarlas por debajo del 10 %. Si el uso todavía supera el 90 % del límite, cierra circuitos y conexiones en orden de antigüedad del dato más antiguo que tienen en cola, empezando por aquellos que llevan más tiempo sin procesarlo. El objetivo es actuar contra participantes que acumulan datos sin leerlos, un patrón típico de este ataque.

15.2.2 CPU

La CPU es un punto débil de la implementación en C, que procesa la actividad de red en un único hilo principal. Las operaciones más costosas, los intercambios de creación de circuitos (capítulo 9), se delegan en un conjunto de hilos trabajadores y se ponen en cola. No obstante, si llegan demasiados intercambios, la cola crece y los circuitos legítimos agotan sus tiempos de espera. Tor no mide directamente el uso de CPU, sino que la protege indirectamente limitando operaciones que la consumen, incluidas la creación de circuitos y las introducciones de servicios onion.

15.2.3 Ancho de banda

Los operadores pueden limitarlo con opciones como `RelayBandwidthRate`, pero esas restricciones sirven principalmente para no superar las condiciones de su proveedor de conectividad; no constituyen defensas por sí mismas. Frente al consumo anómalo de ancho de banda, Tor utiliza el control de flujo y congestión (capítulo 11) y una planificación equitativa de celdas entre circuitos.

15.3 Límites por dirección

La defensa más eficaz frente a inundaciones poco sofisticadas es sencilla. Un guardia ve la dirección IP del cliente y puede limitar lo que solicita cada dirección. Es el único punto de la red donde pueden aplicarse esos límites, porque los saltos posteriores desconocen

deliberadamente el origen. Tor aplica tres familias de límites a los clientes (no a los repetidores autenticados), configurables mediante opciones `DoS…`:

Conexiones (`DoSConnection…`) Número máximo de conexiones TLS simultáneas y tasa máxima de conexiones nuevas por dirección. Se cierran las conexiones que superan esos umbrales.

Circuitos (`DoSCircuitCreation…`) Un *depósito de fichas* (*tokens*) por dirección (*token bucket*): cada circuito nuevo consume una ficha. Estas se reponen a ritmo fijo hasta un máximo (*capacidad de ráfaga* (*burst*)). Se marca una dirección que ha agotado el depósito *y* mantiene al menos cierto número de conexiones; sus solicitudes de circuitos posteriores se rechazan durante un período.

Encuentros de un salto (`DoSRefuseSingleHopClientRendezvous`) Rechaza solicitudes de encuentro recibidas en circuitos de un solo salto, como los empleados por puertas de enlace Tor2web para acceder a servicios onion sin anonimato del cliente. Permitirían generar carga a bajo coste.

15.3.1 El código y el consenso

Estas defensas están **desactivadas** por defecto en el código de tor (`DOS_CC_ENABLED_DEFAULT 0` en `src/core/or/dos.h`). La configuración de la red procede de los parámetros de consenso. El consenso público del 23 de septiembre de 2026 contiene:

Parámetro	Consenso	Valor del código
`DoSCircuitCreationEnabled`	1	0
`DoSCircuitCreationMinConnections`	2	3
`DoSCircuitCreationRate`	2	3
`DoSCircuitCreationBurst`	60	90
`DoSConnectionEnabled`	1	0
`DoSConnectionMaxConcurrentCount`	50	100
`DoSRefuseSingleHopClientRendezvous`	1	—

Los valores de la red son más estrictos que los predeterminados del código: reflejan años de ataques y ajustes posteriores. Poder modificarlos mediante el consenso, en unas horas y sin actualizar miles de repetidores, es una de las herramientas más valiosas del Tor Project durante un ataque.

Advertencia: Límites de los límites por dirección

Un adversario con muchas direcciones IP, como una botnet, puede eludir los límites por dirección. A la inversa, unos límites excesivamente estrictos perjudican a usuarios legítimos que comparten una dirección, por ejemplo detrás de un NAT de una universidad o un operador móvil. Los valores son siempre un compromiso.

15.4 Servicios onion bajo ataque

Los servicios onion son especialmente vulnerables. Cada solicitud de introducción es barata para el cliente: un único `INTRODUCE1` (apartado 14.6). Para el servicio es costosa: debe descifrarla, realizar un intercambio y crear un circuito de varios saltos hasta el punto de encuentro indicado. Esto produce **amplificación**: con relativamente pocos recursos, un adversario puede obligar al servicio y a los repetidores de sus circuitos a trabajar mucho más. El servicio no puede bloquear las direcciones IP de los atacantes porque nunca las ve.

15.4.1 Límites en los puntos de introducción

Una primera defensa consiste en limitar las solicitudes en los puntos de introducción. Con `HiddenServiceEnableIntroDoSDefense` y las opciones relacionadas `…RatePerSec` y `…BurstPerSec`, el servicio utiliza una extensión `DOS_PARAMS` de `ESTABLISH_INTRO` para indicar a cada punto cuántos mensajes `INTRODUCE2` por segundo está dispuesto a recibir. Esto protege el servicio, pero no distingue atacantes de usuarios legítimos: durante un ataque pueden rechazarse ambos.

15.4.2 Prueba de trabajo

La defensa más reciente, propuesta en Propuesta 327 y disponible desde tor 0.4.8.4 (agosto de 2023), es la **prueba de trabajo** (*proof of work*, PoW) [hspow-spec § Motivation]. Los clientes deben demostrar que han efectuado cierto cálculo antes de recibir servicio; las peticiones que muestran mayor esfuerzo reciben prioridad. Unos segundos de trabajo pueden ser aceptables para un usuario legítimo. Para un atacante que envía miles de solicitudes por segundo, multiplicar ese coste por todas ellas puede resultar prohibitivo.

El mecanismo consta de cuatro componentes [hspow-spec § Common protocol]:

1. **Parámetros del descriptor.** En la segunda capa del descriptor (apartado 14.5), el servicio publica una línea `pow-params` con el esquema (`v1`), una *semilla* aleatoria de 32 bytes, el *esfuerzo sugerido* y la caducidad de la semilla. La semilla se renueva cada 105–120 minutos.

2. **El cliente resuelve el problema.** Elegido el esfuerzo E, busca una solución Equi-X para el desafío

   $$\texttt{"Tor hs intro v1\0"} \,\|\, A' \,\|\, \mathit{seed} \,\|\, \mathit{nonce} \,\|\, E,$$

   donde A' es la clave cegada del servicio. Una solución Equi-X no basta. El cliente calcula además un hash Blake2b de 32 bits, R, a partir del desafío y la solución; solo se acepta si $R \cdot E \leq 2^{32} - 1$. De lo contrario cambia el nonce y vuelve a intentarlo.

3. **Prueba en el mensaje de introducción.** El cliente incluye nonce, esfuerzo, los primeros cuatro bytes de la semilla y la solución en una extensión de la parte cifrada de `INTRODUCE1`, legible solo por el servicio.

4. **Cola de prioridades.** El servicio verifica la prueba, mucho más rápido de lo que se tarda en encontrarla, y sitúa la solicitud en una cola ordenada por esfuerzo. Atiende primero las de mayor esfuerzo; las que carecen de prueba tienen esfuerzo cero. Si la cola se llena, descarta inmediatamente la mitad de menor esfuerzo.

En profundidad: ¿Por qué Equi-X?

Equi-X, diseñado por tevador para esta finalidad, es un problema *asimétrico*: encontrar una solución requiere memoria y cálculo apreciable, pero verificarla cuesta muy poco. Se basa en HashX, familia de funciones hash cuya implementación cambia con cada semilla y aprovecha las capacidades de ejecución paralela de las CPU modernas. Eso dificulta acelerar el proceso mediante hardware especializado, limitando la ventaja de un adversario frente a un usuario con un ordenador corriente. Existen dos implementaciones, una en C incluida en tor y otra en Rust para Arti, que se mantienen compatibles mediante pruebas automatizadas. El código C utiliza la licencia LGPL: para incluirlo, tor debe compilarse con `--enable-gpl`. Una compilación sin esa opción carece de PoW: `tor --list-modules` indica `pow: no`, en lugar de `pow: yes`, como veremos en capítulo 17. No es lo habitual:

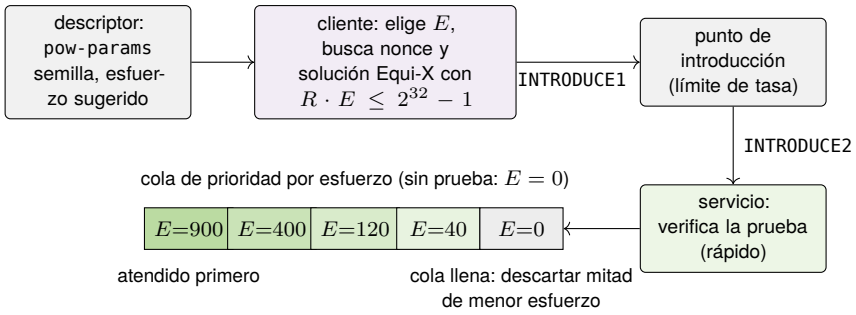


Figura 15.1: Prueba de trabajo de un servicio onion. El cliente lee la semilla y el esfuerzo sugerido del descriptor, encuentra una solución que supera la comprobación y la envía en la parte cifrada de `INTRODUCE1`. El servicio la verifica y ordena las solicitudes por esfuerzo.

tanto la imagen del laboratorio como el paquete `tor` de Debian se compilan con esa opción y contienen el módulo.

15.4.3 Esfuerzo lineal

¿Por qué añadir la comprobación Blake2b? Encontrar una solución Equi-X individual tiene un coste aproximadamente fijo. Una solución aleatoria satisface $R \cdot E \leq 2^{32} - 1$ con probabilidad cercana a $1/E$, por lo que el cliente necesita probar, en promedio, E soluciones. Así, el esfuerzo crece *linealmente* con E y puede ajustarse con precisión. Muchos sistemas PoW exigen en cambio cierta cantidad de bits cero iniciales en un hash, duplicando el coste con cada bit añadido: una escala demasiado gruesa para un control fino. Además, el esfuerzo se declara *antes* de buscar una solución y forma parte del desafío. El cliente no puede obtener un resultado afortunado y luego afirmar que invirtió más esfuerzo del real.

15.4.4 Ajustar el esfuerzo

El esfuerzo sugerido no permanece fijo. El servicio comienza con valor cero: la defensa está inactiva. Cada cinco minutos modifica la sugerencia según el estado de la cola: aumenta si ha permanecido llena o si se han descartado solicitudes con el esfuerzo sugerido, y la reduce en caso contrario. La lógica está inspirada en el control de congestión de TCP (incremento aditivo, reducción multiplicativa). A su vez, los clientes aumentan su esfuerzo cuando un intento de conexión agota su tiempo de espera. El resultado es un sistema de realimentación de dos bucles, en el que los usuarios más persistentes pueden invertir más esfuerzo para ganar prioridad.

15.5 Laboratorio: activar las defensas

Laboratorio 9: Límites anti-DoS y prueba de trabajo en la red de laboratorio

Objetivo. Activar límites de creación de circuitos en un repetidor y superarlos; activar PoW en un servicio onion y leer sus parámetros; estudiar la comprobación Blake2b del esfuerzo mediante `labs/es/codigo/torlab/pow.py`.
Requisitos previos. Laboratorios 1–8; red de laboratorio en funcionamiento, con imagen compilada mediante `--enable-gpl`.
Tiempo estimado. 2 horas.

Advertencia: Solo en tu propia red

Este laboratorio genera deliberadamente tráfico anómalo. Ejecútalo exclusivamente en la red privada del libro. Hacer lo mismo contra repetidores o servicios públicos perjudica a otras personas y constituye un ataque ilegal en numerosas jurisdicciones (capítulo 25).

Paso 1: activar límites en un repetidor. Añadimos al `torrc` de `test003r` límites muy bajos para que se activen rápidamente y le ordenamos recargar la configuración con la señal `HUP`:

```
DoSCircuitCreationEnabled 1
DoSCircuitCreationMinConnections 2
DoSCircuitCreationRate 1
DoSCircuitCreationBurst 10
DoSCircuitCreationDefenseType 2
DoSCircuitCreationDefenseTimePeriod 60 seconds
```

El tipo de defensa 2 significa «rechazar circuitos nuevos».

Paso 2: superar los límites. Abrimos uno o dos canales y solicitamos rápidamente 30 circuitos mediante **`CREATE_FAST`**. Para disponer de contadores independientes en cada ejecución, originamos conexiones desde diferentes direcciones de bucle local (127.0.0.2 y 127.0.0.3), utilizando el parámetro `source_address` de `open_channel`:

```
origen 127.0.0.2, 2 canales: 9 correctos, 21 DESTROY(5); primer rechazo: 10
origen 127.0.0.3, 1 canal: 30 correctos
origen 127.0.0.1, 2 canales: 9 correctos, 21 DESTROY(5); primer rechazo: 10
```

Se desprenden tres observaciones:

- El repetidor rechaza las solicitudes con una celda **`DESTROY`** de razón 5, `RESOURCELIMIT`.
- Con un *capacidad de ráfaga* (*burst*) de 10, tienen éxito nueve circuitos y no diez. El código (`cc_has_exhausted_circuits`, en `src/core/or/dos.c`) marca al cliente en cuanto el depósito llega a cero, así que la décima solicitud ya se rechaza.
- Con un único canal no se activa la restricción: se necesitan al menos `MinConnections` conexiones simultáneas. Así se evita penalizar a un cliente corriente, que normalmente mantiene una sola conexión abierta hacia su guardia.

Habíamos previsto que las conexiones procedentes de 127.0.0.1, dirección de los repetidores del laboratorio, estarían exentas. El experimento refutó esa hipótesis, y `dos.c` confirma que no existe semejante excepción para los circuitos. Al finalizar, restauramos el `torrc` original.

Paso 3: PoW del servicio onion. Añadimos la siguiente línea después de las opciones `HiddenService…` en el `torrc` de `test009h`:

```
HiddenServicePoWDefensesEnabled 1
```

Después de la señal `HUP`, el servicio publica un descriptor nuevo. Al descargarlo y descifrarlo con el laboratorio 8 aparece esta línea en la segunda capa:

```
pow-params v1 tx2Mtmzw5GHgWjcDoivTvWzwQPNZONwMwSMddOc+ZVQ= 0 2026-09-23T18:20:53
```

Contiene el esquema `v1`, la semilla base64, el esfuerzo sugerido **cero** (todavía no hay congestión) y la caducidad, una hora y cincuenta minutos tras la publicación, dentro del intervalo de 105–120 minutos.

Paso 4: el esfuerzo es lineal. El módulo `pow.py` implementa la comprobación Blake2b y simula soluciones Equi-X con bytes aleatorios, cuya distribución sirve para esta prueba:

```
def check_value(service_id, seed, nonce, effort, solution):
    challenge = P_STRING + service_id + seed + nonce + struct.pack(">I", effort)
    h = hashlib.blake2b(challenge + solution, digest_size=4).digest()
    return int.from_bytes(h, "big")

def valid_effort(r, effort):
    return r * effort <= UINT32_MAX
```

La media de intentos hasta el éxito en 400 ensayos por nivel de esfuerzo fue:

```
esfuerzo     1: 1.0
esfuerzo    10: 9.7
esfuerzo   100: 102.3
esfuerzo  1000: 1068.8
```

El número de intentos crece proporcionalmente al esfuerzo. El valor esperado es el esfuerzo porque cada intento tiene probabilidad $1/effort$ de éxito y el número de intentos sigue una distribución geométrica. Su desviación típica es aproximadamente igual a la media. Por ello, la media de 400 pruebas tiene una desviación relativa típica de $1/\sqrt{400} = 5\,\%$. Dos ejecuciones pueden diferir un diez por ciento aunque nada haya cambiado: al repetirlas obtuvimos 969,9 donde el listado indica 1068,8. Una media muestral debe interpretarse junto con su incertidumbre. Otro detalle: `blake2b` con `digest_size=4` no equivale a los primeros cuatro bytes de un Blake2b de 64 bytes, porque la longitud de salida modifica el estado inicial de la función. La especificación subraya este matiz, fácil de pasar por alto.

Resumen

- En Tor, los ataques DoS también amenazan el anonimato: deshabilitar repetidores a voluntad o hacerlos fallar puede redirigir tráfico hacia un adversario.
- La memoria se protege mediante `MaxMemInQueues`; al superarlo, tor vacía cachés y cierra circuitos que acumulan datos.
- Los guardias imponen límites por dirección a las conexiones y los circuitos de clientes mediante depósitos de tokens. Desactivadas en los valores predeterminados del código, estas defensas se activan y ajustan con el consenso.
- Los servicios onion sufren amplificación: iniciar una solicitud cuesta poco, atenderla cuesta mucho.
- La prueba de trabajo v1 combina Equi-X con Blake2b para producir esfuerzo lineal. El servicio prioriza solicitudes de mayor esfuerzo y ajusta el valor sugerido según la cola.

Ejercicios

Ejercicio 15.1 ⋆ Con `DoSCircuitCreationRate` igual a 2 y `Burst` igual a 60, como en el consenso público, ¿cuántos circuitos puede crear un cliente en un minuto partiendo con el depósito lleno? ¿Y en una hora? Supón que la defensa ya se ha activado: la dirección mantiene al menos `DoSCircuitCreationMinConnections` conexiones simultáneas. Por debajo de ese umbral no se aplica el límite.

Ejercicio 15.2 ⋆ ¿Por qué los límites por dirección se aplican solo a clientes y solo en el primer salto? ¿Qué sucedería si un intermedio intentara limitar circuitos «por dirección»?

Ejercicio 15.3 ⋆⋆ Estima la proporción entre el trabajo del cliente y el de la red para una solicitud de introducción: ¿cuántos circuitos, intercambios y celdas genera la respuesta del servicio?

Ejercicio 15.4 ⋆⋆ Compara la prueba de trabajo de Tor con un esquema que exija k bits cero iniciales en un hash. ¿Qué valores de esfuerzo puede expresar cada uno? ¿Por qué es preferible una escala lineal para el control mediante realimentación?

Ejercicio 15.5 ⋆⋆ Un cliente declara esfuerzo $E = 50$, pero por suerte encuentra de inmediato una solución con un valor R tan pequeño que también satisfaría $E = 5000$. ¿Puede presentarla afirmando que dedicó un esfuerzo de 5000? ¿Por qué?

Ejercicio 15.6 ⋆⋆⋆ En la red de laboratorio, establece `MaxMemInQueues` en su valor mínimo en un repetidor intermedio y construye un circuito a través de él para descargar un archivo grande *sin leer* los datos entrantes y sin enviar SENDME. ¿Qué ocurre con las celdas acumuladas? Busca mensajes del administrador de memoria en los registros del repetidor.

Ejercicio 15.7 ⋆⋆⋆ Escribe un simulador pequeño de la cola de prioridad de un servicio onion. Llegan solicitudes con esfuerzo aleatorio (muchas con cero y unas pocas con valores altos); el servicio atiende cierta cantidad por segundo y descarta la mitad de menor esfuerzo cuando la cola supera un umbral. Mide el tiempo medio de espera según el esfuerzo.

Lecturas complementarias

- *Tor denial-of-service mitigations* (dos-spec) y *Onion service proof-of-work* (hspow-spec) [Thee].
- R. Jansen, F. Tschorsch, A. Johnson y B. Scheuermann, *The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network* [Jan+14].
- El repositorio Equi-X de tevador y su diario de desarrollo, citados en la especificación.

Capítulo 16

Censura, puentes y transportes conectables

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; pt-spec v1; obfs4proxy 0.0.14 (Debian); documentación del Tor Project

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir las distintas maneras en que un censor puede bloquear Tor;
- explicar qué son los puentes y por qué resulta difícil distribuirlos;
- describir el protocolo por el que tor se comunica con un transporte conectable;
- distinguir las estrategias de obfs4, meek, Snowflake, WebTunnel y Conjure;
- operar un puente obfs4 en la red de laboratorio y observar lo que puede ver un censor (laboratorio 10).

Todo lo estudiado hasta ahora presupone que el cliente puede alcanzar la red. En muchos países no es posible: las conexiones a Tor están bloqueadas, a veces únicamente durante períodos de tensión política y a veces de forma permanente. Este capítulo concluye la parte II con las técnicas para **eludir la censura**, uno de los campos de actividad más intensos del Tor Project en los últimos años.

16.1 Cómo se bloquea Tor

Un censor, normalmente un gobierno que controla los proveedores de conectividad de un país, dispone de distintas herramientas:

Bloqueo por dirección La lista de repetidores es pública: constituye el consenso (capítulo 12). Un censor puede descargarlo y bloquear todas las direcciones IP, o simplemente las de las autoridades y los repetidores de donde los clientes obtienen el consenso.

Bloqueo de sitios web Bloquear `torproject.org` impide descargar Tor Browser, aunque la red Tor en sí siga accesible.

Reconocimiento del protocolo La *inspección profunda de paquetes* (*deep packet inspection*, DPI) puede identificar conexiones Tor por características del intercambio TLS, los tamaños de paquete o los tiempos, incluso si todavía no se conoce la dirección de destino.

Sondeo activo Si la DPI sugiere que un servidor podría ser un repetidor no publicado, el censor puede conectarse e intentar hablar el protocolo Tor. Si recibe respuesta, puede bloquearlo.

Lista de permitidos En casos extremos, un censor autoriza únicamente los protocolos o destinos incluidos expresamente en una lista y bloquea todo lo demás.

Cada técnica tiene una contramedida, y la competencia entre censores y desarrolladores continúa. Proyectos como OONI miden dónde y cómo se bloquea Tor (capítulo 24).

16.2 Puentes

La respuesta al bloqueo por direcciones es el **puente** (*bridge*): un repetidor que no aparece en el consenso público. Actúa como guardia (apartado 13.3). El cliente lo utiliza como primer salto y el circuito continúa después normalmente por la red. Como su dirección no es pública, un censor no puede bloquearla simplemente leyendo una lista. Los puentes publican descriptores ante una **autoridad de puentes**, que comprueba su accesibilidad y mantiene una lista sin hacerla pública. Las estadísticas publicadas utilizan el resumen *de un resumen* de la identidad para no exponerla.

Un cliente configura un puente mediante una *línea de puente*. En la red de laboratorio, el cliente `test011bc` tiene esta configuración en `torrc`:

```
UseBridges 1
Bridge obfs4 127.0.0.1:9909 2557828B596B33AB02075FE7EAD86F2DC3CEAFD5 cert=4sKDQih...
    iat-mode=0
ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy
```

La línea indica el tipo de transporte, la dirección, la huella de identidad del puente (para verificar la contraparte, capítulo 7) y los parámetros del transporte. El valor `cert=4sKDQih...` está *abreviado* para su presentación: no debe copiarse como configuración funcional. Hay que obtener el certificado completo y la huella de la instancia de puente creada por el lector.

16.2.1 El problema de la distribución

Un puente resulta útil mientras el censor desconoce su dirección. Pero esta debe facilitarse a los usuarios, y un censor puede hacerse pasar por uno de ellos. No existe solución perfecta: el distribuidor debe entregar suficientes puentes a muchos usuarios legítimos y revelar el mínimo posible al censor. El Tor Project los distribuye mediante varios canales, cada uno con defensas contra la recopilación automatizada [Theg]:

- la página web `https://bridges.torproject.org/`;
- correo electrónico a `bridges@torproject.org`;
- un bot de Telegram (`@GetBridgesBot`);
- directamente desde Tor Browser, que detecta los bloqueos y solicita puentes adecuados para el país del usuario (*Connection Assist*, mediante el canal denominado *moat*);
- canales privados administrados por organizaciones que conocen personalmente a sus usuarios.

Durante años, BridgeDB asignó los puentes a los distintos canales. Fue sustituido por **rdsys**, cuya migración terminó en octubre de 2024 [The24b]. Los operadores pueden seleccionar un canal mediante `BridgeDistribution`.

16.2.2 Lox

Lox [TG23] aborda la distribución mediante un sistema de *reputación*. Cada usuario nuevo recibe un puente y una credencial anónima. Si ese puente permanece sin bloquearse, la credencial gana niveles con el tiempo, hasta un máximo de 4; desde el nivel 2 permite además invitar a otras personas. Si se bloquea el puente, el usuario puede obtener otro, pero pierde reputación. La premisa es que un censor que bloquea todos los puentes recibidos no puede progresar, mientras que un usuario honesto sí. Las credenciales utilizan técnicas criptográficas que impiden incluso al propio sistema saber quién invitó a quién, protegiendo el *grafo social* de los usuarios. Lox se está probando en versiones alfa de Tor Browser.

16.3 Transportes conectables

Ocultar una dirección no basta para evitar el reconocimiento del protocolo: el propio tráfico debe tener otro aspecto. En lugar de modificar `tor` cada vez que un censor aprende a reconocer una característica nueva, el Tor Project definió una interfaz para programas externos que transforman el tráfico: los **transportes conectables** (*pluggable transports*, PT) [pt-spec § Architecture Overview].

16.3.1 La arquitectura

Un PT es un proceso independiente iniciado por `tor` y, por ello, *gestionado*. Del lado del cliente, proporciona a `tor` un proxy SOCKS local. `tor` entrega al PT la conexión hacia el puente y el programa transforma el tráfico antes de enviarlo. Del lado del puente, el PT escucha en el puerto público, invierte la transformación y reenvía el tráfico al ORPort local de `tor`. Puede utilizar un pequeño protocolo adicional, el *Extended ORPort*, para indicar la dirección real del cliente a efectos estadísticos.

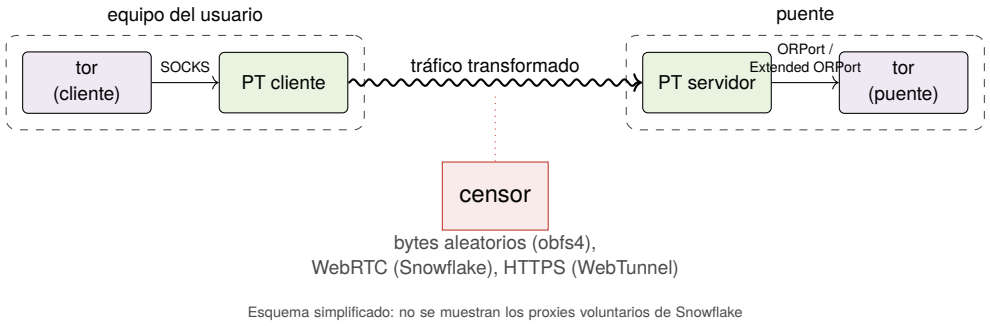


Figura 16.1: Arquitectura esquemática de un transporte conectable. En el cliente, tor se comunica con el PT mediante un proxy SOCKS local; en el puente, el PT entrega el tráfico reconstruido a tor mediante ORPort o, si está configurado, Extended ORPort. Se omiten los proxies voluntarios intermedios de Snowflake. El censor ve tráfico transformado en el enlace observado, pero aún puede identificarlo.

16.3.2 El protocolo entre tor y el PT

`tor` configura el PT con variables de entorno y el PT responde escribiendo líneas en su salida estándar [pt-spec § Specification]. Podemos comprobarlo iniciando manualmente `obfs4proxy` en el contenedor de laboratorio, como haría `tor`. En modo cliente:

```
$ env TOR_PT_MANAGED_TRANSPORT_VER=1 TOR_PT_STATE_LOCATION=/tmp/pt \
      TOR_PT_EXIT_ON_STDIN_CLOSE=1 TOR_PT_CLIENT_TRANSPORTS=obfs4 obfs4proxy
VERSION 1
CMETHOD obfs4 socks5 127.0.0.1:40057
CMETHODS DONE
```

El PT declara la versión del protocolo y anuncia el transporte `obfs4` mediante un proxy SOCKS5 en un puerto local elegido por él. En modo servidor, con variables que indican la dirección pública de escucha y el ORPort de destino:

```
$ env ... TOR_PT_SERVER_TRANSPORTS=obfs4 \
      TOR_PT_SERVER_BINDADDR=obfs4-127.0.0.1:19999 \
      TOR_PT_ORPORT=127.0.0.1:5104 obfs4proxy
VERSION 1
SMETHOD obfs4 127.0.0.1:19999 ARGS:cert=5
    W6vcbWbFGGZgabwm4C3WipezwnAfadoBhqfNVpY72xnkjB/Szczbxg8fLWJJ1uRlyjPSg,iat-mode=0
SMETHODS DONE
```

Los parámetros que siguen a `ARGS:` son los que necesita el cliente; aparecen en la línea de puente. El PT también los escribe en `obfs4_bridgeline.txt` dentro de su directorio de estado. La variable `TOR_PT_EXIT_ON_STDIN_CLOSE` ordena al PT terminar cuando `tor` cierra su entrada, evitando que sobreviva al proceso que lo inició. Por un descuido, el ejemplo de la especificación afirma que el PT escribe sus mensajes en «stdin»; en realidad, como acabamos de ver, los escribe en la salida estándar.

El comando del servidor con `env ...` es *esquemático*: los puntos suspensivos representan otras variables de entorno necesarias y no deben introducirse literalmente. El puerto SOCKS5 de la salida del cliente procede de una ejecución de referencia y no es fijo.

16.3.3 Lyrebird

El programa de transporte utilizado por Tor Browser es **lyrebird**, derivado del antiguo `obfs4proxy`. Implementa obfs4, meek, Snowflake y WebTunnel [Theg]. El laboratorio emplea `obfs4proxy` 0.0.14, disponible en los paquetes Debian y compatible todavía con obfs4.

16.4 Estrategias

Los transportes actuales siguen dos estrategias opuestas: *no parecerse a nada reconocible* o *parecerse a algo permitido*.

16.4.1 obfs4: no parecerse a nada reconocible

obfs4, diseñado por Yawning Angel, transforma el tráfico en una secuencia de bytes indistinguibles de datos aleatorios: sin cabecera reconocible, intercambio TLS ni tamaños de paquete constantes. Tres decisiones lo hacen eficaz:

- Su intercambio de claves es una variante de ntor (apartado 9.6) donde las claves públicas Curve25519 se codifican mediante Elligator2, técnica que hace que un punto de la curva sea indistinguible de una cadena aleatoria.
- El cliente debe conocer el parámetro `cert` del puente, que contiene su identidad y clave pública. Sin él, un observador no puede construir un intercambio válido y el puente no responde. Así se resiste al **sondeo activo**: un censor que se conecta por curiosidad no obtiene ninguna respuesta reconocible.

- El parámetro `iat-mode` (*inter-arrival time*) puede introducir retardos aleatorios y fragmentación para ocultar también los tiempos, a costa de rendimiento.

La limitación de obfs4 es precisamente su fortaleza: un censor que emplea listas de permitidos o bloquea «todo lo que parezca aleatorio» puede detenerlo. Esto ocurre en ciertos países.

16.4.2 meek: detrás de un gran servicio

meek [Fif+15] utiliza *domain fronting*. El cliente abre una conexión HTTPS hacia un gran proveedor de distribución de contenidos con un dominio inocuo en la parte visible de la conexión y especifica el verdadero destino —un servidor que sirve de puerta de entrada a Tor— solamente dentro de los datos cifrados. Bloquear meek obligaría a bloquear todo el servicio, con daños colaterales importantes. Sin embargo, la técnica depende de que los proveedores la permitan, y muchos la han imposibilitado en los últimos años. meek también es lento y costoso porque todo el tráfico atraviesa los servidores del proveedor.

16.4.3 Snowflake: numerosos proxies efímeros

Snowflake [Boc+24] utiliza miles de voluntarios que ejecutan pequeños proxies, por ejemplo como extensión del navegador o programa en un servidor. El cliente contacta con un *broker* mediante un canal difícil de bloquear, como domain fronting, para obtener un proxy; después se conecta con él mediante WebRTC, el protocolo utilizado en videollamadas del navegador. El proxy reenvía el tráfico a un puente Snowflake administrado por el Tor Project. Los proxies son numerosos y cambian constantemente. Bloquear todas sus direcciones es poco práctico y bloquear WebRTC supone bloquear también videollamadas.

16.4.4 WebTunnel: parecerse a un sitio web

WebTunnel, anunciado por el Tor Project en marzo de 2024, envuelve el tráfico en una conexión HTTPS con comportamiento semejante a WebSocket [The24a]. El puente puede situarse detrás de un sitio web ordinario, compartiendo dominio, dirección y puerto. Un proxy inverso reenvía al puente solo las solicitudes dirigidas a una ruta secreta y muestra el sitio normal a los demás. WebTunnel está destinado a censores que utilizan listas de permitidos, frente a los que obfs4 no funciona: presenta la «forma» de un protocolo permitido.

16.4.5 Conjure: ocultarse en el espacio de direcciones

Conjure [Fro+19] es un sistema de *refraction networking*. En vez de un puente en una dirección conocida, utiliza estaciones instaladas en proveedores de conectividad colaboradores. Estas estaciones observan tráfico dirigido a rangos de direcciones sin utilizar y lo desvían hacia Tor. El cliente se conecta a una dirección «fantasma» elegida entre millones de posibilidades. Bloquear Conjure obligaría al censor a bloquear partes enteras de Internet. Según la documentación del Tor Project, Conjure se ha probado con éxito, pero aún necesita trabajos de estabilización y está disponible solo en versiones alfa de Tor Browser y ciertas aplicaciones [Theg].

Advertencia: Ningún transporte funciona en todas partes

Todos los transportes tienen debilidades y su eficacia cambia según el país y el mes. En el foro del Tor Project, por ejemplo, algunos usuarios señalaron en abril de 2025 que obfs4, meek y Snowflake no funcionaban en China y que WebTunnel se conectaba, pero era

bloqueado rápidamente [Tor25]. Por ello Tor Browser ofrece distintos transportes y una función que sugiere el más adecuado para cada país.

16.5 Laboratorio: un puente obfs4 visto por un censor

Laboratorio 10: Un puente obfs4 en la red de laboratorio

Objetivo. Iniciar una red con un puente obfs4 y un cliente que lo utilice; capturar y comparar tráfico de una conexión Tor ordinaria y otra obfs4.
Requisitos previos. Red de laboratorio (capítulo 19). La imagen incluye `obfs4proxy` y `snowflake-client` de los paquetes Debian.
Tiempo estimado. 1–2 horas.

Paso 1: la red con puente. El archivo `labs/es/laboratorio/chutney/networks/book-bridge` define una topología que añade a la red del libro una autoridad de puentes (`test003ba`), un puente obfs4 (`test009br`) y un cliente que lo utiliza (`test011bc`). Desde la raíz del repositorio:

```
$ make -C labs/es/laboratorio clean
$ make -C labs/es/laboratorio start NETWORK=book-bridge
```

El registro de una ejecución de referencia anterior contiene:

```
[notice] new bridge descriptor 'test009br' (fresh): $2557828B...~test009br
[warn] Only one bridge (transport: 'obfs4') is configured. You should
       have at least two for conflux, for any transport that is not 'snowflake'.
[notice] Bootstrapped 100% (done): Done
```

El cliente obtuvo el descriptor del puente a través del propio puente y completó la conexión. La advertencia recuerda que Conflux (capítulo 31) utiliza dos circuitos paralelos y no puede hacerlo con un único puente. Una solicitud web a través del puerto SOCKS 9011 de este cliente funciona igual que en el cliente ordinario.

Paso 2: qué ve el censor. Dentro del contenedor capturamos el tráfico con `tcpdump` en el puerto obfs4 (9909) y en un ORPort normal (5104), mientras el cliente usa el puente y abrimos un canal hacia `test004r` con el laboratorio 1. Analizamos la captura mediante `tshark`. Este es el resultado de una de nuestras sesiones:

```
== ORPort 5104 (TLS), primeros bytes enviados por el cliente ==
1533  16030105f8010005f403033e1234a0401520d1c52b...
== obfs4 9909, primeros bytes enviados por el cliente ==
1957  b566475349099523c53005e1004a392160f15d473e...
== paquetes reconocidos como TLS ==
puerto 5104: 83
puerto 9909: 0
== tamaños de segmentos obfs4 (cliente -> puente) ==
1957 1677 1888 1891 1716 1716 3436 1395 1716 1748 1500 1194 ...
```

En el ORPort, los primeros bytes son inequívocos: `16 03 01` inicia un registro de intercambio TLS y Wireshark reconoce la conversación completa como TLS. El censor ve una conexión TLS que puede inspeccionar para localizar características específicas de Tor, y la dirección del repetidor también aparece en el consenso. Hacia el puente obfs4, los bytes carecen de

estructura reconocible, ningún paquete se identifica como TLS y los tamaños de segmentos no reflejan las celdas de 514 bytes. El censor solo ve una secuencia aparentemente aleatoria hacia una dirección desconocida.

Paso 3: sondear el puente. Intentamos conectar al puerto 9909 del puente mediante el laboratorio 1, como haría un censor que sospecha que allí funciona un repetidor:

```
TimeoutError: The handshake operation timed out
```

El puente no responde *en absoluto*: no envía mensajes de error ni bytes, y nuestro cliente abandona después de 15 segundos. Para obfs4, el TLS ClientHello enviado constituye un intercambio inválido, cuya respuesta es el silencio. Un censor que sondea la dirección no aprende nada, ni siquiera que allí escucha un servicio que habla otro protocolo. Sin el parámetro `cert`, tampoco puede construir un intercambio obfs4 válido.

Resumen

- Un censor puede bloquear Tor por dirección, reconocer el protocolo, sondear servidores sospechosos o admitir solo tráfico expresamente autorizado.
- Los puentes son repetidores no publicados. Su distribución, gestionada actualmente mediante rdsys y explorada con el mecanismo de reputación Lox, debe equilibrar la disponibilidad para los usuarios y el riesgo de descubrimiento por los censores.
- Los transportes conectables son procesos externos que transforman el tráfico; `tor` los configura mediante variables de entorno y lee sus respuestas (`CMETHOD`, `SMETHOD`).
- obfs4 hace que el tráfico parezca aleatorio y resiste el sondeo activo; meek, Snowflake, WebTunnel y Conjure lo ocultan detrás de servicios, protocolos o direcciones cuyo bloqueo tendría un coste elevado para el censor.
- Ningún transporte funciona en cualquier lugar y para siempre.

Ejercicios

Ejercicio 16.1 ⋆ ¿En qué se diferencia un puente de un guardia? ¿Por qué un cliente que utiliza puentes no utiliza también un guardia público?

Ejercicio 16.2 ⋆ Clasifica obfs4, meek, Snowflake, WebTunnel y Conjure según su estrategia («no parecerse a nada reconocible» o «parecerse a algo permitido») y según lo que tendría que bloquear un censor para detener cada uno.

Ejercicio 16.3 ⋆⋆ Describe cómo podría descubrir un censor un puente obfs4 observando únicamente el tráfico, sin sondearlo activamente. ¿Qué características estadísticas podría emplear y qué hace `iat-mode` para contrarrestarlas?

Ejercicio 16.4 ⋆⋆ ¿Por qué un censor no puede avanzar por los niveles de reputación de Lox? ¿Qué podría hacer para eludir el mecanismo y qué lo haría costoso?

Ejercicio 16.5 ⋆⋆ Calcula la entropía de Shannon (bits por byte) de los primeros 10 KB capturados de una conexión obfs4 y una TLS ordinaria en el laboratorio. ¿Qué resultados esperas y por qué podrían diferir poco? Entonces, ¿qué hace reconocible a TLS?

Ejercicio 16.6 ⋆⋆⋆ Intenta ejecutar el laboratorio 1 contra el puerto obfs4 de tu puente. ¿Qué error se produce? Después utiliza `obfs4proxy` como cliente con un parámetro `cert` incorrecto: ¿qué observas en los registros del puente?

Lecturas complementarias

- *Pluggable Transport Specification (Version 1)* (pt-spec) [Thee].
- L. Tulloch e I. Goldberg, *Lox: Protecting the Social Graph in Bridge Distribution* [TG23].
- D. Fifield y otros, *Blocking-resistant communication through domain fronting* [Fif+15].
- C. Bocovich y otros, *Snowflake, a censorship circumvention system using temporary WebRTC proxies* [Boc+24].
- S. Frolov y otros, *Conjure: Summoning Proxies from Unused Address Space* [Fro+19].
- El artículo del Tor Project que presenta WebTunnel [The24a] y las páginas de asistencia sobre puentes [Theg].

Parte III

Tor en la práctica

Capítulo 17

Instalación y torrc

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; manual `tor(1)` de la misma versión

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- elegir la forma más apropiada de instalar Tor según sus necesidades;
- leer y escribir un archivo `torrc` y localizar la documentación de cada opción;
- configurar tor como cliente, con puertos SOCKS, aislamiento de flujos y un puerto de control protegido;
- comprobar una configuración antes de utilizarla y diagnosticar problemas mediante registros.

Con este capítulo comienza la parte práctica del libro. En los capítulos anteriores hemos explicado cómo funciona el protocolo; ahora aprenderemos a utilizar y configurar el programa que lo implementa. Todas las opciones mencionadas están documentadas en el manual de tor (`man tor`), que constituye la referencia autorizada: los valores predeterminados pueden cambiar entre versiones.

17.1 ¿Qué Tor conviene instalar?

Tor Browser Para navegar por la web, casi siempre es la elección apropiada. Incluye una versión de tor ya configurada y un navegador modificado para evitar revelar información que debilitaría las protecciones de la red (capítulo 23). Utilizar tor con un navegador convencional, en cambio, expone al usuario a muchas formas de identificación.

El paquete `tor` Para utilizar Tor con otras aplicaciones o administrar un repetidor o servicio onion. Las distribuciones Linux lo incluyen, aunque no siempre en su versión más reciente. El Tor Project mantiene un repositorio de paquetes para Debian y Ubuntu y distribuye el *Tor Expert Bundle*, que incluye tor y transportes conectables, para Windows y macOS.

Compilación desde el código fuente Como en el laboratorio del libro (apartado 19.3), para estudiar tor o instalar una versión exacta.

Arti El cliente escrito en Rust (capítulo 30), cada vez más maduro, pero con una configuración diferente.

Sea cual sea el método elegido, descarga el programa únicamente de la página oficial y verifica su firma. Una versión de tor modificada por un adversario anularía sus protecciones.

17.2 El archivo torrc

tor lee su configuración de un archivo de texto, normalmente `/etc/tor/torrc` cuando funciona como servicio del sistema. El formato es sencillo: una opción por línea, seguida de sus valores. Las líneas que comienzan por # son comentarios. Algunas opciones pueden aparecer varias veces, como los distintos puertos SOCKS. También pueden indicarse opciones en la línea de órdenes con dos guiones iniciales:

```
$ tor --SocksPort 9150 --DataDirectory /tmp/tor-test
```

El manual documenta cientos de opciones; `tor --list-torrc-options` las enumera todas. Las distribuciones pueden definir un segundo archivo de valores predeterminados mediante `--defaults-torrc`. El archivo indicado con `-f` tiene prioridad.

17.2.1 Comprobar antes de utilizar

Tres órdenes evitan muchas sorpresas:

```
$ tor --verify-config -f /etc/tor/torrc     # comprobar si la configuración es válida
Configuration was valid
$ tor --dump-config short -f /etc/tor/torrc # opciones diferentes de los valores
    predeterminados
$ tor --list-modules                        # módulos incluidos al compilar
relay: yes
dirauth: yes
dircache: yes
pow: yes
```

Como vimos en capítulo 15, la última orden permite saber, por ejemplo, si está disponible la prueba de trabajo para servicios onion.

17.3 Una configuración de cliente

Un cliente apenas necesita configuración. De hecho, el archivo mínimo está vacío: por defecto, tor abre un puerto SOCKS 9050 en localhost. Este es un ejemplo más explícito y comentado para un servicio tor cuyo usuario tiene permisos en `/var/lib/tor` y `/var/log/tor`; no puede ejecutarse sin más como cualquier usuario sin privilegios:

```
1  # Directorio de estado, claves y caché
2  DataDirectory /var/lib/tor
3
4  # Puerto SOCKS para aplicaciones (¡solo localhost!)
5  SocksPort 127.0.0.1:9050
6
7  # Puerto de control, protegido con cookie (capítulo 18)
8  ControlPort 127.0.0.1:9051
9  CookieAuthentication 1
10
11 # Registro al nivel notice
12 Log notice file /var/log/tor/notice.log
```

Advertencia: No expongas el puerto SOCKS

El manual es explícito: SOCKS no está cifrado ni autenticado. Un `SocksPort` que escuche en una dirección accesible desde la red convierte el equipo en un proxy abierto y expone

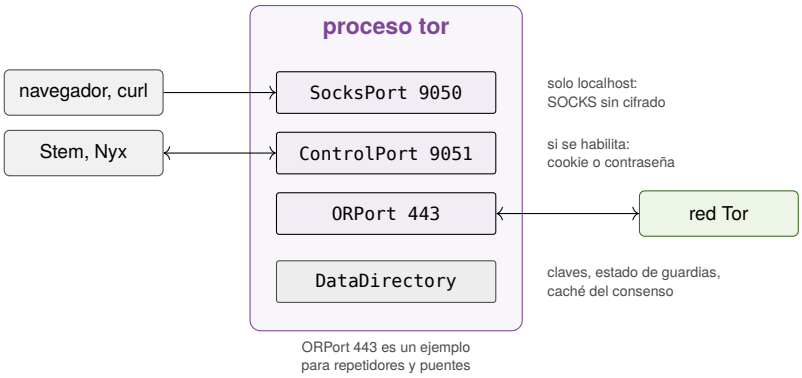


Figura 17.1: Ejemplo de los puertos y el directorio de un proceso tor. SOCKS en 9050 atiende aplicaciones locales; ControlPort 9051 es opcional y requiere autenticación. Ninguno debe exponerse a la red. DataDirectory es un directorio, no un puerto. ORPort 443 es una opción para un repetidor o puente, no un número obligatorio.

en texto claro las solicitudes de las aplicaciones dentro de la red local.

17.3.1 Aislamiento de flujos

Varios flujos pueden compartir un circuito. Es eficiente, pero permite que una salida relacione actividades distintas del mismo usuario. Si un correo personal y un foro seudónimo comparten circuito, la salida puede deducir que proceden de la misma persona. Por eso tor ofrece **aislamiento de flujos**: reglas que determinan cuáles pueden compartir circuito. Se expresan como opciones de `SocksPort`:

`IsolateClientAddr` Los flujos de diferentes direcciones de cliente no pueden compartir circuitos (activado por defecto).

`IsolateSOCKSAuth` Los flujos con diferentes credenciales SOCKS no pueden compartir circuitos (activado por defecto).

`IsolateDestAddr`, `IsolateDestPort` Aislamiento por destino.

`IsolateClientProtocol` Aislamiento por protocolo (SOCKS4, SOCKS5, DNS...).

Los diferentes puertos SOCKS también están aislados entre sí por defecto: a cada puerto de escucha se le asigna un *grupo de sesión* independiente. La opción `SessionGroup` puede modificar este comportamiento y permitir que compartan circuitos. El mecanismo más utilizado es el aislamiento mediante autenticación SOCKS: Tor Browser envía credenciales distintas para cada sitio visitado y obtiene así un circuito diferente por sitio. Lo observaremos en capítulo 18.

Los circuitos no se utilizan indefinidamente. Cuando transcurre `MaxCircuitDirtiness`, diez minutos por defecto desde el primer uso, el circuito deja de aceptar flujos nuevos y tor construye otro.

17.3.2 Elegir repetidores: precaución

Las opciones `EntryNodes`, `ExitNodes` y `ExcludeNodes` permiten imponer o excluir repetidores mediante huella, sobrenombre, dirección o código de país. `StrictNodes` hace obligatoria la exclusión mediante `ExcludeNodes` incluso si impide el funcionamiento. Estas herramientas

son útiles para experimentos y determinadas situaciones. Sin embargo, emplearlas simplemente para «sentirse más seguro» suele empeorar las cosas: restringir la selección reduce la diversidad de rutas, hace al usuario distinguible y debilita las protecciones de los guardias y de la selección ponderada (capítulo 13).

17.3.3 Otros puertos

`DNSPort` tor responde solicitudes DNS y resuelve nombres a través de la red (apartado 11.1.4).

`TransPort` Acepta conexiones TCP *transparentes* redirigidas por el cortafuegos del sistema. Sistemas como Whonix emplean este mecanismo para canalizar tráfico de aplicaciones por Tor (capítulo 23).

`AutomapHostsOnResolve` Asigna direcciones virtuales a nombres `.onion` solicitados por DNS, lo que permite que aplicaciones sin compatibilidad SOCKS accedan a servicios onion mediante `TransPort`.

17.4 El puerto de control

`ControlPort` permite a otros programas controlar tor mientras se ejecuta: consultar su estado, solicitar circuitos nuevos o crear servicios onion dinámicamente. Es extremadamente potente y debe protegerse. Existen dos métodos de autenticación:

Cookie Con `CookieAuthentication 1`, tor escribe un archivo de 32 bytes aleatorios en el directorio de datos. Cualquiera que pueda leerlo podrá autenticarse. La seguridad depende, por tanto, de los permisos de archivo.

Contraseña Con `HashedControlPassword`, tor guarda un resumen de contraseña generado con `tor --hash-password`:

```
$ tor --hash-password test
16:458528FE0C0AF36260B5071446823FB789212185D7EE7F615BC120D55D
```

El hash mostrado corresponde a la contraseña de demostración `test` y se ha comprobado utilizando una sal fija para este ejemplo impreso. Una ejecución real utiliza una sal aleatoria, por lo que normalmente da un resultado distinto incluso con la misma contraseña. No reutilices la contraseña ni el hash del ejemplo para un servicio real.

Igual que SOCKS, el puerto de control nunca debe exponerse a la red. El protocolo de control es el tema de capítulo 18.

17.5 Registros

tor registra su actividad con diversos grados de detalle: `err`, `warn`, `notice`, `info` y `debug`. El nivel `notice` basta para el funcionamiento diario. `info` y `debug` son útiles para investigar el protocolo, como hicimos en los laboratorios, pero producen muchos datos. La opción `SafeLogging`, activada por defecto, sustituye direcciones y destinos por `[scrubbed]`. Un archivo de registro olvidado no debe convertirse en el historial de actividad del usuario. La red de laboratorio desactiva deliberadamente esta protección (`SafeLogging 0`) para examinar todos los detalles.

Los mensajes más útiles al iniciar son los del *bootstrap*. Muestran las etapas de conexión a la red y terminan con `Bootstrapped 100% (done): Done`. Si el inicio se detiene en un porcentaje, esa etapa indica dónde investigar. Por ejemplo, quedarse atascado al conectar con el primer repetidor suele apuntar a censura o a un cortafuegos y sugiere utilizar puentes (capítulo 16).

Resumen

- Para navegar por la web utiliza Tor Browser; el paquete `tor` sirve para otras aplicaciones, repetidores y servicios onion.
- Un `torrc` contiene una opción por línea. El manual es la referencia autorizada y `--verify-config` permite validar la configuración.
- Mantén los puertos SOCKS y de control en localhost. Protege el puerto de control con cookie o contraseña.
- El aislamiento impide que actividades distintas compartan circuito. Tor Browser lo consigue mediante credenciales SOCKS diferentes para cada sitio.
- Forzar o excluir repetidores normalmente reduce la seguridad; `SafeLogging` protege los registros.

Ejercicios

Ejercicio 17.1 ⋆ Dentro del contenedor de laboratorio, crea un `torrc` con una errata en el nombre de una opción y compruébalo con `--verify-config`. ¿Qué mensaje recibes? Después ejecuta `--dump-config short` sobre el `torrc` del cliente de laboratorio.

Ejercicio 17.2 ⋆⋆ Un usuario conecta un cliente de correo con su identidad real y un foro seudónimo a través del mismo `SocksPort`. ¿Qué podría deducir una salida? Propón dos configuraciones para evitarlo.

Ejercicio 17.3 ⋆⋆ ¿Por qué `ExitNodes {it}`, que obliga a utilizar salidas italianas, reduciría el anonimato de un usuario italiano que visita un sitio de Italia? Razona según el modelo de amenazas de capítulo 4.

Lecturas complementarias

- El manual de tor, `man tor`, y `doc/man/tor.1.txt` en el código fuente.
- Las páginas de asistencia del Tor Project sobre instalación y repositorios de paquetes, `https://support.torproject.org/`.

Capítulo 18

Puerto de control y Stem

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; control-spec; Stem 1.8.2

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- comunicarse con tor mediante el puerto de control, incluso manualmente;
- utilizar los comandos principales: `GETINFO`, `SETCONF`, `SIGNAL`, `ADD_ONION` y las suscripciones a eventos;
- escribir programas Python con la biblioteca Stem;
- observar el aislamiento de flujos y el efecto de una identidad nueva en un sistema en funcionamiento (laboratorio 11).

Tor Browser muestra el circuito utilizado por la página actual; herramientas como Nyx indican el ancho de banda y las conexiones de un repetidor; los sistemas de medición de red crean circuitos a petición. Todas estas aplicaciones utilizan la misma interfaz: el **protocolo de control** de tor, accesible a través de `ControlPort` (apartado 17.4).

18.1 El protocolo de control

Se trata de un protocolo de texto semejante a los utilizados por el correo electrónico [control-spec § Message format]. El cliente envía un comando por línea y tor responde con líneas que comienzan por un código de tres cifras. `250` significa éxito. Un guion después del código (`250-`) indica que la respuesta continúa y un espacio (`250 `) indica su última línea; el signo `+` introduce un bloque multilínea terminado por una línea que solo contiene un punto. Los códigos `5xx` indican errores y `650` señala eventos asíncronos.

Podemos probarlo en el cliente de la red de laboratorio mediante una conexión TCP sencilla al puerto 8008:

```
> PROTOCOLINFO 1
250-PROTOCOLINFO 1
250-AUTH METHODS=COOKIE,SAFECOOKIE COOKIEFILE="/lab/net/nodes/008c/control_auth_cookie
    "
250-VERSION Tor="0.4.9.13"
250 OK
> AUTHENTICATE 52977550adebb73908610d7fa3355aba3fbdbdaef01d5ef86523d3232b87e5fa
250 OK
> GETINFO status/bootstrap-phase
250-status/bootstrap-phase=NOTICE BOOTSTRAP PROGRESS=100 TAG=done SUMMARY="Done"
```

```
250 OK
```

La transcripción procede de una ejecución de referencia anterior: las rutas, la versión y la cookie dependen de cada instancia. **Nunca publiques una cookie de control real**: permitiría autenticarse a quien la obtuviera. `PROTOCOLINFO` es uno de los pocos comandos permitidos antes de autenticarse (los demás son `AUTHCHALLENGE`, `AUTHENTICATE` y `QUIT`). Informa sobre los métodos admitidos y la ruta del archivo de cookie. Con la autenticación `COOKIE`, el cliente envía el contenido del archivo en hexadecimal. `SAFECOOKIE` evita transmitir la propia cookie: cliente y tor intercambian valores aleatorios y demuestran conocerla mediante HMAC, lo que impide que un programa que suplante a tor pueda robarla.

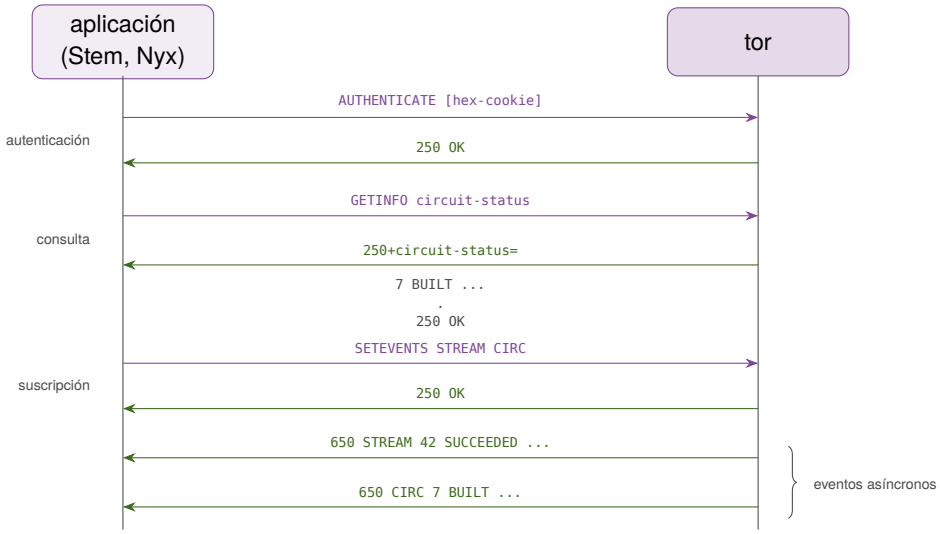


Figura 18.1: Sesión simplificada del protocolo de control: tras la autenticación COOKIE (representada con un marcador), GETINFO circuit-status puede devolver datos multilínea delimitados por un punto aislado y terminados con 250 OK. SETEVENTS recibe 250 OK; los eventos 650 son asíncronos y pueden intercalarse entre respuestas a otros comandos. Los identificadores y datos son ilustrativos.

18.1.1 Los circuitos vistos por tor

`GETINFO circuit-status` enumera los circuitos del cliente. Una entrada abreviada:

```
3 BUILT $50160F8F...~test004r,$12415FEB...~test005r,$9449F36E...~test007e
  BUILD_FLAGS=NEED_CAPACITY,NEED_UPTIME PURPOSE=CONFLUX_LINKED
  CONFLUX_ID=DE224696F58F72950D70DDA2D7981726 CONFLUX_RTT=42538
```

Cada repetidor se representa mediante la huella RSA de su identidad y un sobrenombre. Los circuitos Conflux aparecen por parejas con el mismo `CONFLUX_ID`; las dos «ramas» comienzan en guardias distintos. tor mide sus tiempos de ida y vuelta (`CONFLUX_RTT`, expresado en microsegundos) para decidir por cuál transmitir los datos (capítulo 31). Un detalle del código (`src/feature/control/control_fmt.c`) merece atención: `CONFLUX_ID` expone únicamente *la mitad* del identificador secreto de la pareja. Quien conociera el valor completo podría añadir ramas desde una salida e inyectar datos.

18.1.2 Comandos principales

Comando	Función
`GETINFO`	consulta información: versión, estado de conexión, circuitos, flujos, consenso, descriptores…
`GETCONF`, `SETCONF`	consulta y modifica la configuración en ejecución
`SIGNAL`	envía señales: `NEWNYM` (identidad nueva), `RELOAD`, `SHUTDOWN`…
`SETEVENTS`	se suscribe a eventos: `CIRC`, `STREAM`, `BW`, `HS_DESC`…
`EXTENDCIRCUIT`	crea un circuito con la ruta elegida por el controlador
`ATTACHSTREAM`	asigna un flujo a un circuito (con `__LeaveStreamsUnattached`)
`ADD_ONION`, `DEL_ONION`	crea y elimina servicios onion sin modificar `torrc`
`HSFETCH`	descarga un descriptor de servicio onion (utilizado en apartado 14.9)

Advertencia: Controlar el puerto es controlar tor

El puerto de control permite ordenar a tor qué repetidores utilizar, crear circuitos de un solo salto y revelar el destino de cada flujo. Un programa malicioso que pueda acceder a él puede destruir el anonimato del usuario. Sistemas como Tails y Whonix colocan por ello un filtro (onion-grater) entre las aplicaciones y el puerto de control, permitiendo exclusivamente los comandos necesarios.

18.2 Stem

Stem es la biblioteca Python del Tor Project para el protocolo de control. Oculta los detalles del formato de transmisión, interpreta respuestas y eventos y proporciona objetos prácticos. Su versión más reciente en la fecha de referencia, 1.8.2, es de junio de 2023. Funciona con las versiones actuales de tor y se emplea en numerosas herramientas, pero lleva tiempo sin una versión nueva. Algunas funciones recientes requieren comandos en bruto (`Controller.msg`). Arti dispone de otra interfaz, Arti RPC (capítulo 30).

Este es un ejemplo mínimo, ya utilizado en apartado 19.7.3:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                  # encuentra la cookie o pide la contraseña
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.purpose, [name for _, name in circ.path])
```

Registramos una función que recibirá eventos. El siguiente ejemplo es un *programa autónomo*: registra el listener mientras el controlador permanece abierto y espera diez segundos para recibir eventos. Ese tiempo es solo ilustrativo:

```
import time
from stem.control import Controller, EventType

def on_circuit(event):
    print(event.id, event.status, event.purpose)

with Controller.from_port(port=8008) as c:
    c.authenticate()
    c.add_event_listener(on_circuit, EventType.CIRC)
    time.sleep(10)
```

Stem ejecuta la función desde su hilo de eventos. Solo dispone de uno y llama secuencialmente a todas las funciones registradas: una devolución de llamada lenta retrasa los eventos posteriores. Si el procesamiento requiere mucho cálculo, delégalo en otro hilo.

18.3 Laboratorio: aislamiento de flujos en directo

Laboratorio 11: Aislamiento de flujos e identidad nueva

Objetivo. Utilizar Stem con el cliente de laboratorio para comprobar qué solicitudes comparten circuito según sus credenciales SOCKS y qué cambia después de `SIGNAL NEWNYM`.
Requisitos previos. Red de laboratorio en ejecución y servidor web local en el puerto 8080. El script es `labs/es/laboratorio/aislamiento.py`, que se ejecuta dentro del contenedor (incluye Stem y `curl`).
Tiempo estimado. 1 hora.

Paso 1: registrar flujos. Nos suscribimos a los eventos `STREAM` y, cuando un flujo tiene éxito, anotamos a qué circuito quedó vinculado. Filtrar por destino es esencial: tor abre flujos también para tareas propias, como descargar documentos de directorio.

```
asignaciones = []
def al_recibir_flujo(evento):
    if es_flujo_de_prueba(evento):
        asignaciones.append((evento.id, evento.circ_id, evento.target))

controlador.add_event_listener(al_recibir_flujo, EventType.STREAM)
```

Este fragmento procede de `labs/es/laboratorio/aislamiento.py`: el script completo define `es_flujo_de_prueba` y mantiene abierto el controlador `controlador` dentro de `ejecutar()`. Ejecuta el script completo, no el fragmento por separado.

Paso 2: solicitudes con credenciales diferentes. Enviamos cuatro solicitudes con `curl`: dos con credenciales SOCKS `site-a.example:x1`, otra con `site-b.example:x2` y una sin credenciales. Tor Browser utiliza el dominio principal del sitio como nombre de usuario SOCKS para cada pestaña:

```
$ curl --proxy-user site-a.example:x1 --socks5-hostname 127.0.0.1:9008 http
    ://127.0.0.1:8080/
```

Paso 3: resultados. Esta salida es una transcripción de referencia de una ejecución anterior; los identificadores de Conflux, las rutas y el retraso de NEWNYM no son valores fijos.

```
credenciales site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credenciales site-a.example  -> conflux AADCE71E  ['test003r', 'test004r', 'test007e']
credenciales site-b.example  -> conflux 31B1E3E0  ['test003r', 'test004r', 'test007e']
credenciales -               -> conflux 35FD16D0  ['test003r', 'test004r', 'test007e']
(NEWNYM aplazado 6 s)
después de NEWNYM, site-a.example -> conflux F7011ABE
```

Las dos solicitudes del «sitio A» utilizan el mismo túnel aunque se realizaron en momentos distintos. Las del «sitio B» y sin autenticación utilizan otros. La ruta mostrada corresponde a

una de las dos ramas del túnel; aquí coinciden porque la red de laboratorio tiene pocos repetidores. Como veremos, debemos comparar el túnel y no un circuito individual. Las etiquetas de esta transcripción están localizadas conforme a `labs/es/laboratorio/aislamiento.py`; no se ha vuelto a ejecutar el experimento.

En profundidad: Un circuito no basta: túneles Conflux

Si repetimos el laboratorio muchas veces, las dos solicitudes con credenciales idénticas aparecen ocasionalmente en circuitos con números distintos. No es un fallo de aislamiento. Al consultar la lista completa de circuitos descubrimos que casi todos los del cliente tienen propósito `CONFLUX_LINKED` y aparecen *por parejas* con el mismo `CONFLUX_ID`. Son las dos ramas de un mismo túnel (apartado 31.4), y un evento `STREAM` puede señalar cualquiera de ellas. Por tanto, el aislamiento separa *túneles*, no ramas individuales. Cuando está disponible, el script del libro agrupa los circuitos por `CONFLUX_ID`.
Dos precauciones adicionales pueden verificarse en los registros:

- `SIGNAL NEWNYM` se acepta como máximo una vez cada diez segundos (`MAX_SIGNEWNYM_RATE` en `src/core/mainloop/mainloop.c`). Una solicitud anterior se retrasa y tor registra «Rate limiting NEWNYM request: delaying by N second(s)». Si no esperamos, el NEWNYM del experimento anterior podría aplicarse a mitad del siguiente.
- Los eventos `STREAM` también incluyen flujos que tor abre para sí mismo, como las descargas de documentos de directorio. Debemos filtrarlos por destino.

Incluso con estas precauciones, en una red pequeña sometida a gran carga tor puede seleccionar un túnel recién creado para la segunda solicitud. Por ello el verificador automatizado del libro repite el experimento.

Advertencia: Un error en el laboratorio

En el primer intento, las cuatro solicitudes utilizaron el mismo circuito, aparentemente porque el aislamiento no funcionaba. El problema estaba en nuestro script: habíamos puesto `--proxy-user` justo después de la opción `-o` de `curl`, que interpretó el argumento como nombre de archivo de salida. Nunca se enviaron credenciales. Antes de concluir que un sistema contradice su especificación, comprueba que el experimento hace realmente lo que crees.

Resumen

- El protocolo de control se basa en texto: comandos por línea, códigos de respuesta de tres cifras y eventos asíncronos con código 650.
- Autentícate con cookie, preferiblemente `SAFECOOKIE`, o contraseña. Quien controla el puerto controla tor.
- Stem proporciona una interfaz Python para comandos y eventos. Es estable, pero no ha publicado nuevas versiones desde 2023.
- El aislamiento de credenciales SOCKS proporciona un túnel distinto por sitio, que con Conflux está formado por dos circuitos. `NEWNYM` cambia a túneles nuevos como máximo una vez cada diez segundos.

Ejercicios

Ejercicio 18.1 ⋆ Interpreta la respuesta `552 Unrecognized key "pippo"` al comando `GETINFO pippo`. ¿En qué se distingue un código `5xx` de un evento `650`?

Ejercicio 18.2 ⋆⋆ Con Stem, crea un servicio onion temporal mediante `create_ephemeral_hidden_service` en el cliente de laboratorio, reenviando el puerto 80 al servidor local en el 8080. Accede a él mediante `curl`. ¿Dónde se guardan sus claves?

Ejercicio 18.3 ⋆⋆ Escribe un programa que muestre cada segundo el ancho de banda entrante y saliente del cliente mediante eventos `BW` mientras descargas un archivo grande por la red de laboratorio.

Ejercicio 18.4 ⋆⋆⋆ Con `__LeaveStreamsUnattached` 1, `EXTENDCIRCUIT` y `ATTACHSTREAM`, dirige una solicitud por una ruta elegida por ti, por ejemplo `test004r`, `test005r`, `test006e`. ¿Por qué es peligrosa para el anonimato esta capacidad en un uso real?

Lecturas complementarias

- *Tor's control protocol* (control-spec) [Thee]: todos los comandos y eventos.
- Documentación de Stem, `https://stem.torproject.org/`.

Capítulo 19

Una red Tor privada: Chutney y Shadow

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; commit de Chutney `6cc1588`; Shadow 3.3.0

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar por qué los experimentos con Tor deben ejecutarse en una red privada y no en la pública;
- construir e iniciar, mediante dos órdenes distintas, la red de laboratorio utilizada a lo largo del libro;
- identificar las opciones de configuración que distinguen una red de prueba de la red real;
- explorar los archivos de estado de un nodo, incluidas claves, documentos de consenso y registros;
- inspeccionar circuitos del cliente mediante Stem y enviar tráfico web y de servicios onion por la red;
- comprender cuándo conviene emplear un simulador como Shadow en vez de una red emulada como Chutney.

Este capítulo aparece en la parte III, pero conviene leerlo pronto: todos los laboratorios del libro, comenzando por el laboratorio 1 en capítulo 7, funcionan en la red que crearemos aquí. Quien siga la parte II puede leer primero apartados 19.1 y 19.3 y volver más tarde a los restantes apartados.

19.1 ¿Por qué una red privada?

La red Tor pública está formada por miles de repetidores administrados por voluntarios y la utilizan a diario millones de personas, muchas de las cuales dependen de ella para su seguridad. Experimentar en esta red plantea dos problemas: uno ético y otro práctico.

El problema ético. Cualquier experimento que mida, modifique o simplemente observe tráfico de la red real implica a usuarios que no han dado su consentimiento y podrían sufrir daños. Incluso algo aparentemente inocuo, como un repetidor que almacene más información de la necesaria, puede generar datos que permitan desanonimizar a alguien si llegan a divulgarse. Por ello, el Tor Project mantiene un *Tor Research Safety Board* que revisa los planes

experimentales de investigadores y recomienda un principio general: cuando una pregunta de investigación pueda resolverse sin involucrar a usuarios reales, así debe hacerse. Retomaremos estas pautas en capítulo 29.

El problema práctico. Tenemos poco control sobre la red pública: no podemos predecir qué repetidores se seleccionarán, los circuitos cambian, el consenso se actualiza cada hora y un experimento no puede repetirse en condiciones idénticas. En una red privada, en cambio, podemos elegir la cantidad de nodos, leer registros detallados, detener y reiniciar toda la red en minutos y repetir las pruebas hasta comprender los resultados.

Advertencia: Regla del laboratorio

Todos los ejercicios prácticos del libro que envíen celdas construidas manualmente, modifiquen el comportamiento de los nodos o capturen tráfico deben ejecutarse **exclusivamente** en la red privada. El código educativo `torlab` no protege el anonimato y nunca debe utilizarse contra la red pública.

19.2 ¿Emulación o simulación?

Existen dos maneras de construir una red Tor de laboratorio; ambas son utilizadas por el Tor Project.

Emulación Muchos procesos `tor` reales se ejecutan en un mismo ordenador y se comunican por la interfaz de bucle local. El tiempo es real, el código es real y el sistema operativo proporciona la red. Es el enfoque de **Chutney**, herramienta del Tor Project creada para pruebas de integración de tor y Arti.

Simulación Los programas reales se ejecutan dentro de un simulador de eventos discretos, que intercepta llamadas al sistema y proporciona una red y un reloj virtuales. Es el enfoque de **Shadow** [JH12], desarrollado para estudiar Tor a gran escala.

	Chutney (emulación)	Shadow (simulación)
Escala habitual	Unos pocos nodos o varias decenas	Cientos o miles de nodos
Tiempo	Real	Virtual y determinista
Red	Bucle local, sin latencia ni pérdidas	Topología con latencia y ancho de banda configurables
Reproducibilidad	Limitada por la planificación del sistema operativo	Completa con la misma semilla aleatoria
Uso habitual	Desarrollo, pruebas y enseñanza	Investigación sobre rendimiento y ataques
En este libro	Todos los laboratorios	apartado 19.9 y parte V

Cuadro 19.1: Comparación de Chutney y Shadow.

Chutney es la opción natural para aprender el protocolo. Es ligero, se inicia en pocos minutos y permite interactuar con nodos activos. Por eso nos centraremos aquí en Chutney y presentaremos Shadow al final.

19.3 El laboratorio del libro

Para evitar diferencias entre sistemas operativos, todo el laboratorio se distribuye como una imagen Docker descrita por el archivo `labs/es/laboratorio/Dockerfile`. La imagen:

- compila tor **desde el código fuente oficial** en la versión de referencia del libro, 0.4.9.13, con `--enable-gpl` para incluir el módulo de prueba de trabajo de los servicios onion (capítulo 15);
- instala Chutney en una revisión fijada, Stem y las herramientas de captura (`tcpdump` y `tshark`) utilizadas en capítulo 20.

Fijar las versiones importa: un libro que describe los mensajes de protocolo byte por byte debe procurar que los lectores observen los mismos bytes.

En profundidad: Cuando aparece otra versión de tor

Las salidas que se muestran en este capítulo y en la parte II proceden de la red de laboratorio con tor 0.4.9.13, la versión de referencia del libro. Algunas se registraron con su predecesora 0.4.9.12, publicada pocos días antes, y después se reprodujeron: el script `scripts/verify_labs.py`, que vuelve a ejecutar en vivo todos los laboratorios y comprueba sus resultados, obtiene los mismos resultados con ambas versiones (actualmente 53 comprobaciones, más 14 del laboratorio del puente). El script ejecuta los laboratorios de la edición original italiana, en `labs/it`, de cuyo código se ha traducido el de esta edición; en los listados, los nombres y los mensajes siguen el código de esta edición. Solo cambian los elementos que dependen de cada ejecución: claves, huellas, direcciones `.onion`, marcas temporales y tamaños de los certificados, que varían incluso de un repetidor a otro. El mismo script te permite comprobar, con la versión que utilices, que lo descrito en el libro sigue siendo cierto.

19.3.1 Requisitos

Se necesita Linux con Docker y aproximadamente 1 GB de espacio libre en disco. Docker Desktop funciona en macOS y Windows, pero el modo de red del anfitrión utilizado aquí se comporta de otra manera; en ese caso ejecuta los experimentos dentro del contenedor (apartado 19.7.4). La red utiliza puertos del anfitrión a partir de 5100, 7100, 8000 y 9000. Comprueba que estén libres.

19.3.2 Construir la imagen e iniciar la red

Desde la raíz del repositorio:

```
$ make -C labs/es/laboratorio image        # compilar tor y preparar Chutney (solo la
    primera vez)
$ make -C labs/es/laboratorio start        # configurar e iniciar la red, esperar al
    arranque
```

Estos comandos están definidos en `labs/es/laboratorio/Makefile`. La red utiliza puertos compartidos con el anfitrión: no inicies a la vez las redes de dos ediciones.

La construcción de la imagen lleva unos minutos. En el equipo utilizado para redactar el capítulo, iniciar la red tardó unos dos minutos y medio y terminó con el siguiente resumen abreviado:

```
  test006e:
    bootstrap progress: 100 (SUCCESS), done, Done
    desc publishing: 100 (SUCCESS), all nodes, DESC MD MD_CONS NS_CONS
  test008c:
    bootstrap progress: 100 (SUCCESS), done, Done
  test009h:
    bootstrap progress: 100 (SUCCESS), done, Done
    oniondesc publishing: 200 (ONIONDESC_PUBLISHED), hidden service v3
```

Todos los nodos completaron el arranque, todos los repetidores vieron sus descriptores publicados en el consenso y el servicio onion publicó su descriptor. La orden `make -C labs/es/laboratorio verify` ejecuta entonces la prueba de Chutney: transmite datos a través de una salida y un servicio onion y comprueba que lleguen intactos:

```
  not-done:0 successes:4 failures:0
Transmission: Success
```

`make -C labs/es/laboratorio status` muestra el estado de los nodos. `make -C labs/es/laboratorio shell` abre una consola dentro del contenedor, `make -C labs/es/laboratorio stop` detiene la red y `make -C labs/es/laboratorio clean` elimina también los datos generados.

19.4 Topología de red

La red está descrita en un breve archivo Python, `labs/es/laboratorio/chutney/networks/book`, escrito en el lenguaje de configuración de Chutney:

```
Authority = Node(tag="a", authority=1, relay=1)
Middle    = Node(tag="r", relay=1)
Exit      = Node(tag="e", relay=1, exit=1)
Client    = Node(tag="c", client=1, launch_phase=2)
Onion     = Node(tag="h", hs=1, launch_phase=2)

NODES = (Authority.getN(3) + Middle.getN(3) + Exit.getN(2)
         + Client.getN(1) + Onion.getN(1))
ConfigureNodes(NODES)
```

Cada `Node` es una plantilla y `getN` crea múltiples instancias. El cliente y el servicio onion se inician en una segunda fase (`launch_phase=2`), una vez que las autoridades han producido un consenso. Chutney numera los nodos por orden y los nombra combinando número y etiqueta: `test000a`, …, `test009h`. Los números también determinan los puertos, como muestra figura 19.1.

Nodo	Función	ORPort	ControlPort	Otros
`test000a`–`002a`	autoridad de directorio	5100–5102	8000–8002	DirPort 7100–7102
`test003r`–`005r`	repetidor intermedio	5103–5105	8003–8005	
`test006e`–`007e`	salida	5106–5107	8006–8007	
`test008c`	cliente	—	8008	SocksPort 9008
`test009h`	servicio onion	—	8009	puerto virtual 5858

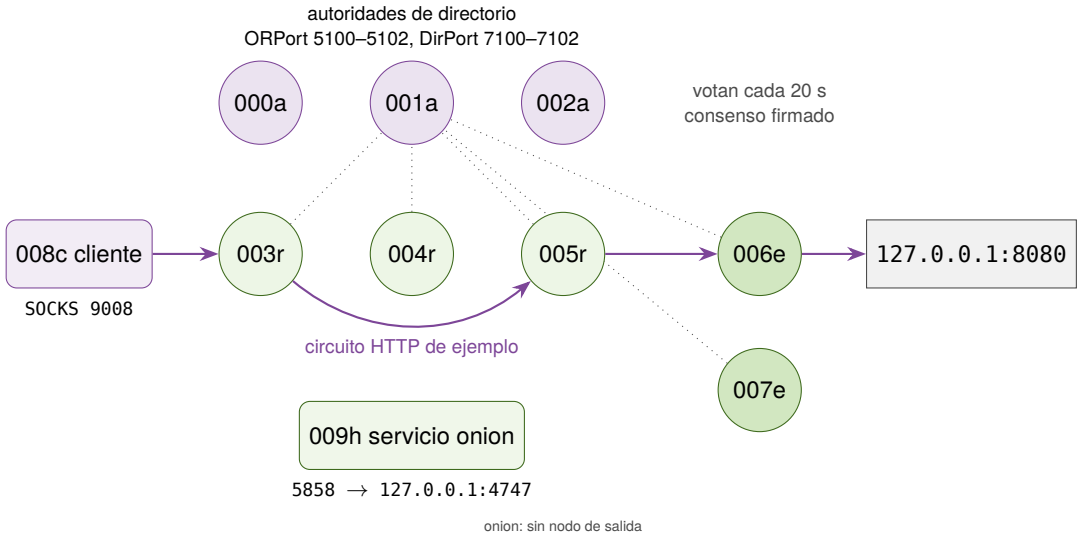


Figura 19.1: Topología del laboratorio Chutney ligada a `127.0.0.1`: tres autoridades de directorio/repetidores, tres repetidores intermedios, dos salidas, un cliente y un servicio onion. ORPort $5100 + n$ solo se utiliza en autoridades y repetidores, incluidas las salidas; todos los nodos disponen de ControlPort $8000 + n$. Las autoridades tienen DirPort $7100 + n$, el cliente SocksPort 9008, y el servicio onion el puerto virtual 5858 hacia `127.0.0.1:4747`. El circuito HTTP de ejemplo a través de una salida es independiente del servicio onion, que no usa salidas. El loopback no constituye una red de contenedor aislada.

En profundidad: ¿Por qué tres autoridades?

Un consenso requiere firmas de la mayoría de las autoridades (capítulo 12). Con una sola, no podríamos observar votaciones; con tres podemos ver el proceso y estudiar en un ejercicio qué sucede cuando una se detiene. La red pública tiene nueve, además de una autoridad de puentes independiente. Su lista está incorporada al propio tor (`src/app/config/auth_dirs.inc`).

19.5 Una red de prueba no es la red real

Chutney genera un `torrc` para cada nodo. Estos archivos revelan exactamente qué es necesario modificar para que funcione una red Tor en miniatura y, por tanto, qué supuestos de la red pública no se cumplen. Estas son las opciones más importantes del torrc de una autoridad (`net/nodes/000a/torrc`):

```
TestingTorNetwork 1
AuthoritativeDirectory 1
V3AuthoritativeDirectory 1
AssumeReachable 1
V3AuthVotingInterval 20 seconds
V3AuthVoteDelay 4
V3AuthDistDelay 4
TestingDirAuthVoteGuard *
TestingDirAuthVoteHSDir *
```

`TestingTorNetwork` Activa el modo de prueba. Solo puede utilizarse junto con una lista explícita de autoridades (`DirAuthority`), evitando que los nodos de prueba se incorporen

accidentalmente a la red pública. También cambia numerosos valores predeterminados para acelerar las operaciones.

`V3AuthVotingInterval` En la red real, las autoridades votan cada hora un consenso nuevo. Aquí lo hacen cada veinte segundos, como muestra el propio consenso:

```
valid-after 2026-09-23 14:37:40
fresh-until 2026-09-23 14:38:00
valid-until 2026-09-23 14:38:20
```

`AssumeReachable` Un repetidor normalmente verifica su accesibilidad externa antes de anunciarse. Esa prueba carece de sentido en la interfaz de bucle local y se omite.

`TestingDirAuthVoteGuard *` En la red real, los repetidores obtienen el indicador Guard tras semanas de estabilidad y suficiente ancho de banda. Aquí las autoridades lo asignan a todos, de modo que **cada repetidor del consenso es un guardia potencial**, a diferencia de la red pública.

Los `torrc` de las salidas también contienen:

```
ExitPolicyRejectPrivate 0
ExitPolicy accept private:*
```

Por defecto, las salidas rechazan conexiones a direcciones privadas y de bucle local para no exponer sus redes locales. En este laboratorio, `127.0.0.1` es el único «Internet» disponible, por lo que debe invertirse esa política.

Advertencia: No copies estas opciones fuera del laboratorio

Las opciones `Testing…`, `AssumeReachable` y, especialmente, `ExitPolicy accept private:*` son exclusivas de redes de prueba. Utilizarlas en un repetidor público sería incorrecto o peligroso para la red del anfitrión.

19.6 Dentro del directorio de un nodo

Cada nodo dispone de un directorio de datos propio en `labs/es/laboratorio/net/nodes/NNNx/`. El directorio de una autoridad contiene, entre otros, los siguientes archivos:

Archivo o directorio	Contenido
`torrc`	configuración generada por Chutney
`keys/`	claves de identidad y onion del nodo (capítulos 5 y 6)
`fingerprint`, `fingerprint-ed25519`	huellas RSA y Ed25519 de identidad
`cached-consensus`, `cached-microdesc-consensus`	las dos variantes de consenso (capítulo 12)
`cached-microdescs.new`	microdescriptores descargados
`v3-status-votes`	votos de autoridades (solo autoridades)
`sr-state`	estado del protocolo de aleatoriedad compartida (solo autoridades)
`notice.log`, `info.log`	registros de nivel *notice* e *info*
`control_auth_cookie`	cookie de autenticación del puerto de control

El directorio del servicio onion también contiene `hidden_service/`, con sus claves y un archivo `hostname` que guarda su dirección `.onion`, por ejemplo:

```
rpns2r36zn5u46wctlgnvtwjotxaf7iflsc3k5eas3lqoxeesv4zwyad.onion
```

La dirección cambia al configurar la red desde cero, porque se genera una clave nueva (capítulo 14).

Un consenso es un archivo de texto legible. Al extraer las líneas `r`, que identifican repetidores y direcciones, y las `s`, con los indicadores, obtenemos estas dos entradas:

```
r test000a H+9DDHx4rEo6I6Jdn8gIyKyi3jk ... 127.0.0.1 5100 7100
s Authority Fast Guard HSDir Running Stable V2Dir Valid
r test006e dEPHoFFHamQyy6pRzlsLB9/ilzU ... 127.0.0.1 5106 0
s Exit Fast Guard HSDir Running Stable V2Dir Valid
```

Observa que todos los repetidores llevan `Guard`, como esperábamos, pero únicamente las dos salidas llevan `Exit`. Los campos se explican en capítulo 12.

19.7 Utilizar la red

19.7.1 Navegar a través de la red privada

Ejecuta los dos ejemplos HTTP *dentro del contenedor del laboratorio*, en `/lab`, entrando desde otra terminal del anfitrión con `make -C labs/es/laboratorio shell`. El parámetro `--noproxy` con una cadena vacía obliga a curl a usar SOCKS incluso para `127.0.0.1` si existe `NO_PROXY`. El servidor del puerto 4747 ofrece el mismo `index.html` creado en el primer ejemplo.

El cliente `test008c` ofrece SOCKS en el puerto 9008. Cualquier programa compatible con SOCKS5 puede utilizarlo. Inicia un servidor web local pequeño y accede por Tor:

```
$ echo "Hola desde la red privada" > index.html
$ python3 -m http.server 8080 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 http://127.0.0.1:8080/index.html
Hola desde la red privada
```

La solicitud atraviesa un circuito de tres repetidores antes de que una salida conecte con el servidor. Utilizar `--socks5-hostname` en lugar de `--socks5` hace que la salida resuelva los nombres de destino en vez del propio sistema. En la red real es esencial para evitar que el DNS local conozca los sitios visitados (capítulo 11).

19.7.2 Acceder al servicio onion

Chutney configura el servicio onion con:

```
HiddenServicePort 5858 127.0.0.1:4747
```

Esto asocia el puerto virtual 5858 de la dirección `.onion` con el puerto local 4747. Solo necesitamos un servidor que escuche allí:

```
$ python3 -m http.server 4747 --bind 127.0.0.1 &
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    http://$(cat net/nodes/009h/hidden_service/hostname):5858/index.html
Hola desde la red privada
```

Esta vez no participa ninguna salida. Cliente y servicio se encuentran en un punto de encuentro dentro de la red (capítulo 14).

19.7.3 Observar los circuitos con Stem

El puerto de control permite inspeccionar un nodo activo. Con Stem (capítulo 18), incluido en la imagen, podemos enumerar los circuitos del cliente:

```
from stem.control import Controller

with Controller.from_port(port=8008) as c:
    c.authenticate()                    # usa automáticamente la cookie
    print(c.get_version())
    for circ in c.get_circuits():
        print(circ.id, circ.status, circ.purpose,
              [name for _, name in circ.path])
```

Un extracto de la salida, tras ejecutar `make -C labs/es/laboratorio verify`:

```
0.4.9.13 (git-3c575400909efe65)
10 BUILT GENERAL ['test004r', 'test005r', 'test007e']
124 BUILT CONFLUX_LINKED ['test006e', 'test004r', 'test007e']
114 BUILT CONFLUX_UNLINKED ['test007e', 'test004r', 'test006e']
128 BUILT CIRCUIT_PADDING ['test004r', 'test001a', 'test007e', 'test006e']
163 BUILT HS_CLIENT_REND ['test004r', 'test001a', 'test006e']
166 BUILT HS_CLIENT_HSDIR ['test004r', 'test006e', 'test007e', 'test003r']
181 BUILT HS_VANGUARDS ['test004r', 'test007e', 'test006e']
```

Incluso en una red tan pequeña, la lista anticipa buena parte del libro:

- Casi todos los circuitos comienzan en `test004r`: el **guardia** del cliente, elegido una vez y reutilizado (capítulo 13). Las excepciones son las ramas de túneles Conflux que comienzan en el segundo guardia (capítulo 18).
- Los circuitos `GENERAL` de tres saltos transportan tráfico ordinario; los de un salto, omitidos aquí, descargan información de directorio.
- Los circuitos `CONFLUX_LINKED` se agrupan en parejas utilizadas conjuntamente para el mismo tráfico: Conflux (Propuesta 329, capítulo 31).
- `HS_VANGUARDS`, `HS_CLIENT_HSDIR` y `HS_CLIENT_REND` identifican circuitos empleados para acceder a servicios onion (capítulo 14).
- Según la especificación de control, `CIRCUIT_PADDING` identifica un circuito mantenido para ocultar cuándo dejó realmente de utilizarse. Por ello su cierre no revela necesariamente cuándo terminó la actividad del usuario.

19.7.4 Trabajar dentro del contenedor

El contenedor utiliza la red del anfitrión, por lo que todos los puertos son accesibles directamente desde el equipo anfitrión. Si esto no funciona, por ejemplo con Docker Desktop, `make -C labs/es/laboratorio shell` abre una consola en el contenedor, con tor, Chutney, Stem, Python y las herramientas de captura instaladas; los archivos del laboratorio están disponibles en `/lab`.

19.8 Primer contacto con un repetidor

El siguiente fragmento Python importa el paquete didáctico `torlab` desde `labs/es/codigo/`; el contenedor Chutney solo monta `labs/es/laboratorio/` como `/lab`, por lo que

no importa automáticamente el paquete. En Linux con red del anfitrión, guarda el fragmento en un archivo y ejecútalo desde la raíz del repositorio mediante `PYTHONPATH=labs/es/codigo python3 script.py`; también puedes facilitar expresamente ese paquete al contenedor. Este ejemplo es solo para la red privada.

Volvamos a capítulo 8. Con `labs/es/codigo/torlab/cells.py` del laboratorio 2, que ya está presente en el repositorio, podemos abrir una conexión TLS hacia el ORPort de un repetidor del laboratorio, enviar una celda **`VERSIONS`** y descodificar la respuesta. El laboratorio 1 de capítulo 7 proporciona el código completo con verificación de certificados. Aquí observamos únicamente el intercambio inicial:

```
import socket, ssl
from torlab.cells import *

ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE   # solo red privada; certificados TLS SIN verificar
s = ctx.wrap_socket(socket.create_connection(("127.0.0.1", 5103)))
s.sendall(encode(versions_cell([3, 4, 5]), 0))

buf, version, received = b"", 0, 0
while received < 4:
    chunk = s.recv(4096)
    if not chunk:
        raise ConnectionError("TLS channel closed early")
    buf += chunk
    while (r := decode(buf, version)) is not None:
        cell, buf = r
        received += 1
        print(describe(cell, version))
        if cell.command == CellCommand.VERSIONS:
            version = negotiate_version([3, 4, 5], cell.body)
s.close()
```

Atención: esta demostración breve desactiva expresamente la validación de certificados TLS con `ssl.CERT_NONE` y *no* verifica la identidad del repetidor a partir de la celda CERTS. No la uses contra repetidores públicos. El laboratorio 1 muestra la validación completa.

El siguiente resultado procede de una sesión anterior de referencia: identificadores de circuito, huellas, direcciones onion, marcas temporales y tamaños de certificados pueden variar.

El repetidor `test003r` respondió en la red de referencia:

```
CircID=0x0000 VERSIONS cuerpo=6 bytes
CircID=0x00000000 CERTS cuerpo=1451 bytes
CircID=0x00000000 AUTH_CHALLENGE cuerpo=36 bytes
CircID=0x00000000 NETINFO cuerpo=509 bytes
```

Es exactamente la secuencia descrita en apartado 8.8: primero **`VERSIONS`** con un CircID de dos bytes; después de negociar la versión 5, las demás celdas utilizan CircID de cuatro bytes. **`CERTS`** y **`AUTH_CHALLENGE`** son de longitud variable y conservan exactamente los cuerpos de sus mensajes; **`NETINFO`** es de longitud fija, con sus 509 bytes. Observa que se asigna `version = ...` justo después de **`VERSIONS`**: la descodificación de las celdas siguientes depende de la versión recién negociada.

19.9 Shadow: una red dentro de un simulador

Chutney no basta para responder preguntas sobre el comportamiento de la red *en su conjunto*. ¿Cómo afectaría otro algoritmo de control de congestión a la latencia? ¿Cuánto tráfico necesitaría un adversario para un ataque concreto? Son preguntas que requieren miles de nodos, latencias realistas entre continentes y, sobre todo, experimentos reproducibles.

Shadow, desarrollado originalmente por Rob Jansen y Nicholas Hopper [JH12], responde a estas necesidades. Las versiones actuales ejecutan programas reales, incluido tor, como procesos Linux normales, interceptando las llamadas al sistema relacionadas con red y tiempo. Un grafo virtual suministra la latencia y el ancho de banda configurados por el usuario, y el tiempo avanza solo cuando el simulador decide. Así, una hora simulada puede tardar más o menos de una hora real según la potencia de cálculo, y repetir un experimento con la misma semilla aleatoria produce el mismo resultado.

Para crear redes Tor realistas con Shadow, los investigadores utilizan **tornettools**. A partir de los datos públicos de Tor Metrics (capítulo 24), genera una red reducida con distribuciones de anchos de banda, indicadores y funciones semejantes a las de la red real.

Utilizaremos Shadow en un experimento guiado de capítulo 29. Por ahora basta saber cuándo es la herramienta adecuada.

Resumen

- Ejecuta experimentos de Tor en redes privadas. En la pública podrían implicar a usuarios sin consentimiento y no serían reproducibles.
- Chutney emula una red Tor con procesos `tor` reales mediante bucle local. Shadow ejecuta el software real dentro de un simulador de eventos discretos, adecuado para redes grandes y experimentos repetibles.
- El laboratorio español (`labs/es/laboratorio/`) compila tor 0.4.9.13 desde las fuentes. El objetivo `make -C labs/es/laboratorio start` inicia una red de diez nodos: tres autoridades, tres intermedios, dos salidas, un cliente y un servicio onion.
- El nodo n utiliza ORPort $5100 + n$, ControlPort $8000 + n$ y SocksPort $9000 + n$.
- Las opciones de prueba (`TestingTorNetwork`, voto cada veinte segundos, asignación de Guard a todos y permiso para conexiones de salida a direcciones privadas) permiten funcionar a la red en miniatura, pero la diferencian de la real. Debemos tener precaución antes de extrapolar mediciones.
- SocksPort 9008 transporta tráfico de aplicación por la red; Stem observa circuitos y `torlab` habla directamente con repetidores.

Ejercicios

Ejercicio 19.1 ⋆ Sin leer el torrc, indica ORPort y ControlPort de `test006e`. ¿Tiene ORPort `test009h`? ¿Por qué?

Ejercicio 19.2 ⋆ Todos los repetidores de la red privada tienen indicador Guard. ¿Qué opción lo provoca y por qué no sucede en la red real? ¿Qué efecto tiene sobre las mediciones de selección de rutas?

Ejercicio 19.3 ⋆ Repite la solicitud `curl` de apartado 19.7 utilizando `--socks5` en lugar de `--socks5-hostname` y `localhost` en lugar de `127.0.0.1`. ¿Qué cambia en lo que recibe la salida? ¿Por qué importa esto en la red pública?

Ejercicio 19.4 ⋆⋆ Observa durante varios minutos el campo `valid-after` del consenso del cliente (`grep valid-after net/nodes/008c/cached-microdesc-consensus`). ¿Cada cuánto cambia? Calcula cuántos consensos se producen al día en la red de laboratorio y en la real.

Ejercicio 19.5 ⋆⋆ Detén una de las tres autoridades (`make -C labs/es/laboratorio shell` y después `kill` al proceso correspondiente según su archivo `pid`). ¿Sigue produciendo consensos la red? ¿Qué pasa si detienes dos? Explica el resultado según capítulo 12.

Ejercicio 19.6 ⋆⋆ Modifica el script Stem de apartado 19.7.3 para mostrar los ORPort de todos los repetidores de cada circuito de tres saltos, obteniéndolos del consenso mediante `get_network_status`.

Ejercicio 19.7 ⋆⋆ Crea `labs/es/laboratorio/chutney/networks/book-large` con cinco autoridades, diez repetidores intermedios y cuatro salidas; después iníciala estableciendo `NETWORK=book-large` en la orden `make`. ¿Cómo se compara el tiempo de arranque con el de la red original?

Ejercicio 19.8 ⋆ ⋆ ⋆ Busca en el archivo `info.log` del cliente los mensajes relativos a la construcción de circuitos, desde la selección de nodos hasta el final. Relaciona cada mensaje con una celda de figura 8.5.

Ejercicio 19.9 ⋆ ⋆ ⋆ Dentro del contenedor, captura el tráfico hacia el puerto 5103 mediante `tcpdump -i lo` mientras ejecutas el script de apartado 19.8. ¿Puedes ver las celdas? ¿Por qué no? ¿Qué sigue siendo observable: tamaños, tiempos? En capítulo 20 veremos cómo profundizar.

Lecturas complementarias

- Chutney, `https://gitlab.torproject.org/tpo/core/chutney`: su README y el directorio `networks/` contienen ejemplos de redes, también con puentes y Arti.
- R. Jansen y N. Hopper, *Shadow: Running Tor in a Box for Accurate and Efficient Experimentation* [JH12].
- Documentación de Shadow y tornettools, `https://shadow.github.io/`.
- Tor Research Safety Board, `https://research.torproject.org/safetyboard/`: recomendaciones para investigar Tor éticamente.

Capítulo 20

Observar el tráfico: Wireshark y registros

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; tshark (Debian trixie); red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- capturar tráfico Tor mediante `tcpdump` y analizarlo con `tshark` o Wireshark;
- distinguir lo que puede ver un observador externo de aquello que permanece oculto;
- descifrar TLS en *su propio* canal utilizando claves de sesión e inspeccionar sus celdas (laboratorio 12);
- seguir paso a paso el protocolo utilizando los registros de tor.

En los capítulos anteriores examinamos el protocolo «desde dentro», escribiendo código capaz de comunicarse con él. Aquí lo observaremos «desde fuera», como lo haría alguien que vigila la red. El ejercicio sirve para dos finalidades opuestas: comprender qué puede deducir un adversario a partir del tráfico y obtener una manera independiente de verificar nuestra propia implementación.

Advertencia: Solo tráfico propio y únicamente en el laboratorio

Capturar el tráfico ajeno sin autorización es ilegal en Italia y en la mayoría de los países (capítulo 25), además de contravenir los principios éticos de la investigación sobre anonimato (apartado 19.1). Todas las capturas de este capítulo se realizan en la red privada del libro o sobre conexiones que abrimos nosotros mismos.

20.1 Qué puede ver un observador

Alguien que observe la conexión entre un cliente y su guardia, por ejemplo un proveedor de Internet, puede ver:

- direcciones IP y puertos: el cliente se conecta a una dirección que, salvo que pertenezca a un puente, aparece en el consenso público y puede reconocerse como repetidor Tor;
- el intercambio TLS: los certificados viajan cifrados con TLS 1.3, pero algunas características del mensaje inicial del cliente siguen siendo visibles;
- los tamaños y tiempos de los registros TLS.

El observador no puede ver el contenido de las celdas, los circuitos ni los destinos. Sin embargo, los tamaños y los tiempos son precisamente los datos utilizados por los ataques de *identificación de sitios web* (*website fingerprinting*) y de correlación (capítulo 26). Que el contenido esté cifrado no significa que el tráfico no revele nada.

En apartado 16.5 comparamos el tráfico Tor ordinario, reconocible como TLS, con obfs4, sin estructura reconocible. Ahora iremos un paso más allá y examinaremos qué contiene TLS.

20.2 Las herramientas

`tcpdump` Captura paquetes desde una interfaz de red y los guarda en un archivo `pcap`. Por ejemplo, dentro del contenedor de laboratorio, para capturar tráfico del ORPort de `test004r`:

```
$ docker exec -u 0 torbook-es-network tcpdump -i lo \
    -w /lab/net/tls.pcap "tcp port 5104"
```

Wireshark y `tshark` Analizan las capturas, el primero mediante interfaz gráfica y el segundo desde la línea de órdenes. Reconocen TLS, pero no existe un disector oficial del protocolo Tor. Incluso después de descifrar TLS, las celdas aparecen como bytes sin interpretar. Nuestro módulo `cells.py` actuará como disector.

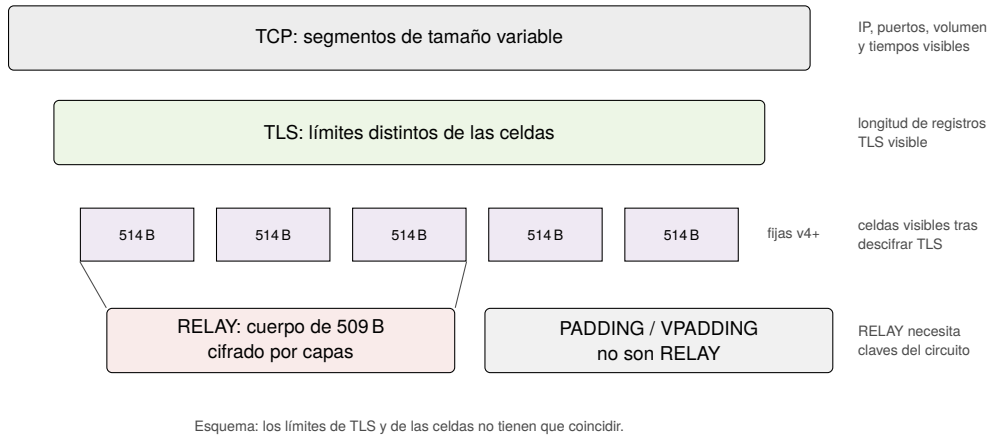


Figura 20.1: Esquema conceptual del canal Tor: los segmentos TCP transportan registros TLS cuyos límites no tienen por qué coincidir con los de las celdas. Se observan las IP, los puertos, las longitudes y los tiempos; descifrar TLS permite ver las celdas. En el protocolo de enlace v4+, una celda de longitud fija ocupa 514 bytes (5 de cabecera y 509 de cuerpo), aunque también existen celdas variables. Solo los cuerpos de las celdas `RELAY` llevan cifrado por capas del circuito; las celdas de relleno no son mensajes RELAY.

20.2.1 Descifrar tu propia conexión TLS

TLS utiliza claves de sesión que los observadores externos no poseen. Sin embargo, una parte que *participa* en la conexión puede exportarlas. Los navegadores y las aplicaciones Python que activan el registro de claves pueden exportarlas a un archivo. Nuestro módulo `channel.py` configura expresamente `SSLContext.keylog_filename` cuando se define `SSLKEYLOGFILE`. Wireshark y `tshark` emplean ese archivo para descifrar una captura. `channel.py`, del laboratorio 1, sigue esta convención: establece la variable antes de ejecutarlo. El archivo contiene líneas como:

```
# TLS secrets log file, generated by OpenSSL / Python
SERVER_HANDSHAKE_TRAFFIC_SECRET 4878224d3f9fb9aedb96d... 0da89dff6feb...
```

Registran el valor aleatorio del cliente y los secretos de cada fase de TLS 1.3 para cada sesión. Las etiquetas son campos estandarizados del archivo de claves, no texto traducible.

En profundidad: Por qué esto no es un ataque contra Tor

Descifrar TLS en tu propio canal no revela nada que el cliente no supiera ya. Incluso con las claves de sesión TLS, las celdas **`RELAY`** permanecen cifradas por capas con las claves particulares de cada salto (capítulo 10). Leerlas requeriría las claves del circuito, conocidas solamente por el cliente y los repetidores implicados. TLS constituye la capa exterior de la cebolla, no su única protección.

20.3 Laboratorio: celdas dentro de TLS

Laboratorio 12: Capturar, descifrar y leer un canal

Objetivo. Capturar un canal abierto desde nuestro cliente hacia un repetidor, descifrarlo con las claves de sesión y reconstruir la secuencia de celdas mediante `labs/es/codigo/torlab/traffic.py`.
Requisitos previos. Laboratorios 1–3, red privada en ejecución y `tcpdump` y `tshark` instalados dentro del contenedor.
Tiempo estimado. 1–2 horas.

Para el laboratorio 12, inicia la red privada y abre dos terminales desde la raíz del repositorio. Ejecuta la siguiente orden de captura *en el anfitrión*: inicia `tcpdump` como root *dentro del contenedor de laboratorio*, porque la consola normal del Makefile utiliza un usuario sin privilegios. Mantén la captura activa mientras ejecutas el cliente en la segunda terminal y deténla con Ctrl+C. El archivo queda en el volumen compartido `labs/es/laboratorio/net/`.

Paso 1: capturar. Inicia la captura en el contenedor. Después, desde el anfitrión, abre un canal con `test004r` y construye un circuito de un salto, exportando al mismo tiempo las claves de sesión:

```
$ PYTHONPATH=labs/es/codigo SSLKEYLOGFILE=labs/es/laboratorio/net/tls-keys.log python3
    -c "
from torlab.laboratory import relay
from torlab.circuit import build
build([relay('004r')], cc=False)"
```

Ejecuta la orden Python desde la *raíz del repositorio* en Linux con red del anfitrión. `PYTHONPATH` permite importar `torlab` desde `labs/es/codigo/`; `labs/es/codigo/torlab/channel.py` consulta expresamente `SSLKEYLOGFILE` y activa el registro de secretos TLS. Este archivo contiene claves de sesión sensibles: no lo publiques y elimínalo después del experimento.

Paso 2: identificar nuestra conexión. Otros repetidores del laboratorio también se conectan al mismo puerto, pero sus claves de sesión son distintas. Pedimos a `tshark` que siga cada flujo TCP después de intentar descifrarlo y nos quedamos con aquel cuyo descifrado funciona:

```
$ docker exec torbook-es-network tshark -r /lab/net/tls.pcap \
    -o tls.keylog_file:/lab/net/tls-keys.log \
    -q -z follow,tls,raw,2
```

El identificador de flujo TCP 2 solo pertenece a una captura de ejemplo anterior: enumera los flujos candidatos con `tshark` y `-T fields -e tcp.stream` antes de elegir el que se descifre correctamente. La salida siguiente es histórica; las longitudes de los certificados y los CircID pueden variar.

El resultado contiene los datos descifrados en hexadecimal de ambas direcciones; una de ellas aparece indentada con una tabulación.

Paso 3: inspeccionar las celdas. La función `cells_from_follow` reconstruye ambos flujos y los pasa al `decode` del laboratorio 2. Cada dirección tiene su propia negociación de versión: la primera celda es **`VERSIONS`** con CircID de dos bytes. Los CircID de cuatro bytes solo se utilizan después de negociar. Resultado de una captura de referencia anterior:

```
client -> relay  CircID=0x0000 VERSIONS cuerpo=4 bytes
relay  -> client CircID=0x0000 VERSIONS cuerpo=6 bytes
relay  -> client CircID=0x00000000 CERTS cuerpo=1443 bytes
relay  -> client CircID=0x00000000 AUTH_CHALLENGE cuerpo=36 bytes
relay  -> client CircID=0x00000000 NETINFO cuerpo=509 bytes
client -> relay  CircID=0x00000000 NETINFO cuerpo=509 bytes
client -> relay  CircID=0xc244f1f6 CREATE2 cuerpo=509 bytes
relay  -> client CircID=0xc244f1f6 CREATED2 cuerpo=509 bytes
```

Observada desde fuera, es exactamente la secuencia de figura 8.5 y capítulo 7. La celda **`VERSIONS`** del cliente tiene un cuerpo de cuatro bytes porque `channel.py` solo ofrece versiones 4 y 5. El identificador de circuito `0xc244f1f6` tiene el bit más significativo igual a 1 porque el cliente abrió el canal (apartado 8.4).

Advertencia: Dos errores de nuestra primera versión

El análisis inicial tenía dos fallos que descubrimos al comparar el resultado con la secuencia prevista. Utilizaba una única versión negociada para las dos direcciones e interpretaba incorrectamente la celda **`VERSIONS`** del repetidor. También asignaba los bloques a la dirección equivocada porque habíamos supuesto cuál de ellas indentaba `tshark`. La regla correcta es sencilla: el primer bloque corresponde siempre a la celda **`VERSIONS`** del cliente.

20.4 Los registros de tor

Una captura de paquetes muestra *qué* se transmite por la red. Los registros de tor explican *por qué*. En el nivel `info`, tor registra las fases de construcción de circuitos, la elección de guardias y la descarga y publicación de descriptores. En `debug`, los informes llegan hasta las celdas individuales. Ya utilizamos registros para verificar qué HSDir consultó el cliente (apartado 14.9) y cuándo rotó los descriptores un servicio onion.

Algunas recomendaciones prácticas:

- `debug` produce muchísimo texto. Actívalo solo para un ámbito concreto, por ejemplo `Log [circ]debug file circ.log`, para limitar el detalle a eventos de circuitos.
- En la red real, `SafeLogging` oculta las direcciones. Los registros del cliente nunca deberían contener los destinos visitados.

- Los registros de repetidores públicos pueden contener datos personales de terceros, con las consecuencias legales de capítulo 25; conserva solo lo estrictamente necesario.

Resumen

- Un observador de una conexión ve direcciones, intercambio TLS, tamaños y tiempos de paquetes, pero no las celdas Tor ni los destinos.
- `SSLKEYLOGFILE` permite descifrar TLS en un canal propio e inspeccionar las celdas; las celdas **`RELAY`** siguen cifradas por capas.
- Analizar una captura proporciona una forma independiente de verificar una implementación.
- Los registros `info` y `debug` de tor explican las decisiones del programa; durante el uso real deben minimizarse y protegerse.

Ejercicios

Ejercicio 20.1 ⋆ En una captura de una descarga por Tor, no todos los registros TLS tienen una longitud múltiplo de 514 bytes. ¿Por qué, si las celdas son de tamaño fijo?

Ejercicio 20.2 ⋆⋆ Amplía el laboratorio 12 descifrando las celdas **`RELAY`** de tu propio circuito con las claves de `circuit.py`. ¿Qué mensajes relay puedes ver durante la creación de un circuito de tres saltos?

Ejercicio 20.3 ⋆⋆ Captura un minuto del canal de un cliente de laboratorio sin generar tráfico de aplicación. Cuenta los registros TLS de cada dirección y relaciona las observaciones con el relleno de canales explicado en capítulo 7.

Ejercicio 20.4 ⋆⋆⋆ Escribe un pequeño disector de Wireshark en Lua que reconozca celdas Tor en datos descifrados y muestre su CircID y comando.

Lecturas complementarias

- Documentación de Wireshark sobre descifrado TLS mediante archivos de claves de sesión.
- Entrada `Log` del manual de tor, sobre ámbitos de registro y niveles de detalle.

Capítulo 21

Publicar un servicio onion

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13; documentación de Onionbalance

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- configurar un servicio onion y proteger sus claves;
- evitar errores de configuración que revelen la ubicación del servidor;
- ofrecer a los visitantes de un sitio web convencional su versión `.onion` mediante Onion-Location;
- restringir el acceso a clientes autorizados mediante descubrimiento restringido (laboratorio 13);
- evaluar las direcciones personalizadas, los servicios de un solo salto y el reparto de carga con Onionbalance.

En capítulo 14 explicamos cómo funcionan los servicios onion. Este capítulo constituye su complemento práctico: publicar un servicio, evitar errores sencillos que comprometan su anonimato y elegir herramientas para necesidades operativas habituales.

21.1 Tu primer servicio onion

Bastan dos líneas en `torrc`:

```
HiddenServiceDir /var/lib/tor/my_service/
HiddenServicePort 80 127.0.0.1:8080
```

La ruta `HiddenServiceDir` debe permitir que el proceso tor lea y escriba; en el laboratorio se encuentra *dentro del contenedor Chutney*. Este primer ejemplo configura un servicio onion *público*: no necesita ningún archivo `alice.auth`. La autorización de clientes y sus archivos `authorized_clients/*.auth` se explican más adelante en este capítulo. La primera indica el directorio donde tor guarda las claves del servicio. La segunda asigna el puerto *virtual* 80 de su dirección `.onion` al servidor local del puerto 8080. Cuando tor se reinicia, crea los siguientes archivos en ese directorio:

`hostname` La dirección `.onion` (apartado 14.2).

`hs_ed25519_public_key`, `hs_ed25519_secret_key` El par de claves de identidad del servicio. **Quien posea la clave secreta controla la dirección**: puede publicar descriptores y suplantar el servicio. Protégela como la clave privada de un certificado y conserva una copia de seguridad segura. Si se pierde, se pierde la dirección para siempre.

El directorio debe ser accesible únicamente al usuario que ejecuta tor; de lo contrario, tor se niega a utilizarlo. Un proceso tor puede alojar varios servicios, cada uno en su propio directorio. En el laboratorio añadimos un segundo servicio a `test009h` mediante otras dos líneas.

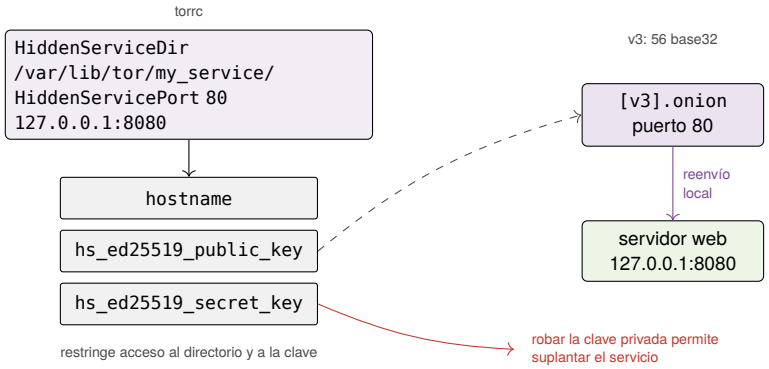


Figura 21.1: Las directivas `HiddenServiceDir` y `HiddenServicePort` configuran un servicio onion v3 público. Tor crea `hostname` y las claves de identidad Ed25519 en el directorio; la dirección contiene 56 caracteres base32 derivados de la clave pública, una suma de verificación y la versión (la figura usa un marcador de posición). El puerto virtual 80 se reenvía a `127.0.0.1:8080`. El robo de la clave privada permite suplantar el servicio; el ejemplo público no necesita `alice.auth`.

21.1.1 Evitar revelar la ubicación del servidor

El protocolo oculta la dirección del servidor, pero una configuración descuidada puede divulgarla. Entre los errores habituales se encuentran:

- **Hacer que el servidor también sea accesible desde el exterior.** Si el servidor web escucha en todas las interfaces y no solo en `127.0.0.1`, alguien que explore Internet puede encontrarlo y comparar su contenido con el servicio onion. Vincula el servidor a localhost o, preferiblemente, utiliza un socket Unix (`HiddenServicePort 80 unix:/path/socket`).
- **Filtrar detalles en las respuestas.** Las páginas de error pueden revelar el nombre del servidor; las cabeceras HTTP pueden contener direcciones internas; los certificados TLS pueden estar emitidos para un dominio público; o pueden quedar accesibles páginas de estado del servidor.
- **Recursos externos.** Las páginas que cargan imágenes o scripts de sitios convencionales envían esas solicitudes fuera del servicio onion. En Tor Browser continúan pasando por Tor, por lo que no revelan la IP del visitante, pero ya no cuentan con la autenticación ni la protección de extremo a extremo del servicio onion: un nodo de salida puede observar los metadatos de conexión y el HTTP sin cifrar, mientras que HTTPS sigue protegiendo el contenido. Las aplicaciones del servidor que contactan con servicios externos para actualizarse o realizar análisis de uso pueden revelar la dirección del servidor.
- **Relojes y horarios.** Un servidor que muestra una hora local asociada a una zona horaria concreta o queda inaccesible cada vez que un servidor público conocido entra en mantenimiento deja indicios que pueden correlacionarse.

En capítulo 28 analizaremos casos reales en los que errores semejantes ayudaron a localizar servicios.

21.2 Onion-Location

Muchos sitios públicos, incluidos grandes medios de comunicación, redes sociales y el Tor Project, ofrecen una versión `.onion`. Un sitio convencional puede anunciarla a quienes lo visiten con Tor Browser mediante la cabecera HTTP:

```
Onion-Location: http://<address>.onion/path
```

También puede utilizarse un elemento `meta` equivalente en la página. Tor Browser muestra entonces un botón en la barra de direcciones que ofrece cambiar a la versión onion. Los visitantes evitan la salida y obtienen una conexión autenticada por la propia dirección; los sitios reciben tráfico Tor sin encaminarlo por salidas, a veces bloqueadas o tratadas como sospechosas.

21.3 Descubrimiento restringido

Un servicio personal, como el acceso SSH a un servidor doméstico o un panel de administración, no tiene por qué ser accesible a todo el que conozca su dirección. El descubrimiento restringido (apartado 14.7) cifra la segunda capa del descriptor de modo que solo los clientes autorizados puedan descifrarla. La configuración utiliza pares de claves X25519 [Thee]:

En el servicio Coloca un archivo `.auth` por cliente en el directorio `authorized_clients/` del servicio. Cada archivo contiene una línea formada por `descriptor:x25519:` seguida de la clave pública del cliente codificada en base32.

En el cliente La opción `ClientOnionAuthDir` indica un directorio con un archivo `.auth_private` por servicio. Cada archivo contiene la dirección sin `.onion`, seguida de `:descriptor:x25519:` y de la clave privada del cliente codificada en base32.

Tor Browser permite introducir la clave privada al visitar un servicio que la exige.

Aviso de seguridad del laboratorio: el script `labs/es/laboratorio/descubrimiento_restringido.py` comprueba primero que no existan directorios de autorización anteriores y se niega a sobrescribirlos. Aun así, ejecútalo *solo* en una red Chutney dedicada y desechable; nunca sobre un servicio real. Para una prueba manual, crea los archivos de autorización antes de recargar los nodos afectados.

Laboratorio 13: Servicio onion con descubrimiento restringido

Objetivo. Configurar un segundo servicio onion en `test009h`, accesible únicamente para un cliente autorizado, y comprobar el acceso con clave y sin ella.
Requisitos previos. Red de laboratorio en ejecución y servidor web local en el puerto 8080.
Tiempo estimado. 1 hora.

Paso 1: el par de claves del cliente. Generamos uno con la biblioteca `cryptography`: Este ejemplo Python es autónomo e imprime la línea de autorización pública. Guarda la clave privada `priv` por separado y de forma segura para escribir más tarde el archivo `.auth_private`; no la divulgues.

```
1 import base64
2 from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
3 from cryptography.hazmat.primitives.serialization import (
4     Encoding, PrivateFormat, PublicFormat, NoEncryption
```

```
)

sk = X25519PrivateKey.generate()
priv = sk.private_bytes(Encoding.Raw, PrivateFormat.Raw, NoEncryption())
pub = sk.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
b32 = lambda b: base64.b32encode(b).decode().rstrip("=")
print("descriptor:x25519:" + b32(pub))
```

Los valores base32 deben omitir los caracteres = del relleno final. El fragmento solo imprime la línea pública; *no guarda* la clave privada. Si se ejecuta como script independiente, `priv` se perderá al terminar el proceso. Para seguir los pasos manualmente, conserva la clave con permisos restrictivos antes de salir o utiliza el laboratorio automatizado, que genera el archivo de autorización del cliente y elimina las claves temporales al finalizar. Son procedimientos alternativos.

Paso 2: el servicio. Creamos `restricted_service/authorized_clients/alice.auth` con la clave pública. Añadimos estas líneas al `torrc` de `test009h`:

```
HiddenServiceDir /lab/net/nodes/009h/restricted_service
HiddenServicePort 80 127.0.0.1:8080
```

Después enviamos `HUP`. tor genera la dirección y publica los descriptores.

La siguiente orden obtiene la dirección onion real del archivo `hostname` que tor crea junto con el servicio: cada red tiene su propia dirección. El error SOCKS y los registros siguientes proceden de una sesión de referencia previa y pueden variar.

Paso 3: sin la clave. Desde el cliente de la red:

```
$ ONION_ADDR=$(cat labs/es/laboratorio/net/nodes/009h/restricted_service/hostname)
$ curl --noproxy "" --socks5-hostname 127.0.0.1:9008 \
    "http://${ONION_ADDR}/index.html"
(curl: SOCKS proxy error 97)
```

El registro del cliente indica:

```
[notice] Fail to decrypt descriptor for requested onion address.
         It is likely requiring client authorization.
```

El cliente encontró y descargó el descriptor, pero no puede descifrar su segunda capa. Ni siquiera conoce los puntos de introducción y no puede intentar conectarse. Los mensajes anteriores son registros originales del programa, no ejecuciones de un laboratorio español.

Advertencia: Un directorio vacío hace inaccesible el servicio

Si existe `authorized_clients/` pero no contiene archivos, en el experimento original el servicio intentó publicar un descriptor que los HSDir rechazaron:

```
[warn] Uploading hidden service descriptor: http status 400 ("Invalid HS
       descriptor. Rejected.") response from dirserver 127.0.0.1:5106.
```

La especificación exige al menos una línea `auth-client` en la capa supercifrada [rend-spec § Hidden service descriptors: encryption format]. Con el descubrimiento restringido activado pero sin clientes autorizados, la configuración observada no generaba ninguna, por lo que el descriptor era inválido. Es un peligro operativo cuando se revoca al último cliente o se crea el directorio antes de incorporar las claves. Un script de laboratorio interrumpido puede dejar también una configuración de prueba incompleta.

Paso 4: con la clave. Escribimos el archivo `.auth_private` en un directorio indicado por `ClientOnionAuthDir`, recargamos la configuración del cliente y repetimos la solicitud. Esta vez se entrega la página. El script `labs/es/laboratorio/descubrimiento_restringido.py` automatiza el experimento y restaura `torrc`.

21.4 Otros requisitos

21.4.1 Direcciones personalizadas

Una dirección que comienza con una palabra reconocible puede ayudar a identificarla. Se generan claves aleatorias hasta obtener el prefijo deseado, por ejemplo mediante una herramienta especializada como *mkp224o*. Cada carácter base32 adicional multiplica por 32 el número esperado de intentos. Un prefijo de seis o siete caracteres puede conseguirse en un tiempo razonable con un ordenador convencional, pero cada carácter sucesivo cuesta decenas de veces más. Como explicamos en apartado 14.2, un adversario también puede generar una dirección con el mismo prefijo. Este facilita reconocerla, pero no autentica el servicio. Protege las claves generadas por programas de terceros tanto como cualquier otra clave secreta.

21.4.2 Servicios de un solo salto

Un sitio público que no necesite ocultar su ubicación, como la edición onion de un periódico conocido, puede sacrificar el anonimato del servidor a cambio de velocidad. Con `HiddenServiceSingleHopMode` y `HiddenServiceNonAnonymousMode`, el servicio construye circuitos de un solo salto hacia los puntos de introducción y encuentro. El cliente permanece anónimo como de costumbre. La publicación del descriptor continúa utilizando circuitos de tres saltos: como comprobamos en apartado 14.9, los HSDir rechazan las solicitudes directas, detalle señalado explícitamente en el manual de tor. El término «NonAnonymous» en el nombre de la opción es intencionado: una instancia tor configurada así no debe utilizarse también como cliente anónimo.

Esta decisión también es difícil de revertir. Cuando un servicio ha operado sin anonimato, su dirección IP puede haber quedado vinculada a la dirección `.onion`. Desactivar simplemente la opción no restaura el anonimato: hacen falta una dirección nueva, claves nuevas, otro `HiddenServiceDir` y una instancia limpia.

21.4.3 Onionbalance

Un solo proceso tor constituye un cuello de botella y un punto único de fallo. Onionbalance [Theb] permite que varios servidores compartan una dirección. Cada uno ejecuta su propio servicio onion con sus puntos de introducción. Onionbalance conserva la clave de la dirección principal y publica un descriptor que combina los puntos de introducción de todos los servidores. Los clientes se distribuyen entre ellos de forma transparente. El Tor Project utiliza Onionbalance para sus propios sitios web.

Resumen

- Un servicio onion necesita dos líneas de configuración. Su clave secreta *controla* la dirección y debe protegerse y respaldarse.
- El servidor debe escuchar solo en localhost o en un socket Unix y evitar revelar información sobre sí mismo en las respuestas o conexiones externas.

- Onion-Location anuncia a los visitantes de Tor Browser la versión onion de un sitio web.
- El descubrimiento restringido utiliza claves X25519: un cliente no autorizado ni siquiera puede descifrar el descriptor.
- Las direcciones personalizadas, los servicios de un salto y Onionbalance responden a necesidades de reconocimiento, velocidad y escalabilidad.

Ejercicios

Ejercicio 21.1 ⋆ ¿Cuántos intentos se necesitan de media para generar un prefijo base32 de seis caracteres? ¿Y de ocho? Si un ordenador prueba diez millones de claves por segundo, ¿cuánto tardaría en cada caso?

Ejercicio 21.2 ⋆⋆ Un servicio onion muestra `Apache/2.4.62 (Debian) Server at 192.0.2.10 Port 80` en sus páginas de error. ¿Qué información revela? ¿Cómo puede evitarse la filtración?

Ejercicio 21.3 ⋆⋆ En el laboratorio 13, ¿cómo revocarías el acceso de Alice? ¿Qué ocurre con los clientes que ya han descargado el descriptor?

Ejercicio 21.4 ⋆⋆⋆ Modifica el servicio de laboratorio para reenviar conexiones a un socket Unix en lugar de un puerto TCP local. ¿Qué ventajas adicionales ofrece, además de evitar un puerto expuesto?

Lecturas complementarias

- Las entradas del manual de tor sobre `HiddenServiceDir`, `HiddenServicePort` y autorización de clientes.
- Documentación del Tor Project sobre servicios onion y Onionbalance [Theb].

Capítulo 22

Administrar un repetidor o un puente

Última verificación: 23 de septiembre de 2026 — tor 0.4.9.13 (manual); red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- elegir entre repetidor intermedio, salida y puente según sus recursos y los riesgos que pueda asumir;
- escribir la configuración esencial de cada uno, incluidos límites de ancho de banda y almacenamiento de claves fuera de línea;
- leer y escribir una política de salida y comprobar sus efectos en la red de laboratorio (laboratorio 14);
- declarar correctamente una familia de repetidores.

La red Tor existe porque miles de personas y organizaciones aportan ancho de banda y equipos. En capítulos anteriores examinamos los repetidores desde la perspectiva del cliente; ahora cambiaremos de lado. Este no es un manual de administración de sistemas —el Tor Project mantiene una guía actualizada para operadores—, sino una explicación de lo que significan las principales decisiones operativas en términos del protocolo.

22.1 ¿Qué tipo de nodo?

Repetidor intermedio Puede acabar convirtiéndose también en guardia. Reenvía celdas entre repetidores sin abrir conexiones a destinos externos de Internet. Es el papel menos exigente. Su dirección aparece en el consenso, pero los sitios web no ven tráfico procedente de ella.

Salida Abre conexiones a destinos en nombre de los clientes. Es especialmente valiosa porque las salidas escasean en relación con la demanda (apartado 13.2), pero también implica mayor exposición: los destinatarios de tráfico abusivo ven la IP de la salida como origen aparente.

Puente Repetidor no publicado que actúa como primer salto para quien no puede acceder a la red pública (apartado 16.2). Incluso una conexión doméstica modesta puede ayudar si se combina con un transporte conectable como obfs4 o WebTunnel.

Advertencia: Antes de comenzar

Administrar una salida tiene consecuencias legales y operativas, como denuncias de abusos y solicitudes de autoridades, examinadas en capítulo 25. Considera empezar con un repetidor intermedio o un puente, informa a tu proveedor de conectividad y evita utilizar la misma conexión para administrar un repetidor y realizar actividades delicadas como cliente Tor.

22.2 Configuración esencial

Un repetidor intermedio solo requiere unas líneas de `torrc`:

```
Nickname        MyRelay
ContactInfo     operator@example.org
ORPort          443
ExitRelay       0
SocksPort       0
RelayBandwidthRate  4 MBytes
RelayBandwidthBurst 8 MBytes
```

`ContactInfo` permite que otras personas contacten con el operador cuando surjan problemas. El manual recuerda que los descriptores se archivan y publican, por lo que esa dirección puede permanecer disponible públicamente indefinidamente. `ExitRelay 0` hace explícito el comportamiento predeterminado de las versiones recientes: con `auto`, tor solo se convierte en salida cuando el operador escribe una política de salida. `RelayBandwidthRate` y `RelayBandwidthBurst` limitan el ancho de banda medio y máximo. En cambio, `AccountingMax` limita el tráfico total durante un período contable, por ejemplo un mes. Cuando se alcanza ese límite, el repetidor deja de aceptar tráfico nuevo hasta el siguiente período.

Nota operativa. Con `ExitRelay` en modo `auto`, las opciones `ReducedExitPolicy` o `IPv6Exit` también pueden activar el papel de salida; no solo lo hacen las reglas `ExitPolicy`. Por el contrario, `ExitRelay 0`, como en el ejemplo, prohíbe todo tráfico de salida. Comprueba la configuración efectiva antes de desplegar un repetidor público.

22.2.1 La vida de un repetidor nuevo

Un repetidor recién iniciado publica su descriptor ante las autoridades, que lo incluyen en el consenso y comienzan a medirlo (apartado 12.4). Al principio recibe poco tráfico: los clientes seleccionan repetidores de forma proporcional a sus pesos de ancho de banda, que siguen siendo pequeños hasta estabilizarse las mediciones. Los indicadores se obtienen con el tiempo (apartado 12.4.3). `Stable` exige funcionamiento prolongado sin interrupciones, y `Guard` requiere además antigüedad suficiente y alta disponibilidad. Al convertirse en guardia, un repetidor puede observar inicialmente una reducción del tráfico: los clientes que ya eligieron guardias no lo seleccionan como intermedio con la misma frecuencia. Después crece gradualmente el tráfico a medida que nuevos clientes lo adoptan (apartado 13.3).

Dos tipos de contratiempos no son equivalentes. Reiniciar con frecuencia sin cambiar la identidad conserva las mediciones de ancho de banda, pero reduce la disponibilidad sostenida necesaria para `Stable` y `Guard`: esos indicadores deben recuperarse. Cambiar las *claves de identidad*, en cambio, crea un repetidor completamente nuevo desde el punto de vista de las autoridades, que vuelve a empezar desde cero incluso en las mediciones de ancho de banda.

22.2.2 Claves

La identidad del repetidor es su clave primaria Ed25519 (apartado 7.2). Por defecto, tor la guarda en el disco del repetidor. Con `OfflineMasterKey 1`, la clave maestra permanece en otro ordenador; al servidor solo se transfieren claves de firma temporales, certificadas mediante `tor --keygen`. Si el servidor se ve comprometido, el adversario obtiene claves que caducan, no la identidad a largo plazo. Este procedimiento exige disciplina: las claves temporales deben renovarse antes de caducar o el repetidor dejará de funcionar.

22.3 Política de salida

Una política de salida es una lista ordenada de reglas `accept` y `reject`, cada una con una dirección o red y un puerto o intervalo de puertos. Tor procesa las reglas **en orden**: se aplica la primera que coincida. Las reglas definidas por el operador preceden a las predeterminadas del manual. Por defecto se rechazan puertos utilizados por correo electrónico (25), NNTP (119), NetBIOS y SMB (135–139, 445) y determinadas aplicaciones de intercambio de archivos; se acepta el tráfico restante. Por separado, tor rechaza por defecto los destinos privados, incluidos `127.0.0.0/8`, `10.0.0.0/8` y `192.168.0.0/16`, además de la dirección pública del propio repetidor. Así protege la red interna del operador (`ExitPolicyRejectPrivate`). Nuestro laboratorio desactiva esa protección para permitir conexiones a `127.0.0.1`.

Orden de las protecciones. Cuando `ExitPolicyRejectPrivate` está activo (valor predeterminado), las reglas que rechazan redes privadas y destinos locales se *anteponen* a las reglas de salida definidas por el operador. La red privada Chutney desactiva esa protección deliberadamente; no copies este ajuste en una salida pública.

Una salida limitada a tráfico web puede utilizar:

```
ExitRelay  1
ExitPolicy accept *:80
ExitPolicy accept *:443
ExitPolicy reject *:*
```

El Tor Project también sugiere `ReducedExitPolicy` como compromiso habitual: permite un conjunto de puertos de servicios frecuentes y excluye otros que generan más quejas por abusos.

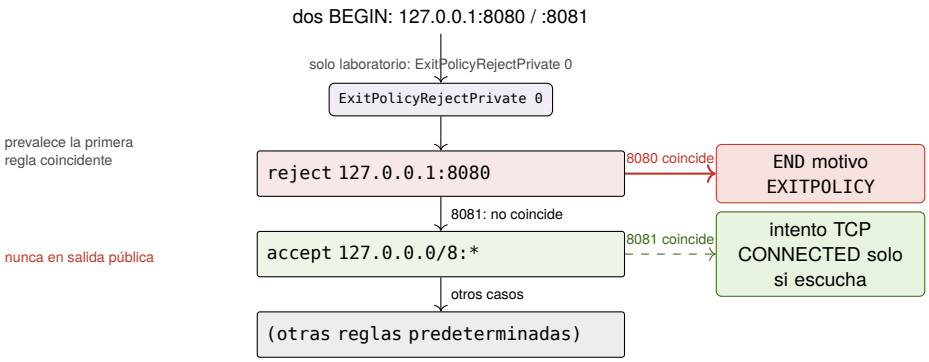


Figura 22.1: Política del laboratorio Chutney, donde `ExitPolicyRejectPrivate 0` se utiliza solo para permitir destinos privados en pruebas; normalmente Tor antepone su rechazo a las reglas del operador. `BEGIN` hacia `127.0.0.1:8080` coincide con `reject` y produce `END EXITPOLICY`; el puerto 8081 omite esa regla y coincide con `accept`. La aceptación permite intentar una conexión TCP, pero no garantiza `CONNECTED` sin un servidor que escuche. Invertir las reglas anula el rechazo específico.

Qué reciben los clientes. El descriptor del repetidor contiene la política completa, mientras el consenso y los microdescriptores transportan un resumen basado solo en puertos, como `p accept 80,443`. El cliente lo utiliza para decidir si una salida podría atender una solicitud (apartado 13.1). Como ese resumen omite las direcciones, puede descubrir un rechazo específico para una dirección únicamente al intentar conectarse, como muestra el laboratorio.

22.4 Laboratorio: comprobar una política de salida

Laboratorio 14: Una regla de salida y sus consecuencias

Objetivo. Añadir una regla a una salida del laboratorio y utilizar `labs/es/codigo/torlab` para observar las respuestas a destinos aceptados y rechazados.
Requisitos previos. Laboratorios 3 y 4; red de laboratorio en funcionamiento.
Tiempo estimado. 1 hora.

Paso 1: la regla. En el `torrc` de `test006e`, insertamos una regla que rechace un puerto antes de las reglas existentes que permiten `127.0.0.0/8`:

```
ExitPolicy reject 127.0.0.1:8080
ExitPolicy accept 127.0.0.0/8:*
```

Estas dos líneas muestran su orden relativo. Al modificar el `torrc` existente, inserta únicamente la nueva regla `reject` antes de la primera línea `ExitPolicy`; no dupliques un `accept` ya presente. El orden importa. Si colocáramos el rechazo después de `accept`, no tendría efecto, porque prevalece la primera regla coincidente. Ese error quedó documentado en el experimento de referencia anterior. Reiniciamos el repetidor y abrimos dos servidores HTTP pequeños en los puertos 8080 y 8081 del contenedor.

Paso 2: tres solicitudes. Con `build` y `stream.py`, construimos un circuito que termina en `006e` y enviamos tres mensajes `BEGIN` hacia los puertos 8080, 8081 y 9999, donde no escucha ningún servidor. Las respuestas, en una de nuestras sesiones (los códigos, la dirección y el TTL pueden cambiar según la versión y el entorno):

```
127.0.0.1:8080 -> END  reason=EXITPOLICY address=127.0.0.1 TTL=1800
127.0.0.1:8081 -> CONNECTED
127.0.0.1:9999 -> END  reason=MISC
```

Paso 3: interpretar las respuestas. El rechazo por política produce la razón `EXITPOLICY`. La celda `END` contiene también la dirección resuelta y un TTL. El cliente puede recordar que esa salida no atiende el destino y elegir otra sin esperar un descriptor nuevo. Por el contrario, el puerto sin servidor disponible produce `MISC`, y no `CONNECTREFUSED`, aunque la especificación contempla esta última razón para conexiones rechazadas [tor-spec § Closing streams].

En profundidad: ¿Por qué `MISC` y no `CONNECTREFUSED`?

La respuesta aparece en los registros `info` de la salida. Seguir su pista es un buen ejemplo de cómo investigar una discrepancia entre especificación e implementación:

```
[info] connection_handle_write_impl(): in-progress connect failed.
       Removing. (Connection refused)
[info] errno_to_stream_end_reason(): Didn't recognize errno 115 (Operation
       now in progress); telling the client that we are ending a stream
```

```
      for 'misc' reason.
```

La salida *sabe* que se rechazó la conexión, pero obtiene la razón a partir de otro valor. Las conexiones hacia destinos no son bloqueantes: `connect()` devuelve inicialmente `EINPROGRESS` (errno 115) y el resultado definitivo está disponible después mediante un evento de escritura. Entonces `connection.c` recupera el error real con `getsockopt(SO_ERROR)`, registra «Connection refused» y llama a `connection_edge_end_errno()`. Sin embargo, esa función vuelve a calcular la razón mediante `errno_to_stream_end_reason(tor_socket_errno(…))`. En Unix, `tor_socket_errno` se define como la macro `(errno)` (`src/lib/net/socket.h`), que hace referencia a la variable global donde todavía figura `EINPROGRESS`. El valor 115 no está entre los reconocidos en `src/core/or/reasons.c`, por lo que se devuelve `MISC`.
La información correcta existe, pero no se utiliza. Las dos ramas del código también difieren: la de conexiones entre repetidores transmite el error recuperado, mientras la correspondiente al flujo lo ignora. Según el informe y el parche conservados en `docs/it/segnalazioni/`, una prueba anterior con tor 0.4.9.13 en una red privada, usando el error correcto en esa rama, cambió la tercera respuesta por `CONNECTREFUSED`. Es un defecto menor sin consecuencias para el anonimato: el cliente sabe que el flujo falló, pero pierde información útil. `CONNECTREFUSED` indicaría que la salida alcanzó el destino y no tendría sentido repetirlo por otra. El análisis y el parche están disponibles en `docs/it/segnalazioni/` para el Tor Project. Este caso también demuestra por qué conviene cotejar especificaciones y comportamiento real.

El script `labs/es/laboratorio/politica_salida.py` implementa las tres pruebas, valida las respuestas y utiliza `finally` para intentar restaurar el `torrc` de `006e`. En una red Chutney dedicada puede invocarse desde la raíz del repositorio con `make -C labs/es/laboratorio politica-salida`; no forma parte de la comprobación Chutney ordinaria `make -C labs/es/laboratorio verify`. No lo ejecutes sobre un repetidor real. Un detalle importante: el circuito necesita tres saltos. Una salida rechaza reenviar tráfico de un circuito de un solo salto y devolvería `END` con razón `TORPROTOCOL` (apartado 15.3).

22.5 Familias de repetidores

Los operadores que administran varios repetidores deben declarar una **familia**. Los clientes nunca incluyen dos repetidores de la misma familia en un circuito (apartado 13.1), porque un operador común podría correlacionar tráfico. El mecanismo histórico es `MyFamily`: cada repetidor enumera las huellas de todos los demás y la pertenencia cuenta únicamente cuando las declaraciones son recíprocas. Administrar diez repetidores exige mantener diez listas coincidentes, todas las cuales deben modificarse al añadir uno.

El mecanismo introducido con tor 0.4.9 utiliza en su lugar una **clave de familia**. El operador crea una mediante `tor --keygen-family MyFamilyKey`, copia el archivo generado `MyFamilyKey.secret_family_key` al directorio de claves (`KeyDirectory`) de cada repetidor, conservando el sufijo `.secret_family_key`, y añade a cada `torrc` la línea `FamilyId` obtenida. La clave compartida debe protegerse y guardarse en una copia de seguridad; si se filtra, hay que sustituirla en toda la familia. Cada repetidor utiliza esa clave para firmar un certificado que demuestra su pertenencia. Los verificadores ya no necesitan comparar listas. Durante la transición, es recomendable configurar ambos mecanismos.

En profundidad: Por qué conviene declarar una familia

Nada obliga técnicamente a declarar los repetidores como familia; un adversario incluso podría evitarlo deliberadamente. Sin embargo, las autoridades y los voluntarios que vigilan la red buscan características compartidas, como redes comunes, versiones, horas de inicio simultáneas y datos de contacto. Una familia no declarada puede atraer investigaciones y hacer que se excluyan sus repetidores del consenso. Declararla forma parte de la relación de confianza en que se basa la red.

No incluyas puentes en listas públicas `MyFamily`: hacerlo podría revelar su relación con repetidores publicados. La guía de Tor aconseja expresamente no utilizar esta opción en puentes. Además, `tor --keygen-family MyFamilyKey` puede sobrescribir archivos de claves que ya existan con ese nombre: utiliza uno nuevo y protege el archivo secreto.

22.6 Administrar un puente

Un puente obfs4 puede configurarse como en el laboratorio 10 (apartado 16.5):

```
BridgeRelay 1
ORPort 9001
ServerTransportPlugin obfs4 exec /usr/bin/obfs4proxy
ServerTransportListenAddr obfs4 0.0.0.0:443
ExtORPort auto
BridgeDistribution any
```

`BridgeDistribution` indica al sistema de distribución, actualmente rdsys, cómo desea el operador distribuir el puente. `any` deja la decisión al sistema; `none` lo excluye completamente, opción útil para compartirlo de forma privada con personas de confianza. Los canales disponibles, como web, correo electrónico y Telegram, figuran en la página de información de puentes. Un puente no aparece en el consenso público, pero su descriptor sí llega a la autoridad de puentes, que comprueba su accesibilidad.

El transporte debe escuchar en un puerto diferente de ORPort y, cuando sea posible, en uno común como el 443. Ese es el puerto que utilizan los clientes y observa el censor (apartado 16.4).

Resumen

- Administrar un repetidor intermedio es la forma menos exigente de contribuir; una salida es especialmente valiosa, pero implica más exposición; un puente ayuda a quienes sufren censura.
- Solo unas pocas opciones son esenciales: identidad, contacto, puertos y límites de ancho de banda. La clave maestra de identidad puede mantenerse fuera de línea.
- La política de salida es una lista ordenada en la que prevalece la primera regla coincidente; los destinos privados se rechazan por defecto.
- El rechazo por política llega al cliente como mensaje `END` con razón `EXITPOLICY`, dirección y TTL.
- Quienes administren varios repetidores deben declararlos como familia mediante `MyFamily` o el mecanismo reciente de claves de familia.

Ejercicios

Ejercicio 22.1 ⋆ Dada la política `accept *:80`, `reject 192.0.2.0/24:*`, `reject *:*`, ¿puede un cliente conectarse por esa salida a `192.0.2.7:80`? Reescribe la política para impedirlo.

Ejercicio 22.2 ⋆ ¿Por qué el resumen basado únicamente en puertos del consenso es insuficiente para decidir si una salida aceptará una conexión? ¿Qué hace el cliente al recibir un `END` con razón `EXITPOLICY`?

Ejercicio 22.3 ⋆⋆ Un operador administra cinco repetidores con `MyFamily` y añade un sexto, pero actualiza la configuración solo en el nuevo. ¿Qué observan los clientes? ¿Qué cambia al utilizar una clave de familia?

Ejercicio 22.4 ⋆⋆ Comprueba la explicación del análisis técnico: activa registros `info` en la salida y repite las tres solicitudes. ¿Qué cambiaría si `connection_edge_end_errno()` recibiera el error devuelto por `getsockopt`? ¿Qué otras razones de cierre de flujos podrían comunicarse correctamente?

Ejercicio 22.5 ⋆⋆⋆ Un repetidor dispone de `AccountingMax 1 TBytes` al mes y 10 MB/s de ancho de banda. ¿Después de cuántos días de funcionamiento a velocidad máxima agotaría su cuota? ¿Cómo afecta a la red un repetidor apagado medio mes frente a otro que funciona todo el mes con la mitad del ancho de banda?

Lecturas complementarias

- La guía para operadores de repetidores del Tor Project, `https://community.torproject.org/relay/`.
- Las entradas del manual de tor sobre `ExitPolicy`, `ReducedExitPolicy`, `FamilyId`, `OfflineMasterKey` y `BridgeDistribution`.
- La especificación de razones de cierre de flujos, [tor-spec § Closing streams].

Capítulo 23

Tor Browser, Tails y Whonix

Última verificación: 23 de septiembre de 2026 — documento de diseño de Tor Browser (2018); manual de Tor Browser; documentación de Tails y Whonix

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar por qué no basta con configurar un navegador convencional para utilizar Tor;
- enumerar los requisitos de seguridad y privacidad en los que se basa el diseño de Tor Browser;
- relacionar el aislamiento por sitio de Tor Browser con el aislamiento de flujos de tor;
- comparar qué protegen Tor Browser, Tails y Whonix y los costes de cada enfoque.

El modelo de amenazas de 2004 indicaba expresamente que Tor no normaliza los protocolos de aplicación (apartado 4.2). Si el navegador revela tu identidad, el encaminamiento cebolla no puede salvarte. En este capítulo examinaremos herramientas diseñadas precisamente para proteger esa capa: Tor Browser, que hace coherente el navegador con las protecciones de la red, y dos sistemas operativos, Tails y Whonix, que extienden el enfoque al equipo completo.

23.1 Por qué no basta con un proxy

Configurar un navegador convencional para utilizar el SocksPort de tor oculta la dirección IP en muchas circunstancias, pero no en todas, y apenas oculta nada más:

- Algunos componentes pueden conectarse fuera del proxy: resolución DNS local, WebRTC, extensiones o programas externos lanzados desde el navegador.
- Las cookies y otros datos del navegador vinculan las visitas de hoy con las de ayer, quizá realizadas sin Tor.
- Un sitio puede recopilar decenas de características del navegador —tamaño de ventana, fuentes instaladas, idioma, zona horaria y detalles del renderizado— y combinarlas en una **huella digital** (*fingerprint*) que distingue a un usuario entre millones.
- Si todas las pestañas utilizan el mismo circuito, un rastreador incrustado en diversos sitios podría observar solicitudes procedentes de la misma salida al mismo tiempo.

El primer problema amenaza la ocultación de la dirección; los restantes amenazan la *imposibilidad de vincular actividades*, uno de los objetivos fundamentales de Tor.

23.2 Tor Browser

Tor Browser es una versión modificada de Firefox, basada en la rama de soporte extendido de Mozilla y distribuida por el Tor Project junto con tor. Su diseño se describe en un documento que, aunque ya no refleje todos los detalles actuales, sigue siendo una declaración importante de principios [Per+18]. Sus requisitos se dividen en dos grupos.

Requisitos de seguridad. Se refieren al uso seguro de Tor; incumplirlos puede revelar inmediatamente a un usuario:

Obediencia al proxy Ningún contenido puede eludir tor.

Separación del estado No puede compartirse información con otros navegadores ni con sesiones externas a Tor.

Sin rastros persistentes Nada que revele la actividad de navegación debe permanecer en el disco ni en la memoria más allá de la sesión, salvo que el usuario elija conservarlo expresamente.

Aislamiento de aplicaciones Todo lo que el navegador deje en el sistema debe poder eliminarse borrándolo.

Requisitos de privacidad. Evitan relacionar actividades diferentes y se organizan en torno al **origen de la barra de direcciones**: el sitio que visita el usuario.

Identificadores Los datos guardados por un tercero, por ejemplo un rastreador integrado en varios sitios, mientras se visita uno no deben quedar disponibles para ese tercero cuando aparece en otro.

Huellas digitales Las características del navegador no deben permitir vincular las visitas a sitios distintos.

Desvinculación a largo plazo El usuario debe poder eliminar todo el estado y obtener una identidad nueva mediante una acción sencilla.

El documento también formula un principio útil: las protecciones no deberían contradecir lo que el usuario espera de un navegador. Su predecesora, la extensión Torbutton, permitía activar y desactivar Tor dentro del mismo navegador, con el riesgo de confundir pestañas abiertas bajo estados diferentes. Una interfaz que induce errores es por sí misma una debilidad de seguridad.

23.2.1 Un circuito por sitio

El aislamiento por origen llega hasta la red. Tor Browser identifica el sitio de la barra de direcciones en las credenciales SOCKS que envía a tor. Por defecto, tor aísla los flujos con credenciales distintas (`IsolateSOCKSAuth`) en circuitos diferentes. Verificamos ese mecanismo en el laboratorio 11 (apartado 18.3). Desde el punto de vista del usuario, como explica el manual [Thed], dos sitios que incluyen el mismo rastreador utilizan circuitos distintos, mientras que las pestañas del mismo sitio comparten uno.

Dos acciones modifican esta disposición:

Nuevo circuito para este sitio Recarga el sitio actual mediante un circuito nuevo, por ejemplo cuando una salida no puede alcanzarlo. No borra el estado del navegador ni cambia los circuitos de los demás sitios.

Nueva identidad Cierra todas las pestañas, elimina cookies, historial y demás estado y obtiene nuevos circuitos para todas las actividades, cumpliendo el requisito de desvinculación a largo plazo.

Ninguna cambia el guardia, que permanece estable durante meses por los motivos explicados en apartado 13.3.

El aislamiento por sitio garantiza una *separación entre sitios*, no que un mismo sitio mantenga siempre un único circuito durante toda la sesión: los circuitos caducan y pueden sustituirse. «Nuevo circuito para este sitio» recarga la pestaña activa; las demás pestañas de ese sitio adoptarán el nuevo circuito cuando se recarguen, mientras que las conexiones ya abiertas no tienen por qué interrumpirse. «Nueva identidad» también detiene la actividad y las descargas en curso. Ninguna de las dos órdenes está destinada a cambiar el guardia, que tor gestiona por separado.

23.2.2 Resistencia a las huellas digitales

La estrategia de Tor Browser frente al fingerprinting no consiste en hacer diferente a cada usuario, sino en hacer que todos se parezcan. Si los usuarios exponen los mismos valores, una huella dice poco más que «esta persona utiliza Tor Browser». De esta idea derivan varias decisiones de diseño visibles. Las ventanas se abren con dimensiones redondeadas y al redimensionarlas aparecen márgenes alrededor del contenido para mantener las dimensiones declaradas en incrementos normalizados (*letterboxing*). La zona horaria declarada es UTC. Muchas interfaces que revelan detalles del sistema operativo están deshabilitadas o normalizadas. Esto explica por qué se desaconseja instalar extensiones o cambiar ajustes: cada diferencia reduce el conjunto de personas entre las que puede ocultarse un usuario, el principio del conjunto de anonimato explicado en capítulo 1.

La meta es *reducir las diferencias observables*, no garantizar huellas literalmente idénticas. La plataforma, el idioma, el nivel de seguridad y otras propiedades aún pueden distinguir grupos de usuarios. Por tanto, se reduce la singularidad, sin prometer una uniformidad perfecta.

23.2.3 Niveles de seguridad

Muchas vulnerabilidades del navegador aprovechan JavaScript y otras funciones complejas. Tor Browser ofrece tres niveles [Thed]:

Estándar Todas las funciones están habilitadas; es el valor predeterminado.

Más seguro JavaScript se desactiva en sitios sin HTTPS; se deshabilitan determinadas fuentes y símbolos matemáticos, y el audio y el vídeo solo comienzan tras un clic.

El más seguro JavaScript queda desactivado en todas partes y también pueden restringirse imágenes y otros contenidos. Los sitios estáticos siguen siendo utilizables.

Elegir un nivel supone equilibrar seguridad y facilidad de uso. La propia elección puede afectar a la huella digital, porque un sitio podría inferir el nivel seleccionado.

En profundidad: Compilaciones reproducibles

Quien descarga Tor Browser necesita confiar en que el binario distribuido corresponda al código publicado. El Tor Project utiliza *compilaciones reproducibles*: el mismo código fuente y el mismo entorno de construcción deberían producir archivos idénticos bit a bit. El documento de diseño describe el trabajo necesario para eliminar causas de diferencias,

como fechas, rutas de archivos y orden de miembros de un archivo comprimido [Per+18]. Una compilación independiente y la comparación de los resultados permiten detectar modificaciones maliciosas introducidas en un entorno de distribución.

23.3 Más allá del navegador

Tor Browser protege su propio tráfico, no el de otras aplicaciones del mismo equipo. Estas pueden seguir conectándose directamente, y un error o una vulnerabilidad explotada del navegador podría revelar la dirección real. Dos sistemas operativos amplían la protección al equipo completo mediante arquitecturas diferentes (figura 23.1).

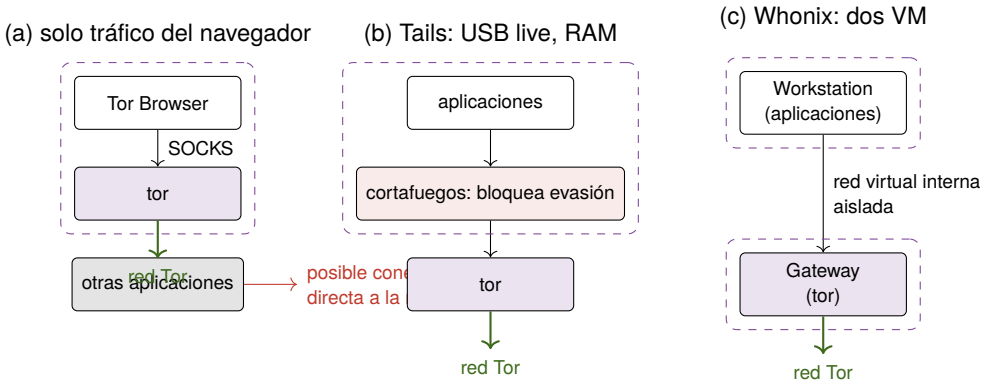


Figura 23.1: Tres modelos de encaminamiento y aislamiento, no garantías absolutas de anonimato. (a) Tor Browser dirige por tor su propio tráfico, mientras que otras aplicaciones del anfitrión pueden conectarse directamente. (b) Tails normalmente impide con reglas de cortafuegos que las aplicaciones eviten Tor; el *Unsafe Browser* para portales cautivos es una excepción explícita no anónima. Tails arranca desde USB, funciona principalmente en RAM y admite almacenamiento persistente opcional. (c) Whonix separa la Workstation con aplicaciones y el Gateway con tor en dos máquinas virtuales comunicadas mediante una red virtual aislada; un anfitrión o hipervisor comprometido aún puede invalidar esa protección.

23.3.1 Tails

Tails es un sistema operativo basado en Debian que se inicia desde una memoria USB [Taia]. Tiene dos propiedades fundamentales:

Todo a través de Tor Se bloquean las aplicaciones que intentan acceder a Internet sin pasar por tor.

Amnesia El sistema se ejecuta en memoria, no escribe en el disco del equipo y olvida su estado al apagarse. Los datos que el usuario desea conservar se guardan opcionalmente en un *almacenamiento persistente* cifrado dentro de la memoria USB.

Tails incluye Tor Browser y otras aplicaciones cuidadosamente configuradas. El proyecto se incorporó al Tor Project en septiembre de 2024 [The24c]. Su documentación es explícita sobre sus limitaciones: Tails no puede proteger frente a hardware manipulado, como un registrador físico de pulsaciones, ni frente a una memoria USB preparada en un ordenador ya infectado.

Estas propiedades describen el funcionamiento habitual de Tails y *no garantizan de forma absoluta* que ningún dispositivo conserve rastros. El usuario puede montar discos expresamente o crear archivos persistentes; el malware, el hardware comprometido y las actividades fuera de Tails también pueden superar el modelo de amenazas.

23.3.2 Whonix

Whonix separa las funciones en dos máquinas virtuales. La **Gateway** ejecuta tor y es la única máquina virtual conectada a la red exterior. La **Workstation** ejecuta las aplicaciones y solo se comunica con la Gateway mediante una red virtual interna. Por arquitectura, todo el tráfico de la Workstation debe pasar por tor y no depende únicamente de configurar cada aplicación. Frente a Tails, proporciona una separación más estricta: incluso un adversario que obtenga privilegios administrativos en la Workstation no puede averiguar directamente la dirección real, porque esa máquina no la conoce. Las contrapartidas son la dependencia de la seguridad del sistema de virtualización y la ausencia de amnesia, salvo que Whonix se ejecute dentro de un entorno que la proporcione.

Separar Gateway y Workstation crea una barrera útil, pero *no una garantía absoluta* frente a una Workstation comprometida. Un atacante aún puede identificar al usuario mediante datos de las aplicaciones y, si compromete el anfitrión, el hipervisor o la Gateway, también puede superar la separación de red. La integridad del cortafuegos y una configuración correcta siguen siendo esenciales.

Un puerto de control filtrado. Algunas aplicaciones de la Workstation, incluida la función Nueva identidad de Tor Browser, necesitan comunicarse con el ControlPort de tor en la Gateway. Pero el protocolo de control (apartado 18.1) también permite solicitar la dirección pública de tor mediante `GETINFO address`, precisamente lo que la Workstation no debería conocer. Por eso Whonix introduce *onion-grater*, un proxy que permite únicamente los comandos expresamente autorizados [Who]. Una función inocua en un contexto puede convertirse en filtración en otro.

Advertencia: Las herramientas no sustituyen los buenos hábitos

Tor Browser, Tails y Whonix reducen los posibles errores técnicos, pero no los conductuales: entrar en una cuenta personal, abrir documentos descargados fuera de un entorno protegido o divulgar datos identificativos. En capítulo 28 examinaremos esa dimensión.

Resumen

- Un navegador ordinario detrás de Tor puede filtrar tráfico fuera del proxy y, sobre todo, vincular actividades mediante estado compartido, huellas digitales o circuitos.
- Tor Browser responde a requisitos explícitos de diseño: obediencia al proxy, separación del estado, persistencia limitada, desvinculación entre sitios y una función Nueva identidad accesible.
- El aislamiento por sitio depende de credenciales SOCKS y del aislamiento de flujos de tor.
- Tor Browser combate las huellas digitales normalizando las características observables de sus usuarios; los niveles de seguridad reducen la superficie de ataque.

- Tails fuerza todo el tráfico a pasar por Tor y evita rastros persistentes; Whonix aísla las aplicaciones en una máquina que no conoce la dirección de red real.

Ejercicios

Ejercicio 23.1 ⋆ En términos de circuitos y estado del navegador, ¿qué diferencia hay entre «Nuevo circuito para este sitio» y «Nueva identidad»? ¿Por qué ninguna acción cambia el guardia?

Ejercicio 23.2 ⋆ Un usuario instala una extensión para bloquear publicidad en Tor Browser porque cree que mejorará la privacidad. ¿Qué podría ganar y perder?

Nota metodológica. El laboratorio 11 simula credenciales SOCKS distintas; no abre tres pestañas reales de Tor Browser. Credenciales iguales pueden reutilizar un circuito, pero no se garantiza en todo momento, porque los circuitos caducan y pueden intervenir otras reglas de aislamiento.

Ejercicio 23.3 ⋆⋆ Utiliza la red de laboratorio y el script del laboratorio 11 para simular tres pestañas: dos que visitan el mismo sitio y otra que visita uno diferente, usando el nombre del sitio como credencial SOCKS. Comprueba mediante el ControlPort qué flujos comparten circuito.

Ejercicio 23.4 ⋆⋆ En cada escenario, identifica qué herramienta entre Tor Browser, Tails y Whonix satisface las necesidades y justifica tu elección: (a) una persona que visita sitios censurados desde su ordenador doméstico; (b) un periodista que procesa documentos confidenciales en equipos ajenos; (c) un investigador que ejecuta varios programas de larga duración que deben utilizar Tor.

Ejercicio 23.5 ⋆⋆⋆ Elige cinco propiedades del navegador que los sitios puedan consultar mediante JavaScript y estima cuántos bits de información revela cada una en un navegador corriente. ¿Cómo las normaliza Tor Browser y qué diferencias siguen existiendo entre sus usuarios?

Lecturas complementarias

- M. Perry, E. Clark, S. Murdoch y G. Koppen, *The Design and Implementation of the Tor Browser* [Per+18].
- El manual de Tor Browser [Thed].
- La documentación de Tails [Taia] y de Whonix, especialmente la página de onion-grater [Who].

Capítulo 24

Medir la red: Tor Metrics y OONI

Última verificación: 23 de septiembre de 2026 — datos de Tor Metrics hasta el 20 de septiembre de 2026; consenso del 23 de septiembre de 2026 a las 15:00; especificaciones OONI ts-023 y ts-030

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- describir los datos que publica el Tor Project y su procedencia;
- explicar cómo se estima el número de usuarios en una red diseñada para no conocerlos y cuál es la incertidumbre de esas estimaciones;
- extraer estadísticas de un consenso real (laboratorio 15);
- explicar cómo mide OONI el bloqueo de Tor desde el punto de vista de los usuarios.

Una red de anonimato plantea un problema peculiar a sus operadores: mejorarla requiere medirla, pero medir implica recopilar información precisamente sobre aquello que debe proteger. El enfoque del Tor Project consiste en medir con moderación, agregar y publicar resultados y documentar cada paso para que cualquier persona pueda reproducir los cálculos [Thec]. En este capítulo seguiremos el recorrido de los datos desde su origen hasta los gráficos y después observaremos la red desde fuera mediante las mediciones de OONI.

24.1 Fuentes de datos

Casi todo lo que se conoce públicamente sobre la red Tor procede de documentos ya estudiados:

Consensos y descriptores (capítulo 12): lista de repetidores, indicadores, pesos de ancho de banda, versiones y políticas de salida.

Descriptores extra-info Publicados por cada repetidor junto con su descriptor, contienen recuentos históricos de bytes transferidos y estadísticas agregadas, entre ellas el número de solicitudes de directorio por país.

Mediciones activas OnionPerf descarga periódicamente archivos a través de Tor desde distintas ubicaciones mundiales y registra tiempos de transferencia y fallos.

CollecTor archiva estos documentos desde los primeros años de la red y permite descargarlos. **Tor Metrics** genera gráficos y tablas a partir de ellos, acompañados de datos CSV descargables y métodos documentados. **Relay Search** facilita consultas sobre repetidores concretos. El consenso empleado en los laboratorios del libro (`fonti/collector`) procede de CollecTor.

24.2 Contar usuarios anónimos

Tor no registra usuarios y ningún repetidor puede saber si dos conexiones pertenecen a la misma persona. ¿Cómo puede, entonces, estimar la red que tiene unos tres millones de usuarios? El método es indirecto (figura 24.1). Cada cliente descarga periódicamente un consenso y las solicitudes al directorio pueden contarse sin identificar al solicitante. Los repetidores que prestan el servicio de directorio cuentan las solicitudes correctas cada día, agrupadas por país deducido de la dirección de origen. Publican los totales agregados en descriptores extra-info, redondeando los recuentos por país al múltiplo de ocho superior para que las cifras pequeñas y exactas revelen menos información [dir-spec § Extra-info document format].

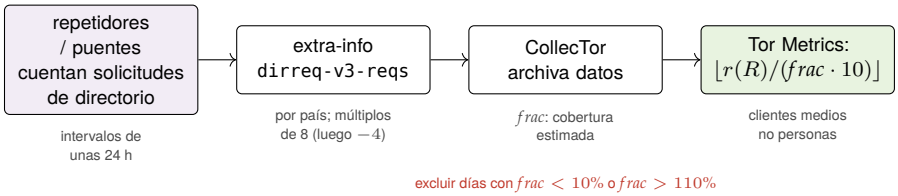


Figura 24.1: De solicitudes correctas de directorio a una estimación indirecta de *clientes conectados de media*, no de personas distintas. Los repetidores que sirven directorios y los puentes producen series agregadas separadas en descriptores extra-info; los recuentos por país de `dirreq-v3-reqs` se redondean al múltiplo de 8 superior. CollecTor archiva los documentos. Tor Metrics corrige el redondeo restando 4 a los valores válidos, estima la fracción de cobertura $frac$ y aplica $\lfloor r(R)/(frac \cdot 10) \rfloor$. El divisor 10 supone una media de solicitudes por cliente y día; se excluyen las fechas con $frac < 10\%$ o $frac > 110\%$.

Tor Metrics recopila esas cifras y aplica tres pasos [Thec]:

1. Compensa el redondeo restando la mitad del intervalo de redondeo a cada valor.
2. Estima la **fracción**, $frac$, de solicitudes de directorio de toda la red cubiertas por los repetidores que aportan datos, teniendo en cuenta cuánto tiempo estuvieron operativos y cuánto tráfico de directorio atendieron.
3. Divide las solicitudes registradas por esa fracción para estimar el total de la red y después lo divide entre diez:

$$r(N) = \left\lfloor \frac{r(R)}{frac \cdot 10} \right\rfloor .$$

El divisor diez es una suposición. Un cliente que permanece conectado continuamente hace aproximadamente quince solicitudes diarias, pero muchos solo están en línea parte del día, y el Tor Project eligió diez como media. Se descartan las estimaciones con una fracción de cobertura inferior al 10 % o superior al 110 %.

Advertencia: Qué significa «usuarios»

El resultado estima el número medio de *clientes conectados*, no de personas individuales. Un cliente conectado dos horas diarias suele generar menos solicitudes que uno activo las veinticuatro horas, pero la proporción no es fija; varios programas que ejecutan Tor en un equipo cuentan como varios clientes. Los cambios de comportamiento del software, como descargar consensos con menor frecuencia, modifican la estimación aunque el número de personas permanezca constante. Los gráficos son especialmente útiles para interpretar *cambios*, no cifras absolutas.

Las cifras en la fecha de referencia del libro. Entre el 1 de agosto y el 20 de septiembre de 2026, los usuarios diarios estimados con conexión directa oscilaron entre 2,8 y 3,5 millones, con fracciones de cobertura del 57 % al 73 %. La media italiana fue de unos 60 000, pero los resultados diarios presentan intervalos esperados muy amplios: el 20 de septiembre, por ejemplo, entre unos 34 000 y 111 000. Cambios dentro de intervalos de esta amplitud no demuestran por sí solos que haya ocurrido un acontecimiento. Ese mismo día, las estimaciones de usuarios de puentes eran de aproximadamente 98 000 para obfs4, 60 000 para Snowflake y 35 000 para WebTunnel.

24.2.1 Detectar la censura

Metrics utiliza las mismas series temporales para señalar posibles episodios de censura. En países con cantidades significativas de usuarios, compara el cambio semanal de cada país con las variaciones habituales en los demás y destaca las fechas en que una cifra queda fuera del intervalo esperado [Thec]. Una caída repentina de usuarios directos acompañada de un aumento de usuarios de puentes es una señal característica de bloqueo. Es un indicio, no una demostración. Para confirmarlo se necesitan mediciones dentro del país, tarea de OONI.

24.3 Rendimiento

OnionPerf mide, desde ubicaciones fijas, el tiempo necesario para descargar archivos de diversos tamaños mediante Tor, los fallos y la latencia de los circuitos: el intervalo entre enviar una solicitud HTTP y recibir las cabeceras de respuesta [Thec]. El 20 de septiembre de 2026, la latencia mediana hacia un servidor público era de unos 110–140 ms desde el punto de medición alemán, 280 ms desde el estadounidense y más de 500 ms desde Hong Kong. Importa la distancia geográfica respecto a la concentración principal de la red, en Europa y Norteamérica. Algunos puntos de medición emplean Conflux, el sistema de rutas múltiples de capítulo 31, lo que permite comparar situaciones similares.

24.4 Laboratorio: leer un consenso real

Laboratorio 15: Obtener estadísticas de red a partir de un consenso

Objetivo. Extraer las estadísticas principales de la red de un consenso real de CollecTor mediante un módulo nuevo, `labs/es/codigo/torlab/metrics.py`.
Requisitos previos. Laboratorio 6 (`directory.py`) y laboratorio 7 (`path_selection.py`); no se necesita una red en ejecución.
Tiempo estimado. 1 hora.

El módulo añade una función `summary` que reutiliza el analizador del consenso del laboratorio 6 y las dos funciones de estimación de usuarios. También ordena los recuentos por nombre cuando los valores coinciden. Sin esta precaución, su orden cambiaría entre ejecuciones debido al orden de iteración de los conjuntos de Python. Para el consenso histórico del 23 de septiembre de 2026 a las 15:00, primero prepara los datos desde la raíz del repositorio con `make -C labs/es data` (se requiere conexión a Internet para descargarlos). Aplicado a ese consenso (salida abreviada):

```
$ PYTHONPATH=labs/es/codigo python3 -m torlab.metrics labs/es/datos/collector/
    consensus-microdesc-2026-09-23-15
relay: 9358
flag: {'Running': 9358, 'Valid': 9358, 'Fast': 8961, 'Stable': 8199,
```

```
       'V2Dir': 7674, 'Guard': 5306, 'HSDir': 4050, 'Exit': 3177,
       'BadExit': 32, 'MiddleOnly': 28, 'StaleDesc': 21, 'Authority': 10}
total_bandwidth: 198927900
exit_bandwidth_share: 0.3308947161257923
guard_bandwidth_share: 0.7010271560701139
versions: [('Tor 0.4.9.12', 6027), ('Tor 0.4.9.11', 2520), ...]
networks16: [('64.65', 771), ('192.42', 636), ('204.8', 598), ...]
```

Algunas observaciones:

- Los 9358 repetidores concuerdan con el gráfico de Tor Metrics, que registra 9416 repetidores y 2333 puentes el 20 de septiembre.
- Aproximadamente un tercio de los repetidores tiene indicador `Exit` y representa cerca de un tercio del ancho de banda medido; los guardias representan el 70 %. Estas fracciones se solapan porque un repetidor puede poseer ambos indicadores. El desequilibrio se corrige mediante los pesos estudiados en apartado 13.2.
- `BadExit` (32 repetidores) y `MiddleOnly` (28) reflejan las decisiones de las autoridades sobre repetidores sospechosos o mal configurados.
- Diez repetidores llevan el indicador `Authority`, pero solo nueve autoridades votan el consenso (apartado 6.1). La décima es `Serge`, la autoridad de puentes, que posee el indicador pero no vota.
- La mayoría de los repetidores ejecutan versiones recientes de la serie 0.4.9.
- Varias redes /16 alojan cientos de repetidores, normalmente pertenecientes a grandes operadores de salidas. La regla de un repetidor por red /16 (apartado 13.1) limita la dependencia de una sola red.

La suma de los pesos de ancho de banda del consenso representa unidades medidas por las autoridades, no tráfico realmente transferido. Para estimar el tráfico, Metrics utiliza historiales de bytes de los descriptores extra-info.

24.5 OONI

El *Open Observatory of Network Interference* (OONI) mide la censura de Internet desde el punto de vista de los usuarios. Voluntarios de todo el mundo instalan OONI Probe, que realiza pruebas periódicas y publica resultados abiertos. Algunas pruebas se refieren específicamente a Tor [OON]:

`tor` Comprueba si son accesibles determinadas autoridades de directorio y puentes. Las pruebas van más allá de abrir una conexión TCP e incluyen acciones específicas del protocolo, como un intercambio obfs4.

`torsf` Intenta iniciar tor mediante Snowflake y registra si el proceso de conexión se completa y cuánto tarda.

Las especificaciones de las pruebas incluyen precauciones para no publicar direcciones privadas de puentes: incorporarlas a los resultados equivaldría a entregárselas a los censores. Metrics y OONI se complementan. Metrics puede mostrar una disminución de usuarios en un país; OONI permite investigar si repetidores o puentes eran realmente inaccesibles desde dentro de ese país e identificar la técnica de bloqueo.

En profundidad: Medir sin perjudicar

Toda estadística publicada por un repetidor implica una compensación. Cuanto más detalle, mayor utilidad, pero también mayor riesgo de revelar información. Por eso los recuentos se redondean y agregan por día y país. La investigación de red sigue los mismos principios (apartado 19.1): recopilar el mínimo necesario, agregar antes de publicar y no conservar datos innecesarios.

Resumen

- Los datos de red proceden de consensos, descriptores y documentos extra-info archivados por CollecTor, y de mediciones activas de OnionPerf. Tor Metrics los analiza mediante métodos documentados y reproducibles.
- Los usuarios se estiman por solicitudes de directorio: solicitudes declaradas divididas por la fracción cubierta y después por diez. Así se estima el promedio de clientes conectados, especialmente útil para estudiar variaciones.
- Los mismos datos pueden sugerir episodios de censura que requieren confirmación mediante mediciones locales.
- Un consenso real proporciona números de repetidores, indicadores, cuotas de ancho de banda, versiones y concentración por redes.
- OONI mide desde dentro de los países la accesibilidad de autoridades y puentes y el funcionamiento de Snowflake.

Ejercicios

Ejercicio 24.1 ⋆ Si todos los clientes comenzaran a descargar consensos con la mitad de frecuencia, ¿qué ocurriría con la estimación de usuarios, suponiendo que el número de personas no cambiara? ¿Cómo podría advertirlo alguien que interpretara los gráficos?

Ejercicio 24.2 ⋆ En un día, los repetidores que proporcionan datos registran veinte millones de solicitudes y la fracción de cobertura estimada es del 60 %. ¿Cuántos usuarios estimaría Metrics? ¿Y si esa fracción fuera del 8 %?

Ejercicio 24.3 ⋆⋆ Amplía `summary` para calcular la cuota de ancho de banda de repetidores con ambos indicadores, `Guard` y `Exit`, y la probabilidad ponderada global de que la salida de un circuito pertenezca a una de las cinco redes /16 más representadas.

Ejercicio 24.4 ⋆⋆ Descarga desde Metrics las series temporales de usuarios directos y de puentes para un país que haya sufrido un episodio documentado de bloqueo de Tor. Describe qué sucede con ambas series en las fechas del bloqueo.

Ejercicio 24.5 ⋆⋆⋆ Los recuentos por país se redondean hacia arriba a múltiplos de ocho. Describe un ataque que ese redondeo dificulta y explica por qué no protege frente a un adversario que controle muchos repetidores.

Lecturas complementarias

- Página *Reproducible Metrics* del Tor Project [Thec], con los métodos utilizados para cada gráfico.

- Especificaciones de pruebas OONI [OON] y OONI Explorer, `https://explorer.ooni.org/`.
- Formato de los descriptores extra-info, [dir-spec § Extra-info document format].

Capítulo 25

Aspectos legales y éticos en Italia y la UE

Última verificación: 23 de septiembre de 2026 — Reglamento (UE) 2022/2065; Decreto Legislativo 196/2003 (texto consolidado publicado por la Autoridad Italiana de Protección de Datos); directrices del Tor Research Safety Board

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- distinguir entre la legalidad de una herramienta y la legalidad de sus usos;
- interpretar la regla europea de «mera transmisión» (*mere conduit*) a la luz del funcionamiento de una salida Tor;
- identificar las cuestiones no resueltas que afrontan los operadores de repetidores en Italia, entre ellas responsabilidad, conservación de datos e incautaciones;
- aplicar los principios de seguridad del Tor Research Safety Board a investigaciones sobre Tor.

Advertencia: Este capítulo no constituye asesoramiento jurídico

La legislación cambia, depende de las circunstancias de cada caso y ha dado lugar a relativamente pocas resoluciones específicamente relacionadas con Tor. Este capítulo presenta las cuestiones pertinentes e identifica las normas examinadas en su texto oficial, señalando expresamente las incertidumbres. Cualquiera que deba adoptar una decisión operativa, como administrar una salida, debería consultar a un abogado especializado en derecho tecnológico.

25.1 Una herramienta y sus usos

En Italia, como en el resto de la Unión Europea, ninguna ley prohíbe de forma general utilizar Tor u otras herramientas que protejan la confidencialidad de las comunicaciones. Por el contrario, la confidencialidad de la correspondencia y otras comunicaciones (artículo 15 de la Constitución italiana) y la libertad de expresión (artículo 21) son derechos fundamentales, al igual que el respeto de la vida privada y la protección de datos personales reconocidos en los artículos 7 y 8 de la Carta de los Derechos Fundamentales de la Unión Europea.

Lo que las personas hacen mediante Tor sigue sometido a las mismas leyes que rigen fuera de Tor. El acceso no autorizado a un sistema informático (artículo 615-*ter* del Código Penal italiano) no se vuelve lícito por atravesar tres repetidores. Distinguir entre una herramienta y

su utilización es esencial en este capítulo y también para quienes proporcionan la infraestructura: los operadores de repetidores aportan capacidad de transmisión, como los proveedores de conectividad, en lugar de seleccionar el contenido transmitido.

25.2 Operadores de repetidores

25.2.1 Mera transmisión

El *Reglamento de Servicios Digitales* (*Digital Services Act*, DSA) europeo, aplicable desde el 17 de febrero de 2024, sustituyó las reglas de responsabilidad de intermediarios establecidas originalmente por la Directiva sobre comercio electrónico de 2000, incorporada al ordenamiento italiano mediante el Decreto Legislativo 70/2003 [22]. Su artículo 4 trata la **mera transmisión** (*mere conduit*): un servicio que transmite información proporcionada por otras personas queda exento de responsabilidad por esa información si no inicia la transmisión, no selecciona a su destinatario y no selecciona ni modifica la información transmitida. La figura 25.1 compara esas condiciones con el comportamiento de una salida Tor al nivel del protocolo de celdas.

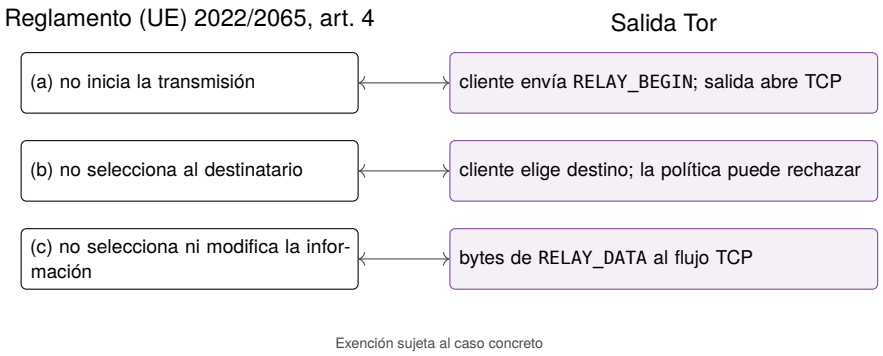


Figura 25.1: Comparación técnica, no conclusión jurídica automática, de los criterios de *mera transmisión* del art. 4 del Reglamento (UE) 2022/2065 y la salida Tor. El cliente indica el destino mediante `RELAY_BEGIN`; la salida abre una conexión TCP y puede rechazarla según su política. Los bytes de aplicación de `RELAY_DATA` pasan al flujo TCP, sin reenviar intactas las celdas Tor. La exención exige valorar el servicio concreto y todos los requisitos legales.

También importan otras dos disposiciones. El artículo 8 excluye una obligación *general* de supervisar la información transmitida o buscar activamente indicios de actividades ilícitas: por regla general, un operador no está obligado a inspeccionar indiscriminadamente el tráfico; un repetidor intermedio no puede leer en claro el contenido de aplicación cifrado. No obstante, el artículo 4(3) conserva la facultad de las autoridades judiciales o administrativas de ordenar a los prestadores de servicios que impidan o pongan fin a infracciones.

La cuestión abierta. El Reglamento se aplica a los «servicios de la sociedad de la información», definidos por el Derecho de la UE como servicios prestados *normalmente a cambio de una remuneración*. ¿Encaja en esa definición un repetidor gestionado gratuitamente por una persona voluntaria? Existen argumentos a favor de una interpretación amplia, y la exención refleja un principio que un tribunal podría considerar: quien se limita a transportar información no debería responder por su contenido. Sin embargo, aquí no se ha establecido la situación jurídica italiana de los repetidores voluntarios de Tor mediante una resolución judicial directamente aplicable. Esta incertidumbre ayuda a explicar por qué algunas salidas las gestionan asociaciones u organizaciones que ya prestan servicios de red.

25.2.2 Responsabilidad civil e investigaciones penales

La exención afecta a la responsabilidad por la información transmitida. No impide que una investigación penal comience a partir de la dirección IP de una salida. Desde la perspectiva de la víctima, un delito cometido mediante Tor parece proceder de la salida. Inicialmente, los investigadores podrían registrar o incautar los equipos del operador. Las preguntas frecuentes legales de la Electronic Frontier Foundation para operadores de repetidores, redactadas según el Derecho estadounidense pero útiles como testimonio de experiencias, indican que no conocen ninguna condena impuesta exclusivamente por administrar un repetidor, aunque describen investigaciones e incautaciones provocadas por malentendidos sobre Tor [Ele]. Algunas precauciones pueden reducir los riesgos prácticos:

- Ejecutar la salida en un servidor dedicado de un centro de datos, y no en casa ni en un ordenador personal de trabajo.
- Indicar claramente que la dirección IP pertenece a una salida, por ejemplo mediante un DNS inverso descriptivo y una página informativa que tor pueda servir desde su DirPort mediante `DirPortFrontPage`.
- Informar previamente al proveedor de conectividad y acordar cómo tramitar las notificaciones de abuso.
- Explicar qué es Tor al responder a dichas notificaciones y conservar la correspondencia.

25.2.3 Conservación de datos

El artículo 132 del Código italiano de Protección de Datos Personales exige a los *proveedores* comprendidos en su ámbito conservar durante doce meses los datos de tráfico telemático con fines de investigación y persecución de delitos [03], frente a los veinticuatro meses del tráfico telefónico. El apartado 5-*bis* mantiene una regla especial: el artículo 24 de la Ley n.º 167, de 20 de noviembre de 2017, amplía el plazo a **setenta y dos meses** para investigaciones relativas a delitos contemplados en los artículos 51(3-*quater*) y 407(2)(a) del Código de Procedimiento Penal, incluidos el terrorismo y otros delitos graves [17]. La parte pertinente del Código de Protección de Datos se aplica a servicios de comunicaciones electrónicas *accesibles al público a través de redes públicas de comunicaciones* (artículo 121).

Si un operador voluntario de repetidores puede considerarse uno de esos proveedores es otra cuestión abierta, que depende también de cómo se ofrece el servicio. Los operadores, especialmente las organizaciones que ya prestan servicios de comunicaciones, deberían solicitar un análisis jurídico antes de administrar una salida. Desde el punto de vista técnico, con la configuración de registros recomendada, tor normalmente no conserva un historial de los destinos de los usuarios. Una salida puede ver los destinos de las conexiones que reenvía, pero desconoce la dirección IP de origen del cliente. Los datos realmente disponibles dependen de la función y configuración del repetidor: no debe suponerse que una salida pueda identificar a los usuarios individuales de Tor.

25.2.4 Los registros son datos personales

Las direcciones IP son datos personales. El Reglamento General de Protección de Datos (RGPD) exige tratar únicamente los datos necesarios y conservarlos solo durante el tiempo indispensable (artículo 5, principio de minimización de datos). Para un repetidor, la opción compatible con la seguridad consiste en registrar únicamente eventos de nivel `notice`, mantener `SafeLogging` activado y evitar capturas de paquetes (apartado 20.4).

25.3 Investigadores

La investigación sobre Tor tiene antecedentes de experimentos éticamente cuestionables, algunos tratados en capítulo 27. Se han administrado repetidores que vigilaban o manipulaban el tráfico de usuarios reales para demostrar ataques. Además de los problemas éticos, esos experimentos podrían estar sujetos al Derecho penal italiano, incluidas las disposiciones sobre interceptación ilícita de comunicaciones informáticas (artículo 617-*quater* del Código Penal), y a las normas de protección de datos.

El **Tor Research Safety Board** es un grupo de investigadores que asesora sobre la seguridad de los experimentos y ha desarrollado directrices [Tor]:

- Utilizar siempre que sea posible una red de prueba y atacar únicamente el tráfico propio.
- Recopilar solo los datos necesarios y únicamente aquellos cuya publicación sería segura.
- Proteger los datos recopilados y las herramientas experimentales.
- Reducir la precisión de los datos mediante redondeo o ruido.
- Considerar conjuntos de datos de terceros que podrían revelar información al combinarse con los propios.
- Preguntarse si los usuarios esperaban que los datos correspondientes permanecieran privados.
- Comprobar que los beneficios previstos justifican los riesgos.

El Board solicita a los investigadores que expliquen qué desean aprender, cómo pretenden hacerlo, qué riesgos introduce el experimento y por qué sus beneficios justifican esos riesgos. Este libro sigue la misma disciplina: los laboratorios que generan tráfico Tor activo utilizan una red privada (apartado 19.1), mientras que los análisis de documentos públicos, como los de capítulo 24, se realizan sin conexión sobre datos agregados y públicos.

Resumen

- Utilizar Tor es lícito por sí mismo; las actividades realizadas mediante Tor siguen regidas por el Derecho ordinario.
- El artículo 4 del DSA exime, bajo ciertas condiciones, a los servicios comprendidos en su ámbito que se limitan a transmitir información ajena. El funcionamiento de una salida es relevante para evaluar esas condiciones, pero sigue abierta la cuestión de si el Reglamento cubre a un repetidor voluntario.
- La exención no impide investigaciones ni incautaciones. Los servidores dedicados, la identificación clara de las salidas y los acuerdos con proveedores de conectividad pueden reducir problemas prácticos.
- Las obligaciones de conservación de datos afectan a proveedores de comunicaciones accesibles al público comprendidos en la norma; su aplicación a repetidores voluntarios requiere una valoración jurídica caso por caso.
- La investigación sobre Tor debe evitar siempre que sea posible experimentos con usuarios reales: utilizar redes de prueba, minimizar los datos, publicar únicamente agregados seguros y valorar los beneficios frente a los riesgos.

Ejercicios

Ejercicio 25.1 ⋆ Para cada una de las tres condiciones del artículo 4 del DSA, describe un cambio de comportamiento de una salida que la vulneraría.

Ejercicio 25.2 ⋆ Utiliza un ejemplo para explicar la diferencia entre la legalidad de una herramienta y la de sus usos, tanto para un usuario Tor como para un operador de repetidores.

Ejercicio 25.3 ⋆⋆ Redacta una respuesta a una notificación de abuso recibida del proveedor de conectividad de una salida después de que un sitio web denuncie intentos repetidos de acceso desde tu dirección IP. ¿Qué puedes explicar y qué medidas prácticas puedes adoptar?

Ejercicio 25.4 ⋆⋆ Una investigadora propone contar los nombres de dominio observados en una salida para estimar cuántos usuarios Tor visitan sitios de noticias italianos. Evalúa la propuesta mediante las directrices del Tor Research Safety Board y sugiere una alternativa más segura.

Ejercicio 25.5 ⋆⋆⋆ Investiga resoluciones judiciales relacionadas con operadores de repetidores Tor dictadas por tribunales italianos y de otros países europeos. Resume la cuestión jurídica y el resultado de cada caso, citando fuentes primarias.

Lecturas complementarias

- Reglamento (UE) 2022/2065, especialmente los artículos 4, 8 y 9 [22].
- Decreto Legislativo 196/2003, artículos 121 y 132, en el texto consolidado publicado por la Autoridad Italiana de Protección de Datos [03].
- Electronic Frontier Foundation, *Legal FAQ for Tor Relay Operators* [Ele].
- Sitio web del Tor Research Safety Board [Tor].

Parte IV

Ataques y defensas

Capítulo 26

Correlación de tráfico, identificación de sitios y ataques Sybil

Última verificación: 23 de septiembre de 2026 — artículos citados en este capítulo; tor 0.4.9.13; red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- clasificar los ataques contra el anonimato en Tor según los puntos de observación del adversario;
- explicar la correlación de tráfico de extremo a extremo y medirla sobre tráfico propio en la red de laboratorio (laboratorio 16);
- describir la identificación de sitios web (*website fingerprinting*) y las limitaciones de sus evaluaciones;
- reconocer los ataques Sybil y las defensas de la red, desde los guardias hasta las mediciones de ancho de banda.

El modelo de amenazas de capítulo 4 dejó claro que Tor no resiste a un adversario que observa ambos extremos de una comunicación. Veinte años de investigación han cuantificado esa advertencia y demostrado que los puntos de observación son más numerosos y accesibles de lo que podría parecer. Este capítulo examina los ataques desde el punto de vista del defensor: qué necesita un ataque, qué consigue y cuáles de los mecanismos estudiados lo dificultan.

26.1 La posición del adversario

Resulta útil clasificar los ataques según lo que puede observar el adversario:

Un extremo La conexión del cliente con el guardia, observada por un proveedor de conectividad, una red local o un censor; o la conexión entre la salida y el destino. En el enlace del cliente al guardia se observan tiempos, volumen y dirección del tráfico cifrado. Entre la salida y el destino, el cifrado de aplicación depende del protocolo. La identificación de sitios puede aprovechar metadatos del lado del cliente sin leer contenido.

Ambos extremos El adversario observa entrada y salida, ya sea operando repetidores en ambas posiciones o controlando redes en las rutas que las conectan. Esto permite correlacionar tráfico.

Dentro de la red Los repetidores hostiles, incluidos los intermedios, pueden medir, retrasar, marcar o descartar celdas.

Por encima de la red La infraestructura subyacente de Internet, incluidos los sistemas autónomos (AS) y los puntos de intercambio de Internet (IXP) atravesados por las conexiones de los repetidores.

26.2 Correlación de extremo a extremo

Tor evita introducir deliberadamente grandes retrasos o mezclas de celdas al estilo de una red mix para seguir siendo utilizable (apartado 2.2). En consecuencia, la *forma* del tráfico —cuántos bytes pasan durante cada intervalo— puede seguir siendo reconocible en ambos extremos, aunque los retrasos, el almacenamiento intermedio, la multiplexación y otros flujos la alteren (figura 26.1). Quien observa ambos extremos puede comparar las formas y decidir si corresponden a los mismos flujos sin romper ningún cifrado.

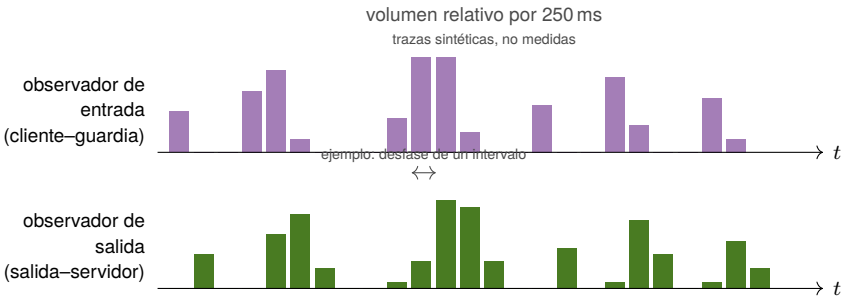


Figura 26.1: Dos trazas *sintéticas*, no mediciones reales, del volumen de tráfico en una dirección en los enlaces cliente–guardia y salida–servidor. Las ráfagas se parecen, pero no son idénticas; el desfase de un intervalo de 250 ms es solo ilustrativo, no un retardo fijo de Tor. El cifrado protege el contenido, pero no necesariamente los patrones temporales y de volumen. La sobrecarga, el almacenamiento intermedio, la multiplexación y otros flujos pueden alterar las trazas; la similitud estadística por sí sola no demuestra una correspondencia.

¿Qué tan fácil es encontrar coincidencias? En 2018, DeepCorr entrenó una red neuronal para reconocer parejas de flujos Tor. Con unos 900 paquetes por flujo, alcanzó un 96 % de exactitud frente al 4 % del método estadístico anterior en las mismas condiciones [NBH18]. Para el adversario, lo más difícil suele ser ocupar los puntos de observación adecuados y no comparar los flujos.

¿Qué probabilidad hay de que el adversario observe ambos extremos? Johnson y sus colaboradores simularon usuarios típicos durante meses con el algoritmo real de selección de rutas de Tor [Joh+13]. Con repetidores maliciosos de ancho de banda moderado, el 80 % de los usuarios de los tipos modelados quedó desanonimizado al menos una vez en seis meses. Frente a un único AS, algunos usuarios de ubicaciones frecuentes se aproximaban al 100 % en tres meses. Estos resultados históricos de simulación no significan que todas las conexiones estén expuestas; muestran que las oportunidades se acumulan con el tiempo. Ese razonamiento motivó guardias de mayor duración y un número menor de guardias por cliente (apartado 13.3).

Internet como adversario. RAPTOR identificó tres mecanismos que permiten a un AS aumentar sus oportunidades [Sun+15]: aprovechar la asimetría de las rutas de Internet, porque puede bastar observar *una* dirección en cada extremo, incluidos los acuses de recibo de TCP; aprovechar los cambios naturales de encaminamiento con el tiempo; y secuestrar o interceptar prefijos mediante BGP. Estas técnicas no modifican Tor. Las defensas incluyen vigilar el encaminamiento y, posiblemente, seleccionar rutas teniendo en cuenta la topología.

Ataques activos. Un adversario que controla un extremo puede *imprimir* deliberadamente un patrón de tráfico característico retrasando o agrupando paquetes y buscarlo en el otro. El ataque de marcado estudiado en la capa de cifrado de apartado 10.3 es una versión extrema de esta idea. Ya en 2005, Murdoch y Danezis demostraron que no siempre resulta necesario observar directamente los enlaces. Un servidor malicioso impone un ritmo al tráfico hacia la víctima; el adversario mide la latencia de sus propios circuitos a través de repetidores candidatos, y aquellos que se ralentizan según ese ritmo podrían formar parte del circuito de la víctima [MD05].

26.3 Laboratorio: correlacionar tráfico propio

Laboratorio 16: Correlación de extremo a extremo

Objetivo. Demostrar en la red de laboratorio que un patrón de tráfico sobrevive al circuito y permite identificar qué flujo observado en la salida corresponde a cada cliente.
Requisitos previos. Laboratorio 11; red de laboratorio en ejecución. Todos los flujos son generados por el propio script del experimento.
Tiempo estimado. 1 hora.

El script `labs/es/laboratorio/correlacion.py` inicia un servidor en el puerto 8095 del contenedor y seis clientes. Cada uno utiliza credenciales SOCKS distintas: con `IsolateSOCKSAuth` activo, sus flujos no pueden compartir circuito (apartado 18.3). Durante veinte segundos los clientes alternan aleatoriamente ráfagas de bloques de 2 KB con períodos de silencio. Dos «observadores» registran datos:

- El observador de entrada registra cuándo escribe datos cada cliente. Esto idealiza lo que podría ver alguien que observara el enlace con el guardia. Allí los datos estarían empaquetados en celdas y TLS, pero los patrones temporales necesarios para correlacionar sobreviven a ambas capas.

- El observador de salida, nuestro servidor, registra cuándo llegan los bytes de cada conexión y recibe un identificador de evaluación de cuatro bytes, excluido del algoritmo de correlación.

Ambos agrupan los bytes en intervalos de 250 ms. Para cada cliente calculamos la correlación de Pearson frente a los seis flujos de salida. El de mayor correlación es la coincidencia propuesta. El script conoce la respuesta correcta, comunicada por el cliente en sus cuatro primeros bytes, pero solo la utiliza para evaluar el resultado, nunca para elegir la coincidencia.

```
$ docker exec -w /tmp torbook-es-network python3 /lab/correlacion.py 1
cliente 0: +0.79 -0.11 +0.12 +0.17 +0.11 -0.01  -> flujo de salida 0 OK
cliente 1: -0.11 +0.80 -0.02 +0.07 +0.18 +0.08  -> flujo de salida 1 OK
cliente 4: +0.13 +0.02 +1.00 +0.21 +0.10 +0.21  -> flujo de salida 2 OK
cliente 3: +0.21 +0.09 +0.33 +0.81 -0.07 +0.03  -> flujo de salida 3 OK
cliente 2: +0.17 +0.05 +0.10 -0.09 +0.85 -0.03  -> flujo de salida 4 OK
```

```
cliente 5: -0.00 +0.01 +0.23 +0.06 -0.13 +0.92  -> flujo de salida 5 OK
correspondencias correctas: 6/6
```

Las columnas identifican los flujos observados en la salida según el orden de llegada de las conexiones, distinto del orden de los clientes. La correlación correcta destaca en cada fila, entre 0,79 y 1,00, mientras los demás valores suelen estar cerca de cero. No hacen falta claves de descifrado ni celdas descifradas: bastan volumen y tiempos. La red de laboratorio es pequeña y está poco cargada, de modo que el retardo es mínimo. Los resultados históricos de DeepCorr indican que clasificadores bastante más sofisticados pueden operar con unos cientos de paquetes en la red real, bajo las condiciones estudiadas. Es un extracto de una ejecución de referencia, no una predicción exacta: el orden, los coeficientes y los aciertos pueden variar, incluso hasta 5/6. El laboratorio forma parte de `make verifica-lab`.

26.4 Identificación de sitios web

Un adversario que solo observa el enlace entre cliente y guardia no puede correlacionar ambos extremos, pero sí puede *reconocer* un sitio web. Cada página genera una secuencia característica de celdas en ambas direcciones: cantidades, orden y ráfagas. El adversario visita los sitios objetivo mediante Tor, registra sus secuencias y entrena un clasificador. Después compara el tráfico de la víctima con los patrones aprendidos.

La investigación de estos ataques ha sido una competición continua entre clasificadores y defensas:

- En 2011, Panchenko y sus colaboradores utilizaron características de volumen, tiempo y dirección para mejorar del 3 % al 55 % el reconocimiento en un conjunto cerrado de 775 sitios. También demostraron un ataque eficaz de *mundo abierto*, donde las víctimas pueden visitar sitios ajenos al conjunto de entrenamiento del adversario [Pan+11].
- En 2018, Deep Fingerprinting utilizó una red neuronal convolucional y superó el 98 % de exactitud sobre 95 sitios web sin defensas, y el 90 % frente a WTF-PAD, una defensa de relleno adaptativo considerada para Tor [Sir+18].

En profundidad: La falacia de la frecuencia base

Las cifras de exactitud extremadamente altas deben interpretarse con cuidado. Juarez y sus colaboradores mostraron que muchos estudios suponían condiciones favorables: una página visitada cada vez, la misma versión de Tor Browser para víctima y adversario y datos de entrenamiento recogidos poco antes del ataque [Jua+14]. Más importante todavía, en un mundo abierto los sitios vigilados representan una fracción diminuta de las visitas totales. Incluso un clasificador con una tasa reducida de falsos positivos puede generar más falsas alarmas que identificaciones correctas entre millones de visitas ajenas. Es la misma trampa estadística que aparece en las pruebas diagnósticas de enfermedades raras; los adversarios reales deben tenerla en cuenta.

Defensas. El relleno de canales (capítulo 7) dificulta identificar los períodos inactivos, pero no está diseñado para impedir la identificación de páginas. tor implementa además un mecanismo más general de *relleno de circuitos*: las máquinas de estados pueden programar celdas ficticias. En la fecha de referencia del texto, los mecanismos desplegados ayudan a que los circuitos de introducción y encuentro de servicios onion se parezcan a circuitos ordinarios (capítulo 14, Propuesta 302). Una defensa ampliamente eficaz contra la identificación de sitios con un coste aceptable de ancho de banda sigue siendo un problema de investigación abierto.

26.5 Sybil: multiplicar identidades de repetidores

Un **ataque Sybil** introduce muchas identidades en una red. En Tor, el adversario añade numerosos repetidores con la esperanza de ser seleccionado frecuentemente como guardia y salida. Tres defensas elevan su coste:

Ancho de banda medido Un repetidor que miente sobre su capacidad podría atraer demasiados clientes. Bauer y sus colaboradores demostraron este engaño basado en descriptores en 2007 [Bau+07]. Actualmente los pesos de consenso dependen de mediciones de las autoridades, no de simples declaraciones (apartado 12.4); por tanto, atraer mucho tráfico exige aportar ancho de banda real.

Restricciones de rutas Dos repetidores de la misma familia o red IPv4 /16 no pueden aparecer en un mismo circuito (apartado 13.1). Los operadores honestos declaran sus familias; los deshonestos al menos quedan limitados por sus direcciones.

Vigilancia de la red Las autoridades pueden excluir repetidores o asignar `BadExit`. Voluntarios e investigadores examinan grupos con características compartidas y vigilan el comportamiento de las salidas. En 2014, por ejemplo, un analizador de salidas descubrió 25 repetidores que manipulaban tráfico o estaban mal configurados, con prácticas como interceptación de HTTPS, alteración de DNS y *sslstrip* [WL14].

En capítulo 27 examinaremos grupos de repetidores maliciosos descubiertos posteriormente y por qué la vigilancia debe mantenerse de forma continua.

Ataques Sybil contra servicios onion. En v2, las posiciones de los HSDir responsables de un servicio eran previsibles. Un adversario podía generar identidades de repetidores elegidas para ocuparlas. Biryukov y sus colaboradores aprovecharon esto en 2013 para medir la popularidad de servicios, dejarlos inaccesibles o facilitar ataques de localización [BPW13]. En v3, las posiciones dependen de claves cegadas y de un valor aleatorio compartido que cambia cada día (capítulo 14), lo que dificulta mucho la colocación dirigida y precalculada; una vez publicados los valores del período se pueden calcular los HSDir responsables, por lo que el riesgo no desaparece.

26.6 Descubrir el guardia

Un guardia protege al cliente, pero también constituye un objetivo atractivo: identificarlo indica al adversario dónde observar. Contra un servicio onion, el atacante puede provocar numerosas construcciones de circuitos, por ejemplo solicitando múltiples conexiones, hasta que un repetidor controlado por él sea seleccionado como intermedio adyacente al guardia. El primer ataque de esta clase, publicado en 2006, contribuyó a motivar el uso de guardias [ØS06]. Los vanguards (apartado 13.4) añaden una capa de repetidores intermedios que se mantienen fijos durante cierto tiempo. Esto dificulta avanzar hacia el guardia real, pero no garantiza que sea imposible descubrirlo.

Resumen

- Los ataques se distinguen por sus puntos de observación: un extremo, ambos extremos, el interior de la red o las rutas subyacentes de Internet.

- La correlación compara la forma del tráfico en ambos extremos sin romper el cifrado. La dificultad para el adversario consiste en ocupar las posiciones adecuadas y las oportunidades se acumulan con el tiempo.
- La identificación de sitios web reconoce páginas a partir del tráfico cifrado observado en un extremo. Las evaluaciones deben considerar las frecuencias base y condiciones realistas.
- Las defensas frente a Sybil incluyen pesos medidos, familias, restricciones /16 y vigilancia de salidas y grupos de repetidores. En v3 la ubicación de HSDir es más difícil de manipular de antemano, aunque se puede calcular para el período vigente.
- Los guardias y vanguards hacen más lento y costoso descubrir el primer salto.

Ejercicios

Ejercicio 26.1 ⋆ Para cada ataque de este capítulo, identifica el punto mínimo de observación que necesita el adversario y la defensa de Tor que lo dificulta.

Ejercicio 26.2 ⋆ Un clasificador reconoce un sitio web vigilado con probabilidad 0,95 y señala erróneamente una visita ajena con probabilidad 0,001. Si solo una de cada $100\,000$ visitas se dirige realmente al sitio vigilado, ¿qué porcentaje de las alertas es correcto?

Ejercicio 26.3 ⋆⋆ Repite el laboratorio 16 utilizando intervalos de dos segundos, después una duración total de cinco segundos y finalmente el mismo ritmo de tráfico para todos los clientes. ¿Qué modificación perjudica más a las coincidencias y por qué?

Ejercicio 26.4 ⋆⋆ Modifica los clientes del laboratorio 16 para enviar bloques a velocidad constante, incluido relleno cuando no tienen datos. ¿Cuánto ancho de banda consume y qué ocurre con la correlación?

Ejercicio 26.5 ⋆⋆⋆ Con `path_selection.py` y el consenso real del laboratorio 15, calcula la probabilidad de que un adversario que controle el 5 % del ancho de banda de guardias y el 5 % del de salidas ocupe ambos extremos de al menos un circuito de un cliente en seis meses. Compara un guardia fijo con cambiar de guardia en cada circuito. Indica un número supuesto de circuitos en seis meses y, solo para el modelo didáctico, usa selecciones independientes ponderadas por ancho de banda; explica sus límites.

Lecturas complementarias

- A. Johnson y otros, *Users Get Routed* [Joh+13].
- Y. Sun y otros, *RAPTOR: Routing Attacks on Privacy in Tor* [Sun+15].
- M. Juarez y otros, *A Critical Evaluation of Website Fingerprinting Attacks* [Jua+14].
- La *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`, para consultar investigaciones posteriores.

Capítulo 27

Casos reales de ataques contra la red

Última verificación: 23 de septiembre de 2026 — comunicados del Tor Project (2014, 2022, 2023); tor 0.4.9.13; red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- reconstruir el ataque de 2014 que utilizó celdas **RELAY_EARLY**, explicar la defensa añadida a tor y verificarla en un laboratorio en funcionamiento (laboratorio 17);
- describir el incidente KAX17 y los trabajos posteriores de vigilancia de repetidores;
- relacionar los ataques de denegación de servicio de 2022–2023 con las defensas explicadas en capítulo 15;
- extraer lecciones más generales sobre cómo proteger una red abierta.

En el capítulo anterior tratamos los ataques como temas de investigación: modelos, experimentos y porcentajes. Aquí examinamos tres incidentes de seguridad de la red real, reconstruidos a partir de comunicados públicos del Tor Project. En el caso KAX17 no se establecieron ni las intenciones del operador ni eventuales actividades maliciosas. Nos centraremos en lo que enseñan sobre el protocolo y la gestión de una red abierta, y no en detalles operativos de los ataques.

27.1 2014: señales mediante tipos de celda

Qué ocurrió. El 4 de julio de 2014, el Tor Project descubrió y eliminó un grupo de repetidores que sospechaba que intentaban desanonimizar a personas que publicaban descriptores o solicitaban información sobre servicios onion, entonces llamados *servicios ocultos*. Los repetidores se habían incorporado el 30 de enero: aproximadamente 115 repetidores rápidos sin función de salida, distribuidos entre dos redes /16 y que representaban conjuntamente cerca del 6,4 % de la capacidad de los guardias. El 30 de julio, el Tor Project publicó un aviso de seguridad donde describía el ataque y sus contramedidas [Din14].

Cómo funcionaba. El ataque combinaba la colocación Sybil (apartado 26.5) con la confirmación activa de tráfico (apartado 26.2). Los repetidores permanecían suficientemente estables para obtener los indicadores `HSDir` y `Guard`. Cuando un HSDir adversario recibía una solicitud de descriptor, respondía con una secuencia de celdas cuyos tipos alternados **RELAY** y **RELAY_EARLY** codificaban el nombre del servicio solicitado (figura 27.1). Si otro miembro del grupo era el guardia del mismo cliente, podía leer la secuencia y vincular al cliente con la solicitud del descriptor del servicio, sin demostrar que llegara a visitarlo.

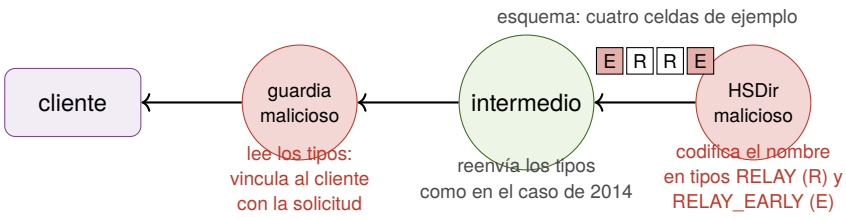


Figura 27.1: El ataque histórico de 2014: el HSDir malicioso codificaba el servicio solicitado en los comandos exteriores RELAY y RELAY_EARLY, visibles para los repetidores del circuito pero protegidos por TLS en los enlaces. El guardia cómplice podía vincular al cliente con una solicitud de descriptor, sin demostrar que visitara el servicio. Las cuatro celdas son esquemáticas, no la codificación completa de una dirección onion.

¿Por qué ese campo? Las celdas **RELAY_EARLY** limitan la longitud de los circuitos: los clientes deben utilizarlas para solicitar extensiones y los repetidores admiten como máximo ocho por circuito ([tor-spec § Handling RELAY_EARLY cells]). Sin embargo, las comprobaciones antiguas solo se aplicaban al tráfico del cliente hacia la red. Normalmente nadie enviaba esas celdas en dirección contraria y nadie comprobaba su presencia. A diferencia del contenido cifrado por capas (capítulo 10), el comando exterior de la celda permanece visible para todos los repetidores. Una señal escrita allí sobrevive al tránsito sin alteraciones. El aviso de seguridad también señaló que la señal codificaba el nombre del servicio de forma legible, por lo que un repetidor que recibiera esas celdas podría interpretar la señal. Un observador externo con simples capturas de red necesitaría antes descifrar el TLS de los enlaces para recuperarla retrospectivamente.

La respuesta. Se adoptaron tres medidas: excluir los repetidores maliciosos; hacer que tor 0.2.4.23 y 0.2.5.6-alpha cerrasen los circuitos al recibir **RELAY_EARLY** dirigidas al cliente; y avanzar hacia la reducción de los guardias de cada cliente de tres a uno, limitando las oportunidades de observar su tráfico. El aviso subrayó que la confirmación *pasiva* de tráfico seguía siendo un problema abierto. El canal basado en el tipo de celda facilitaba confirmar el tráfico, pero no hacía posible por primera vez ese ataque.

27.2 Laboratorio: el límite de RELAY_EARLY

Laboratorio 17: Se permiten ocho; nueve, no

Objetivo. Utilizar `labs/es/codigo/torlab` para verificar cómo aplica un repetidor actual el límite de celdas **RELAY_EARLY**.
Requisitos previos. Laboratorios 4 y 5; red de laboratorio en ejecución.
Tiempo estimado. 30 minutos.

Nuestro `circuit.py` ya se niega a enviar más de ocho celdas **RELAY_EARLY**. El script `labs/es/laboratorio/relay_early.py` omite intencionadamente esa protección en este experimento controlado. Solo permite los casos de ocho y nueve celdas. Construye un circuito de un salto hacia `test004r`, envía n celdas **RELAY_EARLY** con un mensaje `DROP` que el repetidor descarta sin responder y después solicita una conexión de directorio mediante `BEGIN_DIR` para comprobar si el circuito sigue activo. Desde la raíz del repositorio se utiliza el comando siguiente.

```
$ PYTHONPATH=labs/es/codigo python3 labs/es/laboratorio/relay_early.py 8 9
```

```
8 celdas RELAY_EARLY -> CONNECTED
Comprobación: CORRECTA
9 celdas RELAY_EARLY -> circuito destruido, motivo 1
Comprobación: CORRECTA
```

La novena celda provoca el cierre del circuito mediante una celda **DESTROY** de razón 1, PROTOCOL. El listado procede de una ejecución de referencia: direcciones, identificadores de circuito y registros varían entre ejecuciones. Un registro de referencia del repetidor indica:

```
[warn] Received too many RELAY_EARLY cells on circ 4045911407 from
       127.0.0.1:58048.  Closing circuit.
```

La comprobación se encuentra en `src/core/or/command.c`, antes de descifrar la celda. Unas líneas más arriba está la comprobación introducida en 2014: recibir una **RELAY_EARLY** dirigida al cliente produce una advertencia y el cierre del circuito. El comentario del código explica que esas celdas procedieron en el pasado de un fallo de versiones antiguas de tor, pero los repetidores afectados desaparecieron hace mucho. En la actualidad, una celda así se considera «inesperada» y se trata como un ataque. Este laboratorio también forma parte de `make verifica-lab`.

27.3 2020–2021: KAX17 y la salud de la red

Un segundo incidente ilustra una parte menos visible de la defensa: descubrir a los adversarios. El Tor Project lo trató en abril de 2022 en un artículo sobre su equipo dedicado a vigilar la salud de la red [The22].

En diciembre de 2020, los gráficos de repetidores mostraron un patrón extraño. Cada mes, exactamente a medianoche del primer día, decenas de repetidores se incorporaban a la red y después desaparecían lentamente. Parecía un fallo de la lógica de hibernación (`AccountingMax`, apartado 22.2). En septiembre de 2021 comenzaron a aparecer grupos de repetidores aparentemente controlados por un mismo operador, sin declaraciones de familia ni datos de contacto. Las autoridades los excluían, pero aparecían sustitutos. A finales de octubre, una denuncia anónima ayudó a identificar un grupo denominado posteriormente KAX17, que fue eliminado. A continuación cesaron tanto los picos mensuales como las oleadas de repetidores, por lo que el Tor Project consideró plausible atribuir ambos fenómenos al mismo operador. La identidad y los objetivos de este siguen siendo desconocidos.

El artículo es excepcionalmente franco acerca de la respuesta tardía: las herramientas de vigilancia llevaban años desatendidas; la confianza entre voluntarios, empleados y autoridades era débil; y la organización no estaba preparada para ese tipo de trabajo. También explica cómo funciona la gobernanza. Quienes vigilan la red pueden proponer excluir repetidores, pero las autoridades de directorio deciden por mayoría, no el personal del Tor Project. Es la confianza distribuida estudiada en capítulo 12, aplicada a la operación de la red.

En profundidad: Repetidores sin declaración de familia

La señal de alerta inicial del caso KAX17 no fue observar una vulnerabilidad explotada, sino incumplir una norma social: numerosos repetidores aparentemente controlados por el mismo operador sin declarar su familia (apartado 22.5). La lección más general es que un adversario tal vez no necesite romper el protocolo si consigue eludir las convenciones de las que depende la selección de rutas.

27.4 2022–2023: denegación de servicio

Desde el verano de 2022, la red experimentó durante meses ataques de denegación de servicio de diversa naturaleza. En febrero de 2023, el Tor Project comunicó que algunos usuarios no podían cargar páginas o acceder a servicios onion, que las tácticas y los objetivos habían cambiado con el tiempo y que no era posible determinar con certeza la identidad de los atacantes [The23b]. Los servicios onion estaban especialmente expuestos: no podían bloquear las direcciones IP atacantes porque las solicitudes llegaban por circuitos anónimos.

La respuesta a largo plazo fue el sistema de prueba de trabajo introducido en agosto de 2023 con tor 0.4.8 (apartado 15.4) [The23a]. Normalmente inactiva, esta defensa pide a los clientes bajo ataque resolver problemas computacionales de dificultad creciente y da prioridad a quienes han realizado más trabajo. El Tor Project describió costes de unos milisegundos para un cliente normal y de hasta aproximadamente un minuto durante un ataque intenso. El principio es el mismo del laboratorio 9: encarecer para el atacante un ataque que antes resultaba barato sin exigir a los usuarios que se identifiquen.

27.5 Lecciones

- **Todo campo visible constituye un canal potencial.** El ataque de 2014 no rompió la criptografía. Aprovechó un campo que los repetidores podían leer y reenviar. Un protocolo de anonimato debe preguntarse, por cada bit no cifrado, quién puede escribirlo y quién puede leerlo. Esta es una de las motivaciones del diseño más reciente de cifrado relay (apartado 10.4).

- **Los ataques Sybil pueden ser baratos; vigilar la red no lo es.** Añadir repetidores cuesta relativamente poco en comparación con mantener las herramientas, conocimientos y confianza necesarios para detectar comportamientos maliciosos coordinados.

- **Las defensas deben evaluarse a lo largo del tiempo.** Utilizar un solo guardia, mantener los guardias durante más tiempo e introducir vanguards reduce las oportunidades del adversario a largo plazo (apartado 26.2).

- **La transparencia es una defensa.** En los tres incidentes, el Tor Project publicó tanto lo que sabía como lo que desconocía, permitiendo a usuarios, operadores e investigadores evaluar el riesgo.

Resumen

- En 2014, un grupo Sybil cuyos repetidores funcionaban como guardias y HSDir codificó en los tipos de celda los nombres de los servicios onion cuyos descriptores se solicitaron. Actualmente tor cierra los circuitos que reciben **`RELAY_EARLY`** dirigidas al cliente; pasar de tres guardias a uno, y después a dos en 2018, redujo las oportunidades de seleccionar guardias maliciosos (apartado 13.3).

- El límite de ocho celdas **`RELAY_EARLY`** puede verificarse en la red de laboratorio: la novena destruye el circuito.

- KAX17 se identificó después de meses de indicios indirectos, incluidos picos mensuales y familias no declaradas, y de una denuncia anónima. Las autoridades de directorio son responsables de excluir repetidores.

- Los ataques de denegación de servicio de 2022–2023 contribuyeron a introducir la prueba de trabajo para servicios onion.

Ejercicios

Ejercicio 27.1 ⋆ ¿Por qué un guardia no podía leer una señal escrita por un HSDir en el contenido cifrado de una celda, mientras que sí podía leerla en el tipo exterior de la celda?

Ejercicio 27.2 ⋆ Explica cómo reduce un ataque semejante al de 2014 pasar de tres guardias a uno y qué inconveniente introduce.

Ejercicio 27.3 ⋆⋆ Modifica el laboratorio 17 para demostrar que una celda **`RELAY_EARLY`** cuenta para el límite aunque no contenga una solicitud de extensión de circuito. Después examina el código de tor para determinar qué sucede al recibir una **`RELAY_EARLY`** en dirección a un cliente.

Ejercicio 27.4 ⋆⋆ Utilizando datos de CollecTor, ¿qué indicios podrían revelar repetidores gestionados por un mismo operador sin una declaración de familia? Sugiere tres criterios y analiza los falsos positivos.

Ejercicio 27.5 ⋆⋆⋆ Enumera los campos de las celdas y de los mensajes relay que un repetidor intermedio puede ver en claro (capítulo 8). Para cada uno, analiza si un repetidor malicioso podría aprovecharlo como canal hacia otro repetidor del circuito e identifica las defensas existentes o propuestas.

Lecturas complementarias

- Tor Project, *Tor security advisory: «relay early» traffic confirmation attack* [Din14].
- Tor Project, *Malicious relays and the health of the Tor network* [The22].
- Tor Project, *Introducing Proof-of-Work Defense for Onion Services* [The23a].

Capítulo 28

Errores de seguridad operativa y desanonimización: lecciones

Última verificación: 23 de septiembre de 2026 — asistencia del Tor Project; advertencias de Tails; tor 0.4.9.13; red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- distinguir las capas en las que puede filtrarse la identidad e identificar qué herramientas protegen cada una;
- reconocer errores operativos frecuentes, desde filtraciones DNS hasta conjuntos de anonimato demasiado pequeños;
- utilizar tor para comprobar si una aplicación le delega la resolución de nombres (laboratorio 18);
- analizar un incidente real mediante un modelo de amenazas.

Capítulo 26 terminó con una observación incómoda: los ataques contra el protocolo pueden ser costosos y exigir puntos de observación privilegiados. En numerosos casos documentados, el anonimato también se ha perdido por errores en el uso de las herramientas que el protocolo de red por sí solo no puede impedir. Este capítulo examina tales errores, no para explicar cómo identificar personas, sino para aclarar dónde terminan las protecciones de Tor y comienzan las responsabilidades de los propios usuarios.

Advertencia: Alcance de este capítulo

Analizamos errores y prácticas defensivas desde la perspectiva de quienes desean protegerse. No ofrecemos procedimientos para identificar personas. Los incidentes mencionados son públicos y se han reconstruido a partir de las fuentes citadas.

28.1 Cuatro capas

La identidad puede filtrarse en cuatro capas (figura 28.1). La primera, la red, es la que protege tor: en una conexión habitual a través de Tor, el destino no ve la IP original del cliente y un repetidor individual normalmente no conoce ambos extremos. Esta separación no impide todos los ataques de correlación ni otros riesgos. La segunda, la aplicación, está cubierta por Tor Browser, Tails y Whonix (capítulo 23). Las dos últimas, contenido y comportamiento, no quedan automáticamente protegidas por tor. Otras herramientas pueden mitigar algunos

riesgos, pero también importan los datos que los usuarios escriben, suben y las actividades que realizan.

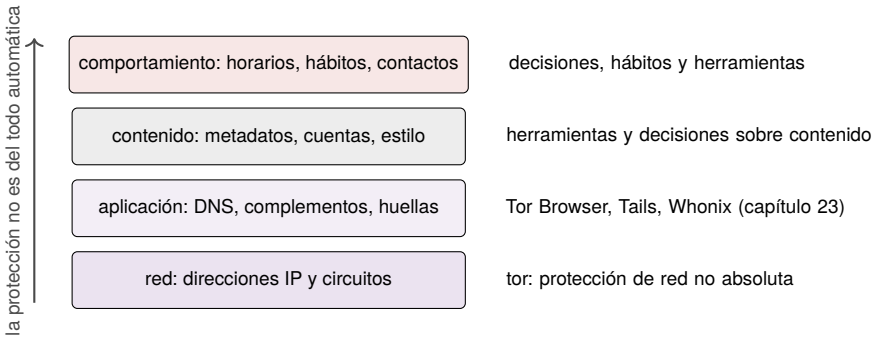


Figura 28.1: Esquema didáctico de cuatro capas donde puede exponerse la identidad. Tor reduce ciertos riesgos de red; las protecciones de aplicación y las herramientas para el contenido ayudan, pero lo que el usuario divulga y hace importa en todas las capas. Ninguna capa garantiza por sí sola el anonimato.

28.2 Errores de aplicación

Tráfico fuera de Tor. tor solo protege las aplicaciones configuradas para utilizarlo [Thef]. El software que establece conexiones directas, incluso para actualizaciones o vistas previas, puede revelar una dirección IP real. La asistencia del Tor Project destaca dos casos conocidos. Las aplicaciones BitTorrent pueden ignorar la configuración del proxy y, además, revelar sus direcciones a los rastreadores. Los documentos descargados que se abren mientras hay conexión también suponen un riesgo: los archivos DOC o PDF pueden incluir recursos remotos que las aplicaciones descargan sin utilizar Tor.

Filtraciones DNS. Un caso más sutil implica SOCKS. Una aplicación puede transmitir a tor el *nombre* del destino y permitir que la salida lo resuelva (apartado 11.1.4), o resolverlo ella misma mediante el DNS del sistema y transmitir a tor únicamente una dirección IP. En el segundo caso, la conexión posterior también utiliza Tor, pero el dominio podría haberse revelado previamente a un resolvedor situado fuera del circuito Tor, asociándolo con la conexión del cliente. No es inevitable una consulta DNS en claro: el resultado puede proceder de una caché, un archivo local o DNS cifrado. tor no puede demostrar una filtración ya ocurrida, pero sí señalar que recibió una solicitud SOCKS con solo una IP, como veremos en el laboratorio.

28.3 Laboratorio: tor detecta un indicio de filtración DNS

Laboratorio 18: SOCKS con nombre de host y sin él

Objetivo. Observar cómo distingue tor una aplicación que le delega la resolución de nombres de otra que la realiza por sí misma, y cómo puede rechazar esta última.
Requisitos previos. Laboratorio 11; red de laboratorio en ejecución.
Tiempo estimado. 30 minutos.

`curl` admite ambos comportamientos. Con `--socks5`, resuelve el nombre localmente y transmite a tor una IP; con `--socks5-hostname`, le transmite el nombre. El script `labs/es/`

`laboratorio/fuga_dns.py` se conecta al ControlPort del cliente `008c` y activa `TestSocks`, que hace que tor registre el tipo de cada solicitud SOCKS. El script escucha los mensajes de registro y comprueba ambas formas con `http://localhost:8080/`.

El laboratorio demuestra el *mecanismo*, no una filtración DNS real observada en el cable. `localhost` se resuelve localmente sin contactar con un servidor DNS, por lo que ninguna consulta abandona el equipo. tor observa una IP en lugar de un nombre de host: un posible indicio de resolución local anterior, no una prueba de fuga DNS (la aplicación podría utilizar directamente una IP). Con un dominio ordinario en lugar de `localhost`, una resolución local semejante podría generar una consulta externa, quizá en claro según la configuración DNS. Evitamos deliberadamente generar ese tráfico y permanecemos en la red privada. El script también fuerza el paso por SOCKS para `localhost` aunque `NO_PROXY` esté configurada. Por último, activa `SafeSocks`, que rechaza solicitudes con una IP aunque no haya existido fuga DNS:

```
--socks5           HTTP 200  tor: Your application (using socks5 to port
                   8080) is giving Tor only an IP address. Applications
                   that do DNS resolves themselves may leak information...
--socks5-hostname  HTTP 200  tor: Your application (using socks5 to port
                   8080) instructed Tor to take care of the DNS resolution
                   itself if necessary. This is good.
--socks5 SafeSocks HTTP 000
```

En la salida ilustrativa ambas solicitudes iniciales funcionan, pero solo la segunda delega la resolución a tor; la primera no demuestra una fuga DNS porque utiliza `localhost`. En el código de tor (`src/core/proto/proto_socks.c`), la primera advertencia se registra con nivel `warn`, limitando su frecuencia para no inundar los registros, y va acompañada de un evento `STATUS_CLIENT` con acción `DANGEROUS_SOCKS` en el ControlPort. Por tanto, un controlador puede advertir al usuario. Con `SafeSocks` activado, esa solicitud se rechaza y `curl` no recibe respuesta. Este laboratorio forma parte de `make verifica-lab` en la edición inglesa.

28.4 Errores de contenido

Metadatos. Muchos archivos contienen datos que sus autores ni siquiera advierten: las fotografías pueden revelar ubicaciones o modelos de cámara; los documentos, autores y fechas de creación. Tails incluye una herramienta para eliminar metadatos y sus advertencias mencionan casos en los que los metadatos de fotografías revelaron dónde se encontraba quien las tomó [Taib].

Cuentas e identidad. Iniciar sesión en una cuenta personal indica al sitio quién eres, independientemente de la dirección IP aparente de la conexión [Thef]. Utilizar dos identidades en el mismo sitio durante una sesión también puede vincularlas mediante cookies, datos de sesión, identificadores de aplicación y otros indicios; el sitio no puede observar directamente el identificador del circuito Tor. Tails recomienda reiniciar entre actividades que deban permanecer separadas [Taib]. El requisito de desvinculación de Tor Browser (apartado 23.2) se refiere a sitios diferentes, no a identidades distintas utilizadas en uno solo.

Estilo de escritura. Incluso sin metadatos, los hábitos de escritura, temas tratados, errores recurrentes y horarios de publicación pueden vincular un texto anónimo con otro firmado. Ninguna herramienta automática puede eliminar todos esos indicios.

28.5 Errores de comportamiento: un conjunto de anonimato demasiado pequeño

Capítulo 1 definió el anonimato mediante la indistinguibilidad dentro de un conjunto. Tor puede reducir algunos rasgos que distinguen la actividad de red, pero no puede garantizar que suficientes personas utilicen Tor en un lugar y momento determinados. Si muy pocas personas lo hacen, conocer *quién utilizó Tor* puede reducir considerablemente las posibilidades.

Un incidente. En diciembre de 2013, una universidad estadounidense recibió falsas amenazas de bomba por correo electrónico poco antes de un examen. El remitente había utilizado un servicio de correo temporal y Tor. Según la declaración jurada del FBI que acompañó a la denuncia, y como informaron varios medios, la universidad identificó quiénes habían utilizado Tor en su red inalámbrica durante las horas anteriores a la llegada de los mensajes [WBU13; San13]. Posteriormente, el estudiante admitió haber enviado los mensajes durante un interrogatorio. No fue necesario romper el cifrado de Tor para reducir el grupo de sospechosos: los observadores locales podían reconocer el uso de Tor sin que ello implicase conocer contenidos ni destinos. Sin embargo, podían saber *quién* estaba utilizando Tor durante ese intervalo y reducir el grupo de sospechosos a unos pocos nombres.

El incidente ilustra tres principios del modelo de amenazas (capítulo 4):

- Tor sin un puente no oculta que se está utilizando: los observadores de la red local pueden reconocerlo [Taib]. Los puentes con transportes conectables (capítulo 16) pueden reducir esa exposición.
- La correlación no exige un adversario global cuando el conjunto de candidatos es pequeño y el suceso ocurre en un intervalo temporal reducido. Es un problema básico de intersección.
- El anonimato mejora con la compañía [DM06]. Cuantas más personas con motivos diferentes utilicen Tor en un mismo entorno, menos información aporta observar que una persona concreta lo utiliza. La asistencia del Tor Project anima expresamente a ampliar su adopción [Thef].

28.6 Una lista de comprobación

La lista no es exhaustiva, pero cubre las capas en orden:

1. Utiliza Tor Browser o un sistema como Tails o Whonix, no un navegador convencional con un proxy configurado manualmente.
2. Evita instalar extensiones o modificar ajustes que hagan destacar a tu navegador.
3. Para otras aplicaciones, verifica que transmitan a tor los nombres de host (`SafeSocks`, `TestSocks`).
4. No abras documentos descargados mientras estés conectado, o utiliza para ello un entorno aislado.
5. Prefiere HTTPS y los servicios onion, porque una salida puede ver el contenido no cifrado.
6. Elimina los metadatos de los archivos antes de compartirlos.

7. No combines identidades que deban permanecer separadas en una misma sesión.
8. Si resulta peligroso que se sepa que utilizas Tor, considera un puente ofuscado apropiado, sin asumir que te vuelva invisible.
9. Recuerda que los horarios, hábitos y estilos de escritura también transmiten información.

Resumen

- La identidad puede filtrarse en las capas de red, aplicación, contenido y comportamiento. tor protege directamente solo la primera.
- Las aplicaciones que eluden Tor, los documentos descargados abiertos con conexión y las filtraciones DNS son errores de la capa de aplicación. tor puede advertir sobre solicitudes SOCKS que contienen únicamente IP y rechazarlas.
- Los metadatos, las cuentas y el estilo de escritura pueden relacionar contenidos con personas.
- Cuando el conjunto de usuarios Tor es pequeño, saber quién lo utilizó y cuándo puede bastar para acotar una identificación. Los puentes y una comunidad amplia de usuarios contribuyen a reducir ese riesgo.

Ejercicios

Ejercicio 28.1 ⋆ Para cada error explicado aquí, identifica su capa en figura 28.1 y la herramienta o el hábito que ayuda a evitarlo.

Ejercicio 28.2 ⋆ Explica por qué una filtración DNS divulga información aunque la conexión posterior atraviese Tor. ¿Quién observa esa información?

Ejercicio 28.3 ⋆⋆ En el laboratorio 18, suscríbete a los eventos `STATUS_CLIENT` mediante Stem y muestra la acción `DANGEROUS_SOCKS` generado por una solicitud que solo contiene una IP. ¿Cómo podría aprovecharlo una interfaz gráfica?

Ejercicio 28.4 ⋆⋆ Reconsidera el incidente de 2013 suponiendo que el estudiante hubiera utilizado un puente obfs4 desde la misma red. ¿Qué podrían haber visto los observadores locales y qué otros datos aún permitirían reducir el grupo de candidatos?

Ejercicio 28.5 ⋆⋆⋆ Elige una persona real o ficticia con una necesidad legítima de anonimato, como una fuente periodística, un activista o un investigador, y redacta un plan operativo que incluya amenazas, capas relevantes, herramientas y hábitos, y riesgos residuales.

Lecturas complementarias

- Recomendaciones de asistencia del Tor Project sobre anonimato [Thef].
- Advertencias de Tails, *Tails is safe but not magic* [Taib].
- R. Dingledine y N. Mathewson, *Anonymity Loves Company* [DM06].

Parte V

Investigación y futuro

Capítulo 29

Investigación académica sobre Tor

Última verificación: 23 de septiembre de 2026 — artículos citados en este capítulo; portal de investigación del Tor Project; consenso del 23 de septiembre de 2026

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- identificar los principales foros donde se publica investigación sobre Tor y los recursos proporcionados por el Tor Project;
- medir cuantitativamente el anonimato, desde la entropía hasta el tiempo transcurrido hasta la primera exposición;
- seleccionar el método experimental apropiado: modelización analítica, simulación, emulación o medición;
- simular miles de clientes durante meses empleando un consenso real (laboratorio 19).

Tor ha estado impulsado por la investigación desde sus comienzos. El encaminamiento cebolla surgió en un laboratorio de investigación (capítulo 3); el diseño de Tor se presentó en una conferencia académica [DMS04]; y muchos de los mecanismos abordados en este libro, desde los guardias hasta el control de congestión, surgieron como respuesta a investigaciones publicadas. Este capítulo se dirige a quienes desean realizar sus propias investigaciones: dónde publicar, cómo medir el anonimato, qué herramientas experimentales utilizar y qué principios de seguridad respetar.

29.1 Dónde se publica investigación sobre Tor

La investigación sobre redes de anonimato se publica principalmente en dos clases de foros:

Privacy Enhancing Technologies Symposium (PETS) y su revista, *Proceedings on Privacy Enhancing Technologies* (PoPETs), un foro especializado: el trabajo pionero sobre medición del anonimato que veremos se presentó en el taller PET de 2002, antes de que existiera la revista PoPETs [Día+02].

Grandes conferencias de seguridad USENIX Security, ACM CCS, IEEE Symposium on Security and Privacy y NDSS. Muchos de los artículos de capítulo 26 se presentaron en estas conferencias.

La *Anonymity Bibliography* de Free Haven recopila publicaciones en este ámbito desde hace más de veinte años y constituye un punto de partida natural para una revisión bibliográfica.

El Tor Project también mantiene un portal de investigación que reúne datos, herramientas, informes técnicos, temas de investigación propuestos e información sobre el Research Safety Board. Sus orientaciones destacan que el protocolo y la red evolucionan continuamente: las mediciones realizadas sin comprender su funcionamiento interno pueden llevar a conclusiones equivocadas. Por ello, se anima a los investigadores a ponerse en contacto con el proyecto desde las primeras fases para conocer experimentos simultáneos y cambios previstos.

29.2 Medir el anonimato

En capítulo 1 definimos el anonimato como indistinguibilidad dentro de un conjunto. Sin embargo, desde la perspectiva de un adversario, no todos sus miembros son igualmente probables. Tras observar tráfico, el adversario podría asignar probabilidades mucho mayores a unos usuarios que a otros. El número de integrantes del conjunto no basta.

Grado de anonimato. En 2002, Díaz y sus colaboradores propusieron medir el anonimato mediante la entropía de la distribución de probabilidades que un adversario atribuye a los posibles remitentes [Día+02]. Si p_i es la probabilidad que el adversario asigna al usuario i como remitente, la entropía es

$$H(X) = -\sum_{i=1}^{N} p_i \log_2 p_i,$$

y alcanza su máximo, $H_M = \log_2 N$, cuando los N usuarios son igualmente probables. El **grado de anonimato** es la razón

$$d = \frac{H(X)}{H_M}, \qquad 0 \leq d \leq 1 :$$

para $N \geq 2$, vale 1 si la distribución posterior es uniforme; esto no demuestra por sí solo la ausencia de información nueva. Vale 0 cuando un remitente tiene probabilidad 1. Para $N = 1$, la razón no está definida ($H_M = 0$).

Una interpretación intuitiva de la entropía es $2^{H(X)}$: el número de usuarios igualmente probables que producirían la misma incertidumbre. Un conjunto de 1000 usuarios en el que el adversario haya reducido los sospechosos a dos, cada uno con probabilidad 0,5, proporciona tanta incertidumbre como un conjunto de dos: $H = 1$ bit.

Límites de las mediciones puntuales. La entropía describe una observación individual, pero la exposición en Tor se acumula con el tiempo, como vimos en apartado 26.2: cada circuito crea otra oportunidad para el adversario. Johnson y sus colaboradores propusieron por tanto medidas *longitudinales*, entre ellas la probabilidad de desanonimización al menos una vez durante cierto período y el tiempo mediano hasta la primera exposición, calculadas simulando el comportamiento de usuarios sobre datos reales de la red [Joh+13]. Seguiremos un enfoque semejante en el laboratorio.

29.3 Cuatro métodos

Modelización analítica Un modelo matemático, como el grado de anonimato o el modelo de ataques Sybil de apartado 26.5. Es rápido y general, pero depende de hipótesis que los sistemas reales inevitablemente incumplen en cierta medida.

Simulación de comportamiento Reproduce un componente simplificado del sistema, como la selección de rutas, utilizando datos reales. TorPS, el simulador de rutas desarrollado por Johnson y sus colaboradores [Joh+13], y nuestro laboratorio 19 siguen este enfoque. Es rápido, pero no puede reflejar nada que el modelo haya omitido.

Emulación y simulación de redes Ejecuta el software real en una red privada, como el laboratorio Chutney utilizado en este libro, o dentro de un simulador de eventos discretos como Shadow [JH12], capaz de ejecutar miles de repetidores y clientes en una sola máquina con tiempo virtual (apartado 19.9). Resulta especialmente útil para investigar el rendimiento, porque permite probar cambios propuestos al protocolo en una red realista antes de desplegarlos.

Mediciones en la red pública Solo la medición directa revela lo que sucede en la red real. También conlleva el mayor riesgo para usuarios reales y exige un diseño experimental cuidadoso (apartado 29.5).

Una investigación sólida suele combinar varios métodos: el análisis matemático sugiere una hipótesis, la simulación la pone a prueba a gran escala y una medición cuidadosamente diseñada examina su realismo.

En profundidad: Medir sin observar a individuos

PrivCount, desarrollado por Jansen y Johnson, muestra cómo pueden coexistir la medición y la privacidad [JJ16]. Los repetidores participantes cuentan sucesos relevantes, pero los contadores se ocultan mediante valores aleatorios compartidos con otros participantes. Se añade ruido según los principios de la *privacidad diferencial*. Solo se hace visible un resultado agregado y deliberadamente perturbado, lo que limita las inferencias sobre usuarios individuales. En 2016, los autores utilizaron PrivCount para estimar aproximadamente $710\,000$ usuarios conectados simultáneamente, de los cuales $550\,000$ estaban activos en un instante determinado, y observaron que el tráfico dirigido a puertos web representaba el 91% de los bytes observados en las salidas participantes (estudio de 2016). Esto lleva un paso más allá los principios de privacidad del redondeo de estadísticas de repetidores (apartado 24.2).

29.4 Laboratorio: tiempo hasta la primera exposición

Laboratorio 19: Meses de circuitos en una simulación

Objetivo. Utilizar un consenso real para estimar cuántos clientes puede desanonimizar con el tiempo un adversario que controla cierta cuota del ancho de banda, con guardias persistentes y sin ellos.
Requisitos previos. Laboratorios 7 y 15; no es necesaria una red en ejecución.
Tiempo estimado. 1–2 horas.

El módulo `labs/es/codigo/torlab/simulation.py` reutiliza los pesos del laboratorio 7. El adversario añade un guardia y una salida que representan cada uno una fracción q del ancho de banda ponderado correspondiente a su posición en el circuito. Simulamos 2000 clientes durante 180 días, cada uno de los cuales crea 20 circuitos diarios.

El modelo sustituye el guardia persistente según un calendario aproximado para un guardia *confirmado*. Según guard-spec y el código de `entrynodes.c`, un guardia confirmado se elimina de la muestra `GUARD_CONFIRMED_MIN_LIFETIME` (60 días) después de la confirmación,

y `CONFIRMED_ON_DATE` se retrotrae aleatoriamente hasta `GUARD_LIFETIME`/10 (12 días). Esto produce en el modelo intervalos de 48–60 días. Los 120 días de `GUARD_LIFETIME` regulan la caducidad de guardias no confirmados en la muestra. La selección real depende además de las listas de guardias, accesibilidad y sustituciones. Una versión temprana de este laboratorio simuló erróneamente rotaciones cada 120 días, reduciendo aproximadamente a la mitad el número de rotaciones y subestimando el riesgo casi en esa misma proporción. Como comparación, también simulamos clientes que eligen un guardia nuevo para cada circuito. Un cliente se considera comprometido el primer día que uno de sus circuitos incluye tanto un guardia como una salida del adversario.

El modelo es intencionadamente sencillo: ignora repetidores intermedios, familias, redes /16, guardias de reserva y fallos. Para mantenerlo rápido, no utiliza `random.choices`, que volvería a calcular miles de pesos en cada selección. En su lugar emplea búsqueda binaria sobre pesos acumulados, lo que permite millones de selecciones sin recalcular todos los pesos; el tiempo depende del equipo.

El consenso público no se incluye en el repositorio: se descarga una sola vez con `make -C labs/es data`, que lo guarda en `labs/es/datos/collector/` comprobando su cabecera, no sus firmas. La simulación es determinista: con el mismo modelo, consenso, proporción y semilla, el resultado es siempre este:

```
$ PYTHONPATH=labs/es/codigo python3 -m torlab.simulation labs/es/datos/collector/
    consensus-microdesc-2026-09-23-15 0.05
adversario: 5% del ancho de banda de guardias y 5% de salidas; 2000 clientes, 180 días
    , 20 circuitos/día
guardia persistente     1d:  2.6%    7d:  4.4%   30d:  4.4%   90d:  9.4%  180d: 18.4%
     mediana: None
guardia por circuito    1d:  4.3%    7d: 27.9%   30d: 76.5%   90d: 98.9%  180d:100.0%
     mediana: 15
```

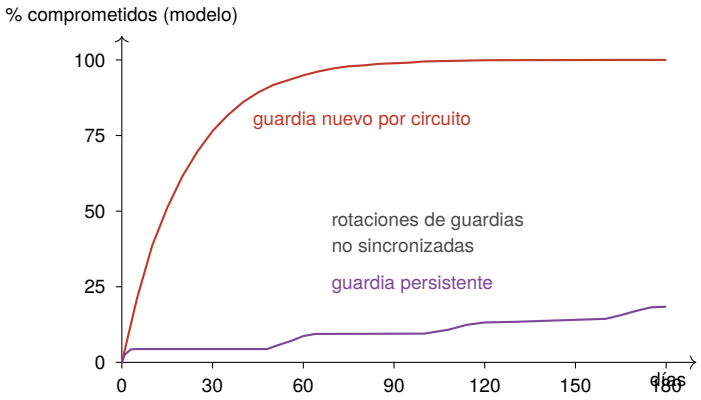


Figura 29.1: Laboratorio 19, simulación simplificada: 2000 clientes, 20 circuitos diarios durante 180 días, con el consenso del 23 de septiembre de 2026; el adversario posee un 5% del ancho de banda ponderado de guardias y un 5% del de salidas. La proporción acumulada contabiliza a quienes usaron al menos un circuito con ambos extremos adversarios en el modelo. Los guardias persistentes rotan en momentos distintos para cada cliente (intervalos simulados de 48–60 días), no en tres fechas comunes. No son observaciones de desanonimización real en la red pública.

La figura 29.1 muestra ambas curvas. Sin guardias persistentes, la mitad de los clientes resulta comprometida en quince días y casi todos en tres meses, ya que cada circuito añade otra oportunidad independiente. Con guardias persistentes, la exposición de un cliente depende en gran medida de la primera selección del guardia. Alrededor del 4% del total

de clientes, los que eligieron un guardia adversario, quedan comprometidos al principio; el resto permanece protegido hasta que rota su guardia. Las rotaciones se distribuyen entre los clientes (aproximadamente cada 48–60 días por cliente en el modelo) y crean nuevas oportunidades de exposición, elevando el total hasta aproximadamente el 18% después de seis meses. Cuando la cuota de ancho de banda adversaria es del 1%, el resultado correspondiente a seis meses cae a aproximadamente el 4%, frente al 31% sin guardias persistentes.

Estos resultados, basados en la instantánea de referencia de septiembre de 2026, ilustran la razón de ser de los guardias. En este modelo simplificado, su efecto principal consiste en redistribuir la exposición entre clientes, sin garantizar una reducción del riesgo en cada circuito. Para la mayoría de los usuarios, el riesgo modelado es inicialmente casi nulo, pero para un grupo menor es alto. La duración de los guardias implica, por tanto, un compromiso. Una rotación más lenta protege durante más tiempo a los usuarios con guardias honestos, pero prolonga la exposición de quienes tienen guardias adversarios. Un adversario paciente que permanezca meses en la red obtiene oportunidades nuevas con cada rotación.

29.5 Investigar de forma segura

Capítulo 25 presentó las directrices del Tor Research Safety Board [Tor]. Al diseñar un experimento, pregúntate primero: *¿requiere realmente mi pregunta de investigación utilizar la red pública?* Muchas preguntas sobre rendimiento y ataques pueden responderse mediante Shadow o una red privada, con experimentos reproducibles y control de variables. Cuando sea indispensable medir la red pública, sigue los principios expuestos: actúa únicamente sobre tu propio tráfico, recoge la mínima cantidad de datos, agrega antes de publicar y consulta al Board y al Tor Project antes de empezar.

La historia reciente ofrece ejemplos contrastantes. Capítulo 27 describió un experimento, o ataque, que afectó a usuarios reales mediante una señal visible para los repetidores receptores, pero no directamente para observadores externos de conexiones TLS cifradas. PrivCount, en cambio, demuestra que es posible medir la red con detalle sin exponer a usuarios individuales. La diferencia no radica solo en las herramientas, sino en el diseño experimental.

Resumen

- La investigación sobre Tor aparece en PETS y en las principales conferencias de seguridad. El Tor Project facilita datos, herramientas y un Research Safety Board.

- La entropía mide el anonimato para una observación individual; el grado de anonimato lo normaliza entre cero y uno. A lo largo del tiempo se necesitan medidas como la probabilidad de compromiso en un intervalo determinado.

- Los modelos analíticos, simulaciones de comportamiento, emulación o simulación de redes y mediciones en la red pública responden a distintas preguntas y entrañan distintos riesgos.

- En la simulación con el consenso de referencia, los guardias persistentes reducen considerablemente la fracción de clientes comprometidos con el tiempo, al concentrar la exposición en menos clientes.

Ejercicios

Ejercicio 29.1 ⋆ Un adversario observa un sistema de cuatro usuarios y asigna las probabilidades $1/2$, $1/4$, $1/8$ y $1/8$. Calcula la entropía, el grado de anonimato y el número equivalente de usuarios igualmente probables.

Ejercicio 29.2 ⋆ Elige y justifica el método más apropiado para cada pregunta: (a) ¿cómo cambiaría la latencia si los circuitos tuvieran cuatro saltos? (b) ¿Cuántos usuarios acceden a Tor desde un país concreto? (c) ¿Reduciría un nuevo algoritmo de selección de guardias el tiempo hasta la primera exposición?

Ejercicio 29.3 ⋆⋆ Ejecuta la función `simulate` del laboratorio 19 con `guard_lifetime` establecido en 30 y 180 días, en vez de los 60 predeterminados, y compara la fracción de clientes comprometidos tras 180 días. Explica los resultados.

Ejercicio 29.4 ⋆⋆ Repite el laboratorio 19 con un adversario que controle el 10% del ancho de banda de salida pero ninguno de guardias, y después con otro que controle el 10% del ancho de banda de guardias pero ninguno de salida. ¿Qué puede conseguir cada uno y por qué?

Ejercicio 29.5 ⋆⋆⋆ Amplía el simulador con tres características que el modelo actual ignora: clientes que reutilizan circuitos durante diez minutos en lugar de crear otros constantemente, la restricción /16 entre guardias y salidas, y un adversario que controle todo el sistema autónomo de una gran red. ¿Qué incorporaciones modifican más los resultados?

Lecturas complementarias

- C. Díaz, S. Seys, J. Claessens, B. Preneel, *Towards Measuring Anonymity* [Día+02].
- R. Jansen, A. Johnson, *Safely Measuring Tor* [JJ16].
- Portal de investigación del Tor Project, `https://research.torproject.org/`.
- La *Anonymity Bibliography*, `https://www.freehaven.net/anonbib/`.

Capítulo 30

Arti: Tor en Rust

Última verificación: 23 de septiembre de 2026 — Arti 2.6.0 (1 de septiembre de 2026); tor 0.4.9.13; red de laboratorio

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- explicar por qué el Tor Project está reescribiendo tor en Rust y evaluar los avances del proyecto;
- comprender la arquitectura de Arti e identificar los conceptos tratados en la parte II;
- interpretar su tabla de subprotocolos admitidos y entender qué características siguen pendientes;
- configurar Arti como cliente de la red privada del libro (laboratorio 20).

A lo largo del libro nos hemos basado en *tor*, la implementación en C iniciada en 2002. Sin embargo, el futuro del cliente Tor es **Arti**, una reescritura completa en Rust. El Tor Project ha designado Arti como cliente preferente para desarrollar nuevas funcionalidades y ha anunciado que todas las funciones nuevas del lado del cliente recibirán una implementación en Arti. Este capítulo presenta Arti desde la perspectiva de quien acaba de escribir un pequeño cliente Tor en Python durante los laboratorios. Muchas de sus decisiones de diseño son las mismas que afrontamos nosotros, implementadas a escala de producción.

30.1 ¿Por qué reescribir tor?

El documento introductorio del proyecto enumera cuatro motivaciones:

Seguridad El código Rust seguro evita muchas clases de errores de memoria, aunque las dependencias, el código `unsafe` y los errores lógicos siguen requiriendo revisión. El Tor Project estima que al menos la mitad de las vulnerabilidades de seguridad registradas en tor habrían sido imposibles en Rust y muchas otras habrían sido muy improbables.

Velocidad de desarrollo Las garantías del lenguaje permiten desarrollar código nuevo con mayor confianza y en menos tiempo.

Flexibilidad tor empezó como proxy SOCKS y las funciones de integración se añadieron después. Arti fue diseñado desde el principio como una biblioteca modular que otras aplicaciones pueden incorporar.

Mantenibilidad Veinte años de desarrollo han dejado relaciones complejas entre los componentes del código de tor. Una reescritura permite separar las responsabilidades con mayor claridad.

Hitos. La primera versión pública de Arti, 0.0.1, apareció en octubre de 2021. La versión 1.0.0, publicada el 1 de septiembre de 2022, declaró Arti estable como cliente. En agosto de 2023, la versión 1.1.7 habilitó por defecto la compilación del soporte de servicios onion del lado cliente, aunque los usuarios todavía debían activar la función. La versión 2.0.0 llegó en febrero de 2026. La versión de referencia de este capítulo es la 2.6.0, publicada el 1 de septiembre de 2026, que mantiene habilitados de forma permanente el control de congestión (apartado 11.3) y el cifrado CGO (apartado 10.4).

Alcance actual. Según la documentación del proyecto en la fecha de referencia, Arti es un cliente de uso general completo que admite puentes y transportes conectables; puede tanto acceder a servicios onion como publicarlos. Todavía no puede funcionar como repetidor, puente o autoridad de directorio. Esas capacidades están en desarrollo y gran parte del trabajo de las versiones recientes se ha centrado en repetidores y autoridades.

30.2 Arquitectura

Arti consta de decenas de *crates* de Rust, las unidades de compilación del lenguaje: más de setenta en la versión 2.6.0. Sus convenciones de nombres distinguen las funciones. Las crates orientadas al usuario comienzan por `arti`, las específicas del protocolo Tor por `tor-`, y las útiles fuera de Tor reciben nombres más generales. Un principio guía el diseño: reducir al mínimo el código que lee o escribe directamente datos de red; la mayoría de los componentes deberían funcionar «como si la red no existiera», facilitando su comprobación.

La figura 30.1 sitúa las principales crates junto a los módulos de `torlab` que cumplen funciones semejantes. La correspondencia es deliberada: ambas arquitecturas siguen las capas del protocolo, desde las primitivas de bajo nivel hasta el cliente de alto nivel.

`tor-llcrypto`, `tor-bytes` Primitivas criptográficas y análisis y serialización binarios. Proporcionan los fundamentos que nuestros módulos `ntor.py` y `encryption.py` obtuvieron de `cryptography` y `struct` en Python.

`tor-cell` Celdas y mensajes relay: el equivalente en Arti de `cells.py` (capítulo 8).

`tor-proto` Canales, circuitos, cifrado por capas y flujos: todo lo que construimos con `channel.py`, `circuit.py` y `stream.py`.

`tor-chanmgr`, `tor-circmgr`, `tor-guardmgr` Administración de múltiples canales y circuitos, selección de rutas y mantenimiento de guardias. Nuestro cliente educativo apenas implementó estas funciones, que figuran entre las más complejas de un cliente real.

`tor-netdoc`, `tor-dirmgr` Lectura y actualización de documentos de directorio, correspondientes a `directory.py`.

`tor-hsclient`, `tor-hsservice` Los dos lados de los servicios onion, para los que `onion.py` solo implementó el descifrado de descriptores.

`arti-client`, `arti` La biblioteca de alto nivel para incorporarla a otras aplicaciones y el programa de línea de órdenes que expone un proxy SOCKS.

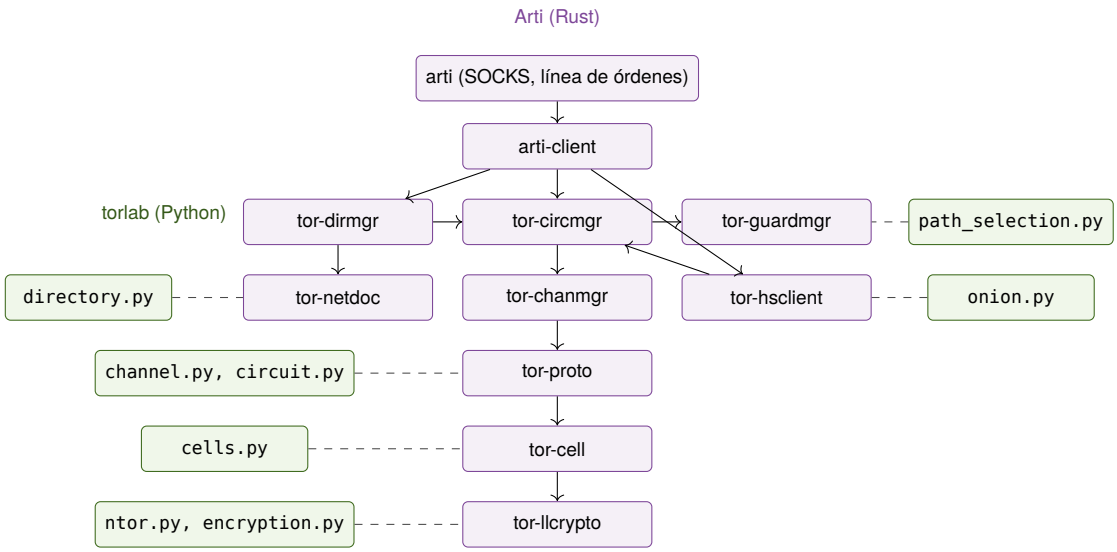


Figura 30.1: Mapa funcional simplificado de crates Arti y módulos didácticos de `torlab`. Las flechas continuas indican relaciones arquitectónicas generales, no el grafo exacto de Cargo; las discontinuas señalan funciones semejantes, no dependencias entre proyectos ni equivalencias completas. Por ejemplo, `onion.py` solo descifra descriptores y no sustituye a un cliente completo de servicios onion; se omiten crates como `tor-bytes` y `tor-hsservice`.

30.2.1 Una comparación: escribir una celda

En el laboratorio 2 escribimos una función de codificación de celdas. La función equivalente de Arti, `write_cell`, ubicada en `crates/tor-cell/src/chancell/codec.rs`, sigue los mismos pasos: escribir el CircID y después el comando. Para una celda de longitud variable, reserva dos bytes para la longitud, codifica el cuerpo y vuelve atrás para escribir la longitud real. Para una de longitud fija, codifica el cuerpo y comprueba que no supere 509 bytes (`CELL_DATA_LEN`). Hay dos diferencias instructivas:

- El CircID se escribe *siempre* con cuatro bytes. Arti no implementa versiones del protocolo de enlace anteriores a la 4, por lo que no necesita el caso especial de CircID de dos bytes (capítulo 8). Solo la celda **`VERSIONS`**, anterior a la negociación, se trata por separado.
- Incumplir el límite de longitud no produce un error del llamante, sino uno *interno*: una celda demasiado grande sería un defecto del código del propio Arti, no de los datos recibidos de otro participante. El tipo de error distingue así un fallo de implementación de una entrada maliciosa.

Arti renuncia expresamente a admitir protocolos obsoletos. Su objetivo es la compatibilidad con las versiones *actualmente recomendadas*, no la conservación de todos los formatos históricos.

30.3 Funciones admitidas

La descripción más precisa de las capacidades de una implementación de Tor es su compatibilidad con **subprotocolos** ([tor-spec § Subprotocol-based versioning]). Cada repetidor y cliente anuncia las versiones que admite para cada familia de funciones: `Link`, `Relay`, `FlowCtrl`, `HSIntro`, etcétera. La documentación de compatibilidad de Arti enumera, entre otras funciones del lado cliente:

- `Link=4` y `Link=5`: CircID de cuatro bytes y relleno moderno de canales (capítulo 7);

- `Relay=4` (ntor-v3, capítulo 9), `Relay=5` (negociación de subprotocolos) y `Relay=6` (cifrado CGO);
- `FlowCtrl=1` (SENDME autenticado) y `FlowCtrl=2` (control de congestión);
- `Desc=4`: familias basadas en claves, como se describe en apartado 22.5.

Advertencia: Un error de desfase en la tabla de Arti

El documento de compatibilidad de Arti citado asigna a los dos subprotocolos de control de flujo los números `FlowCtrl=0` y `FlowCtrl=1`. La especificación, en cambio, asigna el valor 1 a SENDME autenticado y el 2 al control de congestión [tor-spec § Subprotocol-based versioning]; los repetidores del consenso público anuncian `FlowCtrl=1-2`. Se trata de un error de numeración en la documentación, no en la implementación. También ilustra una buena práctica: cuando las fuentes discrepan, hay que examinar la especificación y buscar pruebas independientes en el consenso.

En la fecha de referencia, faltan en el cliente Arti dos capacidades del cliente C. `Conflux=1` permite el funcionamiento multirruta: la crate `tor-proto` lo admite, pero Arti todavía no construye circuitos Conflux. `Padding=2` se refiere a máquinas de estados de relleno destinadas a ocultar la construcción de circuitos de servicios onion (apartado 26.4). Aunque `tor-proto` incluye un marco de relleno basado en la biblioteca *maybenot*, esas máquinas aún no están implementadas. Ningún subprotocolo del lado repetidor figura como admitido: existe gran parte del código correspondiente, pero solo se declararán compatibles las funciones cuando un repetidor funcional pueda ofrecerlas.

30.4 Laboratorio: Arti en la red privada

Laboratorio 20: Un segundo cliente, en Rust

Objetivo. Compilar Arti, configurarlo como cliente de la red privada del libro y utilizar su proxy SOCKS en lugar de `test008c`.
Requisitos previos. Red de laboratorio en ejecución y herramientas de Rust (`cargo`).
Tiempo estimado. 1–2 horas, principalmente de compilación.

Paso 1: compilar. El repositorio no incluye Arti compilado. Desde la raíz del repositorio, descarga sus fuentes en el árbol independiente `labs/es/vendor/arti` y construye una versión optimizada:

```
$ mkdir -p labs/es/vendor
$ git clone https://gitlab.torproject.org/tpo/core/arti.git labs/es/vendor/arti
$ git -C labs/es/vendor/arti checkout arti-v2.6.0
$ (cd labs/es/vendor/arti && cargo build --release -p arti --locked)
$ labs/es/vendor/arti/target/release/arti --version
```

La versión histórica usada en la referencia fue Arti 2.6.0; aquí no se ha repetido la compilación. La primera compilación tarda varios minutos porque construye todo el árbol de crates y dependencias.

Paso 2: configurar. Arti no lee `torrc`: emplea configuración TOML. Para acceder a una red Tor no pública, debe conocer las autoridades de directorio y las cachés alternativas de las que descargar sus documentos iniciales, igual que las líneas `DirAuthority` de los archivos

`torrc` generados por Chutney (apartado 19.6). El generador `labs/es/laboratorio/arti_config.py` lee las huellas RSA, las claves Ed25519, las identidades de certificados v3 y los puertos de escucha de los archivos locales de cada autoridad. Escribe estos valores en `labs/es/laboratorio/net/arti/arti.toml`. El siguiente fragmento es ilustrativo y no un TOML listo para ejecutar: los identificadores se generan en la red local y los puntos suspensivos son marcadores de contenido omitido:

```
[tor_network]
fallback_caches = [
  { rsa_identity = "7B465200C885ED566C6CD4F46DBB0492076D203A",
    ed_identity = "JwhsA9VS01mRVnQseL9xsKZfstoBae7OcgL3jBvDzis",
    orports = ["127.0.0.1:5100"] },
  ...
]
[tor_network.authorities]
v3idents = ["390998046DEFF1E68AC346D7577319791B32C91A", ...]
```

Otros tres cambios de configuración adaptan Arti a una red que funciona enteramente en `127.0.0.1`. La opción `allow_local_addrs` permite conexiones a direcciones locales, que los clientes de la red pública suelen rechazar. Dos longitudes de prefijo imposibles (33 bits para IPv4 y 129 para IPv6) desactivan la restricción de compartir red, que impediría construir cualquier circuito aquí. Por último, `hsdir_interval` acorta el período de servicios onion, tal como hace `TestingTorNetwork` en tor. Estos ajustes reproducen la configuración Arti utilizada por Chutney.

Paso 3: iniciar. Desde la raíz del repositorio, genera primero la configuración con `python3 labs/es/laboratorio/arti_config.py` y después ejecuta `labs/es/vendor/arti/target/release/arti proxy -p 9150 -c labs/es/laboratorio/net/arti/arti.toml`. La duración del arranque depende de la máquina; en una de nuestras ejecuciones:

```
INFO tor_dirmgr::bootstrap: 1: Looking for a consensus.
INFO tor_dirmgr::bootstrap: 1: Downloading certificates for consensus
     (we are missing 3/3).
INFO tor_dirmgr::bootstrap: 1: Downloading microdescriptors (we are missing 8).
INFO tor_dirmgr: Marked consensus usable.
WARN tor_netdir: Requested 60 relays, but only 8 were usable, ...  role=Guard
INFO arti::subcommands::proxy: Sufficiently bootstrapped; proxy now functional.
INFO tor_guardmgr::guard: We have found that guard [scrubbed] is usable.
```

Tres detalles merecen atención. El arranque sigue paso por paso apartado 12.7: primero el consenso, después los certificados de las tres autoridades y finalmente ocho microdescriptores. La advertencia de `tor_netdir` refleja el tamaño reducido de nuestra red: el administrador de guardias preferiría una muestra de 60 candidatos, pero solo ocho son utilizables. La identidad del guardia aparece en el registro como `[scrubbed]`: Arti marca los valores sensibles mediante su crate `safelog` y los oculta en los registros, igual que `SafeLogging` en tor. En otras entradas del registro, durante actualizaciones posteriores, podría aparecer `Applying a consensus diff` (no figura en el fragmento anterior). Arti puede descargar solo los cambios si existe y se negocia una diferencia mediante `DirCache=2`; no está garantizado en cada actualización.

Paso 4: utilizarlo. Arti expone un proxy SOCKS en el puerto 9150. `curl` lo utiliza igual que el de `test008c`:

```
$ curl -s -o /dev/null -w '%{http_code} in %{time_total}s\n' \
       --noproxy "" --socks5-hostname 127.0.0.1:9150 http://127.0.0.1:8080/
200 in 0.047224s
```

El argumento vacío de `--noproxy` evita que `NO_PROXY` excluya el destino local del proxy. El circuito atraviesa los repetidores escritos en C de la red privada: un cliente en Rust y repetidores en C hablan exactamente el mismo protocolo byte a byte estudiado en la parte II. Una última observación se relaciona con el laboratorio 18: las solicitudes SOCKS que solo contienen una dirección IP (`--socks5`) también funcionan y, por defecto, Arti no muestra en sus registros una advertencia análoga a la de tor. Sin embargo, la documentación de `arti-client` advierte a quienes utilizan su API sobre el riesgo de resolver nombres localmente y revelar destinos a un resolvedor DNS, según la configuración. Una solicitud SOCKS con solo IP no demuestra por sí misma una fuga DNS (apartado 28.3).

El laboratorio forma parte de `make verifica-lab` cuando Arti está compilado; en caso contrario, la prueba se omite y se indica.

Resumen

- Arti reescribe tor en Rust, un lenguaje con seguridad de memoria. Es modular, se diseñó como biblioteca, es estable como cliente desde 2022 y constituye el cliente preferente para desarrollar nuevas funciones.

- Su arquitectura sigue las capas del protocolo: primitivas y celdas, canales y circuitos, y administración de directorios, rutas y guardias.

- Las versiones de subprotocolos describen con precisión qué admite Arti. Su cliente implementa ntor-v3, CGO, control de congestión y familias basadas en claves; Conflux y las máquinas de relleno siguen incompletos en la fecha de referencia. El soporte para repetidores continúa desarrollándose.

- Arti se conecta a una red privada mediante autoridades y cachés de directorio configuradas, de modo semejante a un nodo Chutney.

Ejercicios

Ejercicio 30.1 ⋆ Para cada capítulo de la parte II, identifica la crate de Arti que implementa los conceptos principales estudiados.

Ejercicio 30.2 ⋆ ¿Qué ventajas e inconvenientes tiene la decisión de Arti de no implementar versiones obsoletas del protocolo? ¿Quién podría quedar excluido?

Ejercicio 30.3 ⋆⋆ Lee `decode_cell` en `crates/tor-cell/src/chancell/codec.rs` y compáralo con `decode` del laboratorio 2. ¿Cómo se reconoce una celda de longitud variable y qué ocurre si la entrada aún no contiene una celda completa?

Ejercicio 30.4 ⋆⋆ Repite el experimento de aislamiento de flujos del laboratorio 11 utilizando Arti como proxy SOCKS. ¿Impiden las distintas credenciales SOCKS que los flujos compartan circuito en Arti? Distingue el aislamiento de flujos de la idea incorrecta de que cada pareja de credenciales corresponde siempre a un único circuito; investiga su configuración.

Ejercicio 30.5 ⋆ ⋆ ⋆ Escribe un programa pequeño en Rust que utilice `arti-client` para obtener una página del servidor de la red de laboratorio, cargando desde un archivo la configuración del laboratorio 20.

Lecturas complementarias

- El sitio del proyecto, `https://arti.torproject.org/`, y su repositorio de código, especialmente el README, la descripción arquitectónica (`doc/dev/Architecture.md`) y la guía de compatibilidad (`doc/Compatibility.md`).
- El historial de cambios de Arti, que documenta las modificaciones de cada versión mensual.
- La configuración de Arti generada por Chutney (`lib/chutney/arti/config.py`), que inspiró el laboratorio 20.

Capítulo 31

Problemas abiertos y proyectos en curso

Última verificación: 23 de septiembre de 2026 — propuestas de torspec hasta la número 369; tor 0.4.9.13; Arti 2.6.0

Objetivos del capítulo

Al finalizar este capítulo, el lector podrá:

- seguir el proceso mediante el que cambia el protocolo Tor e interpretar el estado de una propuesta;
- explicar por qué la criptografía poscuántica exige modificar formatos de mensajes y no solo algoritmos;
- describir Conflux, Walking Onions y otros proyectos destinados a mejorar rendimiento y escalabilidad;
- identificar maneras concretas de contribuir al proyecto.

Nada de lo descrito en este libro es definitivo. El protocolo examinado aquí en 2026 difiere del de 2016, que carecía de ntor-v3, control de congestión y CGO; volverá a cambiar de aquí a 2036. Este último capítulo mira hacia adelante. Ante cada problema abierto, preguntamos qué todavía no funciona lo suficientemente bien, qué se ha propuesto y cuánto se ha avanzado. Todo lo que sigue es necesariamente provisional: los estados de las propuestas corresponden a los registrados en el repositorio de especificaciones en la fecha de referencia del capítulo.

31.1 Cómo cambia el protocolo

Desde 2007, los cambios de protocolo se desarrollan mediante **propuestas** numeradas, de forma semejante a los RFC del IETF. Una persona contribuyente debate una idea en la lista de correo de desarrolladores, la redacta en el formato establecido y, cuando existe un acuerdo amplio en que merece estudiarse, los editores la incorporan al repositorio con un número. La propuesta se debate y perfecciona hasta detallar suficientemente su implementación. Después, el contenido normativo implementado pasa a las especificaciones. A diferencia de un RFC, una propuesta puede cambiar sin modificar su número y nunca constituye el documento definitivo de una función implementada: ese papel corresponde a las especificaciones (Propuesta 001).

El estado de una propuesta (figura 31.1) debe ser lo primero que compruebe el lector. En este libro hemos encontrado propuestas en situaciones muy distintas. ntor-v3 (Propuesta 332) está *Closed*: implementada e incorporada a las especificaciones. El control de congestión (Propuesta 324) y Conflux (Propuesta 329) están *Finished*: implementadas, pero aún no

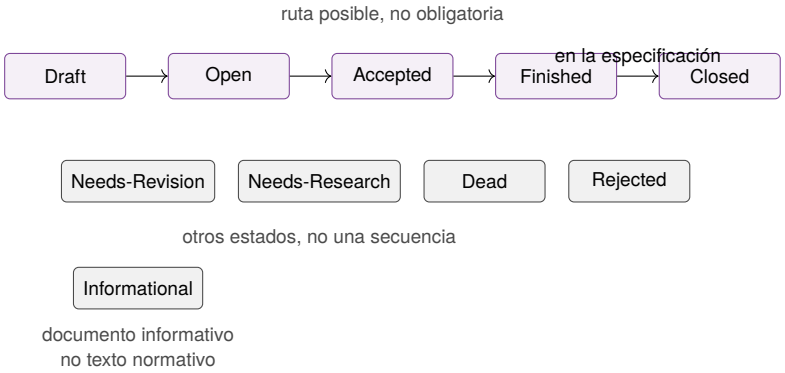


Figura 31.1: Estados de las propuestas Tor según Proposal 001 (esquema simplificado y no exhaustivo). La secuencia *Draft–Open–Accepted–Finished–Closed* es ilustrativa, no obligatoria: *Finished* indica implementación y *Closed* también la incorporación a las especificaciones. *Needs-Revision* y *Needs-Research* describen impedimentos diferentes; *Dead* puede reabrirse y *Rejected* descarta lo propuesto. *Informational* es un documento informativo final, no texto normativo por sí mismo. Se omiten *Meta*, *Reserve*, *Obsolete* y otros estados.

plenamente incorporadas. CGO (Propuesta 359) sigue *Open*, pese a estar ya presente tanto en tor como en Arti. El plan para eliminar TAP (Propuesta 350) figura como *Accepted*, mientras Walking Onions (Propuesta 300) es *Informational*.

31.2 Ordenadores cuánticos

El problema. Todos los mecanismos de clave pública de Tor, incluidos X25519 para los intercambios de claves, Ed25519 para las firmas y los usos restantes de RSA, podrían romperse mediante un ordenador cuántico suficientemente potente (capítulo 5). Nadie sabe si llegará a existir ni cuándo. Sin embargo, para un sistema de anonimato la amenaza puede comenzar antes: un adversario podría registrar hoy tráfico cifrado y descifrarlo en el futuro.

Dos escenarios. La Propuesta 355, de 2025, distingue dos situaciones. En un entorno de *transición* basta con protegerse frente a un ordenador cuántico *futuro*: el tráfico capturado ahora debe resistir el descifrado posterior, mientras las firmas tradicionales siguen siendo aceptables porque falsificarlas activamente requeriría un ordenador cuántico hoy. En un entorno de *nueva generación*, la protección debe cubrir también a un ordenador cuántico que *ya exista* y sea capaz de falsificar firmas o atacar activamente intercambios de claves.

El coste. En 2024, el NIST estandarizó ML-KEM, un mecanismo poscuántico de establecimiento de claves, junto con los algoritmos de firma ML-DSA y SLH-DSA [NIS24]. La propuesta compara sus tamaños con los algoritmos actuales:

Algoritmo	Clave pública	Texto cifrado o firma
X25519	32 bytes	32 bytes
ML-KEM-768	1 184 bytes	1 088 bytes
Ed25519	32 bytes	64 bytes
ML-DSA-65	1 952 bytes	3 309 bytes

En capítulo 8 vimos que un mensaje relay puede transportar como máximo 498 bytes. Por tanto, una sola clave pública ML-KEM-768 necesitaría tres celdas incluso antes de considerar

el resto del intercambio. La criptografía poscuántica no consiste simplemente en sustituir algoritmos: modifica la forma en que los mensajes caben dentro de las celdas. El formato relay actual no permite transmitir directamente un mensaje tan grande: tres celdas representan solo el límite inferior de capacidad si un formato nuevo permite fragmentarlo.

Dirección actual. Las conclusiones de la propuesta son prudentes. Durante la transición, añadir un mecanismo poscuántico a ntor, como sugerían propuestas anteriores, podría ser suficiente. Para un sistema de nueva generación, el diseño preferido utiliza dos mecanismos poscuánticos de establecimiento de claves con el fin de proporcionar también secreto hacia adelante, si el coste resulta aceptable. Incorporar una clave poscuántica a los microdescriptores también compensaría parte del ahorro de ancho de banda conseguido al eliminar claves TAP (Propuesta 350) e introducir familias basadas en claves (apartado 22.5). Las decisiones dependen de mediciones aún pendientes: ¿con qué frecuencia se construyen circuitos desde cero y cuánto ancho de banda y tiempo de CPU consumen los intercambios? Continúa siendo un problema de investigación abierto y una oportunidad para aportar mediciones (capítulo 29).

31.3 Mensajes más grandes y más pequeños

La Propuesta 340, *Open* en la fecha de referencia, aborda directamente la restricción de un mensaje por celda. Separaría los mensajes relay de las celdas: los mensajes largos podrían ocupar varias celdas y varios mensajes cortos podrían compartir una. La motivan tanto los mensajes superiores a 498 bytes, necesarios para intercambios poscuánticos, transporte UDP y Walking Onions, como los mensajes muy breves, por ejemplo `SENDME`, `XON` y `XOFF`. Actualmente consumen una celda entera y podrían ocultarse en el espacio de relleno de otros mensajes, volviéndose invisibles para un observador. El formato nuevo está pensado para funcionar con cifrado relay actualizado, cuyo diseño CGO tratamos en apartado 10.4; coordinar los cambios evitaría multiplicar formatos incompatibles.

31.4 Rendimiento: múltiples rutas

El control de congestión (apartado 11.3) hizo más eficientes los circuitos individuales. El siguiente paso es **Conflux** (Propuesta 329, *Finished*). Un cliente crea dos circuitos hacia la misma salida con guardias e intermedios diferentes, los vincula y distribuye el tráfico según el tiempo de ida y vuelta medido en cada circuito. La idea nació de una investigación de AlSabah, Bauer, Elahi y Goldberg; la versión de Tor la adapta utilizando información del control de congestión y la experiencia de TCP multirruta. Observamos circuitos Conflux en la red de laboratorio (apartado 19.7.3) y mediante Stem (capítulo 18). La propuesta recomienda que las dos ramas no compartan repetidores, salvo la salida, para evitar cuellos de botella compartidos. Es una recomendación, no una regla estricta, y a menudo se incumple en la pequeña red de laboratorio. tor utiliza Conflux desde la versión 0.4.8; Arti, como indicamos en apartado 30.3, todavía no lo hace en esta fecha de referencia. Los puntos de medición de OnionPerf con Conflux y sin él permiten comparar sus efectos en la red real (apartado 24.3).

31.5 Escalabilidad: Walking Onions

Actualmente todos los clientes Tor conocen *todos* los repetidores: descargan el consenso y los microdescriptores, de modo que el tráfico de directorio crece con el número de repetidores. Los microdescriptores y las diferencias de consenso reducen el coste, pero no modifican

su estructura fundamental. El coste inicial también se concentra durante el arranque: un cliente nuevo debe descargar un directorio completo antes de construir su primer circuito (Propuesta 300).

Las soluciones más evidentes tienen inconvenientes conocidos. Mostrar a cada cliente únicamente una parte de la red crea ataques *epistémicos*: las diferencias entre las vistas de distintos clientes pueden identificarlos. Permitir que cada repetidor elija el siguiente, como en algunos sistemas descentralizados, expone las rutas a la captura por repetidores maliciosos. Walking Onions propone otro enfoque: las autoridades publican una estructura firmada que asocia cada repetidor con un intervalo numérico. El cliente elige un número al azar y el repetidor anterior del circuito proporciona una prueba firmada que identifica al repetidor correspondiente a ese intervalo, junto con la extensión. El cliente verifica la firma sin necesitar la lista completa. La propuesta original es *Informational*; una continuación más detallada (Propuesta 323) intenta resolver las cuestiones restantes. Otras propuestas recientes, incluida la relativa a criptografía poscuántica, la tienen en cuenta.

31.6 Otros frentes abiertos

UDP Tor solo transporta TCP. Por tanto, las videollamadas, los juegos y QUIC, el transporte subyacente de HTTP/3, no están admitidos o requieren soluciones alternativas potencialmente inseguras. La Propuesta 348 (*Open*) examina alternativas que van desde túneles a través de salidas hasta transporte nativo de datagramas, junto con riesgos nuevos: huellas de aplicaciones, recopilación de metadatos entre pares y agotamiento de puertos.

Identificación de sitios Terminamos apartado 26.4 con un problema abierto: ninguna defensa general contra la identificación de sitios web ha logrado todavía un coste adicional aceptable. Las máquinas de estados de relleno, desplegadas por tor para circuitos de servicios onion y en desarrollo en Arti mediante *maybenot*, proporcionan un marco para futuras defensas.

Correlación El problema fundamental de capítulo 26 sigue sin resolverse: una red de baja latencia puede conservar patrones temporales y de volumen correlacionables, aunque el almacenamiento intermedio y otros flujos los modifiquen. Las defensas conocidas, como relleno a velocidad constante y retardos añadidos, son costosas; continúa investigándose cómo lograr compromisos más eficientes y seleccionar rutas teniendo en cuenta la topología de Internet.

DoS La prueba de trabajo protege los servicios onion (capítulo 15); el ajuste automático de este mecanismo es objeto de la Propuesta 362.

Arti como repetidor Una vez que Arti pueda funcionar como repetidor y autoridad, la base de código en C podría retirarse con el tiempo. Es uno de los proyectos a largo plazo del Tor Project (capítulo 30).

31.7 Cómo contribuir

Quienes hayan llegado hasta aquí tienen diversas maneras de contribuir, y no todas requieren escribir código:

- **Aportar ancho de banda**: administrar un repetidor intermedio o un puente o, con las precauciones de capítulo 25, una salida (capítulo 22); también es posible ejecutar un proxy Snowflake, incluso desde un navegador.

- **Leer y debatir propuestas**: muchas continúan *Open* porque necesitan comentarios, análisis y mediciones. La lista de discusión de desarrolladores es pública.
- **Escribir código**: tor y Arti se desarrollan en GitLab del Tor Project, con incidencias etiquetadas para personas recién incorporadas. Arti es especialmente importante para las funciones nuevas.
- **Investigar**: utilizar Shadow, Chutney y los datos de Metrics respetando las directrices del Research Safety Board (capítulo 29).
- **Traducir y documentar**: personas voluntarias traducen Tor Browser, su manual y el sitio web. Siempre resulta útil ampliar la documentación a más idiomas, incluido el italiano.
- **Utilizar Tor**: una comunidad más amplia puede aumentar el conjunto de anonimato, pero el número de usuarios no garantiza por sí solo la indistinguibilidad si difieren los tiempos, comportamientos u otros metadatos.

Resumen

- El protocolo evoluciona mediante propuestas numeradas, cuyo estado pasa de *Draft* hacia *Closed* a medida que los cambios se implementan e incorporan a las especificaciones.
- La criptografía poscuántica importa incluso antes de que existan ordenadores cuánticos suficientemente potentes; requiere mensajes mayores que una celda, lo que motiva la propuesta 340 sobre fragmentación y empaquetado.
- Conflux reparte tráfico entre dos circuitos; Walking Onions podría eliminar la necesidad de que cada cliente conozca la red completa.
- UDP, las defensas frente a identificación de sitios y correlación, el ajuste de DoS y la transición de Arti a funcionamiento como repetidor siguen siendo desafíos abiertos.

Ejercicios

Ejercicio 31.1 ⋆ Elige tres propuestas citadas en este libro. Para cada una, identifica su estado actual y explica qué significa: ¿la función ya está desplegada y figura en las especificaciones?

Ejercicio 31.2 ⋆ ¿Cuántas celdas se necesitan para transportar una clave pública ML-KEM-768 y un texto cifrado ML-KEM-768 si cada mensaje relay contiene como máximo 498 bytes y los mensajes separados no pueden compartir celda? ¿Qué cambiaría con la Propuesta 340? Distingue el límite inferior teórico de capacidad (sin sobrecarga de encapsulación) del número real de celdas, que dependería de las reglas de fragmentación de la propuesta.

Ejercicio 31.3 ⋆⋆ Ilustra un ataque epistémico: si distintos clientes conocieran subconjuntos diferentes de la red, ¿cómo podría aprovechar el adversario esas diferencias?

Ejercicio 31.4 ⋆⋆ Utilizando Stem en la red de laboratorio, identifica pares de circuitos Conflux que comparten un `CONFLUX_ID`. Comprueba que ambas ramas terminan en la misma salida y si evitan compartir otros repetidores, como recomienda la Propuesta 329. ¿Qué observas y cómo lo explicas?

Ejercicio 31.5 ⋆⋆⋆ Elige un problema abierto de este capítulo y redacta una propuesta breve con el formato del Tor Project: motivación, diseño, análisis de seguridad, compatibilidad y plan de implementación.

Lecturas complementarias

- El índice de propuestas del repositorio `torspec`, organizado por estado, y el documento de proceso de propuestas (Propuesta 001).
- N. Mathewson, *Options for postquantum circuit extension handshakes* (Propuesta 355).
- *Walking Onions: Scaling and Saving Bandwidth* (Propuesta 300) y su continuación (Propuesta 323).
- El blog del Tor Project, que anuncia y explica los cambios del protocolo.

Apéndice A

Glosario de términos de Tor

Este glosario define los términos técnicos utilizados a lo largo de la edición española. Cada entrada remite también al capítulo donde se introduce el concepto.

Término	Significado	Cap.
análisis de tráfico	Inferencia de información a partir de tiempos, volumen y direcciones de las comunicaciones sin leer su contenido.	1
anonimato	Condición de no ser identificable dentro de un conjunto de sujetos, denominado conjunto de anonimato.	1
ataque Sybil	Presentación de numerosas identidades, por ejemplo varios repetidores, para aumentar las probabilidades de selección.	26
autoridad de directorio	Uno de los pocos servidores que votan y firman el consenso.	12
puente (*bridge*)	Repetidor ausente del consenso público que permite acceder a la red a usuarios afectados por la censura.	16
canal	Conexión TLS entre dos nodos Tor adyacentes que transporta celdas de varios circuitos.	7
celda	Unidad de transmisión de un canal: normalmente 514 bytes, con longitudes variables para ciertos comandos.	8
clave cegada	Clave derivada de la identidad de un servicio onion para un período determinado que oculta su dirección a los HSDir.	14
clave de familia	Clave compartida por repetidores de un mismo operador que certifica su pertenencia a la familia.	22
identificador de circuito	Identificador local a un canal que cambia en cada salto.	8
circuito	Ruta cifrada por capas a través de varios repetidores, construida un salto cada vez.	9
consenso	Documento firmado por las autoridades que enumera repetidores, indicadores y pesos.	12
control de congestión	Ajuste de la cantidad de datos en tránsito en un circuito según el tiempo de ida y vuelta observado.	11
control de flujo	Mecanismo que evita que un emisor desborde al receptor mediante ventanas y mensajes SENDME.	11
correlación de tráfico	Determinación de que dos flujos observados en lugares diferentes pertenecen a la misma comunicación.	26

Término	Significado	Cap.
descriptor	Documento mediante el que un repetidor o servicio onion publica sus claves, direcciones y propiedades.	12
repetidor de salida	Último repetidor del circuito, encargado de abrir la conexión hacia el destino.	6
familia	Grupo de repetidores administrados conjuntamente que no deben aparecer en un mismo circuito.	13
indicador (*flag*)	Propiedad asignada por las autoridades, como `Guard`, `Exit` o `Stable`.	12
secreto hacia adelante	Propiedad por la que comprometer una clave de larga duración no permite descifrar el tráfico pasado.	5
guardia (*entry guard*)	Primer repetidor de los circuitos de un cliente, elegido de forma persistente y reutilizado durante un período prolongado.	13
intercambio de claves (*handshake*)	Intercambio de mensajes con el que dos partes acuerdan claves, por ejemplo ntor o ntor-v3.	9
directorio de servicios onion (HSDir)	Repetidor que almacena y sirve descriptores de servicios onion.	14
repetidor intermedio	Repetidor situado en el centro de un circuito que no conoce ni el cliente original ni el destino.	6
aislamiento de flujos	Asignación de flujos diferentes a circuitos distintos, por ejemplo según las credenciales SOCKS.	18
protocolo de enlace	Protocolo versionado utilizado en un canal, negociado mediante una celda **`VERSIONS`**.	7
mensaje relay	Contenido de una celda **`RELAY`** destinada a cierto salto, por ejemplo `BEGIN` o `DATA`.	8
microdescriptor	Descriptor compacto que contiene solo la información del repetidor que necesita el cliente.	12
modelo de amenazas	Descripción explícita de los adversarios y amenazas frente a los cuales está diseñado un sistema.	4
desvinculación	Imposibilidad de que un observador determine si dos acciones corresponden al mismo sujeto.	1
servicio onion (antes, servicio oculto)	Servicio accesible por Tor cuya dirección codifica su clave pública; anteriormente llamado servicio oculto.	14
relleno (*padding*)	Datos de relleno de celdas o canales destinados a ocultar longitudes y tiempos.	7
pesos de ancho de banda	Coeficientes del consenso que equilibran la selección de repetidores entre las posiciones del circuito.	13
transporte conectable (PT)	Programa separado que transforma el tráfico Tor para dificultar su reconocimiento o bloqueo, como obfs4, Snowflake o WebTunnel.	16
política de salida	Conjunto ordenado de reglas sobre los destinos que acepta un repetidor de salida.	22
prueba de trabajo (PoW)	Desafío computacional que un cliente resuelve para obtener prioridad al acceder a un servicio onion atacado.	15
propuesta	Documento numerado utilizado para debatir y diseñar cambios en el protocolo Tor.	31
seudónimo	Identificador diferente del nombre real de una persona que permite vincular acciones sin revelar la identidad.	1
punto de introducción	Repetidor elegido por un servicio onion para recibir solicitudes iniciales de contacto.	14
punto de encuentro	Repetidor elegido por el cliente donde se unen los circuitos del cliente y del servicio onion.	14
repetidor, enrutador onion	Nodo de la red Tor que reenvía celdas.	6

Término	Significado	Cap.
SENDME	Mensaje que confirma datos recibidos en un circuito y vuelve a abrir una ventana de control de flujo.	11
subprotocolo	Familia versionada de funciones del protocolo anunciadas por repetidores y clientes, como `Link` o `Relay`.	30
flujo (*stream*)	Flujo de datos de una aplicación, como una conexión TCP, transportado por un circuito.	11
vanguard	Repetidor de una capa intermedia mantenido fijo durante un período para ayudar a proteger el guardia de un servicio onion.	13
identificación de sitios web	Reconocimiento de la página visitada únicamente a partir de tamaños y tiempos del tráfico cifrado.	26

Apéndice B

Criptografía complementaria

Última verificación: 23 de septiembre de 2026 — dir-spec y tor-spec (commit de torspec `8e1f245`); biblioteca `cryptography`; red de laboratorio

Capítulo 5 presentó los componentes criptográficos de Tor con los conocimientos matemáticos mínimos. Este apéndice profundiza en tres temas que allí solo se mencionaron brevemente, con ejemplos ejecutables: las claves X25519 y los bits que se ajustan obligatoriamente, la conversión entre las formas Montgomery y Edwards de la misma curva, comprobada contra un certificado real, y la maleabilidad de los cifrados de flujo, que posibilita el ataque de marcado descrito en apartado 10.3.

B.1 Una curva, dos formas

X25519 y Ed25519 utilizan la *misma* curva elíptica sobre el cuerpo de enteros módulo $p = 2^{255} - 19$, expresada de dos maneras:

Forma de Montgomery $v^2 = u^3 + 486662\,u^2 + u$. X25519 utiliza esta forma: una clave pública contiene únicamente la coordenada u de un punto, codificada en 32 bytes little-endian. Esta representación permite un algoritmo de multiplicación escalar sencillo y rápido, la «escalera de Montgomery», cuyo tiempo de ejecución es independiente de la clave secreta.

Forma de Edwards retorcida $-x^2 + y^2 = 1 + d\,x^2y^2$, donde $d = -121665/121666$. Ed25519 utiliza esta forma para firmas: la clave pública es la coordenada y, y el signo de x se almacena en el bit más significativo del último byte.

Una transformación explícita relaciona ambas representaciones; la utilizaremos en B.2.

B.1.1 Ajuste de bits de claves X25519

Una clave privada X25519 comienza como 32 bytes, pero antes de utilizarla la implementación fuerza ciertos bits a valores establecidos: pone a cero los tres bits menos significativos, limpia el bit más alto y activa el siguiente más alto. Limpiar los tres bits inferiores hace que el escalar sea múltiplo de 8, el *cofactor* de la curva, para que los componentes de orden pequeño de un punto recibido no filtren información sobre el escalar. Fijar los bits superiores también fija el número de pasos de la escalera de Montgomery. Podemos comprobarlo con la misma biblioteca usada en los laboratorios:

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives import serialization as s
```

```
import os
def public_key(k):
    return X25519PrivateKey.from_private_bytes(k).public_key().public_bytes(
        s.Encoding.Raw, s.PublicFormat.Raw)
a = bytearray(os.urandom(32)); b = bytearray(a)
b[0] ^= 0x07                    # solo cambian los tres bits inferiores
print(public_key(bytes(a)) == public_key(bytes(b)))      # True
```

Dos secuencias de bytes de claves privadas que solo se diferencian en los bits sobrescritos durante el ajuste producen la misma clave pública.

B.1.2 El punto en el infinito

Durante un intercambio ntor, tanto el cliente como el repetidor comprueban que ninguna de sus operaciones EXP() produzca el punto del infinito. La especificación señala que, en Curve25519, esta comprobación sustituye adecuadamente la validación de que Y pertenece al grupo [tor-spec § The "ntor" handshake]. Sin ella, un adversario que proporcionara un punto público de orden pequeño podría producir un secreto compartido predecible, como una secuencia de ceros. La biblioteca `cryptography` efectúa por sí misma la comprobación: llamar a `exchange` con una clave pública de 32 bytes a cero provoca una excepción (`Error computing shared key`) en vez de devolver un secreto nulo.

B.2 De Montgomery a Edwards: el certificado cruzado ntor

Cada repetidor publica su clave pública ntor, una clave X25519, en el descriptor y demuestra que posee la correspondiente clave privada mediante `ntor-onion-key-crosscert`: certificado Ed25519 de tipo `0a` que certifica la identidad Ed25519 principal del repetidor, pero está *firmado con la clave ntor* [dir-spec § ntor-onion-key-crosscert]. Esto plantea una dificultad: ntor utiliza la forma Montgomery, mientras las firmas Ed25519 necesitan la forma Edwards. La especificación define la conversión [dir-spec § Converting a curve25519 public key to an ed25519 public key]:

$$y = \frac{u - 1}{u + 1} \bmod p.$$

La coordenada u determina y, pero no el signo de x: a cada coordenada de Montgomery corresponden dos claves públicas Edwards simétricas. No es posible recuperar ese signo solo de la clave pública Montgomery, así que el descriptor lo proporciona expresamente como primer argumento del campo del certificado cruzado: 0 para positivo y 1 para negativo. El repetidor lo calcula a partir de su clave privada.

El módulo `labs/es/codigo/torlab/conversion.py` realiza la conversión en unas líneas:

```
P = 2**255 - 19
def montgomery_to_edwards(u_bytes, sign):
    u = int.from_bytes(u_bytes, "little") & ((1 << 255) - 1)
    y = (u - 1) * pow(u + 1, P - 2, P) % P      # división mediante el inverso módulo
    p
    compressed = bytearray(y.to_bytes(32, "little"))
    compressed[31] |= (sign & 1) << 7          # signo de x en el bit más alto
    return bytes(compressed)
```

Después verifica el certificado cruzado mediante el decodificador de certificados del laboratorio 1. Aplicado a los descriptores de la red de laboratorio (clave abreviada para facilitar la lectura; las claves cambian con cada red nueva):

```
$ PYTHONPATH=labs/es/codigo python3 -m torlab.conversion labs/es/laboratorio/net/nodes
    /000a/cached-descriptors test004r
test004r: ntor-onion-key DaxPE+Grnjex2Hmv+5qutl5dIX4La2jF6MIL7IK6+3E
  Clave Edwards (signo 0): c369d08f8083002a15fe22f65dca14bf...
  crosscert válido; identidad certificada: kjPIgDB2OFsGJzuwsT+KMS/El9qrlE2/4P5WvIvmzC8
```

En el ejemplo de referencia, la firma es válida y la identidad certificada coincide con la del archivo `fingerprint-ed25519` del repetidor. Lo mismo sucede con los demás repetidores del laboratorio. Si proporcionamos el signo incorrecto, la verificación falla: es una de las pruebas negativas automatizadas del módulo. La división módulo p se calcula mediante el pequeño teorema de Fermat, multiplicando por $(u+1)^{p-2}$.

B.3 Maleabilidad del cifrado de flujo

AES en modo contador (apartado 5.2) genera una secuencia de bytes que se combina con el mensaje mediante XOR. Quien cambie un bit del texto cifrado altera *el bit correspondiente del texto en claro*, sin conocer la clave; solo necesita conocer o adivinar el contenido original. Este ejemplo en Python lo demuestra:

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os
k, n = os.urandom(16), bytes(16)
cipher = lambda d: Cipher(algorithms.AES(k), modes.CTR(n)).encryptor().update(d)
c = bytearray(cipher(b"BEGIN www.example.org:443"))
delta = bytes(x ^ y for x, y in zip(b"www.example.org", b"www.example.net"))
for i, d in enumerate(delta):
    c[6 + i] ^= d                # modificación dirigida, sin conocer la clave
print(cipher(bytes(c)))           # b'BEGIN www.example.net:443'
```

El cifrado relay original de Tor, tor1, utiliza AES-CTR con una comprobación de integridad solo en el extremo del circuito mediante un resumen de cuatro bytes. Un repetidor malicioso puede, por tanto, alterar de manera predecible una celda y hacer que un cómplice situado más adelante en la ruta reconozca el cambio. Es el ataque de marcado explicado en apartado 10.3 y una de las motivaciones de CGO (apartado 10.4), cuyo diseño hace irrecuperables la celda alterada y las siguientes.

Ejercicios

Ejercicio B.1 $\star$ ¿Cuántos valores diferentes puede adoptar un escalar privado X25519 después del ajuste de bits? Compara el resultado con 2^{256}.

Ejercicio B.2 $\star\star$ Deduce e implementa la transformación inversa desde la coordenada y de una clave pública Ed25519 hasta la coordenada u de Montgomery. Conserva el bit de signo de x de la clave original y utiliza `montgomery_to_edwards` de `conversion.py` para comprobar que ambas conversiones recuperan la clave pública Ed25519 inicial (para $y \neq 1$).

Ejercicio B.3 $\star\star\star$ En tor1, una celda tiene un resumen de cuatro bytes y un campo `recognized` de dos bytes a cero. ¿Qué probabilidad existe de que el último salto acepte como válida una celda modificada aleatoriamente? ¿Por qué puede funcionar, aun así, el marcado?

Apéndice C

Guía de las especificaciones torspec

Última verificación: 23 de septiembre de 2026 — commit de torspec `8e1f245`

Las especificaciones del Tor Project son la fuente primaria de casi todos los capítulos de este libro. Son exhaustivas, pero no se escribieron como un manual didáctico. Durante más de veinte años se mantuvieron como archivos de texto independientes y solo hace relativamente poco se reunieron en un mismo sitio web. Sus propias convenciones reconocen que la coherencia sigue siendo un trabajo en curso. Este apéndice ofrece un mapa para quienes deseen estudiar las especificaciones después de leer el libro.

C.1 Dónde encontrarlas

Las especificaciones se publican en `https://spec.torproject.org/` y se mantienen junto con las propuestas en el repositorio GitLab `torspec` del Tor Project. Allí son archivos Markdown dentro de `spec/`: archivos individuales para temas pequeños y directorios para documentos extensos. `permalinks.md` enumera direcciones estables, como `https://spec.torproject.org/tor-spec`; utilízalas para citarlas. A lo largo del libro citamos los documentos como [document § section], conservando los títulos de las secciones tal como aparecen en el sitio, y las propuestas como «Propuesta NNN».

C.2 Convenciones

El documento introductorio de convenciones establece reglas aplicables salvo indicación contraria:

- `MUST`, `SHOULD`, `MAY` y sus formas negativas conservan el significado de RFC 2119: requisito, recomendación firme y opción permitida, respectivamente.
- Las longitudes se miden en bytes de ocho bits y los prefijos binarios, como «kibibyte», indican múltiplos de 1024.
- Los enteros de varios bytes utilizan orden big-endian, o de «red»: el decimal 4660, en dos bytes, se representa como `12 34`.

Ya encontramos dos excepciones importantes: las claves y los puntos de Curve25519 y Ed25519 se codifican en little-endian (apéndice B), mientras algunos documentos de directorio usan texto legible y datos binarios codificados en base64.

C.3 Mapa de documentos

Documento	Contenido	Capítulos
`tor-spec`	Protocolo principal: canales, celdas, circuitos, protocolos de establecimiento de conexiones y claves (handshakes), cifrado de celdas relay, flujos, control de flujo y de congestión	7–11
`cert-spec`	Certificados Ed25519 de Tor	7
`dir-spec`	Autoridades, votos, consenso, descriptores, microdescriptores y documentos extra-info	12, 24
`path-spec`	Selección de rutas y restricciones	13
`guard-spec`	Algoritmo de guardias	13
`vanguards-spec`	Vanguards completos y ligeros	13
`rend-spec`	Servicios onion v3: claves, descriptores, introducción y encuentro	14, 21
`hspow-spec`	Prueba de trabajo de los servicios onion	15
`dos-spec`	Defensas frente a denegación de servicio	15
`pt-spec`	Interfaz entre tor y los transportes conectables	16
`padding-spec`	Relleno de canales y circuitos	7, 26
`control-spec`	Protocolo de ControlPort	18
`socks-extensions`	Extensiones SOCKS específicas de Tor	11, 28
`param-spec`	Parámetros de consenso y sus valores predeterminados	11, 15
`address-spec`	Nombres especiales, como direcciones `.onion`	14
`srv-spec`	Valor aleatorio compartido de las autoridades	14

C.4 Cómo leer una sección

Algunas recomendaciones extraídas de la redacción del libro:

1. **Comienza por el formato.** Las secciones sobre mensajes binarios ofrecen nombres y tamaños de campos en tablas. Reconstruye el formato sobre papel, como hicimos con los diagramas del libro, antes de leer las reglas.

2. **Distingue lo vigente de lo obsoleto.** Muchos documentos conservan funciones históricas como TAP, las versiones 1–3 del protocolo de enlace y los servicios onion v2. El material obsoleto suele agruparse en apartados o archivos como `obsolete-channels.md` o `historical/`, pero no siempre. Ante la duda, revisa el código para comprobar si sigue existiendo la rama correspondiente.

3. **Compara con las implementaciones.** Las especificaciones expresan la intención; los códigos fuente de tor y Arti revelan el comportamiento efectivo. Durante la investigación para este libro, varias discrepancias aportaron información útil: la ausencia de relleno de canal en circuitos de un salto (capítulo 7) y la razón de cierre observada en una salida (apartado 22.4).

4. **Comprueba el estado de las propuestas.** Una propuesta puede estar implementada antes de incorporarse a la especificación, o encontrarse todavía en discusión sin haberse implementado (apartado 31.1).

5. **Verifica experimentalmente.** Una red privada como la del libro permite comprobar si un campo se comporta como está documentado. Este método fundamenta todos los laboratorios.

C.5 Arti y las especificaciones

El código de Arti suele remitir a las especificaciones en comentarios y documentación e identifica requisitos ambiguos o diferencias frente a tor. Su tabla de subprotocolos admitidos (apartado 30.3) indica qué funciones soportan ambas implementaciones y, por tanto, qué partes de las especificaciones describen el protocolo «vivo».

Apéndice D

Soluciones de los ejercicios

Capítulo 1: anonimato, vigilancia y metadatos

Solución 1.1. Con HTTPS, el proveedor de Internet ve la dirección IP del sitio y posiblemente su dominio, por la resolución DNS y, a menudo, por el campo SNI de TLS. También observa el momento de la conexión, su duración y el volumen de tráfico. Con una VPN, el proveedor solo ve la conexión con el servidor VPN; sin embargo, el operador de la VPN puede observar lo mismo que antes veía el proveedor de Internet. Con Tor, el proveedor observa una conexión con un repetidor o puente Tor, sus tiempos y volumen. Ningún punto individual de la red Tor puede ver simultáneamente al usuario y el destino.

Solución 1.2. Una cuenta creada con un nombre ficticio es seudonimato. Un mensaje sin firma en un tablón constituye anonimato, siempre que ningún otro dato identifique a su autor. Una carta firmada con iniciales representa un seudonimato débil y puede permitir identificar a su autor. Una donación anónima en efectivo es un ejemplo de anonimato.

Solución 1.3. El conjunto de anonimato se reduce de cincuenta usuarios de Tor a quienes hablan italiano y además están despiertos a las tres de la madrugada, quizá solo unas pocas personas. Tor oculta una dirección IP, no el comportamiento. El idioma, los horarios, el estilo de escritura y los datos personales pueden identificar a alguien aunque el tráfico de red no lo haga.

Solución 1.4. Un proveedor VPN ve tanto al usuario como sus destinos. Toda la confianza se concentra en un operador que podría registrar la actividad, verse obligado legalmente a entregar los datos o resultar comprometido. Tor distribuye la confianza entre tres repetidores administrados independientemente, ninguno de los cuales ve ambos extremos. Vincular usuario y destino exigiría comprometer u observar simultáneamente el primer y el último salto.

Capítulo 2: los orígenes

Solución 2.1. La probabilidad al azar es 1 entre 100. Si el observador enviara 99 de los 100 mensajes de un lote, reconocería sus propios mensajes salientes e identificaría el único restante como el de la víctima. Desaparecería el anonimato. Es el *ataque* $n - 1$, al que las redes mix reales se enfrentan mediante lotes más sofisticados, tráfico ficticio y controles sobre mensajes entrantes.

Solución 2.2. Utiliza una *dirección de respuesta anónima*, en el sentido de Chaum. El usuario prepara una «cebolla de retorno» cifrada por capas para una cadena de mezcladores. La capa

interior contiene su dirección y una clave para cifrar la respuesta. El destinatario añade su respuesta a la cebolla; cada mezclador elimina una capa y ninguno, individualmente, conoce el destino final. No se necesita una tabla central de rutas.

Solución 2.3. En cada paso, la solicitud se reenvía a otro miembro del grupo con probabilidad $3/4$ y se envía al sitio con probabilidad $1/4$. El número de miembros atravesados sigue una distribución geométrica con media $1/(1/4) = 4$, contando el primer paso de reenvío. Un miembro no puede saber si quien le remitió la solicitud la originó, porque el reenvío intermedio sigue la misma regla.

Capítulo 3: del encaminamiento cebolla del NRL al Tor Project

Solución 3.1. Con tres saltos, un nodo central malicioso conocía inmediatamente las dos redes organizativas de los extremos porque las entradas y salidas estaban situadas dentro de ellas. Con cuatro saltos, quien comprometiera un nodo cercano a uno de los extremos sabría qué nodo adicional atacar para reconstruir el circuito; con cinco, necesitaría un compromiso más. Tor utiliza tres saltos porque sus repetidores no residen en las redes de los usuarios y porque cada salto adicional incrementa la latencia sin reducir demasiado la principal amenaza de correlación entre entrada y salida. Los guardias ayudan específicamente frente a esa amenaza.

Solución 3.2. Una cebolla única cifrada con las claves públicas permanentes de los nodos no proporciona secreto hacia adelante. Quien consiga después esas claves podrá descifrar las cebollas grabadas. Con construcción telescópica, cada salto realiza un intercambio efímero de claves y los secretos temporales se descartan tras utilizarlos. El cliente también construye el circuito salto a salto y puede identificar inmediatamente a un repetidor que no responde y sustituirlo. Con una sola cebolla, el fallo solo se descubre después de intentar la ruta completa.

Solución 3.3. Es un ejercicio de argumentación abierta. Entre los elementos verificables figuran el código y las especificaciones publicados, las compilaciones reproducibles, los análisis independientes de la literatura científica y una red administrada por operadores independientes. Entre los elementos que todavía requieren confianza están la correspondencia entre los binarios distribuidos y las fuentes publicadas, aunque puede verificarse parcialmente, la posible colusión de las autoridades de directorio y la existencia de vulnerabilidades desconocidas. La seguridad debe evaluarse mediante verificabilidad y confianza distribuida, no suponerse a partir de las fuentes de financiación del proyecto.

Capítulo 4: el modelo de amenazas

Solución 4.1. (a) Sí: el operador de red solo ve una conexión con un repetidor Tor. (b) Sí: el sitio ve la dirección de la salida. (c) No en cuanto a la identidad: el propio usuario se identifica al iniciar sesión en la cuenta. Tor solo oculta desde dónde se conecta. (d) Probablemente no: un adversario que observe ambos extremos, usuario y periódico, podría intentar una correlación temporal, precisamente el escenario excluido del modelo de amenazas de Tor.

Solución 4.2. Un sistema difícil de usar atrae únicamente a unos pocos usuarios muy motivados, a menudo quienes más necesitan protección. El mero hecho de utilizarlo los distingue y deja un conjunto de anonimato pequeño. Un sistema fácil de usar que millones de personas adoptan por motivos ordinarios ayuda a ocultar a quienes tienen necesidades más delicadas.

Solución 4.3. Es un ejercicio abierto. Entre los adversarios posibles figuran el gobierno o empleador de la fuente que observa la red local, el proveedor de correo o mensajería y entidades que controlan redes en uno u otro país. Tor impide que la red local sepa con quién se comunica el periodista y que el servicio conozca su dirección IP. No protege el contenido de los mensajes sin cifrado de extremo a extremo, los metadatos visibles al servicio, como cuentas y marcas temporales, los dispositivos comprometidos ni la correlación realizada por un adversario que observe ambos extremos. Un servicio onion como SecureDrop reduce varios de estos riesgos.

Capítulo 5: criptografía esencial

Solución 5.1. $C_1 \oplus C_2 = P_1 \oplus P_2$, porque la secuencia de cifrado reutilizada se cancela. Si se conoce P_1, puede recuperarse $P_2 = C_1 \oplus C_2 \oplus P_1$.

Solución 5.2. La salida de 16 bytes coincide con los primeros 16 bytes de la salida de 32: SHAKE genera una sola secuencia que puede leerse hasta la longitud deseada. Una KDF puede, por ello, solicitar una secuencia larga y dividirla en partes. Sin embargo, pedir longitudes distintas para usos diferentes *no* proporciona separación de dominios; siguen siendo necesarias etiquetas distintas.

Solución 5.3. Ambos secretos compartidos coinciden. Una clave pública de ceros representa un punto de orden pequeño y produciría un secreto compartido nulo con independencia de la clave privada: un secreto conocido por cualquiera. La biblioteca `cryptography` se niega a devolverlo. ntor exige explícitamente esta comprobación (apartado 9.6); de lo contrario, un adversario podría forzar un secreto predecible.

Solución 5.4. Sin separación de dominios, un valor calculado en un contexto podría aceptarse en otro. Por ejemplo, una MAC generada para autenticar un mensaje de intercambio de claves podría presentarse como clave o prueba en otra fase o protocolo que utilice la misma función y claves. Las etiquetas de dominio hacen que los conjuntos de entradas de los diferentes usos sean distintos por construcción.

Capítulo 6: arquitectura de red

Solución 6.1. Guardia: dirección IP del usuario, horarios y volumen del tráfico, pero no destino. Intermedio: ni usuario ni destino. Salida: destino y contenido no cifrado por la aplicación, pero no usuario. HSDir: ninguna información sobre un usuario que visite un sitio ordinario. Autoridad de directorio: normalmente ninguna información sobre una visita concreta. Puede ver la dirección IP de un cliente que solicite documentos de directorio *directamente*, especialmente durante el arranque. Si los obtiene de un directorio de reserva o una caché, ese servidor y no la autoridad observa la solicitud.

Solución 6.2. `Link=3-5` anuncia protocolos de enlace hasta la versión 5 (capítulo 7). `Relay=2-6` incluye ntor, ntor-v3 (`Relay=4`) y CGO (`Relay=6`). `FlowCtrl=1-2` indica SENDME autenticado y control de congestión. El repetidor puede servir como salida CGO si también tiene indicador `Exit` y negocia el control de congestión junto con CGO (apartado 9.7).

Solución 6.3. 100 KB de datos de aplicación requieren aproximadamente $102400/498 \approx 206$ celdas DATA, o unos $206 \times 514 \approx 106$ KB sobre el enlace. Hay que añadir algunas celdas para la solicitud, la apertura del flujo y los mensajes SENDME (alrededor de siete, uno por cada 31 celdas de datos), además de la sobrecarga de los registros TLS. En total, es aproximadamente un 5–7% por encima de la carga útil, sin contar los intercambios iniciales.

Capítulo 7: canales y protocolo de enlace

Solución 7.1. Un cliente siempre inicia sus conexiones: con su guardia y con los repetidores que ofrecen información de directorio. Nadie necesita contactar directamente con él y escuchar en un puerto facilitaría identificarlo. Un repetidor inicia canales principalmente cuando recibe una solicitud `EXTEND2` y debe alcanzar el siguiente salto del circuito, o cuando, como cualquier nodo, crea circuitos propios para descargar documentos de directorio o comprobar su accesibilidad.

Solución 7.2. `0x000772F7` $= 488183$ horas, correspondientes al 9 de septiembre de 2025 a las 23:00 UTC. Un contador sin signo de 32 bits en segundos se desbordaría en 2106. Contar horas, en cambio, ofrece cientos de miles de años y basta para certificados cuya duración se mide en días o semanas. Una nota de la especificación exige precaución: `cert-spec` afirma que cuatro bytes bastan hasta el año 10136. Es un error identificado explícitamente en Propuesta 338, causado por interpretar el contador como si midiera *minutos*, ya que 2^{32} minutos equivalen aproximadamente a 8166 años.

Solución 7.3. $38 = 32 + 2 + 2 \times 2$ describe un desafío con dos métodos. Si N_Methods vale 2, el cuerpo debe contener $32 + 2 + 4 = 38$ bytes. Un cuerpo de 36 bytes declara más métodos de los que incluye y está mal formado: debe rechazarse. La especificación permite ignorar bytes *adicionales*, no bytes ausentes.

Solución 7.4. Falla la comparación entre la clave certificada por el certificado de tipo 5 y la huella SHA-256 del certificado TLS del adversario: es diferente del certificado del repetidor auténtico. Si el adversario presentara exactamente el certificado TLS legítimo, no podría completar el intercambio TLS sin la correspondiente clave privada.

Solución 7.5. `695661c0` (1 de enero de 2026, 12:00:00 UTC); `04 04 c6336407` (dirección del cliente, 198.51.100.7); `01`; y `04 04 cb007105` (dirección del repetidor, 203.0.113.5), seguidos de relleno de ceros hasta alcanzar 509 bytes.

Solución 7.6. En primer lugar, el guardia vería la misma identidad cuando el usuario cambiara entre las redes doméstica, laboral y móvil, relacionando sesiones y direcciones IP de todos esos entornos. En segundo lugar, la identidad también se facilitaría a los repetidores de los que el cliente obtuviera información de directorio. Repetidores que colaboraran, o un adversario que los observara, podrían vincular la actividad del usuario en distintos nodos. En ambos casos, un identificador persistente destruiría la desvinculación pretendida por Tor.

Solución 7.7. Esquema de solución: analiza el certificado de tipo 2 con una biblioteca X.509, como `cryptography.x509`. Extrae su clave pública RSA, verifica la autofirma y comprueba que su exponente sea 65537. Después analiza el certificado cruzado de tipo 7 conforme a su formato. Asegúrate de que la clave Ed25519 certificada coincide con la identidad ya verificada y comprueba la firma RSA mediante PKCS#1 v1.5 sin identificador OID del algoritmo hash, aplicada al SHA-256 del prefijo y los campos prescritos. La huella del repetidor es el resumen SHA-1 de la representación PKCS#1 de la clave pública RSA codificada en DER.

Solución 7.8. No. El tráfico del cliente `test008c` atraviesa *su propio* canal con el guardia, no nuestro canal. Este último no se ha utilizado para ningún circuito, por lo que `channelpadding_decide_to_pad_channel` decide no añadir relleno. Para observarlo debemos transportar un circuito de varios saltos por nuestro propio canal, como en capítulo 10.

Solución 7.9. TLSSECRETS se deriva de los secretos de la sesión TLS entre A y el intermediario, y la firma de A lo cubre. B recalcula TLSSECRETS a partir de los secretos de *su propia* sesión con el intermediario. Los secretos son distintos, así que la verificación falla aunque el intermediario reenvíe fielmente todas las celdas y SLOG y CLOG coincidan. Sin TLSSECRETS

y SCERT, el intermediario podría presentarse a B como A y reenviar la firma de A, haciendo creer a B que estableció un canal autenticado con A.

Capítulo 8: celdas

Solución 8.1. **`CREATE2`** es una celda de longitud fija: 512 bytes con protocolo de enlace 3 y CircID de dos bytes, o 514 bytes con versión 5 y CircID de cuatro bytes. **`VERSIONS`** utiliza siempre un CircID de dos bytes, por lo que ocupa $2 + 1 + 2 + 4 = 9$ bytes.

Solución 8.2. 0 (**`PADDING`**) es de longitud fija; 7 (**`VERSIONS`**) es variable; 9 (**`RELAY_EARLY`**) es fija; 12 (**`PADDING_NEGOTIATE`**) es fija; y 129 (**`CERTS`**) es variable. El comando 200 tendría longitud variable según la regla ≥ 128, pero no está definido y, por tanto, no está permitido. El receptor puede descartarlo y cerrar el canal.

Solución 8.3. Solo `0x92345678`. El CircID cero queda reservado a celdas ajenas a circuitos. Con protocolo de enlace ≥ 4, la parte que inicia el canal, aquí el cliente, debe elegir CircID con el bit más significativo igual a 1; en `0x12345678` ese bit vale 0.

Solución 8.4. Primera celda: CircID `00000000`, comando `0x81`=129 (**`CERTS`**), por lo que es variable; longitud `0003`, cuerpo `aa bb cc`. Segunda celda: CircID `0x80000007`, comando `0x04` (**`DESTROY`**), de longitud fija, con un cuerpo de 509 bytes. El primer byte del cuerpo, `00`, significa razón NONE. El cuerpo `aabbcc` no constituye por sí mismo un objeto CERTS válido: el ejercicio se refiere únicamente al encuadre de celdas.

Solución 8.5. `02 0000 0005 00000000 012c`: comando DATA (2), Recognized 0, StreamID 5, Digest provisionalmente cero y longitud $300 =$ `0x012c`. Los $509 - 11 - 300 = 198$ bytes restantes son relleno: los cuatro primeros son ceros y los otros 194 son aleatorios.

Solución 8.6. Para una celda todavía cifrada con otra capa, Recognized vale cero con probabilidad 2^{-16}: de un millón de celdas, aproximadamente $10^6/65536 \approx 15$ cumplirían esa condición. Recognized no basta porque hay falsos positivos. Solo la comparación con el resumen acumulativo, que un repetidor no destinatario no puede hacer coincidir salvo por azar (otro factor de 2^{-32}), confirma que la celda está destinada a ese salto.

Solución 8.7. Supongamos que un adversario controla el guardia y la salida. Cuando la salida desea marcar un circuito, lo cierra con una razón poco frecuente, como `NOSUCHSERVICE`. Si el intermedio y el guardia reenviaran sin cambios esa razón hacia el cliente, el guardia podría observar el valor inusual en uno de sus circuitos al mismo tiempo y vincular al cliente conocido por el guardia con el destino conocido por la salida. Si todos los repetidores transmiten y propagan únicamente NONE, el byte de razón no puede transportar esa señal. Sigue siendo posible correlacionar el momento del cierre, pero ese canal es mucho más ruidoso.

Solución 8.8. Formato tradicional: $\lceil 1048576/498 \rceil = 2106$ celdas, que ocupan $2106 \times 514 = 1\,082\,484$ bytes en el canal, un 3,2% de sobrecarga. CGO: $\lceil 1048576/488 \rceil = 2149$ celdas, que ocupan $1\,104\,586$ bytes, un 5,3% de sobrecarga. No se cuentan SENDME ni los mensajes de apertura de flujo.

Solución 8.9. Esquema: utiliza Hypothesis para generar listas de tuplas con CircID, comando y cuerpo. Elige comandos de `CellCommand` y limita a 509 bytes los cuerpos de celdas de longitud fija. Concatena los resultados de `encode`, divídelos en fragmentos de longitud aleatoria e introdúcelos progresivamente en un búfer. Llama repetidamente a `decode` para extraer celdas hasta que no quede ninguna completa. Compara las celdas fijas con cuerpos rellenados con ceros hasta 509 bytes.

Solución 8.10. En Arti 2.6.0, el códec de canal es `ChannelCodec`, situado en `crates/tor-cell/src/chancell/codec.rs`, inicializado con la versión del protocolo mediante `ChannelCodec::new`. Sus métodos `write_cell` y `decode_cell` serializan y extraen celdas. Como nuestro `decode`, `decode_cell` acepta celdas incompletas sin generar un error, devuelve `Ok(None)` hasta disponer de bytes suficientes y consume datos del búfer solo para celdas completas. Arti, en cambio, opera sobre un búfer mutable `BytesMut` y es genérico respecto al tipo de mensaje de canal (`M: ChanMsg`), lo que permite restringir los mensajes aceptados. Otra diferencia es que `decode_cell` lee el comando en el desplazamiento 4 y presupone celdas fijas de 514 bytes, con CircID de cuatro bytes. Arti solo admite protocolos de enlace 4 y 5, según `LINK_PROTOCOLS` en `crates/tor-proto/src/channel/handshake.rs`; la celda inicial **`VERSIONS`**, con CircID de dos bytes, se procesa separadamente durante el intercambio inicial.

Capítulo 9: construcción de circuitos

Solución 9.1. ntor requiere $20 + 32 + 32 = 84$ bytes. ntor-v3 con un mensaje de tres bytes requiere $32+32+32+3+32 = 131$ bytes. HDATA puede contener como máximo $509-4 = 505$ bytes, así que CM puede ocupar como máximo $505 - 128 = 377$ bytes.

Solución 9.2. El secreto de **`CREATE_FAST`** es $X\|Y$, transmitido en claro dentro del canal. Para un segundo salto, X e Y atravesarían el circuito por el primer repetidor. Este conoce las claves de su propia capa, podría leer esos valores, derivar las claves del segundo salto y descifrar todo lo intercambiado entre el cliente y aquel, destruyendo la separación entre saltos.

Solución 9.3. AES-CTR cifra aplicando XOR entre texto claro y una secuencia que depende únicamente de clave y contador. Con la misma clave en ambos sentidos, la celda número n del cliente y la celda número n del repetidor utilizarían la misma secuencia. Un repetidor intermedio que observara ambos textos cifrados podría aplicarles XOR para obtener el XOR de sus textos claros, lo que a menudo permitiría recuperar ambos mensajes.

Solución 9.4. El repetidor `test004r` utiliza ORPort 5104 = `0x13f0`: `03` indica tres especificadores de enlace; el especificador de dirección IPv4 y puerto es `00 06 7f000001 13f0`. El mensaje completo contiene $1 + (2 + 6) + (2 + 20) + (2 + 32) = 65$ bytes de especificadores, más $2 + 2 = 4$ bytes para HTYPE y HLEN y $32 \cdot 3 + 1 + 32 = 129$ bytes de HDATA. El total es 198 bytes, muy por debajo de los 498 bytes disponibles en un mensaje relay.

Solución 9.5. Sin $\mathrm{EXP}(B, x)$, el repetidor no está autenticado. Un intermedio que reenvíe el intercambio podría generar sus propios Y y $AUTH$ con su y, suplantar al siguiente salto y leer el tráfico posterior destinado a él: un ataque de intermediario (*man-in-the-middle*). Sin $\mathrm{EXP}(Y, x)$ se pierde el secreto hacia adelante. Cualquiera que consiga posteriormente la clave onion permanente b podría recalcular $\mathrm{EXP}(X, b)$ a partir de intercambios grabados y derivar todas las claves de sesión.

Solución 9.6. `VER` vincula el intercambio a su contexto, `"circuit extend"` para Tor. Al entrar en todas las derivaciones, un intercambio generado en un contexto no produce claves ni valores $AUTH$ válidos en otro. Sin `VER`, un adversario podría reutilizar un mensaje de intercambio de otro protocolo contra un repetidor Tor, o viceversa, si ambos aceptaran los mismos cálculos. Son ataques *entre protocolos*.

Solución 9.7. En la red de laboratorio, `test003r`, que admite `Relay=2-6`, rechaza una solicitud compuesta solo por `[02 06]` mediante una celda **`DESTROY`** con razón `PROTOCOL`. Acepta una solicitud combinada que incluya `CC_FIELD_REQUEST` y responde con `CC_FIELD_RESPONSE`. El código de tor activa CGO en un salto únicamente cuando está habilitado también el control de congestión: `enable_cgo = supports_cgo && use_congestion_control`, en `src/core/or/`

`extendinfo.c`. Propuesta 359 explica por qué el nuevo formato de mensajes relay, en el que solo ciertos comandos tienen StreamID, es incompatible con el mecanismo anterior de control de flujo SENDME por flujo. Exige el control de flujo de Propuesta 324, es decir `FlowCtrl=2`.

Solución 9.8. Es un ejercicio práctico. `ntor_ref.py` contiene las funciones de cliente y servidor. Utilizar claves fijas en lugar de generar nuevas aleatoriamente produce valores reproducibles que pueden compararse con `ntor.py`.

Solución 9.9. Esquema: `server_part1` recibe HDATA, verifica `ID` y B, calcula $X_b = \text{EXP}(X, b)$, deriva las claves de la primera fase, verifica la MAC y descifra CM. Después `server_part2` genera (y, Y), calcula *key_seed* y *verify*, deriva la clave de SM y calcula $AUTH$. Si se fija y al valor del vector de prueba de la propuesta, la respuesta debe coincidir con `server_handshake`.

Capítulo 10: la cebolla

Solución 10.1. El cliente actualiza el resumen acumulativo hacia delante del segundo salto, cifra con Kf_2 y después con Kf_1. El tercer salto no ve nada: el segundo reconoce la celda y no la reenvía.

Solución 10.2. Cada celda consume 509 bytes de secuencia de cifrado. Después de 1000 celdas, la posición es $509 \times 1000 = 509\,000$ bytes, después del bloque AES $31\,812$, cuyos bloques son de 16 bytes. Reutilizar la clave en un circuito nuevo cuyo contador volviera a cero reutilizaría la secuencia; aplicar XOR a dos textos cifrados con ella revelaría el XOR de sus textos claros. Cada circuito deriva, por tanto, claves nuevas mediante un intercambio con valores efímeros.

Solución 10.3. El resumen acumulativo del repetidor incluiría una celda que el cliente no había contado para ese salto. Desde ese momento, sus estados divergirían, ninguna celda posterior sería reconocida y el circuito quedaría inutilizable. El problema ocurre con probabilidad 2^{-16} para cada celda en tránsito no destinada al repetidor: una de cada $65\,536$ de media, equivalente a unos 33 MB de tráfico.

Solución 10.4. El adversario debería alterar bytes cuyo texto claro esperado se conoce o puede predecirse, para que un cómplice en la salida detecte una modificación concreta: por ejemplo, el comando relay, `02` para `DATA`, StreamID o los cuatro bytes a cero al principio del relleno. Antes de comprobar el resumen, la salida cómplice examina si esos bytes coinciden con la modificación esperada. Una salida honesta solo ve una celda con resumen inválido y destruye el circuito.

Solución 10.5. Con CGO, después de quitar la capa de cifrado de la salida, una celda alterada produce una etiqueta y un contenido seudoaleatorios. No hay una marca reconocible: la modificación se propaga impredeciblemente por toda la celda. Como la etiqueta interviene en el procesamiento de la siguiente celda, las posteriores también quedan imposibles de descifrar y el circuito debe descartarse. Por tanto, el adversario solo puede destruir circuitos, algo que ya podía hacer.

Solución 10.6. Los intervalos observados son 4,1; 3,0; 5,7; 8,7 y 6,8 segundos, con una media de 5,7. Para el máximo de dos variables uniformes independientes en $[a, b]$, el valor esperado es $a + \frac{2}{3}(b - a) = 1{,}5 + \frac{2}{3} \cdot 8 \approx 6{,}8$ segundos. Cinco muestras son insuficientes para que sorprenda la diferencia observada. Además, el temporizador se reinicia cuando cruza el canal *cualquier* celda, no solo las de relleno.

Solución 10.7. En la red de laboratorio, el repetidor intermedio no responde a `DROP`; reconoce el mensaje y lo descarta silenciosamente. La evidencia indirecta es que el circuito sigue

funcionando: después de dos `DROP`, el cliente todavía puede extenderlo hasta la salida. Si el intermedio no hubiera reconocido las celdas, como último salto del circuito en ese momento lo habría destruido. Si cliente e intermedio hubieran contado celdas de forma diferente, sus resúmenes acumulativos se habrían desincronizado y la extensión posterior habría fallado.

Solución 10.8. En la prueba de red privada, cambiar un bit de una celda `EXTEND2` destinada al guardia, entonces único salto del circuito, provoca el fallo de la comprobación del resumen. Como extremo de ese circuito, el guardia responde con una celda **`DESTROY`** de razón 1 (`PROTOCOL`). En un circuito más largo, una modificación realizada por el cliente en una celda destinada al tercer salto atravesaría los dos primeros y solo sería detectada por el tercero.

Capítulo 11: flujos, SENDME y control de congestión

Solución 11.1. El nombre de host debe escribirse en minúsculas: `example.com:443` seguido de un byte cero, es decir `6578616d706c652e636f6d3a343433 00`, y a continuación FLAGS con el bit 1 activado para aceptar IPv6: `00000001`.

Solución 11.2. La dirección IPv4 `c0000201` representa 192.0.2.1; el TTL `0000012c` equivale a 300 segundos. El cliente puede ignorar la dirección. Si la conserva, no debe reutilizarla entre circuitos: la salida podría haber mentido o devuelto una dirección única para reconocer al cliente.

Solución 11.3. Sin datos optimistas, el cliente espera `CONNECTED` antes de enviar la solicitud, lo que cuesta una ida y vuelta adicional de aproximadamente 600 ms en una operación que necesita al menos dos RTT. Si falla la conexión, los datos en cola deben descartarse porque no existe un destino que los reciba. No deben entregarse mediante otra conexión.

Solución 11.4. Un flujo puede tener como máximo 500 celdas en tránsito, aproximadamente $500 \times 498 \approx 249$ KB por RTT. Con un RTT de 400 ms son unos 620 KB/s. Incluso con muchos flujos, el circuito queda limitado a 1000 celdas por RTT, aproximadamente 1,2 MB/s. Con control de congestión, la ventana empieza en solo 124 celdas (unos 62 KB), pero crece hasta que las mediciones RTT indican congestión, sin límite superior fijo.

Solución 11.5. Un cliente malicioso abre muchos flujos hacia un servidor que envía numerosos datos, quizá uno propio, lee muy poco y envía repetidamente mensajes `SENDME` falsificados. La salida sigue transmitiendo y las celdas se acumulan en las colas de guardia, intermedio y cliente, agotando la memoria de los repetidores. `SENDME` autenticado exige demostrar que el cliente recibió la celda que provoca el mensaje. Si el contenido fuera predecible, por ejemplo un archivo conocido, el cliente podría calcular el resumen sin recibirlo; los bytes de relleno aleatorios impiden ese atajo.

Solución 11.6. Esquema: envía `RESOLVE` con un StreamID nuevo y el nombre terminado en cero; interpreta después `RESOLVED` como una secuencia de tuplas con tipo, longitud, valor y TTL. Para un nombre inexistente, la salida envía una entrada de error (`0xF0` o `0xF1`), cuyo contenido debe ignorarse.

Solución 11.7. La salida responde con `END`, razón 4 (`EXITPOLICY`), seguida de la dirección de destino, cuatro bytes para IPv4, y un TTL de cuatro bytes. El cliente puede utilizarlo para no volver a intentar el mismo destino mediante esa salida.

Solución 11.8. Medimos el tiempo que tarda un `SENDME` en llegar a la salida, más el tiempo que necesita la salida desbloqueada para enviar celdas adicionales y que estas lleguen al cliente. Cuando la salida espera `SENDME`, esto aproxima el RTT del circuito observado por el cliente. Una segunda descarga simultánea alarga las colas y aumenta los valores medidos: precisamente la señal utilizada por el control de congestión para reducir su ventana.

Capítulo 12: sistema de directorios

Solución 12.1. El intervalo de votación es una hora, desde `valid-after` hasta `fresh-until`. El cliente elige un instante aleatorio entre $11{:}00 + \frac{3}{4} \cdot 60' = 11{:}45$ y $11{:}45 + \frac{7}{8} \cdot 75' \approx 12{:}50$. Si a las 13:00 aún no tiene un consenso nuevo, el anterior ha caducado formalmente. No obstante, el cliente continúa utilizándolo mientras se considere *razonablemente vigente*, es decir, durante menos de 24 horas después de caducar, conforme a `REASONABLY_LIVE_TIME` en `networkstatus.c`. Puede seguir construyendo circuitos mientras intenta repetidamente obtener un consenso reciente. Aproximadamente 24 horas después de la caducidad deja de confiar en el antiguo.

Solución 12.2. Probablemente `Running` y `Valid`, además de `Fast`, porque 500 KB/s supera los 100 KB/s. Quizá `V2Dir`, si el repetidor funciona también como caché de directorio. No `Stable`, `Guard` ni `HSDir`, que requieren días o semanas de funcionamiento. Tampoco `Exit`, porque su política solo permite el puerto 443, mientras que ese indicador exige tráfico de salida tanto por el 80 como por el 443.

Solución 12.3. La caché entrega a un cliente concreto un consenso de dos horas de antigüedad, todavía válido, que no incluye repetidores añadidos recientemente. Así selecciona rutas de un conjunto ligeramente distinto del utilizado por los demás, lo que podría facilitar reconocer su tráfico. El ataque queda limitado por las tres horas de validez del consenso, por la necesidad de una firma de autoridad legítima (la caché no puede inventar consensos) y por la selección aleatoria de cachés que hacen los clientes. Solicitar microdescriptores mediante su resumen y no mediante la identidad del repetidor impide que la caché entregue a ese cliente una versión diferente de la clave, porque el resumen queda fijado por el consenso firmado.

Solución 12.4. Con tres mediciones, la mediana es 5200. Una medición inflada no la altera. Si la tercera autoridad fuera la única que midiera ese repetidor, no habría tres mediciones suficientes para utilizar su mediana. El consenso utilizaría entonces el ancho de banda anunciado o votado, con `Unmeasured=1` y un límite superior, de modo que la medición exagerada no influiría en el resultado.

Solución 12.5. El voto contiene líneas ausentes o diferentes en el consenso. Entre ellas figuran evaluaciones de ancho de banda `Measured` de las autoridades que lo miden, líneas `m` que identifican métodos de consenso admitidos, indicadores asignados por cada autoridad, sus `flag-thresholds` y compromisos con la aleatoriedad compartida. El consenso conserva únicamente los resultados combinados.

Solución 12.6. Calcula el hash del texto que comienza en la línea `onion-key` del microdescriptor y termina en el salto de línea anterior al siguiente microdescriptor, o al final del documento, incluidas las líneas de la clave RSA codificada en PEM. El resumen SHA-256 en base64, sin caracteres = finales, debe coincidir con la línea `m` del consenso.

Solución 12.7. Con una autoridad detenida, las otras dos siguen votando y producen un consenso con dos firmas, suficientes para la mayoría $\lfloor 3/2 \rfloor + 1 = 2$. Con dos detenidas, la única superviviente no alcanza la mayoría y no se produce otro consenso. Aun así, el cliente puede seguir utilizando el último como *razonablemente vigente* durante unas 24 horas después de su caducidad. En la red de laboratorio un consenso solo dura 40 segundos, pero ese margen de 24 horas sigue vigente, así que la interrupción no resulta visible inmediatamente.

Solución 12.8. Un consenso `ns` normalmente lleva firmas SHA-1; la especificación solo permite ese algoritmo para esa variante. Su línea `r` contiene además el resumen del descriptor completo de cada repetidor y cada entrada tiene también una línea `p` que resume su política de salida.

Capítulo 13: selección de rutas

Solución 13.1. La salida tiene las condiciones más estrictas: debe aceptar el destino solicitado. Elegirla primero garantiza que existe una salida adecuada. Elegir primero el guardia podría eliminar la única salida válida por restricciones de familia o red y obligar a repetir la selección. También existe una razón estadística: las exclusiones afectan a las posiciones siguientes y, por tanto, deberían recaer sobre las que ofrecen el conjunto más amplio de candidatos.

Solución 13.2. Como guardia: $20000 \times 0/10000 = 0$ (`Wgd=0`). Como intermedio: $20000 \times 0 = 0$ (`Wmd=0`). Como salida: $20000 \times 10000/10000 = 20000$ (`Wed=10000`). Con los pesos de este consenso, el repetidor se utiliza exclusivamente como salida.

Solución 13.3. Sin guardias persistentes, la probabilidad de compromiso supera el 50% cuando $1 - (1 - c^2)^C > 0.5$, es decir, $C > \ln 0.5 / \ln(1 - c^2)$: aproximadamente 6932 circuitos con $c = 1\%$ y 17 con $c = 20\%$. Con un guardia fijo, la probabilidad permanece cerca de c, es decir, 1% y 20%, respectivamente. Los guardias son insuficientes frente a un adversario muy poderoso, pero la diferencia sigue siendo considerable.

Solución 13.4. El adversario debe aportar ancho de banda real: las probabilidades de selección dependen del que miden las autoridades, no únicamente de valores anunciados. Las salidas también se vigilan estrechamente, entre otros mecanismos mediante `BadExit`. El límite de dos repetidores por dirección obliga a conseguir numerosas direcciones. Declarar una familia impide que sus nodos aparezcan dos veces en un mismo circuito; no declararla aumenta el riesgo de que las autoridades los descubran y excluyan. Finalmente, los pesos reservan la capacidad de salida escasa para salidas auténticas, donde cuenta el ancho de banda del adversario.

Solución 13.5. Sin límite de muestra, un adversario que controle la red local del cliente podría bloquear las conexiones con todos los guardias honestos. El cliente probaría otros guardias indefinidamente hasta encontrar uno accesible, acabando por elegir uno adversario. Con una muestra limitada, agota sus intentos y deja de añadir guardias, con el posible coste de perder conectividad en lugar de seleccionar automáticamente uno controlado por el atacante.

Solución 13.6. Es un ejercicio práctico. Descarga los microdescriptores de CollecTor junto con los consensos correspondientes. Para cada repetidor, construye el conjunto de identificadores de familia mediante `family-ids` o listas `family` recíprocas, y rechaza un candidato que comparta familia con un repetidor ya seleccionado.

Solución 13.7. Es un ejercicio de investigación. Las familias grandes suelen representar una cuota de probabilidad de selección de salida mucho mayor que cualquier repetidor individual. El riesgo efectivo de concentración depende, por tanto, de los operadores, no solo del número de repetidores. Esto explica la preocupación de la comunidad por la diversidad de operadores y las familias no declaradas (capítulo 27).

Capítulo 14: servicios onion v3

Solución 14.1. $32 + 2 + 1 = 35$ bytes, es decir, 280 bits. Base32 codifica cinco bits por carácter, por lo que se requieren $280/5 = 56$ caracteres sin relleno. Un error de escritura aleatorio tiene una probabilidad aproximada de 2^{-16} de coincidir por casualidad con la suma de comprobación. El byte de versión también debe seguir siendo `0x03`, lo que reduce aún más la probabilidad de un error aleatorio no detectado.

Solución 14.2. A las 09:00 UTC del 1 de enero de 2026 han transcurrido $29\,454\,300$ minutos desde la época Unix. Si restamos el desplazamiento de 720 minutos quedan $29\,453\,580$, así que

$\lfloor 29453580/1440 \rfloor = 20453$. El período $20\,453$ comenzó a las 12:00 UTC del 31 de diciembre de 2025, porque las 09:00 todavía son anteriores al mediodía.

Solución 14.3. El adversario necesita la clave cegada del período y, por tanto, conocer la dirección del servicio; además, el SRV del período y las posiciones de sus propios repetidores en el anillo. Utilizaría esos valores para generar identidades de repetidor situadas inmediatamente después de los índices del servicio. El SRV solo se conoce cuando las autoridades lo publican, aproximadamente medio día antes de que comience el período correspondiente. El adversario no puede preparar con varios días de anticipación repetidores posicionados con precisión, y los nuevos nodos aún necesitan obtener el indicador `HSDir`, que exige al menos 96 horas de funcionamiento.

Solución 14.4. Un HSDir ve una clave cegada, no la dirección del servicio, y recibe solicitudes mediante circuitos anónimos. Un punto de introducción ve una clave de autenticación y un circuito procedente del servicio y puede contar solicitudes de introducción. Un punto de encuentro ve una cookie, dos circuitos y tráfico cifrado que circula entre ambos. El punto de introducción resulta especialmente útil para estimar popularidad porque observa cada intento de conexión, aunque no conozca la dirección correspondiente. Sin embargo, quien ya conozca la dirección podría descargar el descriptor y establecer esa correspondencia.

Solución 14.5. En un circuito de un salto, el HSDir es el primer repetidor y conoce la IP del cliente. Podría relacionar *quién* está buscando *qué* servicio, al menos por la clave cegada y por la dirección si ya la conoce. Rechazar solicitudes no anónimas obliga a los clientes a obtener descriptores mediante otros repetidores.

Solución 14.6. Es un ejercicio práctico. Con más de 2×4 HSDir, el servicio deja de cargar su descriptor en todos. Solicitarlo a cada HSDir mediante un circuito de tres saltos debería encontrarlo exactamente en los nodos calculados para el período actual o el siguiente, ya que el servicio publica descriptores correspondientes a ambos.

Solución 14.7. El valor base64 contiene un contador de especificadores de enlace, seguido de los especificadores en formato `EXTEND2` (apartado 9.2.2). Por ejemplo, el tipo `00` proporciona dirección y ORPort, el tipo `02` una huella RSA y el tipo `03` una identidad Ed25519. Identifica el repetidor por su ORPort, igual a 5100 más el número de nodo. La línea `auth-key` contiene un certificado de tipo 9, firmado por la clave de firma del descriptor, que certifica la clave de autenticación del servicio en ese punto de introducción.

Capítulo 15: defensas frente a DoS

Solución 15.1. Durante el primer minuto, el depósito lleno ofrece 60 tokens y la reposición añade $2 \times 60 = 120$, lo que permite aproximadamente 180 circuitos, quizá algunos menos porque el cliente queda marcado en cuanto el depósito llega a cero. En una hora, a velocidad sostenida, el total es aproximadamente $60 + 2 \times 3600 = 7260$. Un cliente normal solo crea unas pocas decenas de circuitos.

Solución 15.2. Solo el primer repetidor conoce la IP real del cliente. En los saltos posteriores, la «dirección» aparente del canal pertenece al repetidor anterior. Un intermedio que limitara los circuitos por dirección limitaría en realidad todos los circuitos procedentes de un guardia determinado, que puede transportar tráfico de miles de usuarios. Perjudicaría a todos en lugar de restringir al atacante concreto. Por eso los límites por dirección se aplican únicamente a conexiones de cliente no autenticadas.

Solución 15.3. El cliente envía un mensaje `INTRODUCE1` mediante un circuito que puede reutilizar. El servicio debe descifrar la solicitud, efectuar un intercambio hs-ntor y establecer

un circuito nuevo de al menos tres saltos hacia el punto de encuentro. Esto implica varios intercambios ntor con repetidores y las celdas necesarias para crear y extender cada salto. Una sola celda del cliente puede, por tanto, provocar varias operaciones de clave pública y decenas de celdas de red: amplificación de uno o dos órdenes de magnitud.

Solución 15.4. Exigir k bits iniciales a cero solo permite niveles de esfuerzo que sean potencias de dos: 1, 2, 4, etcétera. Cada bit adicional duplica el coste computacional esperado. La comprobación lineal, en cambio, permite cualquier esfuerzo entero E. Un controlador de realimentación que ajuste el esfuerzo con incrementos pequeños necesita esa escala más fina. Con saltos de duplicación, tendería a oscilar entre un esfuerzo demasiado bajo y otro demasiado alto.

Solución 15.5. No. El esfuerzo E forma parte del desafío utilizado para calcular tanto la solución Equi-X como el hash R. Cambiar la afirmación a $E = 5000$ modifica el desafío e invalida la solución encontrada para $E = 50$. El cliente tendría que haber elegido $E = 5000$ antes de iniciar la búsqueda.

Solución 15.6. Es un ejercicio de laboratorio. Las celdas se acumulan en la cola de circuitos del repetidor intermedio hasta que su consumo total de memoria supera `MaxMemInQueues`. Entonces el administrador de memoria cierra los circuitos que llevan más tiempo con datos en cola, incluido el nuestro, y registra en el log del repetidor la memoria liberada.

Solución 15.7. Es un ejercicio de programación. Las solicitudes que ofrecen mayor esfuerzo deberían tener tiempos de espera breves y estables. Las de esfuerzo cero deberían atenderse principalmente cuando la cola está casi vacía, o descartarse cuando se reduce. Es el comportamiento previsto: proteger a usuarios dispuestos a asumir un coste computacional moderado.

Capítulo 16: censura, puentes y transportes conectables

Solución 16.1. Un puente desempeña la misma función que un guardia, primer salto del circuito, pero no figura en el consenso público. El cliente conoce su dirección mediante una línea de puente obtenida por canales privados. Con `UseBridges`, utiliza puentes *en lugar de* guardias públicos. Añadir un guardia público exigiría conectarse directamente con una dirección publicada, precisamente lo que un censor bloquea o vigila.

Solución 16.2. «No parecerse a nada reconocible»: obfs4, cuyo bloqueo exigiría impedir tráfico que no puede reconocerse como protocolo permitido. «Parecerse a algo permitido»: meek, que obligaría a bloquear un gran proveedor de contenidos; Snowflake, que requeriría interferir con WebRTC, su intermediario de señalización y numerosos proxies voluntarios; WebTunnel, que exigiría bloquear HTTPS hacia sitios desconocidos o los sitios que alojan puentes; y Conjure, cuyo bloqueo requeriría impedir el acceso a grandes rangos de direcciones de proveedores de conectividad colaboradores.

Solución 16.3. Un censor podría examinar conexiones duraderas con direcciones sin nombres de host asociados, bytes de entropía inusualmente alta desde el primer paquete, distribuciones reconocibles de tamaños y tiempos, como ráfagas asociadas a celdas Tor, y proporciones de subida y bajada semejantes a navegación web. `iat-mode` introduce retardos aleatorios y fragmentación para dificultar el reconocimiento de patrones temporales y de tamaño, a costa de rendimiento.

Solución 16.4. Un censor que bloquea todos los puentes recibidos no consigue reputación: los puentes bloqueados impiden que sus credenciales alcancen niveles superiores o permisos para invitar. Podría, en cambio, mantener puentes sin bloquear mientras acumula suficientes

credenciales de nivel alto y después bloquearlos simultáneamente. Eso exige mantener muchas identidades pacientes durante meses, mientras los puentes conocidos siguen ayudando a usuarios legítimos. Cada credencial de nivel alto solo permite acceder a un número limitado de puentes.

Solución 16.5. Ambas capturas deberían presentar entropía próxima a ocho bits por byte: después del intercambio inicial, TLS también transporta datos cifrados. La diferencia reside en unos pocos bytes reconocibles, como las cabeceras de registros TLS (`16 03 01`, `17 03 03`, seguidas de la longitud) y el intercambio inicial sin cifrar. La inspección profunda de paquetes puede reconocerlos sin necesidad de pruebas estadísticas de entropía.

Solución 16.6. El laboratorio 1 agota el tiempo de espera (`TimeoutError`) porque el puente no responde. Utilizar un parámetro `cert` incorrecto hace que `obfs4proxy`, del lado del cliente, no complete el intercambio. El puente no reconoce la marca de autenticación del cliente, así que no responde y cierra la conexión después de un retardo. Según el nivel de detalle de los registros, puede aparecer un mensaje de intercambio fallido o ninguno relevante.

Capítulo 17: instalación y torrc

Solución 17.1. tor informa de la opción desconocida mediante un mensaje como `Unknown option '...'. Failing.` y termina sin iniciarse. Ejecutar `--dump-config short` con el `torrc` del cliente muestra solo los valores diferentes de los predeterminados, incluidas opciones propias del laboratorio como `TestingTorNetwork`, las entradas `DirAuthority` y los puertos de escucha.

Solución 17.2. Si el correo y el foro comparten circuito, la salida observa conexiones con el proveedor de correo del usuario, que quizá lo identifique, y con el foro seudónimo a través del mismo circuito y en el mismo intervalo. Podría vincular el seudónimo con la persona. Las soluciones incluyen dos puertos SOCKS separados, uno por aplicación, aislados por defecto salvo que reciban el mismo `SessionGroup`; credenciales SOCKS distintas con `IsolateSOCKSAuth`; o `IsolateDestAddr` en el puerto compartido.

Solución 17.3. Un adversario que vigile redes italianas podría observar tanto al usuario que entra en Tor como a una salida italiana que se conecta con el destino italiano. Ambos extremos estarían en la misma jurisdicción o región de red observable, facilitando la correlación excluida del modelo de amenazas de Tor. Además, las salidas italianas son relativamente escasas y restringir la selección hace al usuario más distinguible y facilita atacarlo con repetidores maliciosos.

Capítulo 18: puerto de control y Stem

Solución 18.1. `552` es un error permanente en respuesta a un comando: la clave `pippo` no existe. En cambio, un mensaje `650` no responde a un comando, sino que es un evento asíncrono enviado porque el cliente se suscribió mediante `SETEVENTS`. Puede llegar en cualquier momento, incluso mientras se espera la respuesta a otro comando.

Solución 18.2. Esquema: llama a `create_ephemeral_hidden_service` del controlador, asociando el puerto virtual 80 al local 8080 e indicando `await_publication`. El método devuelve la dirección onion. Accede mediante `curl --socks5-hostname 127.0.0.1:9008 http://<address>.onion/`. tor conserva en memoria las claves de un servicio efímero y las entrega al controlador, que puede guardarlas para recrear más adelante la misma

dirección. Por defecto, el servicio desaparece al cerrarse la conexión de control, salvo que esté desacoplado.

Solución 18.3. Esquema: registra una función para `EventType.BW`. Recibe cada segundo la cantidad de bytes leídos y escritos como `event.read` y `event.written`. Muestra esos valores mientras descargas un archivo grande. La tasa de entrada aumenta hacia el límite impuesto por el control de congestión y los recursos disponibles.

Solución 18.4. Con `__LeaveStreamsUnattached`, tor no asigna automáticamente flujos a circuitos. `EXTENDCIRCUIT 0 test004r,test005r,test006e` construye un circuito y devuelve su identificador. Cuando llega un evento `STREAM` con estado `NEW`, llama a `ATTACHSTREAM`. La selección manual de rutas es peligrosa porque elude las políticas de guardias y la selección ponderada: una ruta inusual puede distinguir al usuario o un programa malicioso podría dirigir deliberadamente su tráfico por repetidores del adversario.

Capítulo 19: una red Tor privada

Solución 19.1. `test006e` es el nodo número 6, así que su ORPort es 5106 y su ControlPort es 8006. `test009h` no tiene ORPort: un servicio onion no es un repetidor y no acepta conexiones iniciadas por otros nodos de la red. Construye circuitos salientes hacia sus puntos de introducción y encuentro.

Solución 19.2. La opción es `TestingDirAuthVoteGuard *` en los archivos torrc de las autoridades. En la red pública, el indicador Guard exige demostrar estabilidad y ancho de banda a lo largo del tiempo (capítulo 12). En el laboratorio, cualquier repetidor puede actuar como guardia. Por tanto, las estadísticas de selección de primeros saltos no representan las de la red pública.

Solución 19.3. Con `--socks5`, `curl` resuelve `localhost` por su cuenta y entrega una dirección IP al cliente Tor. Con `--socks5-hostname`, transmite el nombre para que lo resuelva la salida. En la red pública, con un dominio de Internet ordinario, la resolución local revelaría los sitios visitados al resolvedor DNS del cliente y a quien lo observe, aunque la conexión posterior utilizara Tor. Es una filtración DNS clásica.

Solución 19.4. Cada veinte segundos, según `V3AuthVotingInterval`. Por tanto, un día contiene $86400/20 = 4320$ consensos de laboratorio, frente a los 24 de la red pública, donde las autoridades votan cada hora.

Solución 19.5. Con una autoridad detenida, la red puede seguir produciendo consensos porque basta una mayoría de las tres, o dos firmas. Con dos detenidas, la restante no puede generar por sí sola un consenso nuevo válido. Sin embargo, los clientes pueden continuar utilizando el anterior como *razonablemente vigente* durante unas 24 horas después de su caducidad (capítulo 12), de modo que aún pueden construir circuitos durante algún tiempo. El consenso de laboratorio solo es válido durante 40 segundos, pero el margen de vigencia razonable sigue siendo de 24 horas.

Solución 19.6. Esquema: `c.get_network_status(fingerprint)` devuelve la entrada correspondiente del consenso con sus atributos `address` y `or_port`. Llámalo para cada huella presente en `circ.path`, tomando el primer elemento de cada tupla.

Solución 19.7. Utiliza la misma estructura de definición de red del libro y sustituye los números de nodos por `getN(5)`, `getN(10)` y `getN(4)`. El tiempo de arranque depende del equipo. Conforme crece la red, buena parte del tiempo adicional se dedica a esperar que los repetidores aparezcan en el consenso y que los clientes descarguen sus microdescriptores.

Solución 19.8. Es un ejercicio exploratorio. Busca mensajes de registro sobre selección de rutas, envío de **`CREATE2`** y `EXTEND2`, recepción de sus respuestas y finalización del circuito. El nivel *info* no muestra cada celda. Activa registros *debug* para ámbitos concretos en el torrc del cliente para observar más detalles.

Solución 19.9. No. Una vez terminada la negociación TLS, las celdas Tor viajan cifradas y `tcpdump` solo puede observar registros TLS. Siguen siendo visibles direcciones, puertos, tamaños y tiempos de registros. Son precisamente las clases de información aprovechadas por ataques de análisis de tráfico (capítulo 26).

Capítulo 21: publicar un servicio onion

Solución 21.1. Un prefijo de seis caracteres requiere aproximadamente $32^6 \approx 1{,}07 \times 10^9$ intentos, unos 107 segundos a diez millones por segundo. Ocho caracteres requieren $32^8 \approx 1{,}1 \times 10^{12}$ intentos, unos $1{,}1 \times 10^5$ segundos, algo más de treinta horas. Cada carácter adicional multiplica por 32 el esfuerzo esperado.

Solución 21.2. La página de error revela el programa y versión del servidor web, información potencialmente útil para identificar vulnerabilidades, y especialmente una dirección IP, 192.0.2.10, que podría corresponder a la dirección pública real del servidor. Así podría localizarse el servicio. Desactiva las firmas del servidor en cabeceras y páginas de error, vincula el servidor únicamente a localhost o a un socket Unix e inspecciona sus respuestas con un cliente de laboratorio.

Solución 21.3. Elimina `alice.auth` y recarga la configuración. El servicio publica un descriptor nuevo sin la entrada de Alice, cifrado mediante una *cookie de descriptor* nueva que ella no puede descifrar. Los clientes que conserven el descriptor anterior pueden seguir utilizando los puntos de introducción conocidos mientras sigan siendo válidos. Para una revocación más completa, rota los puntos de introducción, por ejemplo reiniciando el servicio. Atención al caso límite: si Alice era la única cliente autorizada, queda un directorio vacío y los HSDir rechazan el descriptor (apartado 21.3). Para desactivar por completo el descubrimiento restringido, elimina `authorized_clients/` y recarga la configuración.

Solución 21.4. Configura `HiddenServicePort 80 unix:/path/socket` y haz que el servidor web escuche en ese socket Unix. Además de evitar un puerto TCP abierto, un socket del sistema de archivos puede protegerse con permisos, no resulta accesible desde una interfaz de red e impide exponer accidentalmente el servicio en una interfaz pública.

Capítulo 20: observar el tráfico

Solución 20.1. TLS no alinea registros con celdas Tor. tor escribe celdas en un búfer y la biblioteca TLS las agrupa en registros de hasta 16 KB, tamaño relacionado con el valor `sendme_inc` de 31 celdas (apartado 11.3). TLS añade sus propias cabeceras y etiquetas de autenticación. A su vez, TCP puede dividir o combinar registros TLS en segmentos diferentes. La longitud fija se aplica a celdas Tor, no a paquetes de red.

Solución 20.2. Esquema: después de **`CREATED2`**, descifra las celdas **`RELAY_EARLY`** y **`RELAY`** del circuito quitando sus capas en el orden correcto, con las claves de los saltos almacenadas en `circuit.py`. El estado de la secuencia de AES-CTR debe avanzar exactamente en el orden en que se utilizó. Durante la construcción del circuito pueden verse `EXTEND2` y `EXTENDED2`, seguidos eventualmente por `BEGIN`, `CONNECTED`, `DATA` y `SENDME`.

Solución 20.3. En un canal de cliente que transporta circuitos de varios saltos pueden aparecer registros TLS cada pocos segundos aun sin tráfico de aplicación. Contienen celdas **PADDING** de canal, enviadas a intervalos aleatorios entre 1,5 y 9,5 segundos (apartado 10.5). Por el contrario, un canal que nunca haya transportado circuitos de varios saltos no muestra ese tráfico.

Solución 20.4. Es un ejercicio de programación. Un analizador Lua registrado como *disector heurístico* para datos TLS descifrados puede leer los dos primeros bytes del CircID de la celda **VERSIONS** inicial, o cuatro bytes posteriormente. Después lee el comando y, para celdas variables, el campo de longitud. Mantén estado separado para cada conexión y dirección, como en `traffic.py`.

Capítulo 22: administrar un repetidor o puente

Solución 22.1. Sí. La primera regla coincidente, `accept *:80`, acepta la conexión y las siguientes ya no se consideran. Invierte las dos primeras: `reject 192.0.2.0/24:*`, `accept *:80`, `reject *:*`.

Solución 22.2. El resumen enumera únicamente puertos, no direcciones, y por fuerza es aproximado. Incluye un puerto cuando la salida lo admite para «la mayoría» de direcciones, un umbral técnico preciso definido en dir-spec y no simplemente más del 50%: la salida no debe bloquear más de 2^{25} direcciones IPv4, una red /7, o 2^{112} direcciones IPv6, una red /16, excluyendo direcciones no públicas. Cuando el cliente recibe `END` con razón `EXITPOLICY`, registra que esa salida rechaza la dirección y puerto indicados durante el TTL correspondiente y vuelve a intentar el flujo por otro circuito con otra salida.

Solución 22.3. Con `MyFamily` solo cuentan las declaraciones recíprocas. Los cinco repetidores antiguos siguen formando una familia entre ellos, pero el sexto no forma parte porque ninguno de los anteriores lo enumera. Por tanto, los clientes pueden seleccionarlo junto con un miembro de la familia original. Con una clave de familia basta instalar esa misma clave en el repetidor nuevo: su certificado firmado lo vincula con la familia sin modificar los otros nodos.

Solución 22.4. Los registros muestran sucesivamente «in-progress connect failed …(Connection refused)» y «Didn't recognize errno 115». Si la razón de cierre del flujo utilizara el error devuelto por `getsockopt(SO_ERROR)`, el cliente recibiría `CONNECTREFUSED`. Del mismo modo podrían comunicarse correctamente `TIMEOUT` para `ETIMEDOUT`, `NOROUTE` para `ENETUNREACH` o `EHOSTUNREACH`, y `CONNRESET`, en lugar de `MISC`, para conexiones que fallan después de `EINPROGRESS`. Para el cliente la diferencia práctica es moderada: cambia el mensaje de error, pero el procesamiento del fallo del flujo sigue siendo el mismo.

Solución 22.5. A 10 MB/s, el repetidor transfiere unos 864 GB diarios, contando el máximo entre tráfico enviado y recibido. Una cuota mensual de 1 TB se agotaría en poco más de un día. Un repetidor desconectado durante casi todo el mes pierde los indicadores `Stable` y `Guard` y rompe circuitos al apagarse. Uno más lento pero conectado todo el mes resulta mucho más útil: configura `RelayBandwidthRate` para distribuir la cuota a lo largo de todo el período.

Capítulo 23: Tor Browser, Tails y Whonix

Solución 23.1. Nuevo circuito para este sitio cambia únicamente el circuito utilizado por el sitio de la pestaña activa, sin borrar cookies, historial ni estado de otros sitios; el sitio podría seguir reconociendo al usuario por esos datos almacenados. Nueva identidad cierra

todo, elimina el estado del navegador y obtiene circuitos nuevos para toda la actividad. Ninguna acción cambia el guardia, porque rotarlo con frecuencia aumenta la probabilidad de seleccionar finalmente uno malicioso (apartado 13.3). En cualquier caso, el guardia no puede ver qué sitios visita el cliente.

Solución 23.2. El usuario quizá bloquee la carga de rastreadores adicionales, pero Tor Browser ya los aísla por sitio y limita los identificadores persistentes. Instalar una extensión sacrifica la uniformidad: los sitios podrían detectar su presencia o sus listas de filtrado y distinguir al usuario del conjunto más amplio de personas que utilizan Tor Browser. Además, introduce código adicional con acceso a las páginas visitadas.

Solución 23.3. Con credenciales SOCKS idénticas para dos pestañas que visitan el mismo sitio, tor asigna sus flujos al mismo circuito. Una tercera pestaña con credenciales distintas recibe otro circuito. `GETINFO stream-status` permite ver el identificador de circuito asociado a cada flujo.

Solución 23.4. (a) Tor Browser basta, posiblemente con un puente. (b) Tails: se inicia desde una memoria USB, normalmente no deja rastros en el equipo de otra persona y conserva únicamente datos colocados en el almacenamiento persistente cifrado, con los riesgos propios de hardware comprometido. (c) Whonix: las aplicaciones de la Workstation acceden a la red mediante tor por diseño arquitectónico y permite trabajos prolongados sin exigir amnesia.

Solución 23.5. Es un ejercicio abierto. Entre las propiedades posibles figuran dimensiones de pantalla y ventana, zona horaria, idioma, fuentes disponibles y renderizado de canvas. En un navegador normal pueden aportar desde unos pocos bits, como la zona horaria, hasta muchos, como las características de fuentes y canvas. Tor Browser redondea dimensiones mediante letterboxing, declara UTC y un idioma común, limita fuentes y normaliza o desactiva interfaces reveladoras. Quedan diferencias como el sistema operativo, el nivel de seguridad elegido y las dimensiones redondeadas.

Capítulo 24: medir la red

Solución 24.1. Las solicitudes de directorio se reducirían a la mitad y, con el divisor fijo de diez, la estimación de usuarios también se reduciría a la mitad. Los gráficos mostrarían una caída repentina y aproximadamente simultánea entre países conforme se extendiera la versión nueva de tor. Una caída gradual global correlacionada con la adopción del programa sugeriría un cambio de comportamiento del cliente, no una pérdida equivalente de usuarios. Habría que revisar el divisor del método de estimación.

Solución 24.2. $20\,000\,000/(0{,}6 \cdot 10) \approx 3\,333\,333$ usuarios estimados. Con una fracción de cobertura del 8%, el valor se descarta porque queda por debajo del mínimo del 10%.

Solución 24.3. Esquema: filtra repetidores que tengan ambos indicadores y divide la suma de sus anchos de banda entre el total de la red. Para calcular la probabilidad de selección de salida, utiliza `probabilities(consensus, "exit")` del laboratorio 7 y suma las probabilidades de repetidores cuya red /16 figure entre las cinco más representadas.

Solución 24.4. Es un ejercicio abierto. Un bloqueo documentado suele producir una caída brusca de las estimaciones de usuarios directos el día en que comienza. Las estimaciones de usuarios de puentes, especialmente Snowflake o WebTunnel, pueden aumentar durante los días siguientes conforme las personas afectadas cambian de transporte. Metrics puede señalar la fecha de la caída inicial.

Solución 24.5. Sin redondeo, un adversario que sepa que es el único usuario de un país que llega a cierto repetidor, o conozca exactamente la cantidad de los demás usuarios, podría

consultar sus estadísticas extra-info y confirmar si otra persona de ese país se conectó, mediante una diferencia de uno. El redondeo oculta variaciones pequeñas. Un adversario que controle muchos repetidores no necesita los recuentos publicados porque puede observar directamente las conexiones entrantes de sus propios nodos.

Capítulo 25: aspectos legales y éticos

Solución 25.1. (a) Una salida que iniciara por sí misma solicitudes hacia sitios web, quizá para «mezclar» el tráfico, iniciaría transmisiones. (b) Una salida que redirigiera conexiones a un destino diferente del especificado en la celda `BEGIN` estaría seleccionando al destinatario. (c) Una salida que insertara contenido en páginas HTTP o filtrara contenido por su significado seleccionaría o modificaría información. Son también ejemplos de conductas que pueden dar lugar al indicador `BadExit`.

Solución 25.2. Un cuchillo de cocina es legal, mientras una agresión no lo es. Una persona que lee mediante Tor un periódico censurado puede estar utilizando lícitamente una herramienta legal; esa misma persona comete un delito si accede sin autorización a un equipo, con Tor o sin él. Un operador de repetidores aporta capacidad de transmisión sin seleccionar contenidos, más parecido a un transportista que a un autor, siempre que no participe en actividades ilícitas. Las conclusiones jurídicas concretas dependen de la jurisdicción y de los hechos.

Solución 25.3. Explica que la dirección corresponde a una salida Tor, verificable con herramientas públicas del Tor Project; que el operador no conoce ni registra las identidades de los clientes y no puede identificarlos; y que el sitio reclamante puede, si lo desea, gestionar de forma diferente las conexiones de direcciones públicas de salida. En la práctica, el operador podría añadir una regla de política de salida que rechace la dirección de ese sitio si las ventajas operativas compensan la restricción.

Solución 25.4. La propuesta observa tráfico real de usuarios sin consentimiento, recopila datos que podrían revelar información y no son adecuados para publicar, como los dominios visitados, y ofrece un beneficio limitado. Contradice varios principios de seguridad de la investigación y en Italia podría plantear problemas penales y de protección de datos. Las alternativas más seguras incluyen utilizar agregados públicos de Metrics, medir la accesibilidad de sitios de noticias solo con el propio cliente Tor del investigador o pedir al Research Safety Board que evalúe un diseño de recuento respetuoso con la privacidad mediante agregación y ruido, como en las estadísticas de repetidores.

Solución 25.5. Es un ejercicio de investigación. Busca sentencias en bases jurídicas oficiales, como las decisiones publicadas de tribunales italianos o del Tribunal de Justicia de la Unión Europea, en lugar de depender solo de resúmenes de terceros. Para cada resolución registra tribunal, fecha, legislación aplicable y resultado, y distingue asuntos de operadores de repetidores de otros relacionados con usuarios de Tor.

Capítulo 26: correlación, identificación y ataques Sybil

Solución 26.1. La correlación de tráfico exige observar ambos extremos, mediante repetidores o redes subyacentes; los guardias, las restricciones de rutas y el ancho de banda medido la dificultan. La identificación de sitios web necesita únicamente el enlace cliente–guardia; existen defensas limitadas de relleno y la investigación continúa. Descubrir un guardia requiere un repetidor intermedio y la capacidad de provocar la construcción de circuitos; guardias

y vanguards ayudan. Un adversario Sybil administra muchos repetidores y las defensas incluyen mediciones de ancho de banda, familias, restricciones /16 y vigilancia. Los ataques de encaminamiento exigen influencia sobre un sistema autónomo o un punto de intercambio de Internet; las respuestas incluyen vigilancia BGP fuera del protocolo Tor.

Solución 26.2. Entre $100\,000$ visitas, se esperan aproximadamente 0,95 alertas verdaderas y $0{,}001 \cdot 99999 \approx 100$ falsos positivos. La proporción de alertas correctas es $0{,}95/(0{,}95 + 100) \approx 0{,}9\%$. Casi todas son falsas.

Solución 26.3. Con intervalos de dos segundos durante solo cinco segundos, quedan únicamente dos o tres muestras por serie y la correlación se vuelve inestable. Si todos los clientes siguen el mismo ritmo de tráfico, por ejemplo con la misma semilla aleatoria, las series resultan casi idénticas y se vuelve imposible asociarlas correctamente. Igualar ritmos es especialmente perturbador: es el principio del tráfico a velocidad constante de PipeNet (apartado 2.4).

Solución 26.4. Con una tasa constante de transmisión, las series de tráfico de entrada dejan de variar. La correlación de Pearson es matemáticamente indefinida porque la varianza es cero, o queda dominada por el ruido de red; el ataque falla. El coste es transmitir a la tasa máxima elegida incluso sin datos útiles. En el laboratorio pueden ser decenas o cientos de KB/s por cliente, varias veces el volumen de los datos de aplicación.

Solución 26.5. Con un guardia fijo, el adversario solo triunfa si el guardia seleccionado es malicioso, con probabilidad aproximada del 5%. En ese caso, también controla la salida en alrededor del 5% de los circuitos. Durante seis meses de uso intensivo, la probabilidad de al menos un compromiso se aproxima, por tanto, al 5%, mientras permanece a cero para usuarios con guardias honestos en este modelo simplificado. Con un guardia aleatorio por circuito, la probabilidad por circuito es aproximadamente $0{,}05 \cdot 0{,}05 = 0{,}0025$; después de miles de circuitos, el compromiso es casi seguro. Es la razón de los guardias persistentes y puede verificarse por simulación con la clase `Adversary` del laboratorio 7.

Capítulo 27: casos reales de ataques contra la red

Solución 27.1. En dirección al cliente, cada repetidor añade su propia capa de cifrado a la carga útil de la celda (capítulo 10). El guardia recibe una carga cifrada bajo las claves de los saltos intermedios y no puede leer lo escrito por el HSDir. En cambio, el comando exterior de celda forma parte de una cabecera que todos los repetidores leen y reenvían sin alterar.

Solución 27.2. Con tres guardias, la probabilidad de que al menos uno sea adversario es aproximadamente tres veces mayor y cada guardia malicioso observa parte del tráfico del cliente. Con uno solo, el riesgo de selección es menor y una elección honesta protege al cliente durante mucho tiempo. Las desventajas son que un único guardia malicioso observa todo el tráfico de ese cliente y que uno lento o inaccesible puede perjudicar la conectividad.

Solución 27.3. El laboratorio ya utiliza celdas `DROP` en lugar de solicitudes de extensión. El contador de `command.c` comprueba el comando exterior antes de descifrar, así que no depende del mensaje interior. En sentido opuesto, el código registra una advertencia y cierra el circuito con razón `PROTOCOL`, independientemente de que pertenezca a un cliente o a un repetidor.

Solución 27.4. Entre las señales posibles figuran repetidores incorporados al consenso exactamente en el mismo minuto, versiones poco comunes de tor y sistemas operativos coincidentes en descriptores, direcciones de la misma red o proveedor de alojamiento, puertos y sobrenombres parecidos y falta de contacto. Los falsos positivos incluyen operadores grandes legítimos que actualizan todos sus repetidores a la vez, proveedores populares y despliegues

automatizados idénticos. Ningún indicio demuestra por sí solo una administración común: combina evidencias y contacta con el operador cuando sea posible.

Solución 27.5. Un repetidor intermedio ve metadatos exteriores en claro: CircID, que cambia en cada salto y, por tanto, no atraviesa la ruta sin modificaciones; el comando de celda, incluidos **`RELAY`** y **`RELAY_EARLY`**, canal explotado en 2014, hoy cerrado en la dirección hacia el cliente y limitado a ocho celdas en el sentido opuesto; **`DESTROY`** y su razón; así como el momento y la cantidad de celdas. Los dos últimos continúan disponibles para confirmar tráfico pasivamente sin defensa completa. Las cargas útiles están cifradas por capas, pero con tor1 un repetidor malicioso podía marcarlas para que otro las reconociera (apartado 10.3). CGO (apartado 10.4) hace que tales modificaciones produzcan texto en claro irrecuperable en los saltos posteriores.

Capítulo 28: errores de seguridad operativa

Solución 28.1. El tráfico que elude Tor, los documentos abiertos mientras hay conexión y las filtraciones DNS pertenecen a la capa de aplicación; entre las defensas figuran Tor Browser, Tails o Whonix, `SafeSocks` y abrir archivos sin conexión. Los metadatos, el inicio de sesión en cuentas y la mezcla de identidades pertenecen a la capa de contenido: utiliza herramientas de eliminación de metadatos y sesiones separadas. El estilo de escritura, las horas de publicación y el uso de Tor en un conjunto de anonimato pequeño pertenecen a la capa de comportamiento: presta atención a los hábitos, los puentes y cuándo publicar.

Solución 28.2. Una solicitud DNS local viaja desde el equipo del usuario hasta el resolvedor del sistema, normalmente administrado por el proveedor de conectividad, e incluye el dominio del sitio web. Ese proveedor y cualquiera que observe la ruta al resolvedor pueden conocer qué sitio pretende visitar el usuario, aunque la conexión posterior utilice Tor.

Solución 28.3. Suscríbete a `EventType.STATUS_CLIENT` mediante Stem. Una solicitud que solo contenga una IP genera un evento de gravedad `WARN`, con acción `DANGEROUS_SOCKS` y argumentos `PROTOCOL` y `ADDRESS`. Una interfaz gráfica podría identificar la aplicación y el destino en una advertencia y explicar que es necesario corregir la configuración SOCKS de la aplicación.

Solución 28.4. Con obfs4, un observador local vería una conexión cifrada aparentemente sin estructura a una dirección de apariencia ordinaria y no podría establecer con certeza que era tráfico Tor. Persisten otros indicios: volumen de la conexión, momentos, duración inusual, conexiones semejantes y contenido de los mensajes o posibles motivos, que podrían reducir el conjunto de candidatos a las personas que se esperaba que hicieran aquel examen.

Solución 28.5. Es un ejercicio abierto. Una buena respuesta sigue la estructura de capítulo 4: adversarios y capacidades, qué puede observar cada uno en las capas de figura 28.1, protecciones elegidas y su justificación y lista explícita de riesgos residuales, como correlación por un adversario que observe ambos extremos o incautación de los dispositivos del usuario.

Capítulo 29: investigación académica sobre Tor

Solución 29.1. $H = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + 2\cdot\frac{1}{8}\cdot 3 = 1{,}75$ bits. $H_M = \log_2 4 = 2$ bits, así que $d = 0{,}875$. El número equivalente de usuarios equiprobables es $2^{1{,}75} \approx 3{,}4$.

Solución 29.2. (a) Simular una red con Shadow o emularla a escala pequeña mediante Chutney: se necesita el comportamiento real de tor, pero no implicar a usuarios reales. (b) Medir la red real mediante datos agregados públicos, como hace Tor Metrics (apartado 24.2). (c)

Emplear una simulación de comportamiento como TorPS con consensos reales que abarquen un período largo, posiblemente acompañada de análisis teórico.

Solución 29.3. Con `guard_lifetime` de 30 días, que produce una duración efectiva de 18–30 días tras retrotraer la fecha, un cliente elige guardias unas siete u ocho veces en 180 días y la simulación produce un 32,1% de compromiso. Con los 60 días predeterminados, una duración efectiva de 48–60 días produce unas tres selecciones y el 18,4%. Con 180 días de duración, una o dos selecciones producen el 8,6%. Aproximadamente, con k selecciones independientes de guardia la proporción comprometida se acerca a $1-(1-q)^k$, donde $q=5\%$; $k\approx 7{,}5$ da alrededor del 31%, coherente con la simulación de 30 días. Duraciones mayores implican menos selecciones y exponen a menos clientes nuevos, pero quienes seleccionaron un guardia malicioso siguen expuestos durante más tiempo.

Solución 29.4. Ninguno de los dos adversarios compromete clientes según el criterio del laboratorio, que exige observar ambos extremos. No obstante, sus observaciones son distintas: una salida maliciosa ve los destinos y cualquier tráfico de aplicación que permanezca sin cifrar, pero no las identidades de los clientes. Un guardia malicioso ve las IP de los clientes que lo seleccionaron, pero no sus destinos, y podría intentar identificar sitios web mediante sus patrones de tráfico (apartado 26.4).

Solución 29.5. Es un ejercicio de investigación. Reutilizar circuitos disminuye la cantidad diaria de nuevas selecciones y ralentiza la curva de compromiso sin guardias persistentes. La restricción /16 influye poco salvo que los repetidores adversarios estén concentrados en una misma red. Un adversario que controle un sistema autónomo cambia el modelo de manera más fundamental porque observa tráfico sin administrar repetidores, lo que requiere datos de topología de Internet como los de [Joh+13].

Capítulo 30: Arti

Solución 30.1. Capítulo 5: `tor-llcrypto`. Capítulo 7: `tor-proto` para canales y `tor-chanmgr`. Capítulo 8: `tor-cell`. Capítulos 9–11: `tor-proto` para circuitos, cifrado, flujos y control de flujo. Capítulo 12: `tor-netdoc`, `tor-dirmgr` y `tor-dirclient`. Capítulo 13: `tor-circmgr`, `tor-guardmgr` y `tor-relay-selection`. Capítulo 14: `tor-hsclient`, `tor-hsservice` y `tor-hscrypto`. Capítulo 15: `equix` y `hashx` para pruebas de trabajo. Capítulo 16: `tor-ptmgr`.

Solución 30.2. Ventajas: menos código, menor superficie de ataque, ausencia de casos especiales como CircID de dos bytes y ninguna criptografía débil obsoleta que mantener. Inconvenientes: Arti no puede comunicarse con repetidores muy antiguos. No obstante, la red pública Tor ya excluye versiones no recomendadas, por lo que el efecto práctico concierne sobre todo a redes privadas e instalaciones desactualizadas.

Solución 30.3. `decode_cell` lee el comando del quinto byte y lo utiliza para determinar si la celda tiene longitud variable. En ese caso, los dos bytes siguientes indican la longitud y el tamaño total es $7+$ el valor declarado; de lo contrario, el tamaño es 514 bytes. Si todavía no han llegado suficientes datos, devuelve `Ok(None)` en vez de considerar la celda incompleta un error. El llamante vuelve a intentarlo cuando dispone de más bytes. Es el mismo comportamiento necesario para un decodificador de flujos como el del laboratorio 2.

Solución 30.4. Sí. El proxy SOCKS de Arti aísla por defecto los flujos con credenciales SOCKS diferentes y separa también los distintos puertos de escucha. Observa los circuitos desde el lado de los repetidores, por ejemplo mediante el ControlPort de una salida, o en los registros debug de Arti. Si se utiliza Arti como biblioteca, se ofrece aislamiento explícito

mediante `TorClient::isolated_client` o tokens de aislamiento de `StreamPrefs`. En Arti 2.6.0, `cfg.rs` todavía documenta como trabajo futuro la configuración equivalente por puerto.

Solución 30.5. Es un ejercicio de programación. Carga el archivo TOML del laboratorio en un `TorClientConfig`, inicia el cliente con `TorClient::create_bootstrapped` y abre un flujo con `connect(("127.0.0.1", 8080))`. Envía una solicitud HTTP por ese flujo. La configuración debe permitir direcciones locales, como el archivo generado por el laboratorio 20.

Capítulo 31: problemas abiertos

Solución 31.1. Ejemplos en la fecha de referencia del capítulo: Propuesta 332, sobre ntor-v3, está *Closed*, es decir, implementada e incorporada a las especificaciones. Propuesta 324, sobre control de congestión, está *Finished*, implementada pero pendiente de integrar completamente su texto en las especificaciones. Propuesta 359, sobre CGO, está *Open*: ya presente en el código, pero su propuesta continúa en discusión. Comprueba las cabeceras de cada propuesta y el índice de estados del repositorio para obtener actualizaciones.

Solución 31.2. Con el formato tradicional, el cálculo sencillo de tamaños requiere $\lceil 1184/498 \rceil = 3$ celdas para la clave pública ML-KEM-768 de 1184 bytes y $\lceil 1088/498 \rceil = 3$ más para el texto cifrado de 1088 bytes: seis en total. No obstante, el protocolo actual ni siquiera permite fragmentar de esa forma un mensaje relay individual. Propuesta 340 proporciona el encuadre necesario, permitiendo que un mensaje abarque varias celdas y que varios mensajes compartan una celda. Su cuerpo propuesto de celda relay es de 493 bytes, no 498, porque el formato nuevo reserva 16 bytes para autenticación, $509 - 16 = 493$, como explicamos en capítulo 8. El primer fragmento de un mensaje transporta como máximo 490 bytes útiles porque incluye una cabecera de encuadre. La propuesta mantiene actualmente un límite de 493 bytes para cuerpos ordinarios, así que transportar un `EXTEND2` poscuántico mayor exigirá además una versión nueva del subprotocolo `Relay` que amplíe ese límite. Incluso con esa extensión, la clave pública sigue ocupando tres celdas y el texto cifrado otras tres. Como viajan en *direcciones opuestas*, del cliente al repetidor y de vuelta, no pueden empaquetarse juntos. El total sigue siendo seis celdas.

Solución 31.3. Si cada cliente descargara un subconjunto distinto de repetidores, un adversario que conociera o influyera en el conjunto de cierto cliente podría reconocer sus circuitos. Utilizar repetidores disponibles solo para ese cliente revelaría quién construyó el circuito. Incluso sin conocer previamente el subconjunto, observar los repetidores que utiliza a lo largo del tiempo puede reducir progresivamente el número de clientes posibles.

Solución 31.4. Filtra circuitos con propósito `CONFLUX_LINKED` y agrúpalos por `CONFLUX_ID` (capítulo 18); compara después las rutas de cada pareja. Ambas ramas terminan siempre en la misma salida, como cabía esperar. En la red de laboratorio suelen compartir también otros repetidores. Por ejemplo, `[test007e, test003r, test006e]` y `[test003r, test007e, test006e]` utilizan exactamente los mismos nodos en otro orden. No es un defecto de tor. La red privada tiene pocos repetidores elegibles y Propuesta 329 formula su recomendación como `SHOULD NOT`, no una prohibición absoluta. Compartir así los nodos es mucho menos frecuente en la red pública con miles de repetidores.

Solución 31.5. Es un ejercicio abierto. El formato de referencia está en Propuesta 001: una cabecera con número, título, autor, fecha y estado, seguida de motivación, diseño, análisis de seguridad y compatibilidad, plan de implementación y plan de despliegue.

Apéndice B: criptografía complementaria

Solución B.1. De los 256 bits, cinco tienen valor fijo: los tres inferiores, el más alto y el inmediatamente inferior. Quedan 2^{251} valores diferentes frente a 2^{256} antes del ajuste. Esto no debilita el esquema: el nivel de seguridad de la curva es aproximadamente de 128 bits en cualquier caso.

Solución B.2. De $y = (u-1)/(u+1)$ se deduce $u = (1+y)/(1-y) \bmod p$. En Python, lee y de los 32 bytes little-endian después de borrar el bit de signo, calcula `(1 + y) * pow(1 - y, P - 2, P) % P` y codifica el resultado en 32 bytes little-endian. Aplicar `montgomery_to_edwards` con el bit de signo original de x recupera la clave Edwards inicial (para $y \neq 1$).

Solución B.3. Una celda modificada aleatoriamente supera ambas comprobaciones con probabilidad aproximada $2^{-16} \cdot 2^{-32} = 2^{-48}$: dos bytes de `recognized` a cero y cuatro bytes correctos de resumen. Pero el ataque de marcado no exige que la celda alterada sea *aceptada*. Un cómplice adversario que la observe, o el extremo que la rechaza, puede reconocer la modificación por sus efectos, como el cierre del circuito o contenidos predecibles. Esa observación puede bastar para vincular dos puntos del circuito.

Bibliografía

[03] *Decreto legislativo 30 giugno 2003, n. 196, Codice in materia di protezione dei dati personali*. Texto consolidado publicado por la autoridad italiana de protección de datos (Garante per la protezione dei dati personali). 30 de jun. de 2003. URL: https://www.garanteprivacy.it/ (visitado 23-09-2026).

[17] *Legge 20 novembre 2017, n. 167, art. 24 (termine di conservazione dei dati di traffico per finalità di accertamento e repressione dei reati)*. Gazzetta Ufficiale, Serie Generale n.º 277, 27.11.2017. 20 de nov. de 2017. URL: https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:2017-11-20;167.

[22] *Regolamento (UE) 2022/2065 del Parlamento europeo e del Consiglio, del 19 ottobre 2022, relativo a un mercato unico dei servizi digitali (regolamento sui servizi digitali)*. DO L 277 de 27.10.2022. 19 de oct. de 2022. URL: https://eur-lex.europa.eu/eli/reg/2022/2065/oj.

[Bau+07] Kevin Bauer, Damon McCoy, Dirk Grunwald, Tadayoshi Kohno y Douglas Sicker. «Low-Resource Routing Attacks Against Tor». En: *ACM Workshop on Privacy in the Electronic Society (WPES)*. 2007.

[Boc+24] Cecylia Bocovich, Arlo Breault, David Fifield, Serene y Xiaokang Wang. «Snowflake, a censorship circumvention system using temporary WebRTC proxies». En: *Proceedings of the 33rd USENIX Security Symposium*. 2024.

[BPW13] Alex Biryukov, Ivan Pustogarov y Ralf-Philipp Weinmann. «Trawling for Tor Hidden Services: Detection, Measurement, Deanonymization». En: *IEEE Symposium on Security and Privacy*. 2013.

[Cha81] David L. Chaum. «Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms». En: *Communications of the ACM* 24.2 (1981), págs. 84-88.

[Cha88] David Chaum. «The Dining Cryptographers Problem: Unconditional Sender and Recipient Untraceability». En: *Journal of Cryptology* 1.1 (1988), págs. 65-75.

[DDM03] George Danezis, Roger Dingledine y Nick Mathewson. «Mixminion: Design of a Type III Anonymous Remailer Protocol». En: *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. 2003, págs. 2-15.

[Día+02] Claudia Díaz, Stefaan Seys, Joris Claessens y Bart Preneel. «Towards Measuring Anonymity». En: *Privacy Enhancing Technologies (PET 2002)*. LNCS. 2002.

[Din14] Roger Dingledine. *Tor security advisory: «relay early» traffic confirmation attack*. 30 de jul. de 2014. URL: https://blog.torproject.org/tor-security-advisory-relay-early-traffic-confirmation-attack/ (visitado 23-09-2026).

[DM06] Roger Dingledine y Nick Mathewson. «Anonymity Loves Company: Usability and the Network Effect». En: *Proceedings of the Fifth Workshop on the Economics of Information Security (WEIS 2006)*. 2006.

[DMS04] Roger Dingledine, Nick Mathewson y Paul Syverson. «Tor: The Second-Generation Onion Router». En: *Proceedings of the 13th USENIX Security Symposium*. 2004, págs. 303-320.

[EDG09] Nathan S. Evans, Roger Dingledine y Christian Grothoff. «A Practical Congestion Attack on Tor Using Long Paths». En: *Proceedings of the 18th USENIX Security Symposium*. 2009.

[Ele] Electronic Frontier Foundation. *Legal FAQ for Tor Relay Operators*. URL: `https://community.torproject.org/relay/community-resources/eff-tor-legal-faq/` (visitado 23-09-2026).

[Fif+15] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann y Vern Paxson. «Blocking-resistant communication through domain fronting». En: *Proceedings on Privacy Enhancing Technologies* 2015.2 (2015), págs. 46-64.

[Fro+19] Sergey Frolov, Jack Wampler, Sze Chuen Tan, J. Alex Halderman, Nikita Borisov y Eric Wustrow. «Conjure: Summoning Proxies from Unused Address Space». En: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (*CCS*). 2019.

[GRS96] David M. Goldschlag, Michael G. Reed y Paul F. Syverson. «Hiding Routing Information». En: *Information Hiding, First International Workshop*. Vol. 1174. LNCS. Springer, 1996, págs. 137-150.

[GSU13] Ian Goldberg, Douglas Stebila y Berkant Ustaoglu. «Anonymity and One-Way Authentication in Key Exchange Protocols». En: *Designs, Codes and Cryptography* 67.2 (2013), págs. 245-269.

[Jan+14] Rob Jansen, Florian Tschorsch, Aaron Johnson y Björn Scheuermann. «The Sniper Attack: Anonymously Deanonymizing and Disabling the Tor Network». En: *Proceedings of the Network and Distributed System Security Symposium* (*NDSS*). 2014.

[JH12] Rob Jansen y Nicholas Hopper. «Shadow: Running Tor in a Box for Accurate and Efficient Experimentation». En: *Proceedings of the Network and Distributed System Security Symposium* (*NDSS*). 2012.

[JJ16] Rob Jansen y Aaron Johnson. «Safely Measuring Tor». En: *ACM Conference on Computer and Communications Security* (*CCS*). 2016.

[Joh+13] Aaron Johnson, Chris Wacek, Rob Jansen, Micah Sherr y Paul Syverson. «Users Get Routed: Traffic Correlation on Tor by Realistic Adversaries». En: *ACM Conference on Computer and Communications Security* (*CCS*). 2013.

[Jua+14] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz y Rachel Greenstadt. «A Critical Evaluation of Website Fingerprinting Attacks». En: *ACM Conference on Computer and Communications Security* (*CCS*). 2014.

[Jus14] Just Security. *Video clip of former director of NSA and CIA: «We kill people based on metadata»*. en el Johns Hopkins Foreign Affairs Symposium, 1 de abril de 2014. 2014. URL: `https://www.justsecurity.org/10318/video-clip-director-nsa-cia-we-kill-people-based-metadata/` (visitado 24-09-2026).

[MD05] Steven J. Murdoch y George Danezis. «Low-Cost Traffic Analysis of Tor». En: *IEEE Symposium on Security and Privacy*. 2005.

[NBH18] Milad Nasr, Alireza Bahramali y Amir Houmansadr. «DeepCorr: Strong Flow Correlation Attacks on Tor Using Deep Learning». En: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[NIS24] NIST. *FIPS 203* (*ML-KEM*), *FIPS 204* (*ML-DSA*), *FIPS 205* (*SLH-DSA*). los tres estándares poscuánticos, publicados conjuntamente. 13 de ago. de 2024. URL: https://csrc.nist.gov/pubs/fips/203/final (visitado 24-09-2026).

[OON] OONI. *Specification: Tor* (*ts-023*) *and Tor using Snowflake* (*ts-030*). URL: https://github.com/ooni/spec/tree/master/nettests (visitado 23-09-2026).

[ØS06] Lasse Øverlier y Paul Syverson. «Locating Hidden Servers». En: *Proceedings of the 2006 IEEE Symposium on Security and Privacy*. 2006, págs. 100-114.

[Pan+11] Andriy Panchenko, Lukas Niessen, Andreas Zinnen y Thomas Engel. «Website Fingerprinting in Onion Routing Based Anonymization Networks». En: *ACM Workshop on Privacy in the Electronic Society* (*WPES*). 2011.

[Per+18] Mike Perry, Erinn Clark, Steven Murdoch y Georg Koppen. *The Design and Implementation of the Tor Browser*. Borrador. 15 de jun. de 2018. URL: https://2019.www.torproject.org/projects/torbrowser/design/ (visitado 23-09-2026).

[PH10] Andreas Pfitzmann y Marit Hansen. *A terminology for talking about privacy by data minimization: Anonymity, Unlinkability, Undetectability, Unobservability, Pseudonymity, and Identity Management*. Versión 0.34. 2010.

[RR98] Michael K. Reiter y Aviel D. Rubin. «Crowds: Anonymity for Web Transactions». En: *ACM Transactions on Information and System Security* 1.1 (1998), págs. 66-92.

[San13] Runa A. Sandvik. *Harvard Student Receives F For Tor Failure While Sending «Anonymous» Bomb Threat*. 18 de dic. de 2013. URL: https://www.forbes.com/sites/runasandvik/2013/12/18/harvard-student-receives-f-for-tor-failure-while-sending-anonymous-bomb-threat/ (visitado 24-09-2026).

[Sir+18] Payap Sirinam, Mohsen Imani, Marc Juarez y Matthew Wright. «Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning». En: *ACM Conference on Computer and Communications Security* (*CCS*). 2018.

[Sun+15] Yixin Sun, Anne Edmundson, Laurent Vanbever, Oscar Li, Jennifer Rexford, Mung Chiang y Prateek Mittal. «RAPTOR: Routing Attacks on Privacy in Tor». En: *USENIX Security Symposium*. 2015.

[Syv11] Paul Syverson. «A Peel of Onion». En: *Proceedings of the 27th Annual Computer Security Applications Conference* (*ACSAC*). 2011, págs. 123-137.

[Taia] Tails. *How Tails works*. URL: https://tails.net/about/ (visitado 23-09-2026).

[Taib] Tails. *Warnings: Tails is safe but not magic!* URL: https://tails.net/doc/about/warnings/ (visitado 23-09-2026).

[TG23] Lindsey Tulloch y Ian Goldberg. «Lox: Protecting the Social Graph in Bridge Distribution». En: *Proceedings on Privacy Enhancing Technologies* 2023.1 (2023), págs. 494-509.

[Thea] The Tor Project. *History*. URL: https://www.torproject.org/about/history/ (visitado 23-09-2026).

[Theb] The Tor Project. *Onionbalance: documentazione e guida per gli onion service v3*. URL: https://onionservices.torproject.org/apps/base/onionbalance/ (visitado 23-09-2026).

[Thec] The Tor Project. *Reproducible Metrics*. URL: https://metrics.torproject.org/reproducible-metrics.html (visitado 23-09-2026).

[Thed] The Tor Project. *Tor Browser User Manual*. URL: https://tb-manual.torproject.org/ (visitado 23-09-2026).

[Thee] The Tor Project. *Tor Specifications*. URL: https://spec.torproject.org/ (visitado 23-09-2026).

[Thef] The Tor Project. *Tor Support: Staying anonymous*. URL: https://support.torproject.org/faq/staying-anonymous/ (visitado 23-09-2026).

[Theg] The Tor Project. *Tor Support: Using Bridges; Unblocking Tor*. URL: https://support.torproject.org/little-t-tor/circumvention/using-bridges/ (visitado 23-09-2026).

[The22] The Tor Project. *Malicious relays and the health of the Tor network*. 25 de abr. de 2022. URL: https://blog.torproject.org/malicious-relays-health-tor-network/ (visitado 23-09-2026).

[The23a] The Tor Project. *Introducing Proof-of-Work Defense for Onion Services*. 23 de ago. de 2023. URL: https://blog.torproject.org/introducing-proof-of-work-defense-for-onion-services/ (visitado 23-09-2026).

[The23b] The Tor Project. *Tor is slow right now. Here is what is happening*. 7 de feb. de 2023. URL: https://blog.torproject.org/tor-network-ddos-attack/ (visitado 23-09-2026).

[The24a] The Tor Project. *Hiding in plain sight: Introducing WebTunnel*. https://blog.torproject.org/introducing-webtunnel-evading-censorship-by-hiding-in-plain-sight/. 12 de marzo de 2024. 2024.

[The24b] The Tor Project. *Making new connections: from BridgeDB to Rdsys*. 2024. URL: https://blog.torproject.org/making-connections-from-bridgedb-to-rdsys/ (visitado 24-09-2026).

[The24c] The Tor Project. *Uniting for Internet Freedom: Tor Project & Tails Join Forces*. 26 de sep. de 2024. URL: https://blog.torproject.org/tor-tails-join-forces/ (visitado 24-09-2026).

[Tor] Tor Research Safety Board. *Tor Research Safety Board*. URL: https://research.torproject.org/safetyboard/ (visitado 23-09-2026).

[Tor25] Tor Project Forum. *Feedback from China [April 2025]: increased GFW censorship; obfs4/meek/snowflake unusable, webtunnel connects but is quickly blocked*. 2025. URL: https://forum.torproject.org/t/feedback-from-china-april-2025-increased-gfw-censorship-obfs4-meek-snowflake-unusable-webtunnel-connects-but-is-quickly-blocked/18233 (visitado 24-09-2026).

[WBU13] WBUR News. *Student Charged For Bomb Threats At Harvard*. 17 de dic. de 2013. URL: https://www.wbur.org/news/2013/12/17/harvard-bomb-hoax-student-charges (visitado 23-09-2026).

[Who] Whonix. *onion-grater*. URL: https://www.whonix.org/wiki/Onion-grater (visitado 23-09-2026).

[WL14] Philipp Winter y Stefan Lindskog. «Spoiled Onions: Exposing Malicious Tor Exit Relays». En: *arXiv:1401.4917*. incorporado después, con otros autores, en *Privacy Enhancing Technologies Symposium* (PETS 2014). 2014.

Índice analítico

AUTH_CHALLENGE, 39, 44, 47, 49, 56, 60, 165
AUTHENTICATE, 39, 45, 49, 50, 56
AUTHORIZE, 56

CERTS, 33, 39, 41, 43–45, 47, 49, 56, 60, 165, 265
CREATE, 56
CREATE2, 33, 55, 56, 60, 64, 67–69, 71, 75, 76, 265, 275
CREATE_FAST, 28, 56, 70, 71, 73, 76, 132, 266
CREATED, 56
CREATED2, 56, 60, 67–69, 76, 275
CREATED_FAST, 56

DESTROY, 56, 60, 64, 69, 76, 81, 90, 91, 94, 95, 132, 217, 265, 266, 268, 280

iniciador, 37

NETINFO, 39, 45–49, 54, 56, 60, 165

PADDING, 46, 50, 54, 56, 76, 85, 86, 265, 276
PADDING_NEGOTIATE, 56, 265
Propuestas
  001, 243, 248, 282
  214, 55
  216, 71, 75
  220, 38
  224, 118
  236, 112
  271, 112
  289, 58, 83, 92, 95, 96
  291, 112
  300, 244, 246, 248
  302, 212
  323, 246, 248
  324, 59, 73, 92–94, 96, 243, 267, 282
  327, 130
  329, 59, 164, 243, 245, 247, 282
  332, 72, 75, 77, 243, 282
  333, 113
  338, 46, 264
  339, 59
  340, 59, 245, 247, 282
  346, 34
  348, 246
  350, 70, 244, 245
  355, 244, 248
  359, 59, 77, 82, 244, 267, 282
  362, 246

RELAY, 46, 56, 57, 60, 83, 171, 173, 215, 250, 275, 280
RELAY_EARLY, 46, 56, 57, 60, 64, 65, 69, 83, 85, 86, 215–219, 265, 275, 280
respondedor, 37

torrc
  __LeaveStreamsUnattached, 153, 156
  AccountingMax, 182, 217
  AssumeReachable, 162
  AutomapHostsOnResolve, 148
  BridgeDistribution, 136, 186, 187
  ClientOnionAuthDir, 177, 179
  ContactInfo, 182
  ControlPort, 148, 151
  CookieAuthentication, 148
  DirAuthority, 161, 273
  DirPortFrontPage, 203
  DNSPort, 91, 148
  EntryNodes, 147
  ExcludeNodes, 147
  ExitNodes, 147
  ExitPolicy, 187
  ExitPolicyRejectPrivate, 183
  ExitPolicyRejectPrivate 0, 183
  ExitRelay 0, 182
  FamilyId, 187
  HashedControlPassword, 148
  HiddenServiceDir, 175, 176, 179, 180
  HiddenServiceNonAnonymousMode, 179

HiddenServicePort, 123, 176, 180
HiddenServiceSingleHopMode, 179
HSLayer2Nodes, 114
HSLayer3Nodes, 114
IsolateSOCKSAuth, 190, 211
KeepalivePeriod, 38
Log, 173
MaxCircuitDirtiness, 147
MaxMemInQueues, 128, 133, 134, 272
MyFamily, 185–187, 276
OfflineMasterKey, 187
OfflineMasterKey 1, 183
ReducedExitPolicy, 183, 187
RelayBandwidthBurst, 182
RelayBandwidthRate, 128, 182, 276
SafeLogging, 148, 149, 172, 203, 239
SafeSocks, 223, 224, 280
SessionGroup, 147, 273
SigningKeyLifetime, 44
SocksPort, 146, 147, 149
StrictNodes, 147
TestingDirAuthVoteGuard, 162, 274
TestingLinkCertLifetime, 44
TestingTorNetwork, 161, 166, 239, 273
TestSocks, 223, 224
TransPort, 148
UseBridges, 272
V3AuthDistDelay, 102
V3AuthNIntervalsValid, 102
V3AuthVoteDelay, 102
V3AuthVotingInterval, 102, 162, 274

VERSIONS, 39, 47, 48, 53, 54, 56, 60, 62–64, 165, 172, 237, 250, 265, 266, 276
VPADDING, 39, 48, 56